

# ARGUS — Smart Pest Monitoring System

## Technical Manual

Qian Runzhe — Electronic & Computer Engineering, Ngee Ann Polytechnic

Version 1.1 · 11 August 2026

## Table of Contents

|                                                                    |    |
|--------------------------------------------------------------------|----|
| Chapter 1 — System overview and architecture .....                 | 7  |
| 1.1 Role in the system .....                                       | 7  |
| 1.2 Inventory .....                                                | 7  |
| 1.3 Design principles .....                                        | 12 |
| 1.4 Production vision vs the testbed — read this first.....        | 13 |
| 1.5 End-to-end dataflow (the detection rail).....                  | 13 |
| 1.6 The parallel live rail (Kinesis Video Streams).....            | 15 |
| 1.7 Deployment topology — what runs where .....                    | 16 |
| 1.8 The two cameras and the two models.....                        | 17 |
| 1.9 Component map — where everything is documented .....           | 19 |
| 1.10 Operations / reproduction.....                                | 20 |
| 1.11 Cross-references .....                                        | 23 |
| Chapter 2 — AWS cloud backend.....                                 | 24 |
| 2.1 Role in the system .....                                       | 24 |
| 2.2 Inventory .....                                                | 25 |
| 2.3 S3 buckets.....                                                | 28 |
| 2.4 DynamoDB tables .....                                          | 31 |
| 2.5 Lambda: pest-detection-processor (the detection pipeline)..... | 35 |
| 2.6 Lambda: pest-monitoring-api (the dashboard API).....           | 51 |
| 2.7 Lambda: pest-camera-scheduler .....                            | 57 |
| 2.8 Lambda: kvs-hls-handler .....                                  | 57 |
| 2.9 Lambda: pest-model-watchdog .....                              | 57 |
| 2.10 Lambda layer: fyp-pillow.....                                 | 58 |
| 2.11 API Gateway: HTTP API zwpcbivmsj .....                        | 59 |
| 2.12 Cognito .....                                                 | 60 |
| 2.13 CloudFront: E1423RGLAXWNSI.....                               | 61 |

|                                                                                                |     |
|------------------------------------------------------------------------------------------------|-----|
| 2.14 Kinesis Video Streams .....                                                               | 62  |
| 2.15 EventBridge schedules .....                                                               | 62  |
| 2.16 SES .....                                                                                 | 63  |
| 2.17 IAM roles and policies.....                                                               | 64  |
| 2.18 Rekognition Custom Labels (backend view) .....                                            | 67  |
| 2.19 Operations / reproduction.....                                                            | 68  |
| 2.20 Cross-references .....                                                                    | 70  |
| Chapter 3 — Detection models and the training pipeline.....                                    | 71  |
| 3.1 Role in the system .....                                                                   | 71  |
| 3.2 Inventory .....                                                                            | 72  |
| 3.3 Rekognition Custom Labels primer.....                                                      | 75  |
| 3.4 The moth model (inherited).....                                                            | 76  |
| 3.5 The armyworm model ladder.....                                                             | 77  |
| 3.6 The CAG holdout discipline.....                                                            | 83  |
| 3.7 <code>convert_to_manifest.py</code> — YOLO to Ground Truth manifest .....                  | 84  |
| 3.8 <code>convert_other_caterpillar_new.py</code> — the purchased-set variant.....             | 86  |
| 3.9 <code>merge_manifests.py</code> and <code>upload_images.py</code> — combine and ship ..... | 86  |
| 3.10 <code>rank_purchased.py</code> — quality-rank the purchased set .....                     | 87  |
| 3.11 The build scripts — leak-proof dataset assembly (v7.2 → v7.4 → v8 → v9) .....             | 88  |
| 3.12 The evaluate scripts — whole-image holdout arbitration .....                              | 91  |
| 3.13 The manifest line format .....                                                            | 93  |
| 3.14 Training procedure, end to end.....                                                       | 94  |
| 3.15 Model wiring and rollback .....                                                           | 96  |
| 3.16 Inference-side algorithms — where they live and why they exist.....                       | 96  |
| 3.17 Operations / reproduction.....                                                            | 98  |
| 3.18 Cross-references .....                                                                    | 100 |
| Chapter 4 — ARGUS dashboard (web frontend) .....                                               | 101 |
| 4.1 Role in the system .....                                                                   | 101 |
| 4.2 Inventory .....                                                                            | 102 |
| 4.3 Architecture: vanilla JS ES modules, no framework.....                                     | 104 |
| 4.4 <code>config.js</code> .....                                                               | 105 |
| 4.5 <code>auth.js</code> — Cognito sign-in .....                                               | 106 |
| 4.6 <code>api.js</code> — the API client .....                                                 | 107 |

|                                                                             |     |
|-----------------------------------------------------------------------------|-----|
| 4.7 state.js — shared state and the IndexedDB image cache .....             | 108 |
| 4.8 utils.js — helpers .....                                                | 109 |
| 4.9 Gallery (gallery.js + modal.js) .....                                   | 110 |
| 4.10 Bounding boxes and verification (bbox.js) .....                        | 113 |
| 4.11 Cameras and settings (settings.js).....                                | 115 |
| 4.12 Live view (live.js).....                                               | 118 |
| 4.13 Analytics (analytics.js) .....                                         | 119 |
| 4.14 Notifications and toasts.....                                          | 121 |
| 4.15 Dormant module: costs.js .....                                         | 121 |
| 4.16 Design system: liquid glass (styles.css v5.2) .....                    | 122 |
| 4.17 Operations / reproduction.....                                         | 123 |
| 4.18 Cross-references .....                                                 | 126 |
| Chapter 5 — Edge platform: Unitree Go2 + Jetson Orin .....                  | 127 |
| 5.1 Role in the system .....                                                | 127 |
| 5.2 Inventory .....                                                         | 128 |
| 5.3 Hardware and wiring .....                                               | 130 |
| 5.4 DDS environment: setup_go2.sh .....                                     | 132 |
| 5.5 USLAM navigation .....                                                  | 133 |
| 5.6 Map profiles: map_profiles.md .....                                     | 135 |
| 5.7 go2_patrol_gated.py — the gated patrol node .....                       | 137 |
| 5.8 pose.py — one-shot pose grab .....                                      | 143 |
| 5.9 The tests harness (robot/tests/) .....                                  | 144 |
| 5.10 kvs-controller.service — the A8 live-stream producer .....             | 145 |
| 5.11 Gimbal control map (SIYI A8 Mini).....                                 | 147 |
| 5.12 Operations and reproduction .....                                      | 148 |
| 5.13 patrol-scheduler.service — auto-launch on the dashboard schedule ..... | 153 |
| 5.14 Cross-references .....                                                 | 154 |
| Chapter 6 — Edge platform: mini PC (moth camera) .....                      | 156 |
| 6.1 Role in the system .....                                                | 156 |
| 6.2 Inventory .....                                                         | 157 |
| 6.3 Topology and networking .....                                           | 158 |
| 6.4 kvs_controller.py — the streaming daemon.....                           | 158 |
| 6.5 run_kvs_controller.sh — the wrapper script.....                         | 163 |

|                                                                                                     |     |
|-----------------------------------------------------------------------------------------------------|-----|
| 6.6 kvs-controller.service — the systemd unit .....                                                 | 164 |
| 6.7 capture_and_upload_v4_armyworm.py — the still-frame path.....                                   | 164 |
| 6.8 reverse-tunnel-fyp.service — the self-healing reverse tunnel .....                              | 168 |
| 6.9 Access recipe: laptop → Orin → tunnel → VM.....                                                 | 169 |
| 6.10 Operations / reproduction.....                                                                 | 170 |
| 6.11 Cross-references .....                                                                         | 174 |
| Chapter 7 — ARGUS one-click deployer .....                                                          | 175 |
| 7.1 Role in the system .....                                                                        | 175 |
| 7.2 Inventory .....                                                                                 | 176 |
| 7.3 app.py — the desktop shell and the Api bridge .....                                             | 178 |
| 7.4 deploy.py — the stage engine.....                                                               | 181 |
| 7.5 training.py — the in-app training pipeline .....                                                | 202 |
| 7.6 The UI — screens map .....                                                                      | 205 |
| 7.7 build.ps1 — packaging .....                                                                     | 207 |
| 7.8 The two-round rehearsal plan .....                                                              | 208 |
| 7.9 Security doctrine summary .....                                                                 | 210 |
| 7.10 Operations / reproduction.....                                                                 | 210 |
| 7.11 Cross-references .....                                                                         | 213 |
| Chapter 8 — Reproducing the system from scratch .....                                               | 214 |
| 8.1 Role in the system .....                                                                        | 214 |
| 8.2 Inventory .....                                                                                 | 215 |
| 8.3 Prerequisites (both paths).....                                                                 | 217 |
| 8.4 Path A — automated deployment with ARGUS .....                                                  | 218 |
| 8.5 Path A — day-1 post-deploy tasks (worked example: the 2026-08-10/11 production migration) ..... | 225 |
| 8.6 Path B — manual resource creation, in order .....                                               | 228 |
| 8.7 Path B — Lambda code upload from the local mirror .....                                         | 238 |
| 8.8 Device provisioning (checklist level) .....                                                     | 239 |
| 8.9 Account-portability caveats (critical — read before either path) .....                          | 240 |
| 8.10 Smoke-test suite (run after either path).....                                                  | 242 |
| 8.11 Operations — cost profile, idle check, teardown .....                                          | 244 |
| 8.12 Cross-references .....                                                                         | 245 |
| Chapter 9 — Appendix: registries and reference tables .....                                         | 246 |

|                                               |     |
|-----------------------------------------------|-----|
| 9.1 Role in the system .....                  | 246 |
| 9.2 Inventory .....                           | 246 |
| 9.3 AWS resource registry .....               | 247 |
| 9.4 Device registry .....                     | 263 |
| 9.5 Environment-variable tables .....         | 266 |
| 9.6 Repository file map .....                 | 269 |
| 9.7 Camera registry .....                     | 271 |
| 9.8 Glossary of project terms.....            | 274 |
| 9.9 Document conventions and change log ..... | 277 |
| 9.10 Operations / reproduction.....           | 278 |
| 9.11 Cross-references .....                   | 280 |

Prepared for Dr. Yan Li (supervisor), Ngee Ann Polytechnic staff, and the next student cohort: this manual is both the technical reference and the project handover document. Client: Changi Airport Group. Site: Jewel Changi Airport, Shiseido Forest Valley.

**Where things live.** Repository file paths are relative to the project repository root (the repo holding `lambda/`, `web/`, `robot/`, `deployer/`, `datasets/`, `migration/`, `docs/`). AWS resources are located by account, ARN, console path, or CLI command. The system runs on the NP production account 506868652945 (stack deployed 2026-08-10, both models trained there, validated end to end 2026-08-12). The development account 366356442579, where the system was built, is retired from operation and appears only in explicitly historical notes.

**Getting the repository.** The repository is distributed as a snapshot archive stored in the production account — the production account is the single handover point; there is deliberately no external git hosting. Console: account 506868652945 → S3 → `argus-frames-506868652945` → `handover/` → download the newest `argus-repo-snapshot-*.zip`. CLI: `aws s3 cp s3://argus-frames-506868652945/handover/argus-repo-snapshot-20260812.zip . --profile your-profile`. The snapshot holds all code, docs, deployment tooling, dataset scripts and manifests, the evaluation holdout, and this manual. It does not hold bulk training images (already under `training-data/` in the same bucket), the vendor-licensed purchased image set (non-redistributable; obtain from the project owner), or credentials of any kind.

**How to use this manual.** To understand the system, read Chapter 1, then the chapter for your area. To operate the running system, each chapter ends with an Operations section. To reproduce the system on a new AWS account, go straight to Chapter 8; it points back into Chapters 2-7 where depth is needed. Every AWS procedure gives both the console click-path and the CLI command; values you must supply yourself are shown in **bold**, and inside command text they are wrapped in **\*\*double asterisks\*\*** — replace the whole wrapped token, asterisks included.

**Conventions.** The code is ground truth: where a chapter flags “doc lag”, the repository documentation disagreed with the code and the code was followed. No credential values appear anywhere in this manual; credential storage locations are named instead. Non-secret identifiers (account id, ARNs, bucket names, IPs) are included deliberately: they are needed to operate and audit the system. Per-chapter “as of” stamps mark content freshness.

# Chapter 1 — System overview and architecture

This chapter explains what ARGUS is, the design principles behind it, the full end-to-end dataflow, and where every piece of the system physically runs. *As of 2026-08-14 (W19). Repository file paths in this manual are relative to the project repository root. The system runs on the NP production account 506868652945: the stack was deployed there 2026-08-10 by the ARGUS deployer, both models were retrained there, end-to-end validation passed 2026-08-12, and the handover snapshot `argus-repo-snapshot-20260813.zip` was published 2026-08-13. The development account 366356442579 is retired to read-only history — see 1.2.1.*

## 1.1 Role in the system

ARGUS is a smart pest monitoring system built for Changi Airport Group (CAG). The target site is the Shiseido Forest Valley inside Jewel Changi. The pests are moths (adult) and armyworms (larvae). Both damage the indoor planting. Today CAG staff find them by walking the site and looking. ARGUS replaces that with cameras, cloud detection, and email alerts, so an operator only acts when something is found.

The system has three layers. Edge devices capture frames. The cloud decides whether a frame contains a pest. The operator sees results on a web dashboard and gets an SES email alert on a hit. No detection logic runs on the edge. No human is in the capture loop.

The project is a Ngee Ann Polytechnic (NP) final-year ECE diploma project, Apr–Aug 2026, supervised by Dr. Yan Li. The moth detection model is inherited from the predecessor, Wilbur Teo. The armyworm model, the detection pipeline upgrades, the dashboard, and the ARGUS deployer are this project’s own work.

## 1.2 Inventory

Every subsystem this manual covers, with its location and its chapter. Unless a row says otherwise, cloud identifiers below are the NP production account’s; the retired development account’s counterparts are history and live in 1.2.1.

| Component                          | Location                                                                                                                                                                                                       | Purpose                                           |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| AWS accounts                       | NP production<br>506868652945 (IAM user<br>Student_QianRunzhe, CLI<br>profile prod) — the operative<br>account; retired dev<br>366356442579 (CLI profile<br>nbk2, read-only history);<br>both region us-east-1 | Host the cloud stack (see<br>1.2.1 for the split) |
| pest-detection-processor<br>Lambda | lambda/pest-detection-<br>processor.py (mirror of the                                                                                                                                                          | S3-triggered detection<br>pipeline v6.3: tiled    |

| Component                                    | Location                                                                                                                                                                                                                                                                                  | Purpose                                                                                                                                                  |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                              | live function; entry point <code>lambda_handler(event, context)</code> )                                                                                                                                                                                                                  | Rekognition finder + hard-object FP suppression + Sonnet 4.6 judge-every-box gate + post-gate cleanup (see 1.5)                                          |
| pest-monitoring-api Lambda                   | <code>lambda/pest-monitoring-api.py</code>                                                                                                                                                                                                                                                | 21-route HTTP API behind API Gateway <code>vzf17s6z00</code> (the retired dev account's gateway was <code>zwpcbivmsj</code> )                            |
| pest-camera-scheduler Lambda                 | <code>lambda/pest-camera-scheduler.py</code>                                                                                                                                                                                                                                              | Runs per-camera capture/model schedules                                                                                                                  |
| kvs-hls-handler, pest-model-watchdog Lambdas | live account; sources mirrored in <code>deployer/audit/*_src/</code> and <code>lambda/pest-model-watchdog.py</code> (the v6.2 per-camera build the deployer ships)                                                                                                                        | HLS playback URL minting; auto-stop of running Rekognition endpoints (per-camera <code>max_runtime_min</code> , 15-minute EventBridge schedule; see 1.3) |
| S3 buckets                                   | <code>argus-frames-506868652945</code> , <code>argus-processed-506868652945</code> , <code>argus-dashboard-506868652945</code> (retired dev: <code>frames-armyworm-366356442579</code> , <code>processed-images-armyworm-366356442579</code> , <code>pest-dashboard-366356442579</code> ) | Frame ingest, processed images, dashboard hosting                                                                                                        |
| DynamoDB tables                              | <code>pest-monitoring-cameras</code> , <code>-detections</code> , <code>-system-config</code> , <code>-schedule-logs</code> (same names on both accounts)                                                                                                                                 | Camera config, detection records, global settings, schedule logs                                                                                         |
| Rekognition Custom Labels                    | production: <code>project argus-detection</code> (armyworm, v9r-prod-20260810 trained, F1 0.613), <code>project argus-moth-detection</code> (moth-prod-20260811 trained, F1 0.991). Retired dev history: <code>armyworm-detection-v9</code> (v9 retrain v9-20260805-0713),                | The detection models                                                                                                                                     |



| Component                          | Location                                                                                                                                                                                                                                                         | Purpose                                                                               |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Bedrock                            | armyworm-detection (v5/v4), SmartPestProject (Wilbur's moth model)<br>inference profile us.anthropic.claude-sonnet-4-6 (authorized on both accounts; production also has Haiku 4.5)                                                                              | LLM verification stage (Claude Sonnet 4.6; the v4.5-era Haiku 4.5 profile is history) |
| Dashboard (web)                    | web/dashboard_v4/ → <a href="https://d1dtoxef7qmugl.cloudfront.net">https://d1dtoxef7qmugl.cloudfront.net</a> (production); the retired dev copy at <a href="https://d1twcdquexdgj8.cloudfront.net">https://d1twcdquexdgj8.cloudfront.net</a> holds history only | Operator UI: live view, gallery, analytics, settings                                  |
| Cognito                            | production: pool us-east-1_9se1FDHpc, app client 6vebotf45bp8u46cnraddiap1v (retired dev: pool us-east-1_ea0aJdus1, client 4husu6afr835e235eu9dqp8av6)                                                                                                           | Dashboard login (admin-created accounts only)                                         |
| CloudFront                         | production E1YADURLSAVNFA (retired dev: E1423RGLAXWNSI)                                                                                                                                                                                                          | HTTPS in front of the dashboard S3 website                                            |
| Kinesis Video Streams              | armyworm-cam-stream, moth-cam-stream — the names are the same on both accounts, but the stream resources exist only on the retired dev account; production has none (deployed without --live-view; see 1.6)                                                      | Live video rail (parallel to detection)                                               |
| Unitree Go2 EDU + Jetson Orin Nano | dog network 192.168.123.x; Orin at 192.168.123.18                                                                                                                                                                                                                | Demo/testbed patrol platform (NOT production)                                         |
| SIYI A8 Mini gimbal camera         | mini-HDMI -> MS2109 USB capture card on the Orin (since 2026-07-29; the old ethernet address 192.168.144.25 and its                                                                                                                                              | Camera on the Go2 rig (worm_cam)                                                      |

| Component                         | Location                                                                                                                                                                                                                                                                    | Purpose                                                                                                                 |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
|                                   | RTSP URL no longer answer)                                                                                                                                                                                                                                                  |                                                                                                                         |
| Mini PC (Win11 + Ubuntu 22.04 VM) | VM user wilburteo; bridged NIC 192.168.123.99                                                                                                                                                                                                                               | Runs <code>kvs_controller.py</code> for the fixed Hikvision moth camera (192.168.1.66)                                  |
| ARGUS deployer                    | <code>deployer/</code> ( <code>deploy.py</code> , <code>app.py</code> , built exe)                                                                                                                                                                                          | One-click recreation of the whole stack in a fresh AWS account (executed for real on the production account 2026-08-10) |
| Account migration tooling         | <code>migration/</code> ( <code>copy_training_data.py</code> , <code>train_v9r_on_prod.py</code> , <code>migrate_moth.py</code> , <code>migrate_jewel_records.py</code> , <code>diff_accounts.py</code> ; frozen config baseline <code>prod_baseline_20260810.json</code> ) | The scripts that stood up and verified the production account                                                           |
| Docs                              | <code>docs/</code> ( <code>state.md</code> , <code>aws.md</code> , <code>hardware.md</code> , <code>detection.md</code> , <code>dashboard.md</code> )                                                                                                                       | Living project documentation (context; code is ground truth)                                                            |

### 1.2.1 The two accounts — development/reference vs NP production

Until 2026-08-09 everything lived in one account. On 2026-08-10 the project received the NP production account, the ARGUS deployer stood the full stack up there the same day, and the migration is COMPLETE: both models retrained on production (armyworm 2026-08-12, moth 2026-08-11), end-to-end validation passed 2026-08-12, handover snapshot `argus-repo-snapshot-20260813.zip` published to `s3://argus-frames-506868652945/handover/` 2026-08-13. Where the two accounts stand:

**366356442579 — retired development/reference (CLI profile `nbk2`).** The account the system was built and validated in. Every development-era measurement in this manual was taken here unless stated otherwise. It is now history, read-only by policy: nothing operative runs there, no new writes. It still holds the dev-era dashboard at <https://d1twcdquexdgj8.cloudfront.net> and all development detection history, including the threshold-study evidence zones the final report cites — the account is kept until the final report is filed because that evidence lives nowhere else. Every “how to do it now” instruction in this manual points at production.

**506868652945 — NP production (IAM user `Student_QianRunzhe`, CLI profile `prod`).** Stood up 2026-08-10 by the ARGUS deployer itself: the full 15-stage deploy ran headless in 103

seconds with zero errors, which doubles as the deployer's first real-account validation (Chapter 7). This is the operative account. Live as of 2026-08-14:

- Dashboard <https://d1dtoxef7qmugl.cloudfront.net> (CloudFront E1YADURLSAVNFA, Deployed and serving; a Cognito sign-in user exists and is CONFIRMED).
- API <https://vzf17s6z00.execute-api.us-east-1.amazonaws.com> — 21 routes, JWT on all but GET /stream/status. Cognito pool us-east-1\_9se1FDHpc, app client 6vebotf45bp8u46cnraddiapl.
- Buckets argus-frames-506868652945, argus-processed-506868652945, argus-dashboard-506868652945; the four pest-monitoring-\* tables; the five Lambdas under their usual names (the processor at the validated 1024 MB / 600 s sizing with the full v6.3 tuning env); the fyp-pillow layer; the watchdog schedule ENABLED (rate 15 min).
- Rekognition project argus-detection: models are account-bound and cannot be exported, so the armyworm model had to be RETRAINED — the largest migration work item. The training data (36,641 objects / 4.27 GB) was copied server-side with every manifest source-ref repointed at the new bucket, and version v9r-prod-20260810 was submitted for training 2026-08-10. Training completed: Rekognition test-split F1 0.613, and the version ARN is wired into the production camera rows (verified live 2026-08-12; endpoint RUNNING at that read for the validation window).
- Rekognition project argus-moth-detection: moth-prod-20260811 TRAINED 2026-08-11, F1 0.991 (the inherited SmartPestProject scored 0.988 on the same data — treat them as equivalent). The moth capability is fully reconstructed on the new account from data alone: model, camera row, and training set.
- SES: the alert address is verified; the account is still in the SES sandbox.
- Bedrock: Sonnet 4.6 and Haiku 4.5 were already authorized — no use-case-form wait.
- 40 detection records migrated: the complete 36-record Jewel on-site set (the irreplaceable Go2-at-Jewel patrol evidence, frames included, S3 keys byte-identical) plus 4 requested gallery records. Migrated records keep the old account's model ARN on purpose — it is provenance, not configuration.

Deliberately NOT migrated: KVS / live view (the deploy ran without --live-view, so no streams exist on the new account and the Live tab has nothing to play — see 1.6), the Wilbur-era legacy Lambdas/tables/APIs, and the bulk detection history (the post-training validation re-push recreates that class of evidence natively, with the new model's ARN).

Naming differs by design: the deployer parameterizes everything, so production buckets carry the argus- prefix and the deployer seeds the detection camera row as camera-1 (with manual\_upload; the moth migration added moth\_cam). During the 2026-08-11 reconciliation the detection row was aligned with the old account's worm\_cam configuration (label "Worm Cam", 05:40 schedule kept disabled). Verified live 2026-08-12: the production pest-monitoring-cameras rows are worm\_cam, moth\_cam, and manual\_upload — device upload scripts keep their camera ids at cutover.

Closed since: the holdout validation re-push PASSED 2026-08-12 15:22 (26 images through the live production chain; the first same-day attempt had been invalidated by the processor-role IAM gap, see 9.3.14), and `worm_cam.max_runtime_min` is back at 45 (restored 2026-08-12 after a temporary 240). The edge scripts' repo mirrors were repointed at production 2026-08-13 (upload bucket `argus-frames-506868652945`, API base `vzf17s6z00`; camera ids and table names unchanged). Still open: sync those mirrors onto the Orin and the mini PC VM and swap the on-device AWS credentials to a production-account key when the hardware returns, and revoke the three temporary cross-account S3 read grants on the old bucket (each migration script has a `--revoke` flag).

## 1.3 Design principles

**Edge captures, cloud decides, operator acts.** Edge devices only take pictures and upload them. All detection intelligence lives in one Lambda in the cloud. The operator's only jobs are to review the dashboard and respond to alerts. This keeps every edge device dumb, cheap, and replaceable.

**Serverless.** There is no server to patch or keep alive. The stack is S3 + Lambda + DynamoDB + API Gateway + Rekognition + Bedrock + SES + Cognito + CloudFront + Kinesis Video Streams. Everything is pay-per-use. The one billed-per-hour resource is a running Rekognition Custom Labels endpoint (about \$4/hr). Endpoint lifecycle is automated again since the 2026-08-05 scheduling rework: the `pest-model-watchdog` Lambda (schedule `pest-model-watchdog-15min`, re-enabled after a 2026-07-28..08-05 pause during which stops were manual) honours a per-camera `max_runtime_min` field (`worm_cam` = 45), so a scheduled morning start runs one detection round and the endpoint is stopped again within 45–60 minutes. No fixed stop time exists anywhere; the legacy `model-start-schedule` / `model-stop-schedule` entries were deleted.

**One region; one operative account.** Everything runs in `us-east-1`. Historically the whole stack lived in the development account (366356442579, CLI profile `nbk2`); on 2026-08-10 the ARGUS deployer reproduced it in the NP production account (506868652945, CLI profile `prod`), which is the operative account since — see 1.2.1. Within each account there is no cross-account wiring and no cross-region resource (the migration's temporary cross-account S3 read grants are scoped, documented, and revoked at sign-off). The one deliberate exception: the Bedrock model id is a cross-region inference profile (`us.` prefix), which Bedrock itself routes; the caller still only talks to `us-east-1`.

**Config in DynamoDB, not in code.** Per-camera behavior (target label, model ARN, confidence threshold, tiling on/off, LLM verify on/off, schedule) is a row in `pest-monitoring-cameras`. The Lambdas read it at call time. Changing the live model is a one-field write, not a redeploy.

**The detection target is a parameter.** Nothing in the Lambdas or dashboard hardcodes "armyworm". A camera row's `target_label` decides what counts as a detection. This is what makes the ARGUS deployer able to ship a generic product (Chapter 7).

**S3 CLI note:** CloudShell writes are blocked in the dev/reference account by a VPC endpoint. Use a local CLI with profile prod for any write there; CloudShell is reads only.

## 1.4 Production vision vs the testbed — read this first

This distinction is a hard project truth. Getting it wrong misrepresents the whole design.

**Production vision: fixed cameras.** The deployed product is a set of fixed cameras, one at each monitoring waypoint in the Shiseido Forest Valley, each pointed close-range at vegetation. Fixed close-range capture is not just cheaper — it is what the detection model is good at. The armyworm model’s in-domain recall on close-range vegetation is near-total, while wide indoor scenes are its known weakness (Chapter 3). Deployment geometry is a detection lever, not just a logistics choice.

**The Unitree Go2 + SIYI A8 Mini rig is a demo and testbed only.** The quadruped patrols between waypoints, points the gimbal camera, captures, and uploads — a mobile stand-in for cameras that are not installed yet. It exists to prove the pipeline end-to-end and to demo it. It is not part of the production design, and no argument in this manual should be read as “the robot is the product.” The testbed did its job: autonomous patrol was declared complete on 2026-07-30, after three consecutive 5/5 waypoint runs at Jewel, app-free (route and settings in robot/map\_profiles.md; Chapter 5).

The cloud cannot tell the difference, by design. A frame from the Go2’s camera and a frame from a future fixed camera land in the same S3 prefix, hit the same Lambda, and follow the same rules. Swapping the testbed for fixed cameras changes nothing in the cloud.

## 1.5 End-to-end dataflow (the detection rail)

The full chain, camera to operator:

camera frame (JPEG)

```
-> S3 argus-frames-506868652945 key: frames/<camera_id>/<...>.jpg  
    (retired dev account: frames-armyworm-366356442579)
```

```
-> S3 ObjectCreated event (prefix frames/)
```

```
-> Lambda pest-detection-processor (v6.3)
```

```
    1. read camera row from pest-monitoring-cameras (model ARN, target_label,
```

```
       min_confidence = the CANDIDATE floor (10), tiling_enabled,  
       llm_verify_enabled, post_verify_floor = the display floor (33))
```

```
    2. Rekognition DetectCustomLabels (worm_cam runs v9r-prod-20260810,  
       a high-recall front end)
```

```
       (per-camera tiling: grid + overlap + per-tile upscale + NMS;  
       ON for worm_cam – tiles gather candidates down to  
       TILE_MIN_CONFIDENCE=8; the W15 whole-image ruling is SUPERSEDED)
```

```
    3. hard-object FP suppression (v4.3): a DetectLabels pass finds  
       person/vehicle/furniture/machinery regions; any target box mostly  
       covered by one is dropped; boxes on plants are never touched
```

4. LLM denoiser gate (per-camera opt-in): Bedrock Claude Sonnet 4.6 judges EVERY candidate box as a padded, upscaled crop (LLM\_VERIFY\_ALL\_BOXES=true, up to LLM\_VERIFY\_MAX\_BOXES=120); the verify prompt describes the larva (elongated, clearly segmented body, typically yellow-and-black striped)
  5. post-gate cleanup: NMS + containment dedupe, area cap (a box covering more than 5% of the frame is dropped), and the post\_verify\_floor (33)
  6. write ONE DynamoDB record to pest-monitoring-detections (unconditional – clean frames write a record too)
  7. on a confirmed detection: SES email alert to the configured recipient
- > dashboard (reads records via pest-monitoring-api, draws boxes client-side)
- > operator

Key properties of this rail:

- **The frame upload is the API.** An edge device needs exactly one AWS permission: put an object into the frames bucket. Everything after that is event-driven.
- **put\_item is unconditional.** A clean frame (zero detections) still writes a record. The Go2 patrol gate depends on this: the patrol script uploads, then wait\_for\_detection() polls a DynamoDB query on image\_id (every 1.5 s, 150 s budget, fail-open on timeout) until the record appears, then moves to the next waypoint. The gate condition is record EXISTENCE, not detection: no detection is not a timeout; it is a normal record.
- **The detection record is the single source of truth.** Primary key image\_id is the exact S3 object key. The record stores structured bboxes plus a verifications map (operator TP/FP verdicts). The processor no longer bakes boxes into images; the dashboard draws boxes on a canvas overlay from the stored coordinates.
- **Who finds and who judges is settled: Rekognition finds, Sonnet 4.6 judges.** The detector is deliberately a high-recall front end — its console F1 (0.613 for the production v9r-prod-20260810; the dev v9r scored 0.599) is not pipeline accuracy, because precision comes from the gate and the post-gate cleanup, not from the detector. A whole-frame LLM pass as the finder was A/B-tested and rejected (Bedrock's input-resolution ceiling makes it miss most worms; Chapter 3), and the v5.x experiments that let the LLM lead via its own tile sweep were retired as well: the v6.3 dead-code strip (deployed 2026-08-07, 3266 -> 1545 lines) deleted all of those code paths, leaving exactly the live chain — tiled finder -> v4.3 suppression -> judge-every-box gate -> post-gate cleanup — with the live path byte-identical to v6.2's.
- **Two floors, two meanings.** min\_confidence (10) is the candidate floor feeding the gate; raising it silently strangles recall before the LLM ever sees a box. The user-facing threshold is the per-camera post\_verify\_floor, applied after verification — it is the field the dashboard's threshold control edits. The live value is **33** (verified

against the production row 2026-08-14). History: 49 was decided 2026-08-10 after a measured 34-vs-49 threshold study, then refitted per-build to 33 on 2026-08-13 so a live Test upload draws the same boxes the curated gallery shows. The two knobs cannot be confused since the 2026-08-05 rework.

- **Stage order is fixed:** finder → v4.3 suppression → judge → post-gate cleanup. Each stage only ever removes or confirms what the previous stage produced.

NOTE: version pinning. This chapter matches processor v6.3, deployed 2026-08-07. The repo mirror is `lambda/pest-detection-processor.py`; the pre-strip v6.2 source is archived at `lambda/archive/pest-detection-processor_v6.2_full.py`. The same v6.3 source ships in the ARGUS deployer and runs on the production account. Chapter 2 documents the processor version history in full.

## 1.6 The parallel live rail (Kinesis Video Streams)

Detection is asynchronous. The live rail exists so the operator can also watch the cameras in real time. It is fully parallel to the detection rail and shares nothing with it except the cameras.

camera RTSP feed

```
-> GStreamer pipeline (rtspsrc -> rtph264depay -> h264parse -> kvssink)
    running on the edge device (Orin for worm_cam, mini PC VM for moth_cam)
-> Kinesis Video Streams (armyworm-cam-stream / moth-cam-stream, 2 h retention)
-> Lambda kvs-hls-handler (GET /video-playback) mints an HLS session URL
-> dashboard Live tab plays the HLS stream
```

Streaming is on-demand, controlled from the dashboard. The edge daemons (`kvs_controller.py` on both the Orin and the mini PC VM) poll `GET /stream/status` every 5 seconds — this is the single unauthenticated API route — and start or stop their GStreamer pipeline to match the `stream_enabled` flag on their camera row. `armyworm-cam-stream` is BROKEN as of 2026-07-29: the SIYI A8 was moved off ethernet onto the MS2109 USB capture card, so the RTSP source the GStreamer pipeline expects no longer exists. `kvs-controller.service` on the Orin still runs, but streaming will fail the moment it is enabled. On `moth-cam-stream` the sources split across layers: the mini PC's `kvs-controller.service` is healthy and polls with `stream_enabled=true` on the `moth_cam` row, and HLS playback was verified on the dashboard at some earlier point, but `docs/aws.md` (newer) records the stream as parked with the Hikvision camera repurposed. Treat the moth stream as parked; closing this definitively needs a live check (the row's `stream_enabled` flag plus KVS PutMedia activity).

On the NP production account the live rail does not exist: the 2026-08-10 deploy deliberately ran without `--live-view`, so no KVS stream resources were created. The production `worm_cam` row does carry `kvs_stream_name` `armyworm-cam-stream` with `stream_enabled` `false` (verified live 2026-08-14) — the name matches the old account's, but there is no stream resource behind it. Nothing in production used streaming anyway

(stream\_enabled was already false on the old account), but the dashboard Live tab and the devices' kvs\_controller have nothing to talk to until streams are created — an open decision, driven by whether the demo needs live view.

## 1.7 Deployment topology — what runs where

**Cloud — NP production (AWS 506868652945, us-east-1) — the operative stack.** The ARGUS-deployed implementation of the design: the five Lambdas under their usual names (processor v6.3 at 1024 MB / 600 s), the four tables, the three argus-\* buckets, API Gateway vzfl7s6z00, Cognito us-east-1\_9se1FDHpc, CloudFront E1YADURLSAVNFA, the fyp-pillow layer, Rekognition projects argus-detection + argus-moth-detection, and the watchdog schedule (rate 15 min, ENABLED; the watchdog is the v6.2 per-camera build, which reads max\_runtime\_min per camera — shipped by the deployer). No KVS (see 1.6). SES is in sandbox mode. Chapter 2 covers all of it.

**Cloud — retired dev/reference (AWS 366356442579, us-east-1) — history only.** The same design, where it was built: five Gen-2 Lambdas, the four tables, the three old-name buckets, the dev Rekognition projects, Bedrock, SES, Cognito, CloudFront, API Gateway zwpcbivmsj, both KVS streams, and the EventBridge Scheduler entries pest-model-watchdog-15min and the runtime pest-sched-\* rules (the 05:40 SGT morning-start rule exists only while the camera row's schedule is enabled; it was left disabled). The legacy model-start-schedule / model-stop-schedule / frame-extraction-schedule entries were deleted 2026-08-05. Kept read-only for the evidence it holds (1.2.1).

**Jetson Orin Nano (on the Go2, 192.168.123.18, SSH user unitree).** Ubuntu 20.04, ROS 2 Foxy. Runs the patrol script (repo mirror robot/go2\_patrol\_gated.py, deployed on the Orin under ~/go2/; its main() walks the waypoint list — per scan waypoint: USLAM goal, capture\_frame(), upload\_frame() to frames/worm\_cam/<waypoint>/<ts>.jpg, then wait\_for\_detection() polls the detections table every 1.5 s for up to 150 s until the processor's unconditional record appears, fail-open on timeout — the repointed mirror already uploads to argus-frames-506868652945), the SIYI A8 Mini capture/control code (siyi\_sdk over UDP port 37260 for gimbal control; frames via the mini-HDMI -> MS2109 USB capture card since 2026-07-29 — the old RTSP path is gone), the S3 uploader, the compiled ARM64 KVS producer, and its kvs\_controller. It does NOT run USLAM — navigation and SLAM run on the Go2's sport MCU (192.168.123.161, DDS only, no SSH). The Orin talks to USLAM over plaintext std\_msgs/String topics /uslam/client\_command and /uslam/server\_log. Chapter 5.

**Mini PC (fixed moth camera site).** Windows 11 host running an Ubuntu 22.04 VM (account wilburteo). The VM runs kvs-controller.service (systemd, Restart=always), which polls /stream/status and pipes the Hikvision camera at 192.168.1.66 into moth-cam-stream. Remote access is a self-healing reverse SSH tunnel from the VM to the Orin (reverse-tunnel-fyp.service, -R 2222:localhost:22), key-auth both ways. This box is the closest thing in the current build to the production fixed-camera node. Chapter 6.



**Browser (operator).** The dashboard is static files — `index.html`, `styles.css`, 13 ES modules — served from S3 bucket `argus-dashboard-506868652945` behind CloudFront `E1YADURLSAVNFA` at `https://d1dtoxef7qmugl.cloudfront.net`, its `js/config.js` templated at deploy time to the production API and Cognito client (the retired dev copy: `pest-dashboard-366356442579` behind `E1423RGLAXWNSI` at `https://d1twcdquexdgj8.cloudfront.net`). Login is Cognito (email sign-in, admin-created accounts, 12 h tokens). All API calls carry a Bearer JWT; enforcement lives in API Gateway, not in the JS. Chapter 4.

**ARGUS deployer (any Windows machine).** A built exe (`deployer/`, built from `deploy.py / app.py` via PyInstaller, spec `ARGUS.spec`) that recreates the whole cloud stack in a fresh AWS account from the audited bill of materials in `deployer/STACK_MANIFEST.md`. Entry point: `main()` in `deployer/deploy.py`, which runs the module-level STAGES list — 15 (name, stage\_fn) pairs in dependency order, stage\_iam through stage\_ses — via `run_plan()`, emitting a structured event per stage and stamping `deploy:last_stage` into saved state so a died run can report where it stopped and resume; --only `s3,lambda` runs a named subset. `app.py` is the WebView UI wrapper around the same plan. It ships a generic, detection-target-agnostic product: no dataset, no trained model (Rekognition Custom Labels models are account-bound and cannot be exported), parameterized names, and a guided training step for the customer's own data. No longer only a rehearsal artifact: the 2026-08-10 NP-production-account deploy was executed by this deployer (15 stages, 103 s, zero errors). Chapter 7. Rehearsal procedure: `deployer/REHEARSAL.md`.

**Development laptop.** A Windows 11 machine holding the working copy of the project repository. Joins the dog network via a USB Ethernet adapter at static `192.168.123.50/24`. All AWS writes go through CLI profiles `prod` (NP production, the operative one) and `nbk2` (retired dev/reference) from here.

## 1.8 The two cameras and the two models

The `pest-monitoring-cameras` table holds three rows: `worm_cam`, `moth_cam`, and `manual_upload` (a hidden fallback that backs the dashboard's Test-upload feature). The ids are the same on both accounts since the 2026-08-11 reconciliation (see 1.2.1). The two real cameras, production values (checked against the live `worm_cam` row 2026-08-14):

|                 | worm_cam                                                                                        | moth_cam                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Physical camera | SIYI A8 Mini on the Go2 (testbed)                                                               | Hikvision fixed camera at 192.168.1.66                                                     |
| Edge host       | Jetson Orin Nano                                                                                | Mini PC Ubuntu VM                                                                          |
| Target label    | armyworm-larva                                                                                  | Moths                                                                                      |
| Model           | v9r-prod-20260810 in project argus-detection (own work; retrained on production 2026-08-10; the | moth-prod-20260811 in project argus-moth-detection (rebuilt on production from the data of |

|                                 | worm_cam                                                                                                                                                                                                                               | moth_cam                                                                                                                                    |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Model F1                        | dev-account v9 retrain is its direct ancestor, same data and recipe)<br>0.613 — low by design: a high-recall front end feeding the LLM gate (the dev v9r scored 0.599 on the same recipe; v5's 0.852 is history; see the caveat below) | Wilbur Teo's inherited SmartPestProject)<br>0.991 (the inherited SmartPestProject scored 0.988 on the same data — treat them as equivalent) |
| Candidate floor / display floor | min_confidence 10 / post_verify_floor 33 (49 decided 2026-08-10, refitted per-build to 33 on 2026-08-13)                                                                                                                               | min_confidence only (no gate)                                                                                                               |
| KVS stream                      | kvs_stream_name = armyworm-cam-stream, stream_enabled false; no stream resource on production, and the dev-era stream is BROKEN as of 2026-07-29 (see 1.6)                                                                             | moth-cam-stream (name only on production; see 1.6)                                                                                          |
| Tiling                          | ON (tiling_enabled=true; the W15 whole-image ruling is superseded)                                                                                                                                                                     | off                                                                                                                                         |
| LLM crop-verify                 | ON (llm_verify_enabled=true)                                                                                                                                                                                                           | OFF — the verify prompt asks about larvae; running it on adult moths would misjudge every box                                               |
| Endpoint auto-stop              | max_runtime_min 45 (watchdog-enforced)                                                                                                                                                                                                 | —                                                                                                                                           |

Model ARNs (both resolve at call time from the camera row's custom\_model\_arn field — nothing hardcodes an ARN). Operative, on production:

- armyworm (live; wired into worm\_cam + manual\_upload, verified 2026-08-12):  
arn:aws:rekognition:us-east-1:506868652945:project/argus-detection/version/v9r-prod-20260810/1786377372187
- moth (live; wired into moth\_cam): arn:aws:rekognition:us-east-1:506868652945:project/argus-moth-detection/version/moth-prod-20260811/1786412382515

Development-era history, all on the retired dev account (kept for provenance — the migrated Jewel detection records deliberately keep these ARNs, because the ARN records which model produced that detection):

- armyworm v9 retrain (the dev-era final; live on the dev account 2026-08-07..08-10; a 2026-08-05 retrain of v9 with added data augmentation — flips, rotations, exposure jitter; note the separate project name): `arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection-v9/version/v9-20260805-0713/1785913987295`
- armyworm v9 first cut (superseded by the retrain; endpoint STOPPED): `arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection-v9/version/v9-20260725-1746/1785001598671`
- armyworm v5 (old rollback; endpoint STOPPED): `arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection/version/v5-2026-07-07/1783394123547`
- armyworm v4 (older history): `arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection/version/armyworm-detection.2026-05-21T12.46.19/1779338780450`
- moth: Wilbur’s SmartPestProject, native in the dev/reference account (its data, not the model, seeded the production rebuild — Rekognition models are account-bound).

Honest caveat on v5’s 0.852: that F1 was measured on the purchased-domain test set and did not transfer to the CAG holdout images (v5 ties v4 there at the operational threshold). The v9 family’s 0.599–0.613 is not a regression — it is deliberate: v9 is tuned for recall and the Sonnet 4.6 gate removes the false positives. The full model history, the holdout methodology, and why domain — not model capability — is the binding constraint are in Chapter 3.

Rollback between armyworm versions is one field write. Beware: the ARN contains colons, and a raw `aws dynamodb update-item` from PowerShell mangles the quoting — use `boto3` or a JSON parameter file (Chapter 3 has the procedure).

## 1.9 Component map — where everything is documented

| Subsystem                                                                                        | Chapter   |
|--------------------------------------------------------------------------------------------------|-----------|
| Cloud stack: Lambdas, S3, DynamoDB, API Gateway, SES, Bedrock, KVS, IAM, watchdog                | Chapter 2 |
| Detection models: armyworm v1–v9 history, moth model, holdout methodology, Rekognition CL limits | Chapter 3 |
| Dashboard: modules, auth, deploy, gallery/analytics/live tabs                                    | Chapter 4 |
| Go2 + Jetson Orin: USLAM navigation, patrol                                                      | Chapter 5 |

| Subsystem                                                                                              | Chapter   |
|--------------------------------------------------------------------------------------------------------|-----------|
| script, SIYI A8 Mini control, capture-upload gate                                                      |           |
| Mini PC: VM, kvs-controller service, reverse tunnel, Hikvision camera                                  | Chapter 6 |
| ARGUS deployer: exe, deploy.py, stack manifest, rehearsal                                              | Chapter 7 |
| Reproduction on a fresh AWS account end to end (executed for real on NP's 506868652945, 2026-08-10/11) | Chapter 8 |
| Registries: every ID, ARN, IP, endpoint, and credential location in one place                          | Chapter 9 |

## 1.10 Operations / reproduction

The procedures below verify that you are looking at the right account and that the top-level stack is alive. Deep operations live in the per-subsystem chapters.

### 1.10.1 Verify the account and region

There are now two accounts (1.2.1) — first make sure you are in the one you intend.

Console: sign in, open the account menu (top right) and confirm the account ID reads 5068-6865-2945 (NP production, the operative account) or 3663-5644-2579 (retired dev/reference); confirm the region selector (top right) reads **N. Virginia (us-east-1)**.

CLI:

```
aws sts get-caller-identity --profile **prod**
aws sts get-caller-identity --profile **nbk2**
```

Expect "Account": "506868652945" for prod and "Account": "366356442579" for nbk2. If you are reproducing on your own account (Chapter 8), substitute your own profile name and expect your own account ID everywhere this manual shows either number.

Note: in the dev/reference account, CloudShell cannot write (VPC endpoint block). Do all writes from a local CLI with the nbk2 profile.

### 1.10.2 Confirm the core Lambdas exist

Console: **Lambda → Functions** (region us-east-1). You should see, among others: pest-detection-processor, pest-monitoring-api, pest-camera-scheduler, kvs-hls-handler, pest-model-watchdog. The production account carries exactly those five, plus a previous student's unrelated Amplify relics — leave those alone. The retired dev/reference account also carries ~11 legacy Gen-1 functions — do not delete anything based on this list; Chapter 9 classifies every function.

CLI:

```
aws lambda list-functions --profile **prod** --region us-east-1 --query "Functions[].FunctionName" --output table
```

### 1.10.3 Confirm the camera configuration

Console: **DynamoDB** → **Tables** → **pest-monitoring-cameras** → **Explore table items**.

Expect three rows: `worm_cam`, `moth_cam`, `manual_upload` (same trio on both accounts since the 2026-08-11 reconciliation; see 1.2.1). Check `custom_model_arn`, `target_label` (armyworm-larva), `min_confidence` (candidate floor, 10), `post_verify_floor` (display floor, 33), `tiling_enabled` (true), `llm_verify_enabled` (true), `max_runtime_min` (45) on `worm_cam`.

CLI:

```
aws dynamodb scan --table-name pest-monitoring-cameras --profile **prod** --region us-east-1
```

### 1.10.4 Confirm the ingest trigger

Console: **S3** → **argus-frames-506868652945** → **Properties** → **Event notifications** (retired dev: frames-armyworm-366356442579). Expect one notification: `s3:ObjectCreated:*`, prefix `frames/`, destination `pest-detection-processor`.

CLI:

```
aws s3api get-bucket-notification-configuration --bucket argus-frames-506868652945 --profile **prod**
```

### 1.10.5 Smoke-test the whole detection rail

The cheapest full-rail test needs no robot and no camera: the dashboard **Settings** → **Test upload** panel uploads an image via a presigned URL into `frames/manual_upload/...`, which fires the real pipeline end to end. A test image with a known pest should produce a gallery card with boxes within ~10–30 seconds (longer if the Rekognition endpoint has to be running — check **Rekognition** → **Custom Labels** → **Projects** → **argus-detection** or `aws rekognition describe-project-versions` first; a stopped model means zero custom detections. Since 2026-08-05 the watchdog auto-stops the endpoint `max_runtime_min` (45) minutes after it comes up, so a stopped endpoint is the NORMAL resting state, not a fault).

Equivalent CLI upload:

```
aws s3 cp **path/to/test.jpg** s3://argus-frames-506868652945/frames/manual_upload/**test-name**.jpg --profile **prod**
```

Then check the record:

```
aws dynamodb get-item --table-name pest-monitoring-detections --key "{\"image_id\":{\"S\":\"frames/manual_upload/**test-name**.jpg\"},\"detection_time\":{\"S\":\"**<from a query>**\"}}\" --profile **prod** --region us-east-1
```

(The table's sort key is `detection_time`; if you do not know it, query on `image_id` instead of `get-item`.)

### 1.10.6 Open the dashboard

URL to hand out: **`https://d1dtoxef7qmugl.cloudfront.net`** (NP production — the one to give people) or **`https://d1twcdquexdgj8.cloudfront.net`** (retired dev/reference, history only). Login accounts are Cognito admin-created only, per account and pool (creation commands in Chapter 4). The raw S3 website URL also works but is HTTP-only — never hand it out; passwords must not travel over plain HTTP.

### 1.10.7 Reproducing on a fresh account

Two supported paths, both in later chapters:

- **ARGUS deployer (Chapter 7):** the built exe recreates the whole parameterized stack from `deployer/STACK_MANIFEST.md`. This path is proven: it built the NP production account for real on 2026-08-10 (15 stages, 103 s).
- **Manual (Chapter 8):** the step-by-step console + CLI build, in the dependency order IAM → DynamoDB → S3 → Lambda layer → Lambdas → S3 notification → Cognito → API Gateway → CloudFront → config write-back → KVS → Scheduler → Rekognition → seeds → SES.

Two hard constraints to know before starting: Rekognition Custom Labels models are account-bound — they cannot be exported or copied, so a fresh account must retrain from labeled images. And a fresh account's SES starts in sandbox mode — it can only send to verified addresses until a manual production-access request is approved.

One expensive lesson from the real migration, learned before it could waste a multi-hour training run: **after any labelling done in the Rekognition console, the S3 manifest file is stale — the console dataset is the source of truth.** Rebuilding a dataset on another account from the copied manifest silently loses console-added boxes. Export the live entries with `ListDatasetEntries`, repoint every source-ref at the new bucket, upload that as the new manifest, and only then create the datasets (scripted in `migration/train_v9r_on_prod.py`).

### 1.10.8 Security note for anyone touching the repo

No file in the repo or this manual may contain a secret: no AWS secret access keys, no passwords, no tokens. Non-secret identifiers (account ID, ARNs, bucket names, API IDs, IPs) are fine and are used freely above. One known exception exists as a caution: `datasets/archive/experiments/pre_v3_abandoned/download.py` contains an inline Roboflow API key from an abandoned experiment — credential stored there, not reproduced here. Do not copy that file anywhere public. Device and RTSP passwords are documented separately, outside the repo (see `docs/hardware.md` header).

## 1.11 Cross-references

- Chapter 2 — every cloud resource in 1.5–1.7, at recreate-level detail, including the processor version history (v4.0 → v6.3).
- Chapter 3 — both models, the v1–v9 training history, the holdout evaluation discipline, and the tiling history (the W15 whole-image ruling and its supersession).
- Chapter 4 — the dashboard modules, Cognito user management, redeploy command, emergency auth rollback.
- Chapter 5 — Go2/Orin: USLAM command sequence, patrol gate loop, SIYI A8 Mini control mapping and its firmware quirks.
- Chapter 6 — mini PC VM, `kvs-controller.service`, the reverse-tunnel access path.
- Chapter 7 — the ARGUS deployer and the product-scope ruling (generic, no dataset ships).
- Chapter 8 — full reproduction runbook for a fresh account (executed for real on the NP production account, 2026-08-10/11).
- Chapter 9 — registries: the complete ID/ARN/IP/endpoint tables, and the PROD/DEMO/DEAD classification of every live resource.

## Chapter 2 — AWS cloud backend

This chapter documents every production cloud resource of ARGUS: S3, Lambda, DynamoDB, API Gateway, Cognito, CloudFront, Kinesis Video Streams, EventBridge, SES, IAM, and the Bedrock and Rekognition touch points, with the configuration and code needed to operate and reproduce each one.

*As of 2026-08-14.* (Facts with a different date are marked with that date. The live camera-row values quoted in 2.4/2.5 were read from the production DynamoDB table on 2026-08-14. `migration/prod_baseline_20260810.json` is the deploy-day freeze; it predates the 2026-08-13 floor refit and still shows floor 49. File paths in this chapter are relative to the project repository root.)

**The system runs on the NP production account.** ARGUS was built and validated on the development account **366356442579** (CLI profile `nbk2`, now retired to history/evidence duty). On 2026-08-10 the ARGUS deployer stood the full stack up on the NP production account **506868652945** (IAM user `Student_QianRunzhe`, CLI profile `prod`): all 15 stages in 103 seconds. Both models were then retrained there (`moth-prod-20260811` F1 0.991, trained 2026-08-11; `armyworm v9r-prod-20260810` F1 0.613, trained 2026-08-12, ARN wired into the camera rows), the end-to-end validation run passed 2026-08-12, and the handover snapshot `argus-repo-snapshot-20260813.zip` was published to `s3://argus-frames-506868652945/handover/` on 2026-08-13. This chapter documents the production account as the operative system; development-account identifiers are kept as marked history (2.2.1). KVS/live view is the one part not stood up on production (2.14).

### 2.1 Role in the system

The cloud backend is the middle of the end-to-end chain. Upstream, a camera (the SIYI A8 Mini on the Unitree Go2 during demos, fixed cameras in the production concept, or the dashboard's Test upload panel) puts a JPEG frame into an S3 bucket under `frames/{camera_id}/{waypoint_id}/{filename}`. Downstream, the vanilla-JS dashboard reads results over an authenticated HTTP API and draws bounding boxes on the original frame.

Everything between those two points is serverless and event-driven. An S3 event notification invokes the `pest-detection-processor` Lambda for every new frame. That Lambda runs AWS Rekognition Custom Labels detection (tiled or whole-frame per camera), filters false positives with a Rekognition `DetectLabels` pass, adjudicates candidate boxes with a Bedrock multimodal model, writes one DynamoDB record per frame (clean frames included), and sends an SES alert when a pest is found. A second Lambda, `pest-monitoring-api`, serves all dashboard routes behind an API Gateway HTTP API with a Cognito JWT authorizer. Three small Lambdas handle scheduled model start/stop, KVS HLS playback URLs, and a cost watchdog that stops any Rekognition endpoint left running.



The design principle that shapes the whole backend: the DynamoDB detection record is the contract. The Go2 patrol gate, the dashboard gallery, and the model test scripts all poll or query the same pest-monitoring-detections table keyed by the exact S3 object key. Nothing depends on annotated images being generated (none are, since processor v4.1); the dashboard draws boxes client-side from stored coordinates.

Production account: **506868652945**, region **us-east-1**, CLI profile prod (IAM user Student\_QianRunzhe, AdministratorAccess via group CAG\_Proj, no permission boundary). Every command in this chapter targets this account unless it is explicitly marked as history. The production account is not empty — a previous student’s Amplify “moobusapps” relics (13 roles, one deployment bucket) exist there; leave them alone. The retired development account (history and evidence only): **366356442579**, CLI profile nbk2. It is shared with unrelated projects; only the resources mapped in 2.2.1 belonged to ARGUS. Its one CLI quirk, kept for anyone auditing the old evidence: CloudShell writes are blocked by a VPC endpoint there — use a local CLI with `--profile nbk2` for writes, CloudShell for reads only.

## 2.2 Inventory

| Component                | Location / identifier                                  | Purpose                                                                                                                                             |
|--------------------------|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Frames bucket            | argus-frames-506868652945                              | Frame ingest; S3 event triggers the processor; also holds the migrated training sets under training-data/ and the handover snapshot under handover/ |
| Processed bucket         | argus-processed-506868652945                           | Legacy annotated-image store; empty since processor v4.1, kept because the API’s S3_PROCESSED_BUCKET env var and the delete path reference it       |
| Dashboard bucket         | argus-dashboard-506868652945                           | Static website hosting for web/dashboard_v4/ (public read)                                                                                          |
| Layer bucket             | lambda-layers-366356442579 (dev account only, history) | Held the unused Gen-1 ffmpeg layer zip only; no production counterpart exists or is needed                                                          |
| pest-detection-processor | Mirror lambda/pest-detection-processor.py              | S3-triggered detection pipeline (tiling, FP suppression, LLM denoiser gate, post-gate cleanup,                                                      |

| Component                              | Location / identifier                                                                                                                               | Purpose                                                                                                                  |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| pest-monitoring-api                    | Mirror lambda/pest-monitoring-api.py                                                                                                                | DDB write, SES)<br>All dashboard HTTP routes; DynamoDB read/write, model start/stop, schedules, identities, cost, delete |
| pest-camera-scheduler                  | Mirror lambda/pest-camera-scheduler.py                                                                                                              | Executes scheduled Rekognition model start/stop from EventBridge rules                                                   |
| kvs-hls-handler                        | Mirror deployer/audit/kvs-hls-handler_src/lambda_function.py                                                                                        | Generates KVS HLS playback URLs for the dashboard Live tab                                                               |
| pest-model-watchdog                    | Mirror lambda/pest-model-watchdog.py (v6.2, per-camera limit; the pre-v6.2 audit copy is deployer/audit/pest-model-watchdog_src/lambda_function.py) | Auto-stops Rekognition endpoints after a runtime limit                                                                   |
| Lambda layer fyp-pillow                | arn:aws:lambda:us-east-1:506868652945:layer:fyp-pillow:1                                                                                            | Pillow 12.2.0 for the processor's tiling and cropping                                                                    |
| DynamoDB pest-monitoring-cameras       | table, PK camera_id                                                                                                                                 | Per-camera config incl. model ARN, tiling_enabled, llm_verify_enabled                                                    |
| DynamoDB pest-monitoring-detections    | table, PK image_id + SK detection_time, GSI by-pest-time                                                                                            | One record per processed frame                                                                                           |
| DynamoDB pest-monitoring-system-config | table, PK config_key                                                                                                                                | Single-row global settings (email recipients, capture flags)                                                             |
| DynamoDB pest-monitoring-schedule-logs | table, PK log_id + SK timestamp                                                                                                                     | Audit rows from the scheduler Lambda                                                                                     |
| API Gateway HTTP API                   | vzf17s6z00 (pest-monitoring-api-gateway)                                                                                                            | 21 routes, Cognito JWT authorizer, one open route                                                                        |
| Cognito                                | pool us-east-1_9se1FDHpc, client                                                                                                                    | Dashboard login, admin-create-only                                                                                       |

| Component                 | Location / identifier                                                                     | Purpose                                                                  |
|---------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
|                           | 6vebotf45bp8u46cnraddiap1v                                                                |                                                                          |
| CloudFront                | E1YADURLSAVNFA →<br>https://d1dtoxef7qmug1.cloudfront.net                                 | HTTPS front for the dashboard bucket, CachingDisabled                    |
| KVS                       | armyworm-cam-stream, moth-cam-stream (dev account only; not created on production — 2.14) | Live-view ingest from the Jetson Orin GStreamer producer                 |
| EventBridge Scheduler     | pest-model-watchdog-15min + runtime pest-sched-* rules                                    | Watchdog cadence; per-camera model schedules                             |
| SES                       | identity<br>rex2956550768@gmail.com (sandbox)                                             | Detection alert email, sender and default recipient                      |
| IAM                       | 5 execution roles + scheduler invocation role                                             | Per-function policies (2.17)                                             |
| Policy files              | lambda/cors.json, lambda/ddb-policy.json, lambda/bedrock-policy.json                      | Source JSON applied to API GW CORS and the processor/api roles           |
| Rekognition Custom Labels | projects argus-detection (armyworm), argus-moth-detection (moth)                          | The detector models; ARNs stored per camera in DynamoDB, never hardcoded |

### 2.2.1 Development account (366356442579) identifiers — retired 2026-08-10

ARGUS was built and validated on the development account, then stood up on production by the ARGUS deployer (deployer/deploy.py, run 2026-08-10 with --prefix argus). Table/Lambda/role/layer names are unchanged across accounts, and so are the DynamoDB table names and KVS stream names; the account-suffixed and generated identifiers differ. The development column below is history — use it only to read old evidence, never in a “how to do it now” command:

| Resource         | Development<br>(366356442579)          | Production (506868652945)    |
|------------------|----------------------------------------|------------------------------|
| Frames bucket    | frames-armyworm-366356442579           | argus-frames-506868652945    |
| Processed bucket | processed-images-armyworm-366356442579 | argus-processed-506868652945 |
| Dashboard bucket | pest-dashboard-                        | argus-dashboard-             |

| Resource              | Development<br>(366356442579)                                 | Production (506868652945)                                                                                                                                                                    |
|-----------------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API Gateway HTTP API  | 366356442579<br>zwpcbivmsj                                    | 506868652945<br>vzf17s6z00<br>(https://vzf17s6z00.execute-api.us-east-1.amazonaws.com)                                                                                                       |
| Cognito pool / client | us-east-1_ea0aJdus1 /<br>4husu6afr835e235eu9dqp8a<br>v6       | us-east-1_9se1FDHpc /<br>6vebotf45bp8u46cnraddiap<br>lv                                                                                                                                      |
| CloudFront            | E1423RGLAXWNSI →<br>https://d1twcdquexdgj8.c<br>loudfront.net | E1YADURLSAVNFA →<br>https://d1dtoxef7qmugl.c<br>loudfront.net                                                                                                                                |
| Rekognition projects  | armyworm-detection-v9,<br>SmartPestProject                    | argus-detection<br>(armyworm; version v9r-<br>prod-20260810, trained<br>2026-08-12, F1 0.613),<br>argus-moth-detection<br>(version moth-prod-<br>20260811, trained 2026-08-<br>11, F1 0.991) |
| fyp-pillow layer      | ...layer:fyp-pillow:2                                         | ...layer:fyp-pillow:1<br>(fresh publish)                                                                                                                                                     |
| KVS streams           | armyworm-cam-stream,<br>moth-cam-stream                       | none (live view not migrated)                                                                                                                                                                |

Device-side consequence of the rename: the Orin / mini-PC upload target and API base had to move to argus-frames-506868652945 and the vzf17s6z00 API. The repo mirrors (robot/go2\_patrol\_gated.py, robot/capture\_4k\_hdmi.py, robot/kvs\_controller.py, minipc/kvs\_controller.py, minipc/capture\_and\_upload\_v4\_armyworm.py) were repointed 2026-08-13; syncing them onto the devices and swapping the on-device AWS credentials (both still carry old-account keys) is pending the hardware's return.

## 2.3 S3 buckets

### 2.3.1 argus-frames-506868652945 (the ingest bucket)

Purpose. Every frame enters the system as an object under frames/{camera\_id}/{waypoint\_id}/{filename}. The bucket also stores non-frame material under other prefixes — the migrated training sets under training-data/v9/ and training-data/moth/, and the handover snapshot under handover/ — which is why the delete path in the API is prefix-gated (2.6).

Configuration (created by the ARGUS deployer to the same shape as the audited development bucket; the dev-account snapshot deployer/audit/s3\_\_frames-

armyworm\_\_\*.json is the template): - Versioning ON, SSE-S3 AES256, all four Block Public Access flags TRUE, no bucket policy. - CORS (needed because the dashboard PUTs test uploads and GETs frames with presigned URLs directly from the browser): AllowedMethods GET, PUT, POST, HEAD; AllowedHeaders \*; ExposeHeaders ETag; MaxAge 3000; AllowedOrigins: - https://d1dtoxef7qmug1.cloudfront.net - http://argus-dashboard-506868652945.s3-website-us-east-1.amazonaws.com - http://localhost:5500, http://127.0.0.1:5500, http://localhost:5501, http://127.0.0.1:5501 - Event notification: s3:ObjectCreated:\*, key prefix filter frames/, target Lambda pest-detection-processor. The prefix filter keeps training-data and handover uploads from triggering detection (publishing the 188.9 MiB snapshot to handover/ fired zero invocations). Note that the processor's EXIF write-back (2.5.1 step 5) rewrites the same frames/ key, so a rotated upload re-triggers the Lambda once; the second pass finds an already-upright image, does not rewrite again, and the loop terminates after one extra invocation.

Note on lambda/cors.json: that file is the API Gateway CORS document (AllowHeaders content-type, authorization), not this bucket's CORS. The bucket CORS lives in the audit JSON above.

Inspect: - Console: S3 → Buckets → argus-frames-506868652945 → Permissions tab (CORS) and Properties tab → Event notifications. - CLI: aws s3api get-bucket-cors --bucket argus-frames-506868652945 --profile prod and aws s3api get-bucket-notification-configuration --bucket argus-frames-506868652945 --profile prod

Recreate (fresh account, after the processor Lambda exists): 1. aws s3api create-bucket --bucket \*\*frames-<prefix>-<ACCOUNT\_ID>\*\* --profile \*\*PROFILE\*\* (in us-east-1, do NOT pass --create-bucket-configuration; in any other region you must). 2. Grant S3 permission to invoke the processor BEFORE wiring the notification, or the put fails: aws lambda add-permission --function-name pest-detection-processor --statement-id s3-invoke --action lambda:InvokeFunction --principal s3.amazonaws.com --source-arn arn:aws:s3:::\*\*frames-<prefix>-<ACCOUNT\_ID>\*\* --source-account \*\*ACCOUNT\_ID\*\* --profile \*\*PROFILE\*\* 3. aws s3api put-bucket-notification-configuration --bucket \*\*frames-<prefix>-<ACCOUNT\_ID>\*\* --notification-configuration file://\*\*path-to-notification.json\*\* --profile \*\*PROFILE\*\* (template: deployer/audit/s3\_\_frames-armyworm\_\_notification.json). 4. aws s3api put-bucket-cors --bucket \*\*frames-<prefix>-<ACCOUNT\_ID>\*\* --cors-configuration file://\*\*path-to-cors.json\*\* --profile \*\*PROFILE\*\* (update the origins to the new CloudFront domain and website endpoint). Console path for the notification: bucket → Properties → Event notifications → Create event notification → prefix frames/, event type All object create events, destination Lambda.

Gotcha. The retired development account carried two duplicate S3 invoke statements on the processor (one console-added, one CLI-added). Harmless, but emit only one when recreating; the deployer emits one.

### 2.3.2 argus-processed-506868652945

Purpose. On the development account, Gen-1 and processor v4.0 wrote annotated images to the processed bucket; since v4.1 the processor writes nothing to it, and the production bucket has been empty from day one. The bucket must still exist because pest-monitoring-api requires the S3\_PROCESSED\_BUCKET env var at cold start (os.environ["S3\_PROCESSED\_BUCKET"], a hard KeyError if absent), and DELETE /detection deletes legacy processed\_image\_key objects from it. The processor does NOT read this env var anywhere in its code; it is merely set on the live function config and ignored. (deployer/STACK\_MANIFEST.md item 6 overstates this as a processor cold-start requirement; the code contradicts it. NOTE: doc lag there, not here.)

Configuration: Versioning ON, SSE-S3, PAB all TRUE, private, CORS localhost:5500 only, no notification.

### 2.3.3 argus-dashboard-506868652945 (static website)

Purpose. Hosts the built dashboard files (index.html, styles.css, js/\*.js). Served publicly two ways: the raw website endpoint over HTTP, and CloudFront E1YADURLSAVNFA over HTTPS (the URL that is handed out).

Configuration: - Versioning OFF, SSE-S3, Block Public Access all FALSE. - Bucket policy PublicReadDashboard: Principal \*, s3:GetObject on arn:aws:s3:::argus-dashboard-506868652945/\* . - Static website hosting: Index document index.html, Error document index.html (this is the SPA fallback; CloudFront has no custom error responses).

Redeploy after any dashboard edit (the --cache-control flag is required; without it browsers heuristically cache the JS and users keep the old build):

```
aws s3 sync web/dashboard_v4 s3://argus-dashboard-506868652945 --delete --
cache-control "no-cache, must-revalidate" --exclude "*.md" --exclude
".claude/*" --profile prod
```

The repo copy of web/dashboard\_v4/js/config.js keeps the development reference values; the deployer rewrites HTTP\_API / COGNITO\_CLIENT\_ID in memory at upload time. If you sync by hand, first set them to the production values (https://vzf17s6z00.execute-api.us-east-1.amazonaws.com, 6vebotf45bp8u46cnnraddiavl) in your working copy.

Because CloudFront runs the CachingDisabled policy, this sync IS the full redeploy. No CloudFront invalidation step exists or is needed.

Recreate order trap: put-public-access-block (all false) must come BEFORE put-bucket-policy, or the policy call is rejected. Then put-bucket-website, then sync. - Console: S3 → bucket → Permissions → Block public access → Edit; Permissions → Bucket policy; Properties → Static website hosting. - CLI: aws s3api put-public-access-block - -bucket \*\*BUCKET\*\* --public-access-block-configuration BlockPublicAcls=false,IgnorePublicAcls=false,BlockPublicPolicy=false,RestrictPublicBuckets=false --profile \*\*PROFILE\*\*, then aws s3api put-bucket-policy --

```
bucket **BUCKET** --policy file:///**policy.json**, then aws s3api put-bucket-website --bucket **BUCKET** --website-configuration '{"IndexDocument":{"Suffix":"index.html"},"ErrorDocument":{"Key":"index.html"}}'.
```

### 2.3.4 lambda-layers-366356442579 (dev account only, history)

Held only the Gen-1 ffmpeg-layer.zip (~59 MB), which no current Lambda uses. The fyp-pillow layer (7.9 MB) publishes directly without S3 staging. The production account has no layer bucket and does not need one.

## 2.4 DynamoDB tables

All four production tables: billing PAY\_PER\_REQUEST, TTL off, PITR off, no streams, TableClass STANDARD, DeletionProtection false. Tags: Project=armyworm-v2, Owner=Runzhe-armyworm, ManagedBy=Runzhe.

Inspect any table: - Console: DynamoDB → Tables → table name → Overview / Indexes / Explore table items. - CLI: `aws dynamodb describe-table --table-name **TABLE** --profile prod --region us-east-1`

### 2.4.1 pest-monitoring-cameras

PK camera\_id (S). No GSI. One row per camera; this table is the single runtime home of the Rekognition model ARN and of the per-camera feature flags. All three detection-path Lambdas resolve custom\_model\_arn from here at call time; nothing hardcodes an ARN.

Live rows: 3, after the 2026-07-14 camera\_id migration (armyworm\_go2\_a8mini→worm\_cam, moth\_cam\_01→moth\_cam; test row person\_cam deleted 2026-07-13).

Live item (worm\_cam, read from the production table 2026-08-14):

```
{
  "camera_id": "worm_cam",
  "label": "Worm Cam",
  "target_label": "armyworm-larva",
  "model_type": "custom",
  "custom_model_arn": "arn:aws:rekognition:us-east-1:506868652945:project/arg
us-detection/version/v9r-prod-20260810/1786377372187",
  "min_confidence": 10,
  "post_verify_floor": 33,
  "max_runtime_min": 45,
  "model_running": false,
  "tiling_enabled": true,
  "llm_verify_enabled": true,
  "stream_enabled": false,
  "kvs_stream_name": "armyworm-cam-stream",
  "default_waypoint_id": null,
  "schedule": {"enabled": false, "days": [], "start_time": "05:40", "updated_
```

```
at": "..."}  
}
```

Three of these fields form the tuned detection ladder: `min_confidence` 10 is the CANDIDATE floor before the LLM gate (raising it strangles recall — the exact trap found 2026-08-05), `post_verify_floor` is the display/denoise floor applied AFTER the LLM verdict (per-camera override of the processor env `POST_VERIFY_FLOOR`), and `max_runtime_min` 45 is the per-camera watchdog auto-stop window (v6.2). The floor's history matters when reading old runs: 49 was decided and flipped live 2026-08-10 on the development account, and the deploy-day baseline `migration/prod_baseline_20260810.json` still shows 49; after the production model retrain, the floor was refitted per-build to **33** on 2026-08-13 (camera row AND Lambda env together) so a live Test upload draws the same boxes the curated gallery shows. **The current live value is 33.** The floor is a property of a model build, not a constant — refit it whenever the detector is retrained. The schedule map no longer carries `end_time`: since API v6.2 scheduling is start-only and the watchdog closes the run (2.6, 2.9).

`moth_cam` mirrors this with `target_label` "Moths", `model_type` custom, the production moth ARN (`arn:aws:rekognition:us-east-1:506868652945:project/argus-moth-detection/version/moth-prod-20260811/1786412382515`, seeded by `migration/migrate_moth.py`), `kvs_stream_name` "moth-cam-stream", `max_runtime_min` 45, and `llm_verify_enabled` FALSE on purpose — the moth model is accurate enough on its clean trap background that verification tokens buy nothing, and the verify prompt asks about larvae while this camera's target is adult moths. `manual_upload` is the hidden fallback row used whenever an S3 key does not parse to a known camera; it also backs the dashboard Test upload. The deployer seeds it as `armyworm-larva/custom` (the development account's row was `Person/general/80`) — behaviour is identical because its `custom_model_arn` is empty, which routes the processor down the generic path.

NOTE: doc lag. `deployer/audit/dynamodb__pest-monitoring-cameras__seed-scan.json` is a 2026-07-08 development-account snapshot with 4 rows under the old camera ids and the v5 model ARN. The 3-row layout above has been current since the 2026-07-14 `camera_id` migration.

Seeding history worth keeping: the ARGUS deployer seeds camera rows with the tuned values (`min_confidence` 10, `llm_verify_enabled` true, `post_verify_floor` 49 as of deploy day — see the floor history above; `max_runtime_min` 45). That seeding is a 2026-08-10 fix: an earlier deployer seeded `min_confidence` 30 and no `llm_verify_enabled` at all, which would have silently disabled the whole LLM layer on a fresh stack. Two post-deploy row repairs on 2026-08-11: `worm_cam.label` restored to "Worm Cam" (the deployer had written the `--deployment-name` into the label, and since the dashboard shows the label, not the `camera_id`, the row looked missing), and `worm_cam.schedule` restored to the 05:40-daily-disabled setting so re-enabling it does the right thing. `worm_cam.kvs_stream_name` carries the same stream name as on the development account, but no KVS stream actually exists on production (2.14); `stream_enabled` is false, so nothing reads it.



Gotcha. Writing a model ARN with `aws dynamodb update-item` from PowerShell mangles the quoting (the ARN contains colons). Use boto3 or a JSON file (`datasets/migrate_camera_ids.py` shows the pattern).

## 2.4.2 pest-monitoring-detections

PK `image_id` (S) = the exact S3 object key. SK `detection_time` (S), ISO 8601 with Z suffix. GSI by-pest-time: `pest_type` HASH + `detection_time` RANGE, projection ALL. The production table holds the curated handover gallery (64 records after the 2026-08-13 curation, 28 with a detection) plus the 36 migrated Jewel on-site records; the development table (~907 rows at retirement) keeps the threshold-study evidence.

One record per processed frame, written unconditionally (clean frames too, so the patrol gate never dead-locks). Example item — a 2026-07-29 development-account record, kept because it shows every field the schema has carried; production rows differ as noted below. Shaped by the processor's `db_item` (2.5.1 step 12):

```
{
  "image_id": "frames/worm_cam/wp3/20260729_101502.jpg",
  "detection_time": "2026-07-29T02:15:07Z",
  "pest_type": "armyworm-larva",
  "source": "live-detection",
  "created_at": "2026-07-29T02:15:07Z",
  "bucket": "frames-armyworm-366356442579",
  "original_image_key": "frames/worm_cam/wp3/20260729_101502.jpg",
  "camera_id": "worm_cam",
  "waypoint_id": "wp3",
  "target_label": "armyworm-larva",
  "target_detected": true,
  "target_confidence": "81.6",
  "min_confidence_used": 60,
  "label_count": 2,
  "labels": [{"name": "armyworm-larva", "confidence": 81.6}],
  "bboxes": [
    {
      "label": "armyworm-larva", "confidence": "81.6",
      "top": "0.4412", "left": "0.6120", "width": "0.0231", "height": "0.0184",
      "verified_by_llm": "true", "llm_reason": "elongated segmented larva with spots"
    }
  ],
  "verifications": {},
  "model_type": "custom",
  "model_arn": "arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection-v9/version/v9-20260725-1746/1785001598671",
  "llm_verify_model": "us.anthropic.claude-sonnet-4-6",
  "llm_scan": {"ran": false, "cells": "", "recovered": 0}
}
```

Field notes. Rows written by processor v6.3+ (2026-08-07) no longer carry the `llm_scan` map or `source=llm_scan` recovery boxes — those belong to the removed whole-frame

scan (2.5.6) and survive only on historical rows. Production rows carry bucket argus-frames-506868652945 and the production model ARN; the 36 migrated Jewel records deliberately keep the development account's model\_arn (provenance — it records which model produced that detection) plus migrated\_from/migrated\_at stamps. bboxes is only populated when target\_detected is true; a clean frame writes the record with an empty list. Bbox coordinates are stringified normalized floats (DynamoDB's Number type rejects the high-precision floats Rekognition emits; strings also match the Wilbur-era migrated data), confidence is formatted to one decimal, and llm\_reason is truncated to 120 characters by boxes\_to\_db\_format. verifications is a map filled by dashboard TP/FP clicks, keyed by bbox index as a string. labels is a legacy JSON string the dashboard still reads; since v4.5 its target-label entries are filtered by the same detection floor as bboxes (a real bug fix, 2.5.6), while non-target entries pass through unfiltered as context. Duplicate rows per image\_id are possible (at-least-once S3 events write a new row with a different detection\_time); DELETE /detection removes all of them. Old migrated rows use space-separated timestamps; /history handles both formats.

### 2.4.3 pest-monitoring-system-config

PK config\_key (S). Single live row detection\_settings:

```
{
  "config_key": "detection_settings",
  "email_enabled": true,
  "recipient_email": "rex2956550768@gmail.com",
  "additional_recipients": [],
  "auto_capture": true,
  "capture_interval": 60
}
```

Read by the processor for alert recipients and by the API for /settings and /identities.

### 2.4.4 pest-monitoring-schedule-logs

PK log\_id (S, uuid4) + SK timestamp (S, ISO Z). Audit rows written by pest-camera-scheduler:

```
{
  "log_id": "0f6f8b8e-...",
  "timestamp": "2026-07-20T01:00:03Z",
  "camera_id": "worm_cam",
  "action": "start",
  "status": "success",
  "message": "Model start initiated for worm_cam",
  "schedule_id": "b71b...",
  "trigger_time": "09:00"
}
```

## 2.4.5 Dead tables (do not recreate)

pest-monitoring-config (Gen-1 config, PK config\_id, only referenced by the archived Flutter app) and websocket-connections (the WebSocket push store retired when the dashboard moved to polling in v3.7).

Recreate the four production tables. Detections (the only one with a GSI):

```
aws dynamodb create-table --table-name pest-monitoring-detections --
attribute-definitions AttributeName=image_id,AttributeType=S
AttributeName=detection_time,AttributeType=S
AttributeName=pest_type,AttributeType=S --key-schema
AttributeName=image_id,KeyType=HASH
AttributeName=detection_time,KeyType=RANGE --billing-mode PAY_PER_REQUEST --
global-secondary-indexes '[{"IndexName":"by-pest-
time","KeySchema":[{"AttributeName":"pest_type","KeyType":"HASH"}, {"Attribute
Name":"detection_time","KeyType":"RANGE"}], "Projection":{"ProjectionType":"AL
L"}}]' --profile **PROFILE** --region **REGION**
```

The other three are plain creates with their key schemas from 2.4.1/2.4.3/2.4.4 and no GSI. Console: DynamoDB → Create table → set keys → Customize settings → On-demand capacity.

## 2.5 Lambda: pest-detection-processor (the detection pipeline)

Purpose. Turns one uploaded frame into one detection record and, when warranted, one alert email. This is the largest and most-evolved piece of the backend (~1,630 lines after the v6.3 dead-code strip; ~3,270 before it). Python 3.12, **1024 MB / 600 s** live (the 512 MB / 180 s recommendation in the file header is the minimum, not the deployment), layer fyp-pillow:1 (production; the dev account's live layer was :2 — 2.10), role pest-detection-processor-role, handler lambda\_function.lambda\_handler. Trigger: the frames-bucket S3 notification (2.3.1).

Version status (updated 2026-08-11): - The mirror lambda/pest-detection-processor.py is **v6.3** and matches the deployed code: the v6.3 dead-code strip was deployed 2026-08-07 (Runzhe's pre-migration order). Removed: the v4.6 whole-frame scan, v4.8 cluster merge, v5.1 LLM-FIRST, v5.2 LLM-LEAD, v5.6 LLM-PLAIN, v5.7 LLM-AGENT, v6.1 picture-in-picture composite, and the \_\_rek- detector override — ~1.6k lines of experiment paths production never ran. The live path is unchanged: tiling → v4.3 suppression → LLM denoiser gate (LLM\_VERIFY\_ALL\_BOXES=true) → post-gate cleanup. The \_\_confN and \_\_llm- key overrides stay (the dashboard Test upload uses both). Full pre-strip source: lambda/archive/pest-detection-processor\_v6.2\_full.py. The earlier open item about proving mirror-vs-deployed byte equality is closed by this deploy. Two oversized-frame fixes were added and deployed 2026-08-12 (detect\_whole\_frame + \_full\_frame\_bytes, documented in 2.5.3); the mirror carries both. - Live configuration: Timeout **600 s**, MemorySize **1024 MB**, LLM\_VERIFY\_MODEL\_ID=us.anthropic.claude-sonnet-4-6, LLM\_VERIFY\_ALL\_BOXES=true (denoiser mode: every box judged),

LLM\_VERIFY\_MAX\_BOXES=120, LLM\_VERIFY\_MAX\_TOKENS=300, LLM\_VERIFY\_WORKERS=3, LLM\_VERIFY\_PAD=0.6, TILE\_MIN\_CONFIDENCE=8, POST\_NMS\_IOW=0.1, POST\_NMS\_CONTAIN=0.1, POST\_MAX\_BOX\_AREA=0.05, POST\_VERIFY\_FLOOR=33 (floor history: 34→49 on 2026-08-10, refitted 49→**33** on 2026-08-13 for the retrained production model; the per-camera post\_verify\_floor DDB field overrides it and reads 33 too — verified against the live row 2026-08-14; the frozen deploy-day baseline migration/prod\_baseline\_20260810.json still shows 49). The retired dev account also still carries dead LLM\_SCAN/LLM\_MERGE/LLM\_LEAD/LLM\_FIRST/LLM\_AGENT/LLM\_PLAIN/LLM\_VERIFY\_COMPOSITE env vars whose code paths v6.3 deleted; the production account omits them (harmless either way). - deployer/audit/lambda\_\_pest-detection-processor.json is a 2026-07-08 snapshot (Timeout 60, no LLM env vars). Stale on those fields; still correct for role, layer, trigger, and the five base env vars. The frozen baseline above supersedes it for configuration. - The bedrock-verify inline policy is now dumped: deployer/audit/iam\_\_pest-detection-processor-role\_\_inline\_\_bedrock-verify.json. The live policy grants bedrock:InvokeModel + bedrock:Converse on wildcarded inference-profile families (us.anthropic.\*, global.anthropic.\*, us.amazon.\*, us.meta.\*, us.mistral.\*) plus the matching region-wildcarded foundation-model ARNs, so Sonnet 4.6 is covered. lambda/bedrock-policy.json (Haiku 4.5 + Nova Lite only, no Sonnet 4.6) is the older, narrower source file — recreating the role from it while the env points at Sonnet 4.6 reproduces the total-gate-failure mode in 2.5.6/2.5.9. Use the audit dump. - A second, previously undocumented inline policy is also dumped: deployer/audit/iam\_\_pest-detection-processor-role\_\_inline\_\_s3-frames-write.json — s3:PutObject scoped to frames/\* on the frames bucket. It exists for exactly one code path: the v4.8.3 EXIF orientation write-back (2.5.1 step 5). Without it, rotated phone uploads keep their EXIF tag and every box draws in the wrong place.

Base environment variables (the LLM/tiling tunables are documented inline below): SENDER\_EMAIL (required — the code's only hard os.environ[] read, live rex2956550768@gmail.com), TABLE\_DETECTIONS, TABLE\_CAMERAS, TABLE\_SYSTEM\_CONFIG, S3\_PROCESSED\_BUCKET (config-only: set on the live function but never read by the processor code).

## 2.5.1 Handler flow, end to end

lambda\_handler(event, context) in plain steps:

1. Parse the S3 event record: bucket + URL-decoded object key. Non-image extensions (anything not .jpg/.jpeg/.png) return 200 “Skipped”.
2. parse\_s3\_key(object\_key): splits the key on / and accepts it as standard only when there are at least 4 segments and the first is literally frames; then camera\_id = segment 2, waypoint\_id = segment 3 with everything from the first \_\_ stripped (so per-request overrides can ride in the key, step 4), filename = the remaining segments rejoined. Any other shape logs the key and returns the manual\_upload fallback pair — the function never fails, it degrades.

3. `get_camera_config(camera_id)` reads the camera row with a three-step fallback chain: the requested id, then the `manual_upload` row, then hardcoded defaults (Person / general / `min_confidence` 80) if even that read fails — a DynamoDB outage degrades detection rather than killing it. `get_system_config()` reads the global row (empty dict on any error); `collect_recipients()` merges `recipient_email` + `additional_recipients` into a deduped, order-preserving list (a comma-separated string is tolerated and split).
4. Per-request overrides parsed from the key, both stateless (no DDB writes, so parallel A/B runs never fight over config):
  - `__conf<N>` (N 10-100) overrides `min_confidence` for this run only. Used by the dashboard Test upload confidence slider.
  - `__llm-<alias>` (v4.9) picks the Bedrock verification model for this run from `LLM_MODEL_ALIASES` (v6.3 keeps two: `sonnet46`, `haiku45`).  
`set_active_llm_model()` is called on EVERY record and resets a one-slot holder, because Lambda reuses warm containers and a stale model id would silently attribute one model's results to another. Resolution order (v6.3), most specific first: the `__llm-` key alias, then the camera row's `llm_model_id` (the dashboard's per-camera model picker — stored as an ALIAS, not a raw model id, so a typo can never reach Bedrock as an unknown id), then the `LLM_VERIFY_MODEL_ID` env default. Only known aliases are honoured at any level: an unknown model id does not fail loudly at the call site, it fails as `AccessDenied` on EVERY crop, which the gate reads as “no verdict”.
5. EXIF orientation normalisation (v4.8.3, 2026-07-28): download the frame, `ImageOps.exif_transpose`, and if the image was rotated, write the upright pixels BACK to the same S3 key. Reason: browsers honour the EXIF Orientation tag, PIL and the coordinate maths here do not, so phone uploads had every box drawn in the wrong place. Baking the rotation in once makes Rekognition, the crops, and the dashboard agree. Non-fatal on any error.
6. Detection (2.5.2 / 2.5.3): tiled or single `detect_custom_labels` for custom cameras, `detect_labels` for general ones.
7. v4.3 hard-object suppression (2.5.5).
8. LLM gate (2.5.6): `apply_llm_verify_gate` — since v6.3 the one and only gate (the scan/merge/lead/first experiment dispatches are deleted). Only runs for custom-model cameras with `llm_verify_enabled: true` in DynamoDB and `LLM_VERIFY=true` globally. `hybrid_gate_ran = n_verified > 0`, never “the call did not raise” (a 2026-07-22 review bug: every internal fail-open path returns normally having verified nothing).
9. Post-gate cleanup (v5.0, extended v5.9/v6.2): `apply_post_gate_cleanup` over gate survivors, only when the gate verified something. Four knobs, all live-configured: `POST_MAX_BOX_AREA` (live 0.05 — a box over 5% of the frame cannot be a larva whatever the verifier said; grounded in the hand-labelled maximum of 4.39%, and the guard against a frame-spanning box the verifier confidently called a caterpillar),

POST\_NMS\_IoU (live 0.1) plus POST\_NMS\_CONTAIN (live 0.1 — IoU divides by the union, so a small box wholly inside a big one scores near zero and survives IoU-NMS; the containment test asks how much of the SMALLER box is covered), and the post-verify confidence floor: env POST\_VERIFY\_FLOOR overridden per camera by the DDB field post\_verify\_floor (v6.2; both live 49). All default 0 = off in code.

10. Decide target\_detected: detection\_floor = 0 if hybrid\_gate\_ran else min\_confidence. A gate-approved sub-threshold box is a legitimate detection; re-applying min\_confidence would delete it.
11. extract\_bounding\_boxes + boxes\_to\_db\_format build the bboxes list (2.5.8).
12. Unconditional put\_item to pest-monitoring-detections. Clean frames write a record; the Go2 patrol completion gate polls get\_item by S3 key and depends on it.
13. SES alert if target\_detected and email enabled and recipients exist (2.5.10).

## 2.5.2 Camera config lookup and the detection branch

is\_custom = model\_type == "custom" and custom\_model\_arn set and not REPLACE\_\*. Custom cameras with tiling\_enabled: true on the row (AND global env TILING\_ENABLED, default true) go through run\_tiled\_detection. Custom without tiling do a single detect\_custom\_labels on the S3 object with MinConfidence=30 (deliberately low; filtering happens later so the record keeps full label data). General cameras call detect\_labels (MaxLabels=50, MinConfidence=50). Any tiling failure falls back to the single S3Object call, so tiling can never break detection.

## 2.5.3 The tiling path (v4.2)

Functions: get\_tiling\_config, compute\_tile\_regions, \_crop\_upscale\_bytes, \_detect\_custom\_on\_bytes, \_tile\_label\_to\_global, nms, run\_tiled\_detection.

Geometry.compute\_tile\_regions(w, h, cols, rows, overlap) slices the frame into a TILE\_COLS x TILE\_ROWS grid (default 4x4), expands every tile by TILE\_OVERLAP (0.15) of the base tile on each side, and clamps to the frame. Overlap guarantees a target straddling a boundary lands whole in at least one tile. With TILE\_INCLUDE\_FULL\_FRAME=true a full-frame pass is prepended: 16 tiles + 1 = the 17-call geometry ("quadrant/17-tile" in project shorthand; cols/rows are env-tunable, so a 2x2 quadrant grid is the same code with different numbers).

Each tile is cropped and Lanczos-upscaled until its long edge reaches TILE\_UPSCALE\_LONG\_EDGE (1920; roughly the W6-validated ~4x zoom for a 1080p frame), JPEG-encoded, and sent to detect\_custom\_labels as raw bytes with MinConfidence=TILE\_MIN\_CONFIDENCE (code default 30; LIVE VALUE 8 since the denoiser era, which is what feeds the LLM gate its volume). \_detect\_custom\_on\_bytes retries throttles three times with backoff. \_tile\_label\_to\_global converts each tile-normalized box back to global normalized coordinates. nms greedy-deduplicates per class at TILE\_NMS\_IoU 0.5, keeping the highest-confidence box per overlap cluster (overlapping tiles produce duplicates by construction). Tiles run in a ThreadPoolExecutor

(TILE\_MAX\_WORKERS 4). The output list is shaped exactly like Rekognition CustomLabels, so nothing downstream knows tiling happened.

History note. Tiling was ruled OFF in W15 (“whole-image is the deployment mode”) and turned back ON for worm\_cam by Runzhe’s own call 2026-07-27/28 when the architecture flipped to “Rekognition finds (high recall), the LLM verifies (precision)”.

docs/detection.md’s “tiling is OFF in production” paragraph is the W15-era record. NOTE: doc lag.

## 2.5.4 Rekognition calls

Three Rekognition APIs are used: detect\_custom\_labels (the armyworm/moth models; per tile as bytes, or once per frame as an S3Object reference), detect\_labels (twice over: as the general-camera detector, and as the v4.3 suppression pass), and, from the other Lambdas, start/stop\_project\_version + describe\_projects/describe\_project\_versions. The endpoint must be RUNNING or detect\_custom\_labels fails; see 2.18.

**Where the detection JSON goes.** This is the most common question about the pipeline, so it is worth stating exactly. Rekognition does not write a result file anywhere, and nothing is “sent” to the processor. detect\_custom\_labels is a **synchronous HTTPS request** made by the Lambda through boto3. The Lambda puts the image in the request — either as raw bytes (Image={"Bytes": ...}, used for every tile) or as a pointer (Image={"S3Object": {"Bucket": ..., "Name": ...}}, used for the single whole-frame call) — and the JSON result comes back **in the response of that same call**, parsed by boto3 into a Python dictionary:

```
resp = rekognition.detect_custom_labels(  
    ProjectVersionArn=model_arn,  
    Image={"Bytes": tile_bytes},  
    MinConfidence=TILE_MIN_CONFIDENCE,  
)  
labels = resp["CustomLabels"]      # list of dicts, in memory, nothing on disk
```

Each entry in resp["CustomLabels"] is {"Name": <label>, "Confidence": <0-100>, "Geometry": {"BoundingBox": {"Left", "Top", "Width", "Height"}}}, with the box normalised 0-1 against the image that was sent. For a tiled frame that means 17 separate responses, each with coordinates relative to its own tile; \_tile\_label\_to\_global converts them to whole-frame coordinates before anything else touches them.

From that point the result exists **only as a Python object inside the running Lambda**. It is passed by reference through suppress\_nonveg, apply\_llm\_verify\_gate and apply\_post\_gate\_cleanup, each of which filters the list in memory. Nothing is persisted until the very end, when detections\_table.put\_item writes the surviving boxes into the DynamoDB record. There is no intermediate S3 object, no queue, no file, and no way to inspect the raw model output after the fact except in the CloudWatch log lines the



processor prints as it goes. If a raw copy is ever needed for debugging, it has to be added deliberately, for example by logging `json.dumps(resp)` before filtering.

The same holds for `detect_labels`: its response is consumed in-process by `get_suppression_regions` and discarded.

### 2.5.5 v4.3 DetectLabels hard-object suppression

Problem it solves: Rekognition Custom Labels object detection cannot be trained on negative images (proven empirically and against AWS documentation, 2026-07-13; see `docs/detection.md`). The model's learned prior is "plant close-up means armyworm"; out of domain it boxed people, a robot arm, and a jeep wheel at 90%+. So non-vegetation false positives are filtered at the application layer.

Functions: `get_suppression_regions`, `_covered_fraction`, `suppress_nonveg`.

Algorithm: 1. Runs only for custom models, only when the model actually produced at least one box (clean frames never pay the extra call), and only when `SUPPRESS_NONVEG=true` (default). 2. `get_suppression_regions` runs `detect_labels` on the frame (`MaxLabels=50`, floor `SUPPRESS_MIN_CONF 55`) and collects instance bounding boxes whose label is in `SUPPRESS_LABELS` (person, vehicle, wheel, tire, machinery, furniture, building, wall, and ~40 more) and NOT in `SUPPRESS_PROTECT`. 3. The plant-label exemption: `SUPPRESS_PROTECT` (plant, leaf, flower, tree, vegetation, grass, foliage, soil, ground, ...) can never generate a suppression region. Worms live on plants; a DetectLabels "Plant" box overlapping a worm proves nothing. 4. Coverage test: `suppress_nonveg` drops a custom box when `_covered_fraction` — the fraction of the WORM BOX's own area lying inside a hard-object region, not IoU — reaches `SUPPRESS_COVERAGE (0.5)`. Boxes without geometry pass through untouched. 5. Non-fatal at every level: a DetectLabels error returns zero regions and detection proceeds unfiltered.

All knobs env-tunable: `SUPPRESS_NONVEG`, `SUPPRESS_MIN_CONF`, `SUPPRESS_COVERAGE`, `SUPPRESS_LABELS`, `SUPPRESS_PROTECT`. Required an IAM add of `rekognition:DetectLabels` to `pest-detection-processor-policy` (the role previously had only `DetectCustomLabels`). Verified pre-deploy: the 65% jeep-tire FP on `cag_armyworm_103` (94% covered by a wheel region) drops while the real worms in 104/105 stay (`datasets/verify_suppression.py`).

### 2.5.6 The Bedrock LLM verification gate (v4.4 → v4.7)

Shared plumbing: - `get_bedrock_client()` builds the bedrock-runtime client lazily under a lock (crop verdicts run in a thread pool; unlocked concurrent construction on boto3's default session can raise inside a worker, which fail-open would silently score as "unverified, keep"). Client config: read timeout `LLM_VERIFY_TIMEOUT (12 s)`, 5 retries in adaptive mode (client-side rate limiting; a large sweep can fire 144 calls per frame). - `crop_box_bytes(pil_img, bbox, pad, long_edge)`: crops a normalized box with `LLM_VERIFY_PAD (0.6)` padding as a fraction of the box's own size, floored at `LLM_VERIFY_MIN_CONTEXT_PX (32 px)` so a tiny box still gets real surrounding context, then



upscales toward LLM\_VERIFY\_LONG\_EDGE (672) capped at LLM\_VERIFY\_MAX\_UPSCALE (8x) so a 6x4 px box is not blown into pure interpolation noise (a real 2026-07-21 finding). - `verify_one_crop(crop_bytes)`: one Bedrock Converse call, raw JPEG bytes + LLM\_VERIFY\_PROMPT, maxTokens LLM\_VERIFY\_MAX\_TOKENS (code default 100; live 300 for Sonnet 4.6, see the thinking-model gotcha in 2.5.9). Temperature is only sent when LLM\_VERIFY\_TEMPERATURE is explicitly set (sampling parameters are a 400 on newer Claude models). Refusal/guardrail stop reasons return None (a safety decline is not a verdict about the crop). The reply text is taken from the first text block, skipping the reasoning blocks that thinking-always-on models emit. - `_extract_json_object` + `parse_llm_verdict`: balanced-brace scan honouring string quoting (a greedy regex broke on any brace after the object), then strict parsing of `{"is_larva": bool, "reason": str}` with tolerant bool coercion. Anything unparseable returns None = “unverified”, which KEEPS the box. - The prompt (LLM\_VERIFY\_PROMPT) is the v5.9 NEUTRAL rewrite (2026-08-04, wording refined 2026-08-07): it opens with “What is in this photo?”, states that most such crops contain only leaves/stems/wood/soil/shadows, describes the target as an elongated soft body with clear segmentation “typically carrying yellow-and-black stripes” (Runzhe’s wording), lists the common non-larva look-alikes, and demands strict JSON with true ONLY if a larva is genuinely present at the crop centre. Two earlier prompt generations are retired for measured reasons: the recall-biased ending (“if unsure, answer true”) let wood planks and dried leaves survive as confirmed detections, and the primed opening (“an automated detector flagged this region as possibly containing a larva”) was a leading question — of 28 surviving false boxes the primed prompt killed 2, the neutral wording killed 5, and 5 false verdicts literally cited hand-drawn ink marks as their evidence. Do not reintroduce a preamble that asserts a larva may be present.

The v4.5 hybrid gate (deployed 2026-07-22; still the code’s default behaviour when LLM\_VERIFY\_ALL\_BOXES=false): - Only target-label boxes BELOW the camera’s `min_confidence` are adjudicated. A box at/above it is trusted outright and never sent to Bedrock — its fate cannot change, so verifying it would only spend money and latency. This changed the meaning of `min_confidence` from “detection floor” to “the point above which Rekognition’s word is final”. - An explicit “not a larva” verdict DROPS the box; a positive verdict or ANY failure keeps it. Fail-open at every level: missing Bedrock permission, model error, malformed reply, throttle — none of them ever deletes a box. - Candidates are sorted by confidence and capped at LLM\_VERIFY\_MAX\_BOXES (`_crop_verdicts`), judged in parallel (LLM\_VERIFY\_WORKERS, code default 4, live 3). - Per-camera opt-in via the DynamoDB flag `llm_verify_enabled` (mirrors `tiling_enabled`). Not ceremony: the Lambda is shared by every custom camera, and `moth_cam`’s target is adult moths while the prompt asks about larvae. Only `worm_cam` opts in. Global kill switch LLM\_VERIFY (default true). - Results land per box as `verified_by_llm` (“true”/“false”) and `llm_reason` (written by `boxes_to_db_format` from the internal `_llm` field), plus `llm_verify_model` on the row. - Two pre-production bugs found by adversarial review 2026-07-22, both fixed and regression-tested: (1) gate success used to be judged by “did the call not raise”, which let all-fail-open runs collapse the detection floor to 0 with nothing verified; the gate now returns `n_verified` and the floor only collapses when `n_verified > 0`. (2) The legacy

labels field dumped raw Rekognition output down to its ~30-50% internal gather floor whenever the gate did not run, and the dashboard trusts target-label entries there as confirmed detections; it is now filtered by the same detection floor as bboxes.

The v4.6 whole-frame scan is HISTORY as of v6.3: it ran one extra Bedrock pass over the downsampled frame asking for positive grid cells and could recover a synthetic box from an un-boxed positive cell (fail-closed, crop-confirmed). It was never enabled in production (LLM\_SCAN=false throughout) and the v6.3 strip deleted the code; old detection records still carry its llm\_scan {ran: false} observability map, which is why the field exists. The key measurement that killed it stands: whole-frame LLM vision missed 8/14 larvae that a zoomed crop caught (2026-07-21; a methodology A/B of the verifier on internal images — not a detection-recall statistic; see Chapter 3) — a whole-frame model's silence is not evidence.

The v4.7 denoiser mode (LLM\_VERIFY\_ALL\_BOXES=true, THE LIVE MODE since 2026-07-29): the min\_confidence exemption is removed; EVERY target box is crop-judged and the model may delete even a high-confidence box. Rationale: with tiling on, Rekognition recall is near-total but the noise boxes (leaf/shadow/soil/flower) also come back at high confidence, so trusting high confidence lets that noise through. Survival is fail-CLOSED per box: a target box survives only with a positive verdict; rejected boxes AND un-judged boxes (over the cap, crop failure, throttle) are both dropped. The one exception is TOTAL GATE FAILURE — candidates existed, zero verdicts came back. That is infrastructure down, not evidence about the frame, so everything is kept, the log shouts, and the caller falls back to plain min\_confidence thresholding. The guard exists because the real thing happened: the model id was switched to Sonnet 4.6 before the role had Bedrock permission for it, producing 492 AccessDenied calls and 44/44 images in that test batch reporting zero detections. Pair the mode with a raised LLM\_VERIFY\_MAX\_BOXES (live 120) so tiling's volume is actually covered, or unchecked boxes die fail-closed. Measured live 2026-07-29: a clean SIYI A8 frame produced 61 boxes at 8-14%, the gate judged all 61 and dropped all 61, Lambda duration 35.4 s (the timeout budget was 180 s that day, 600 s since; a tiled frame plus up to 120 crops measures 24-54 s).

### 2.5.7 Retired experiment gates (removed in v6.3)

Between v4.8 and v6.2 the mirror accumulated experiment modes that production never enabled: v4.8 cluster merge (LLM\_MERGE — union-find over fragment boxes, one union-crop verdict per cluster), v5.1 LLM-FIRST (the model finds first, Rekognition matched against it), v5.2 LLM-LEAD (Sonnet sweeps an 8x8 native-resolution tile grid and Rekognition may only sharpen geometry), v5.6 LLM-PLAIN, v5.7 LLM-AGENT, the v6.1 picture-in-picture composite, and the \_\_rek- per-request detector override. The v6.3 dead-code strip (2026-08-07) deleted them all; the corresponding env vars are inert if still present on a function config. Two things to know remain: - The full pre-strip source is archived at `lambda/archive/pest-detection-processor_v6.2_full.py`; the experiments' measured results live in `docs/detection.md`. Do not re-deploy the archive. - v5.0 post-gate cleanup is the one survivor — it graduated into the live path and is documented in 2.5.1 step 9. Its

floor is deliberately applied AFTER the LLM verdict (the same number as an input floor does nothing useful).

### 2.5.8 Output shaping

`extract_bounding_boxes(labels, target_label, floor, model_type)` filters by exact case-insensitive name match and the passed floor, and normalizes both custom-label geometry and general-label Instances into {Left, Top, Width, Height, Confidence, Name} dicts (a general label above threshold with no instance becomes a `no_box` entry, which the DB formatter skips). `boxes_to_db_format` stringifies coordinates and attaches `verified_by_llm / llm_reason / source=llm_scan` when present. The dashboard reads `bbox` fields by name and ignores unknown keys, so new fields never require a frontend change.

### 2.5.9 Known gotchas and failure modes

- The Bedrock model id MUST be the cross-region inference profile (`us.` prefix), never the bare foundation-model id. The bare id fails with “Invocation of model ID ... with on-demand throughput isn’t supported. Retry your request with the ID or ARN of an inference profile”. List valid ids: `aws bedrock list-inference-profiles --profile prod --region us-east-1`.
- Pointing `LLM_VERIFY_MODEL_ID` at a thinking-always-on model without raising `LLM_VERIFY_MAX_TOKENS` makes every verdict silently fail open: the 100-token budget is consumed by reasoning and the JSON never arrives. The code logs the specific symptom (`stopReason=max_tokens` with no parsable verdict).
- Switching the model id before adding IAM for it = total gate failure (2.5.6). Loud in logs, invisible on the dashboard except as suspicious cleanliness.
- Bedrock Converse limits: 20 images per message, each  $\leq 3.75$  MB and  $\leq 8000 \times 8000$  px; do not pre-base64 image bytes, boto3 handles it.
- `put_item` is unconditional. If no record appears for an upload within  $\sim 30$  s, the Lambda itself failed; check CloudWatch `/aws/lambda/pest-detection-processor`.
- Changing a Lambda’s runtime or name requires delete + recreate, which wipes the resource policy; re-grant the S3 invoke permission (and for the API, the API Gateway permission) after.
- The fyp-pillow layer must contain the compiled `PIL/_imaging*.so`; layer version :1 lacked it and every PIL import failed (no tiling, no crops).

### 2.5.10 SES alerting from the processor

Subject [FYP Alert] {target\_label} detected at {waypoint\_id} ({confidence}). Body lists pest, confidence (or “LLM whole-frame scan” for recovery-only), camera, zone, model type, the `s3://` image path, timestamp, a recovery note when applicable, and all labels. Send failures are logged and non-fatal (the classic cause: an unverified recipient in SES sandbox mode, 2.16).

Inspect / deploy: - Console: Lambda → Functions → pest-detection-processor → Configuration (env vars, layer, trigger) / Monitor → Logs. - CLI inspect: `aws lambda get-function-configuration --function-name pest-detection-processor --profile prod --region us-east-1` - CLI deploy code (the mirror must be renamed inside the zip): `zip pest-detection-processor.py as lambda_function.py, then aws lambda update-function-code --function-name pest-detection-processor --zip-file fileb://**path-to-zip** --profile prod` - CLI recreate: `aws lambda create-function --function-name pest-detection-processor --runtime python3.12 --handler lambda_function.lambda_handler --role arn:aws:iam:**ACCOUNT_ID**:role/pest-detection-processor-role --memory-size 1024 --timeout 600 --layers **fyp-pillow-layer-arn** --zip-file fileb://**path-to-zip** --environment "Variables={SENDER_EMAIL=**SENDER_EMAIL**,TABLE_DETECTIONS=pest-monitoring-detections,TABLE_CAMERAS=pest-monitoring-cameras,TABLE_SYSTEM_CONFIG=pest-monitoring-system-config,S3_PROCESSED_BUCKET=**PROCESSED_BUCKET**}" --profile **PROFILE**` — then add the detection-tuning env vars from the frozen baseline (Sonnet 4.6, `LLM_VERIFY_ALL_BOXES=true`, `LLM_VERIFY_MAX_BOXES=120`, `LLM_VERIFY_MAX_TOKENS=300`, `TILE_MIN_CONFIDENCE=8`, `POST_NMS_IΟΥ=0.1`, `POST_NMS_CONTAIN=0.1`, `POST_MAX_BOX_AREA=0.05`, `POST_VERIFY_FLOOR=49`). Sizing matters: one tiled frame plus up to 120 verify crops measures 24-54 s, so a 60 s timeout would fail intermittently on real frames (a real ARGUS deployer bug, fixed 2026-08-10). The ARGUS deployer (deployer/deploy.py) now sets all of this automatically.

### 2.5.11 Function reference: every function in the processor

The sections above follow the frame through the pipeline. This table is the other view: each function in `lambda/pest-detection-processor.py`, what it is responsible for, and the actual rule or algorithm inside it. Read it when you are looking at the source and need to know why a function does what it does. Order follows the file.

| Function                                             | Job                                             | Rule or algorithm inside                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------------------------|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>active_llm_model / set_active_llm_model</code> | Decide which Bedrock model verifies THIS record | Most-specific-wins resolution: an <code>__llm-&lt;alias&gt;</code> segment in the S3 key beats the camera row's <code>llm_model_id</code> , which beats the <code>LLM_VERIFY_MODEL_ID</code> env default. The alias is looked up in a table of allowed ids, so a bad alias cannot inject an arbitrary model id. Stored in a module-level global for the life of one invocation |
| <code>iso_now</code>                                 | One timestamp format everywhere                 | UTC, ISO 8601, Z suffix. Every record and log line                                                                                                                                                                                                                                                                                                                             |

| Function               | Job                                                       | Rule or algorithm inside                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| parse_s3_key           | Turn the object key into (camera, waypoint, filename)     | <p>uses it, so string comparison sorts chronologically and the /history date filter can be a plain string range</p> <p>Splits frames/{camera}/{waypoint}/{file}; strips a __suffix from the waypoint segment so per-request overrides (__conf50, __llm-haiku) do not create phantom waypoints. Anything that does not match falls back to the manual_upload camera rather than failing</p> |
| get_camera_config      | Fetch the per-camera row                                  | Two DynamoDB get_item calls: the requested id, then manual_upload as fallback. Read failures are caught, so a table hiccup degrades to the fallback instead of killing the invocation                                                                                                                                                                                                      |
| get_system_config      | Fetch the single global row                               | One get_item on the config key; returns {} on failure so callers can use .get() defaults                                                                                                                                                                                                                                                                                                   |
| collect_recipients     | Build the alert address list                              | Merges recipient_email with additional_recipients, de-duplicates while preserving order                                                                                                                                                                                                                                                                                                    |
| extract_bounding_boxes | Turn the raw Rekognition reply into the internal box list | Filters on two conditions at once: label name matches the camera's target_label (case-insensitive) AND confidence is at or above the threshold. Carries no_box for labels with no geometry and __llm for the verdict attached later                                                                                                                                                        |

| Function                                          | Job                                           | Rule or algorithm inside                                                                                                                                                                                                                                                         |
|---------------------------------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>boxes_to_db_format</code>                   | Shape boxes for DynamoDB                      | Stringifies every coordinate. DynamoDB's Number type rejects the high-precision floats Rekognition returns, and the dashboard parses them back to float anyway                                                                                                                   |
| <code>get_tiling_config</code>                    | Decide whether this frame is tiled, and how   | Per-camera opt-in ( <code>tiling_enabled</code> ) gated by a global env switch; grid size, overlap, upscale and gather floor all come from env vars so tuning never needs a code deploy                                                                                          |
| <code>compute_tile_regions</code>                 | Produce the tile rectangles                   | Divides the frame into a cols x rows grid, expands every tile by overlap of a base cell on all four sides, then clamps to the frame edges. The overlap is what stops a worm sitting on a seam from being cut in half; the clamp is why edge tiles are smaller than interior ones |
| <code>_encode_jpeg /<br/>_full_frame_bytes</code> | Produce JPEG bytes<br>Rekognition will accept | Encodes at a set quality, then shrinks in a loop until the payload is under the service limit                                                                                                                                                                                    |
| <code>detect_whole_frame</code>                   | The single full-frame<br>detection call       | Prefers the S3Object form (no download). On <code>ImageTooLargeException</code> it falls back to downloading, shrinking and sending bytes, so an oversized phone photo degrades instead of failing the frame                                                                     |
| <code>_crop_upscale_bytes</code>                  | Prepare one tile for the<br>detector          | Crops the region, then Lanczos-upscases until the long edge reaches the configured target. This is the zoom that gives a small target more pixels to be                                                                                                                          |

| Function                             | Job                                | Rule or algorithm inside                                                                                                                                                                                                                                                                |
|--------------------------------------|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>_detect_custom_on_bytes</code> | Call the detector on raw bytes     | found in<br>Wraps <code>detect_custom_labels</code> with retry and backoff on throttling, so a burst of 17 tile calls does not lose a tile to a transient limit                                                                                                                         |
| <code>_tile_label_to_global</code>   | Put a tile hit back on the frame   | Rescales the tile-normalised box by the tile's pixel origin and size, divided by the frame size. <code>region=None</code> means the full-frame pass, whose geometry is already global. Returns <code>None</code> for a label with no usable geometry                                    |
| <code>_iou</code>                    | Overlap measure for the tile merge | Intersection area divided by union area of two boxes; zero when they do not touch, one when identical                                                                                                                                                                                   |
| <code>nms</code>                     | Merge duplicates across tiles      | Greedy per-class non-maximum suppression: sort by confidence descending, walk the list, keep a box only if its IoU against every already-kept box of the same class is below the threshold. Greedy means each box is compared with what survived, never with the original list          |
| <code>run_tiled_detection</code>     | Orchestrate the whole tiled pass   | Builds the regions, runs the tile calls in a small thread pool plus one full-frame pass, converts every hit to global coordinates, then de-duplicates with <code>nms</code> . Returns a list shaped exactly like a plain Rekognition reply, so nothing downstream knows tiling happened |

| Function                             | Job                                    | Rule or algorithm inside                                                                                                                                                                                                                                                                                              |
|--------------------------------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>get_suppression_regions</code> | Find hard non-vegetation objects       | One <code>detect_labels</code> call; keeps instance boxes whose label is in the suppression list and not in the protect list. Plant labels can never suppress, because worms live on plants. Returns <code>[]</code> on any error so detection is never blocked by this call                                          |
| <code>_covered_fraction</code>       | How much of a box sits inside a region | Intersection area divided by the BOX's own area, not by the union. The question here is "is this box on top of that object", not "are these the same object"                                                                                                                                                          |
| <code>suppress_nonveg</code>         | Drop boxes sitting on hard objects     | Drops a target box when <code>_covered_fraction</code> against any suppression region reaches <code>SUPPRESS_COVERAGE</code> (0.5). Boxes with no geometry pass through untouched                                                                                                                                     |
| <code>get_bedrock_client</code>      | Build the Bedrock client once          | Lazy construction behind a lock. Verification runs in a thread pool, and without the lock the first verified frame after a cold start could build the client from several threads at once                                                                                                                             |
| <code>crop_box_bytes</code>          | Cut the crop the verifier will see     | Pads the box by a fraction of its OWN size with a pixel floor, clamps the padded rectangle to the frame, crops, then Lanczos-upscases to a target long edge with a hard cap on the scale factor. The pad gives context to compare against; the cap stops a six-pixel box being blown up into pure interpolation noise |



| Function                           | Job                               | Rule or algorithm inside                                                                                                                                                                                                                                                                                                     |
|------------------------------------|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>_extract_json_object</code>  | Find the JSON in a model reply    | Scans for the first balanced <code>{...}</code> , tracking string quoting so a brace inside a quoted value does not end the scan early. Replaced an earlier greedy regex that broke on exactly that case                                                                                                                     |
| <code>parse_llm_verdict</code>     | Turn the reply into a verdict     | Extracts <code>{"is_larva": bool, "reason": str}</code> . Returns None when the reply cannot be trusted, which the caller treats as “no verdict” rather than as a negative                                                                                                                                                   |
| <code>verify_one_crop</code>       | Ask Bedrock about one crop        | One Converse call carrying an image block and the prompt text. Sampling parameters are omitted unless explicitly configured, because newer Claude models reject them. A safety refusal ( <code>stopReason</code> of <code>refusal</code> or <code>guardrail_intervened</code> ) is treated as no verdict, not as a rejection |
| <code>_crop_verdicts</code>        | Judge every candidate             | Sorts candidates by confidence, caps the count at <code>LLM_VERIFY_MAX_BOXES</code> , then crops and judges them through a small thread pool. Returns ( <code>label</code> , <code>verdict-or-None</code> ) pairs                                                                                                            |
| <code>apply_llm_verify_gate</code> | Decide what survives verification | In the delivered denoiser mode every target box needs a positive verdict to live; rejected and un-judged boxes are dropped. One deliberate exception first: if candidates existed but ZERO verdicts came back, the model is unreachable,                                                                                     |

| Function                | Job                                | Rule or algorithm inside                                                                                                                                                                                                                                                                                                                          |
|-------------------------|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                         |                                    | so nothing is dropped, the log shouts, and the Lambda falls back to a plain confidence floor                                                                                                                                                                                                                                                      |
| apply_post_gate_cleanup | Final tidy of what the gate passed | Runs four rules in a fixed order: area cap, display floor, then a pairwise pass that drops a box clashing with any already-kept box by either IoU or containment. Only boxes matching the target label are touched. A per-camera post_verify_floor overrides the env default, which is how the dashboard threshold control works without a deploy |
| _bbox_contain           | The overlap IoU cannot see         | Intersection divided by the area of the SMALLER box. Two tiles marking one worm can share almost no union, so IoU calls it 0.02 while this calls it 1.0                                                                                                                                                                                           |
| _bbox_iou               | IoU for the cleanup pass           | Same measure as _iou, on Rekognition-style {Left, Top, Width, Height} boxes                                                                                                                                                                                                                                                                       |
| lambda_handler          | The whole flow                     | Parses the event, reads config, fixes orientation, detects, suppresses, verifies, cleans up, writes one DynamoDB record unconditionally, and sends SES mail only on a confirmed detection. Every stage is wrapped so a non-fatal failure degrades that stage rather than losing the frame                                                         |

## 2.6 Lambda: pest-monitoring-api (the dashboard API)

Purpose. A single Lambda serving every dashboard route through API Gateway vzf17s6z00. Python 3.12, 256 MB, 30 s, role pest-monitoring-api-role, no layers. ~1,095 lines (mirror lambda/pest-monitoring-api.py), merged from Gen-1 pest-control-api v3.6 + pest-history-query v3. WebSocket broadcast was removed; the dashboard polls.

Env vars: S3\_FRAMES\_BUCKET (required), S3\_PROCESSED\_BUCKET (required), SCHEDULE\_EXECUTOR\_ARN (the pest-camera-scheduler function ARN; two-phase deploy, set after that function exists), plus the four table names.

Router. lambda\_handler reads method + path from the HTTP API v2 requestContext.http payload and dispatches. OPTIONS \* returns CORS headers unconditionally. Every response goes through cors\_response (Allow-Origin \*, Allow-Headers Content-Type, Authorization) with a Decimal-safe JSON encoder (\_json\_default).

Write protection. Two allowlists guard DynamoDB writes against arbitrary key injection. GLOBAL\_ALLOWED: email\_enabled, recipient\_email, additional\_recipients, auto\_capture, capture\_interval. CAMERA\_ALLOWED: label, target\_label, model\_type, custom\_model\_arn, min\_confidence, model\_running, default\_waypoint\_id, kvs\_stream\_name, stream\_enabled, tiling\_enabled, llm\_verify\_enabled, llm\_model\_id, post\_verify\_floor, max\_runtime\_min. Additions in order: tiling\_enabled 2026-07-13 (the dashboard “Zoom scan” toggle), llm\_verify\_enabled 2026-07-22, then the v6.2/v6.3 trio — post\_verify\_floor (the dashboard threshold control edits THIS, the display/denoise floor after the LLM check, never min\_confidence, the candidate floor before it), max\_runtime\_min (per-camera watchdog window), and llm\_model\_id (per-camera verification-model alias; a Test upload’s \_\_llm- key still wins for that one run).

Routes and their data access:

| Route             | Handler              | What it touches                                                                                                                                                                   |
|-------------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET /settings     | handle_get_settings  | Reads the system-config row + a full cameras scan (list_cameras_as_map, paginated); returns the old nested-map shape {...global, cameras: {id: {...}} for dashboard compatibility |
| POST /settings    | handle_post_settings | Allowlisted update_item on one camera row ({camera_id, fields}) and/or the global row                                                                                             |
| POST /model/start | handle_model_start   | Resolves custom_model_arn from the camera row,                                                                                                                                    |

| Route                  | Handler                              | What it touches                                                                                                                                                                                                                                                                         |
|------------------------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                        |                                      | <code>rekognition.start_project_version</code> (1 inference unit), sets <code>model_running=true</code>                                                                                                                                                                                 |
| POST /model/stop       | <code>handle_model_stop</code>       | <code>stop_project_version</code> , sets <code>model_running=false</code>                                                                                                                                                                                                               |
| GET /model/status      | <code>handle_model_status</code>     | Resolves the PROJECT ARN by prefix-matching <code>describe_projects</code> (the project id is not derivable from the version ARN by string-splitting), then <code>describe_project_versions</code> for the real RUNNING/STOPPED status, per camera or for all                           |
| GET /presigned-url     | <code>handle_presigned_url</code>    | method=PUT: presigned POST to the frames bucket (1 B - 25 MB content-length condition), used by Test upload. method=GET: bucket chosen by key shape — keys starting <code>frames/</code> from the frames bucket, prefix-less keys from the processed bucket (legacy annotated images)   |
| POST /detection/verify | <code>handle_verify_detection</code> | Resolves the composite key by querying the first row for the <code>image_id</code> ; per-bbox verdicts into the verifications map ( <code>SET verifications.#idx = "TP"/"FP"</code> , REMOVE to clear, with an <code>if_not_exists</code> init) or the legacy image-level verified bool |
| DELETE /detection      | <code>handle_delete_detection</code> | Permanent delete; detailed below                                                                                                                                                                                                                                                        |
| GET /history           | <code>handle_get_history</code>      | Filtered scan of detections (camera, zone, detected, pest_type, source,                                                                                                                                                                                                                 |

| Route              | Handler                  | What it touches                                                                                                                                                                                                                             |
|--------------------|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                    |                          | date_from/date_to, model substring), paginated with a 10,000-scanned cap, sorted desc by detection_time, limit max 500. The date filter is a plain string range, which works for BOTH the old space-separated and new ISO timestamp formats |
| GET /cost          | handle_cost              | Cost Explorer<br>get_cost_and_usage grouped by SERVICE (daily or monthly), plus a month-to-date total; touches no project tables                                                                                                            |
| GET /identities    | handle_get_identities    | Merges system-config recipients, then SES get_identity_verification_attributes for each                                                                                                                                                     |
| POST /identities   | handle_post_identity     | ses.verify_email_identity (sends the confirmation email) + appends to additional_recipients                                                                                                                                                 |
| DELETE /identities | handle_delete_identity   | ses.delete_identity (non-fatal) + removes from the list; refuses to remove the primary recipient                                                                                                                                            |
| GET /schedule      | handle_get_schedule      | Reads the schedule map from camera rows                                                                                                                                                                                                     |
| POST /schedule     | handle_post_schedule     | Creates/removes the START rule (v6.2, below) + stores the schedule map on the camera row; deletes any legacy stop rule on every save                                                                                                        |
| DELETE /schedule   | handle_delete_schedule   | Deletes both rules, sets schedule={enabled: false}                                                                                                                                                                                          |
| GET /schedule-logs | handle_get_schedule_logs | Scan of schedule-logs, optional camera filter, newest first, limit max 500                                                                                                                                                                  |

| Route                               | Handler                  | What it touches                                                                                                                                 |
|-------------------------------------|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| POST /stream/start,<br>/stream/stop | handle_stream_start/stop | Toggle stream_enabled on the camera row. Never touches KVS; the Orin-side kvs_controller polls the flag and starts/stops the GStreamer producer |
| GET /stream/status                  | handle_stream_status     | Read-only stream flags per camera. THE one unauthenticated route (2.11); the Orin/mini-PC controller polls it without a JWT                     |

DELETE /detection (v4.3 of the API, added 2026-07-07, route id glwyqo0, JWT-gated) in detail. DELETE /detection?image\_id=<key>: 1. Query ALL rows for the image\_id (duplicates possible). Zero rows = 404 and NOTHING is touched — the endpoint can never become an arbitrary S3 key deleter. 2. Delete S3 objects FIRST. Frame keys are deleted only under the frames/ prefix, gated in code AND in IAM (s3:DeleteObject in pest-monitoring-api-policy is scoped to frames/\*), so training assets under assets/ and datasets/ are unreachable both ways. Legacy processed\_image\_key objects are deleted from the processed bucket. 3. Any S3 failure aborts with 500 BEFORE rows are removed. The record is the only index to the object, so it must outlive it. The call is safely retryable (delete\_object on a gone key is a no-op). 4. Then delete every DynamoDB row.

Dynamic schedules — START-ONLY since v6.2 (Runzhe's call). handle\_post\_schedule creates ONE classic EventBridge rule per camera, pest-sched-{camera\_id}-start, cron built by \_cron\_expression (cron(mm hh ? \* MON,TUE.. \*), UTC), targeting SCHEDULE\_EXECUTOR\_ARN with an Input payload {camera\_id, action, schedule\_id, trigger\_time} and a per-rule lambda:AddPermission (StatementId sched-invoke-<rule>, Principal events.amazonaws.com, SourceArn built from the account id that \_account\_id\_from\_context extracts from the invocation context). No stop rule exists any more: the model starts at the chosen time and the watchdog closes it after the camera's max\_runtime\_min (2.9). Every save deletes any legacy -stop rule so old pairs die off; \_delete\_scheduled\_rule removes targets, rule, and permission, each best-effort. The schedule map on the camera row accordingly has no end\_time field.

Gotchas: - /history and /schedule-logs are table scans. Acceptable at project volume (hundreds of rows), not at scale; the by-pest-time GSI exists but these handlers do not use it. - GET /settings returns every camera field to any authenticated user, including the model ARN. Non-secret, but know it is exposed. - /cost needs AWSBillingReadOnlyAccess on the role; without it the route returns 500. - Schedule times are entered in Singapore local time and converted to UTC **by the API, not by the caller**. \_cron\_expression subtracts a fixed 8 hours (SGT has no daylight saving) and, when that crosses midnight,

shifts every day in the list back by one — Monday 05:40 SGT becomes `cron(40 21 ? * SUN *)`. Do not pre-convert on the dashboard side or the schedule will fire eight hours early. An earlier version passed the SGT time straight through and every schedule fired eight hours late; that is the bug this conversion exists to fix.

**Function reference: the helpers behind the routes.** The route table above names the handler for each path. These are the shared functions those handlers call.

| Function                                              | Job                                      | Rule or algorithm inside                                                                                                                                                                                                                           |
|-------------------------------------------------------|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>cors_response</code>                            | The single exit point for every response | Wraps status and body with the CORS headers and serialises with <code>_json_default</code> . Nothing returns a raw dict, so no route can accidentally omit CORS                                                                                    |
| <code>_json_default</code>                            | Make DynamoDB data JSON-safe             | Converts <code>Decimal</code> to <code>int</code> when it has no fractional part and to <code>float</code> otherwise. DynamoDB returns every number as <code>Decimal</code> , which <code>json.dumps</code> refuses                                |
| <code>_account_id_from_context</code>                 | Learn our own account id at run time     | Splits it out of the invocation context's function ARN, so the EventBridge SourceArn is built for whatever account the stack was deployed into. Nothing is hard-coded                                                                              |
| <code>get_camera /<br/>list_cameras_as_map</code>     | Read camera rows                         | <code>get_item</code> for one; a paginated scan for all, reshaped into <code>{camera_id: {...}}</code> because the dashboard was written against that older nested shape                                                                           |
| <code>update_system_config /<br/>update_camera</code> | Write config safely                      | Build an <code>UpdateExpression</code> from only the keys present in the allowlist, with <code>ExpressionAttributeNames</code> for every field so a reserved word cannot break the call. A key outside the allowlist is silently ignored, which is |

| Function                            | Job                                                   | Rule or algorithm inside                                                                                                                                                                                                                                                                                                                  |
|-------------------------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                     |                                                       | what stops arbitrary attribute injection                                                                                                                                                                                                                                                                                                  |
| <code>_camera_arn</code>            | Resolve a camera's model ARN                          | Reads the row and raises on an unknown camera, so the model start/stop routes fail loudly instead of calling Rekognition with an empty ARN                                                                                                                                                                                                |
| <code>_rule_name</code>             | Name the EventBridge rule                             | <code>pest-sched-{camera_id}-{action}</code> , one deterministic name per camera, so a save can find and replace its own rule without storing an id                                                                                                                                                                                       |
| <code>_cron_expression</code>       | Build the UTC cron from a local time                  | Subtracts a fixed 8 hours for SGT and shifts the day list back one when that crosses midnight (see the gotcha above)                                                                                                                                                                                                                      |
| <code>_put_scheduled_rule</code>    | Create the rule and let EventBridge invoke the target | Puts the rule, sets the scheduler Lambda as target with a JSON input payload, then adds a per-rule <code>lambda:AddPermission</code> whose <code>SourceArn</code> is built from the account id discovered at run time. Refuses to run if <code>SCHEDULE_EXECUTOR_ARN</code> is unset, because a rule with no target is worse than no rule |
| <code>_delete_scheduled_rule</code> | Remove a rule completely                              | Removes targets, then the rule, then the permission, each wrapped so a missing piece does not abort the rest. This is why a half-created rule can always be cleaned up by saving again                                                                                                                                                    |

Inspect: Console Lambda → pest-monitoring-api; CLI `aws lambda get-function-configuration --function-name pest-monitoring-api --profile prod`.



## 2.7 Lambda: pest-camera-scheduler

Purpose. The executor behind the dynamic pest-sched-\* rules. ~167 lines. Python 3.12, 128 MB, 60 s, role pest-camera-scheduler-role, no static trigger of its own (invoked by the rules and, in principle, directly via SCHEDULE\_EXECUTOR\_ARN).

Handler flow: validate camera\_id and action ("start"/"stop") from the event payload → get\_camera → resolve custom\_model\_arn (missing or placeholder = logged failure row + 400) → rekognition.start\_project\_version(MinInferenceUnits=1) or stop\_project\_version → update\_camera sets model\_running → write\_schedule\_log writes the audit row (status success/failure, message, schedule\_id, trigger\_time). ResourceInUseException (already in the target state, e.g. a start rule firing on a running model) is treated as a soft success so schedules do not spam failure rows.

Env vars: TABLE\_CAMERAS, TABLE\_SCHEDULE\_LOGS.

Gotcha: SCHEDULE\_EXECUTOR\_ARN on pest-monitoring-api must point at this function or POST /schedule fails with "deploy pest-camera-scheduler first".

## 2.8 Lambda: kvs-hls-handler

Purpose. Turns a KVS stream name into a playable HLS URL for the dashboard Live tab. Kept separate from pest-monitoring-api on purpose: its IAM surface is only two kinesisvideo read actions (no DynamoDB, S3, or SES), and KVS is a separable phase. Python 3.12, 128 MB, 30 s, role kvs-hls-handler-role, NO env vars. Source mirror: deployer/audit/kvs-hls-handler\_src/lambda\_function.py (rescued during the 2026-07-08 audit; it was never in lambda/).

Route: GET /video-playback?stream=<kvs\_stream\_name> on API zwpcbivmsj, integration 33f69kt, JWT-gated.

Handler flow: get\_hls\_url(stream) calls kinesisvideo.get\_data\_endpoint(APIName="GET\_HLS\_STREAMING\_SESSION\_URL") — the endpoint is per-account and per-stream, always resolved at runtime, never hardcoded — then builds a kinesis-video-archived-media client against that endpoint and calls get\_hls\_streaming\_session\_url(PlaybackMode="LIVE", Expires=3600). Errors return 503 with a hint: the stream does not exist in this account, or it exists with no live fragments (the producer is not pushing). The /stream/start|stop|status control routes stay in pest-monitoring-api; they only toggle a DynamoDB flag and never touch KVS.

## 2.9 Lambda: pest-model-watchdog

Purpose. Cost guard. A running Rekognition Custom Labels endpoint bills per hour; this function guarantees no model runs forever, no matter HOW it was started (dashboard, schedule, console, or CLI). Python 3.12, 128 MB, 30 s. Role: pest-model-watchdog-role on the production account, created by the ARGUS deployer (2.17). Historical note: the retired development account carried a console-generated variant, pest-model-watchdog-

role-78asdw2b under a /service-role/ path; the deployer deliberately drops the suffix and path so every fresh deployment gets the plain name. Source mirror: lambda/pest-model-watchdog.py (v6.2; the pre-v6.2 audit rescue at deployer/audit/pest-model-watchdog\_src/lambda\_function.py lacks the per-camera limit).

Trigger: EventBridge Scheduler pest-model-watchdog-15min, rate(15 minutes) (2.15; the ARGUS deployer creates the same schedule on the production account). Env: TABLE\_CAMERAS, MAX\_RUNTIME\_MIN=75 live — but since v6.2 that env value is only the FALLBACK: the per-camera DDB field max\_runtime\_min wins when present, and worm\_cam carries **45** (set 2026-08-10, restored from a temporary 240 used for the 08-07 test window). The dashboard writes max\_runtime\_min through the settings API, and it is what makes start-only scheduling (2.6) safe: a scheduled morning run closes itself ~45 minutes after start.

NOTE: doc lag. The function docstring says “every 10 minutes” and a 60-minute default; live truth is a 15-minute schedule with the per-camera/env limits above. Worst case a model runs its limit + 15 minutes before being stopped.

Handler flow, per camera row with a real custom\_model\_arn: 1. project\_arn\_for\_version resolves the project ARN by listing describe\_projects and prefix-matching on project/<name>/, because the project id in the project ARN is not present in the version ARN (same trick as the API's /model/status). 2. model\_status reads the authoritative status from describe\_project\_versions. 3. Not RUNNING → remove any stale model\_started\_at stamp so the next start measures fresh. 4. RUNNING with no stamp → self-stamp model\_started\_at = now (first sighting). Self-stamping is what makes the watchdog start-method-agnostic. 5. RUNNING past the limit → stop\_project\_version, set model\_running=false, remove the stamp.

Handler flow addendum (v6.2): step 5's limit is `int(cam.get("max_runtime_min", MAX_RUNTIME_MIN))` with a try/except fallback to the env value on a malformed field.

Operational consequence to remember during long test sessions: the watchdog WILL stop your endpoint mid-run after the camera's max\_runtime\_min (worm\_cam: ~45 minutes). This is working as designed, not a fault — the armyworm endpoint was auto-stopped exactly this way after the 2026-08-07 test window. For a longer session, raise the camera's max\_runtime\_min for the day (POST /settings) or temporarily disable the schedule (2.15), and put it back after. Do not reflexively stop endpoints yourself between test runs; check state first (GET /model/status or the CLI in 2.18).

## 2.10 Lambda layer: fyp-pillow

Provides Pillow 12.2.0 (cp312, manylinux2014\_x86\_64) to the processor. Live ARN `arn:aws:lambda:us-east-1:366356442579:layer:fyp-pillow:2` (7,888,247 bytes). Version :1 is dead: it was built without the compiled `_imaging*.so`, so every PIL import failed and no boxes could be tiled or cropped.

Rebuild for a fresh account: 1. `pip install --platform manylinux2014_x86_64 --implementation cp --python-version 3.12 --only-binary=:all: --target ./python Pillow==12.2.0` 2. VERIFY `python/PIL/_imaging*.so` and `python/pillow.libs/` exist (the exact:1 bug), then zip the python/ folder at archive root. 3. `aws lambda publish-layer-version --layer-name fyp-pillow --zip-file fileb://**layer-zip** --compatible-runtimes python3.12 --compatible-architectures x86_64 --profile **PROFILE**` — capture the returned version number (a fresh account gets :1; do not hardcode :2) and attach it: `aws lambda update-function-configuration --function-name pest-detection-processor --layers **new-layer-arn** --profile **PROFILE**` Console: Lambda → Layers → Create layer; then function → Code → Layers → Add a layer.

## 2.11 API Gateway: HTTP API zwpcbivmsj

pest-monitoring-api-gateway, HTTP API v2, single auto-deploy \$default stage, base URL `https://zwpcbivmsj.execute-api.us-east-1.amazonaws.com` (no stage suffix). The production-account counterpart is `vzf17s6z00` (`https://vzf17s6z00.execute-api.us-east-1.amazonaws.com`), created by the ARGUS deployer 2026-08-10 with the same 21-route/JWT layout; the route ids below are the dev account's. CORS at the API level, from `lambda/cors.json`: `AllowOrigins *`, `AllowMethods GET, POST, DELETE, OPTIONS`, `AllowHeaders content-type, authorization`, `MaxAge 3600`. The authorization entry is what lets the browser send the Cognito JWT.

JWT authorizer `cognito-dashboard`, id `enxa26`: `IdentitySource $request.header.Authorization`, `Issuer https://cognito-idp.us-east-1.amazonaws.com/us-east-1_ea0a1dus1`, `Audience 4husu6afr835e235eu9dqp8av6`. Applied to every route since 2026-07-06 EXCEPT `GET /stream/status` (`AuthorizationType NONE`), which the Orin/mini-PC `kvs_controller` polls unauthenticated. That route returns only stream flags and writes nothing.

Two integrations, both `AWS_PROXY POST`, payload format 2.0, 30 s timeout: `t7ggvzn` → `pest-monitoring-api` (20 routes) and `33f69kt` → `kvs-hls-handler` (`GET /video-playback` only).

All 21 routes with live route ids: `GET /cost 1xu5zhs`; `DELETE /identities 1zwy2v1`; `DELETE /schedule 2dlsspt`; `POST /stream/start 33iv13c`; `GET /presigned-url 58gsnz6`; `POST /detection/verify 7wgkuzm`; `GET /video-playback ao22ksu` [→ `33f69kt`]; `GET /settings b3kl8si`; `GET /stream/status gcc7355` [NO AUTH]; `DELETE /detection glwyqo0`; `POST /stream/stop i8pc1lp`; `POST /settings jedpy5t`; `POST /model/stop krzufjf`; `GET /identities nqy131o`; `GET /schedule-logs odwxz00`; `POST /model/start t4w6czh`; `POST /schedule tos63cu`; `GET /history ttif38k`; `POST /identities w0ivphi`; `GET /schedule wqh3mvj`; `GET /model/status zbh91li`.

Lambda invoke permissions: `pest-monitoring-api` has a broad `SourceArn arn:aws:execute-api:us-east-1:366356442579:zwpcbivmsj/*`; `kvs-hls-handler` is route-scoped to `.../zwpcbivmsj/*/video-playback`.

Inspect: - Console: API Gateway → APIs → pest-monitoring-api-gateway → Routes / Authorization / Integrations / CORS. - CLI: `aws apigatewayv2 get-routes --api-id vzf17s6z00 --profile prod` (also `get-authorizers`, `get-integrations`, `get-api`).

Recreate: `aws apigatewayv2 create-api --name **NAME** --protocol-type HTTP --cors-configuration file:///lambda/cors.json --profile **PROFILE**` → `create-authorizer` (type JWT, issuer/audience from the NEW Cognito ids) → `two create-integration` → `21 create-route` calls (attach the authorizer to all but GET /stream/status) → `create-stage --stage-name '$default' --auto-deploy` → the two `aws lambda add-permission` calls with SourceArns containing the **new API id**. Then write the new base URL into `web/dashboard_v4/js/config.js`.

Emergency authorizer detach (if Cognito ever locks everyone out): `aws apigatewayv2 update-route --api-id vzf17s6z00 --route-id **ROUTE_ID** --authorization-type NONE --profile prod` per route; the prepared commands live in `docs/dashboard.md`.

Dead APIs, do not recreate: REST 3go4jj1698 (PestDetectionAPI, already deleted) and WebSocket j4v2m5cbte (retired when the dashboard moved to polling).

## 2.12 Cognito

User pool `pest-dashboard-users`, id `us-east-1_ea0aJdus1` (production account: pool `us-east-1_9selFDHpc`, client `6vebotf45bp8u46cnraddiaplv`, same admin-create-only design; first production sign-in user created and CONFIRMED 2026-08-11). Email as the username attribute, auto-verified email, MFA off, **AllowAdminCreateUserOnly=true** (nobody can self-register; every account is created by the operator), Cognito-default email sending (no SES dependency for auth mails), no hosted UI, no OAuth, no domain, no Lambda triggers, password policy minimum length 8 with lowercase + numbers required, account recovery via verified email.

App client `dashboard-web`, id `4husu6afr835e235eu9dqp8av6`: public SPA client with NO secret, auth flows `ALLOW_USER_PASSWORD_AUTH` + `ALLOW_REFRESH_TOKEN_AUTH`, id/access token validity 12 h, refresh 30 days, token revocation on. The dashboard signs in with direct `USER_PASSWORD_AUTH` against the client id; the pool id is not referenced by the frontend. The client id is a public identifier and safe to commit.

User management (admin-create-only means these are the only ways users exist): -

Console: Amazon Cognito → User pools → `pest-dashboard-users` → Users → Create user. -

CLI create: `aws cognito-idp admin-create-user --user-pool-id us-east-1_9selFDHpc --username **EMAIL** --temporary-password **TEMP_PASSWORD** --profile prod` - CLI set permanent password: `aws cognito-idp admin-set-user-password --user-pool-id us-east-1_9selFDHpc --username **EMAIL** --password **NEW_PASSWORD** --permanent --profile prod` - CLI delete: `aws cognito-idp admin-delete-user --user-pool-id us-east-1_9selFDHpc --username **EMAIL** --profile prod`

Recreate: `aws cognito-idp create-user-pool --pool-name pest-dashboard-users --username-attributes email --auto-verified-attributes email --admin-create-user-config AllowAdminCreateUserOnly=true --policies "PasswordPolicy={MinimumLength=8,RequireLowercase=true,RequireNumbers=true,RequireUppercase=false,RequireSymbols=false}" --profile **PROFILE**`, then `aws cognito-idp create-user-pool-client --user-pool-id **NEW_POOL_ID** --client-name dashboard-web --no-generate-secret --explicit-auth-flows ALLOW_USER_PASSWORD_AUTH ALLOW_REFRESH_TOKEN_AUTH --token-validity-units "IdToken=hours,AccessToken=hours,RefreshToken=days" --id-token-validity 12 --access-token-validity 12 --refresh-token-validity 30 --profile **PROFILE**`. Feed the new ids into the API Gateway authorizer and config.js (client id + region only). Seed zero users; the operator admin-creates their own. Console: Amazon Cognito → Create user pool.

## 2.13 CloudFront: E1423RGLAXWNSI

Distribution E1423RGLAXWNSI → <https://d1twcdquexdgj8.cloudfront.net>. This is the development-account dashboard URL. The production distribution is E1YADURLSAVNFA → <https://d1dtoxef7qmugl1.cloudfront.net> (Deployed and serving the ARGUS dashboard since 2026-08-11; same design as below with the argus-dashboard-506868652945 website origin).

Configuration: - One origin, id s3-website-pest-dashboard: the S3 STATIC WEBSITE endpoint pest-dashboard-366356442579.s3-website-us-east-1.amazonaws.com with CustomOriginConfig OriginProtocolPolicy=http-only. Both choices are forced: only the website endpoint honours the index/error documents (the SPA fallback), and website endpoints have no HTTPS, so an https origin policy returns 502. No OAC/OAI (the bucket is public-read). - Default behavior: ViewerProtocolPolicy redirect-to-https, methods HEAD+GET only, Compress on, CachePolicyId 4135ea2d-6df8-44a3-9df3-4b5a84be39ad = the AWS-managed **Managed-CachingDisabled** policy (the id is the same in every account; hardcode it). - CachingDisabled rationale: every request passes through to S3, so a plain `aws s3 sync` is a complete redeploy with no invalidation step and no cache-propagation wait. At this project's traffic the lost caching is irrelevant; the removed operational failure mode (users seeing a stale build after a deploy) is not. Browser-side caching is separately defeated by the --cache-control "no-cache, must-revalidate" flag on the sync (2.3.3). - DefaultRootObject index.html, default CloudFront certificate, no aliases, no WAF, no logging, no custom error responses (the S3 website ErrorDocument does the SPA fallback).

Inspect: Console CloudFront → Distributions → E1YADURLSAVNFA; CLI `aws cloudfront get-distribution-config --id E1YADURLSAVNFA --profile prod`.

Recreate: `aws cloudfront create-distribution --distribution-config file://**config.json** --profile **PROFILE**` using `deployer/audit/cloudfront__E1YADURLSAVNFA_distribution-config.json` as the template, with a FRESH CallerReference (CloudFront rejects duplicates; live was pest-dashboard-cf-2026-07-06) and the new website-endpoint origin. Deployment takes

minutes to reach Deployed; capture the new id + \*.cloudfront.net domain and add the domain to the frames-bucket CORS origins and to wherever the dashboard URL is published.

## 2.14 Kinesis Video Streams

KVS exists only on the development account: the 2026-08-10 production deploy ran without --live-view, so the production account has NO streams, worm\_cam.kvs\_stream\_name is empty there, and the dashboard Live tab and the devices' kvs\_controller cannot work against production until streams are created (whether the demo needs them is the project owner's call). Two dev-account streams, both DataRetentionInHours=2, AWS-managed KMS key alias/aws/kinesisvideo (no CMK): - armyworm-cam-stream (ARN arn:aws:kinesisvideo:us-east-1:366356442579:stream/armyworm-cam-stream/1779436101323) — live; fed by the KVS Producer SDK compiled for ARM64 on the Jetson Orin, GStreamer passthrough pipeline rtspsrc → rtph264depay → h264parse → kvssink off the SIYI A8 Mini RTSP feed. - moth-cam-stream — parked (the Hikvision camera was repurposed). - FYP-PROJECT is Wilbur's Gen-1 stream (168 h retention): dead, do not recreate.

Control flow: dashboard → POST /stream/start toggles stream\_enabled in DynamoDB → the Orin kvs\_controller polls GET /stream/status (the unauthenticated route) and starts/stops the producer → the dashboard Live tab calls GET /video-playback → kvs-hls-handler mints an HLS URL.

Inspect: Console Kinesis Video Streams → Streams; CLI `aws kinesisvideo list-streams --profile prod` and `aws kinesisvideo describe-stream --stream-name armyworm-cam-stream --profile prod`. Recreate: `aws kinesisvideo create-stream --stream-name **STREAM_NAME** --data-retention-in-hours 2 --profile **PROFILE**`. Stream names carry no account id. The HLS data endpoint is per-account per-stream and is always resolved at runtime; never write it into config.

## 2.15 EventBridge schedules

Two mechanisms coexist; do not confuse them.

1. EventBridge SCHEDULER (the newer service) — one production schedule, pest-model-watchdog-15min (group default): rate(15 minutes), timezone Asia/Singapore (cosmetic for rate()), State ENABLED, FlexibleTimeWindow OFF, MaximumRetryAttempts 0, target = the watchdog Lambda ARN with no Input. Scheduler invokes Lambda by ASSUMING a role (live: the console-generated Amazon\_EventBridge\_Scheduler\_LAMBDA\_d585792c00 with `lambda:InvokeFunction` on the watchdog), not via a Lambda resource policy.
  - Inspect: Console → Amazon EventBridge → Scheduler → Schedules; CLI `aws scheduler get-schedule --name pest-model-watchdog-15min --profile prod`.



- Disable temporarily (long test session): Console → the schedule → Disable, or  
`aws scheduler update-schedule --name pest-model-watchdog-15min --schedule-expression "rate(15 minutes)" --flexible-time-window Mode=OFF --target file://**target.json** --state DISABLED --profile prod` (update-schedule requires re-passing the full target; read it with get-schedule first).
  - Recreate: create one stably named invocation role (trust `scheduler.amazonaws.com` with condition `aws:SourceAccount = ACCOUNT_ID`; policy `lambda:InvokeFunction` on BOTH the watchdog and pest-camera-scheduler), then `aws scheduler create-schedule --name pest-model-watchdog-15min --schedule-expression "rate(15 minutes)" --flexible-time-window Mode=OFF --target '{"Arn": "**WATCHDOG_ARN**", "RoleArn": "**INVOCATION_ROLE_ARN**", "RetryPolicy": {"MaximumEventAgeInSeconds": 86400, "MaximumRetryAttempts": 0}}' --profile **PROFILE**`
2. Classic EventBridge RULES — created at RUNTIME by pest-monitoring-api for per-camera model schedules: pest-sched-{camera\_id}-start/stop, cron in UTC, target pest-camera-scheduler with an Input payload, invoke permission granted per rule on the function's resource policy (2.6). Nothing to pre-create beyond the api role's events/lambda permissions.

Gen-1 leftovers in the live account (`model-start-schedule`, `model-stop-schedule`, `frame-extraction-schedule`, all still ENABLED and firing daily against dead Lambdas = wasted spend; `ExtractFrameEveryMinute` DISABLED): do not recreate; disabling them in the source account is recommended.

## 2.16 SES

The alert channel. BOTH accounts are in the SES SANDBOX (ProductionAccessEnabled=false, 200 messages/24 h, 1/sec): they can only send FROM and TO verified identities. That is fine here because sender and default recipient are the same verified address, `rex2956550768@gmail.com` (both the processor's `SENDER_EMAIL` env var and `system-config recipient_email`). The address was verified on the production account 2026-08-11 (until the verification link is clicked, `SendingEnabled=false` and no alert mail goes anywhere — the exact state the production account sat in for a day). No SNS, no SQS, no configuration sets. Legacy identities `teowilbur@gmail.com` (Wilbur, dead) and `neobkee@gmail.com` (unrelated) also exist on the development account; ignore them.

Adding a recipient: dashboard → POST /identities, or Console SES → Identities → Create identity → Email address, or CLI `aws ses verify-email-identity --email-address **EMAIL** --profile prod`. The person must click the verification link, or sandbox SES silently refuses delivery; the processor logs “recipient may be unverified” on send failure.

Fresh account: verify sender + recipient (`verify-email-identity` each, poll `aws ses get-identity-verification-attributes --identities **EMAIL** --profile **PROFILE**` until Success), set the processor env var and the system-config row. Leaving the sandbox

is a manual production-access request to AWS Support (about 1-2 business days); at project volume the sandbox is sufficient.

## 2.17 IAM roles and policies

All five Lambda execution roles share the same trust policy (Principal `lambda.amazonaws.com`, `sts:AssumeRole`; `deployer/audit/iam__trust-policy-lambda.json`) and all carry `AWSLambdaBasicExecutionRole` for CloudWatch logs. Raw policy JSONs live in `deployer/audit/iam__*.json`; the checked-in sources for two of them are `lambda/ddb-policy.json` and `lambda/bedrock-policy.json`.

Staleness caveat on the audit JSONs: most are 2026-07-08 snapshots. One known gap remains: `iam__pest-detection-processor-role__inline__pest-detection-processor-policy.json` contains only `rekognition:DetectCustomLabels`; the `rekognition:DetectLabels` action was added 2026-07-13 for v4.3 suppression. Two later processor-role policies ARE now dumped fresh (captured from the live development-account role 2026-08-11): `iam__pest-detection-processor-role__inline__bedrock-verify.json` (wildcarded inference-profile families including `us.anthropic.*`, so Sonnet 4.6 is covered — use this, not the narrower `lambda/bedrock-policy.json`) and `iam__pest-detection-processor-role__inline__s3-frames-write.json` (`s3:PutObject` on `frames/*` for the EXIF write-back). The same 2026-08-11 pass wired both into the ARGUS deployer’s role build (account/region rewritten at deploy). Note the production role’s provenance: it was deployed 2026-08-10, BEFORE that deployer fix, so it came up without `bedrock-verify`, `s3-frames-write`, or `rekognition:DetectLabels`; it was patched 2026-08-12 by `migration/fix_processor_iam.py` after the first validation run exposed the gap (see 9.3.14), and now matches the development role action-for-action. Recreating the processor role verbatim from the 2026-07-08 JSON alone still breaks v4.3 suppression; add `DetectLabels` and the two dumped policies. The table below describes the LIVE state.

| Role                          | Managed policies             | Inline / customer-managed policies (content)                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pest-detection-processor-role | AWSLambdaBasicExecution Role | pest-detection-processor-policy: <code>s3:GetObject</code> on <code>frames/*</code> , <code>s3:Get+PutObject</code> on <code>processed/*</code> , DDB <code>PutItem/GetItem/UpdateItem/Query</code> on <code>detections + index</code> , <code>GetItem</code> on <code>cameras</code> , <code>rekognition:DetectCustomLabels + rekognition:DetectLabels</code> (v4.3 add, 2026-07-13; absent from the 2026-07-08 audit JSON) on <code>*</code> , |



| Role                     | Managed policies                                                           | Inline / customer-managed policies (content)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                          |                                                                            | <p>ses:SendEmail/SendRawEmail (the live policy also grants ses:ListVerifiedEmailAddresses). read-system-config: DDB GetItem on system-config. bedrock-verify (dump: deployer/audit/iam__pest-detection-processor-role__inline__bedrock-verify.json):</p> <p>bedrock:InvokeModel + bedrock:Converse on wildcarded inference-profile families (us.anthropic.*, global.anthropic.*, us.amazon.*, us.meta.*, us.mistral.*) AND on the region-wildcarded foundation-model ARNs (arn:aws:bedrock:*::foundation-model/... — cross-region inference routes to several regions, so the foundation-model statement must be region-wildcarded). s3-frames-write (dump: deployer/audit/iam__pest-detection-processor-role__inline__s3-frames-write.json): s3:PutObject on frames/* for the v4.8.3 EXIF write-back</p> |
| pest-monitoring-api-role | AmazonSESFullAccess, AWSLambdaBasicExecutionRole, AWSBillingReadOnlyAccess | <p>pest-monitoring-api-policy: S3 Get/Put/List/Delete on both armyworm buckets (DeleteObject scoped to frames/*); dynamodb:* on cameras + detections + index; rekognition</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

| Role                       | Managed policies                                                 | Inline / customer-managed policies (content)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                            |                                                                  | Describe/Start/StopProjectVersion; events<br>PutRule/DeleteRule/PutTargets/RemoveTargets/ListTargetsByRule/DescribeRule<br>scoped to rule/pest-sched-*; scheduler CRUD<br>scoped to<br>schedule/default/pest-camera-*; lambda<br>Add/RemovePermission/InvokeFunction on pest-camera-scheduler;<br>iam:PassRole conditioned to scheduler.amazonaws.com;<br>kinesisvideo read. pest-monitoring-ddb-full-access (mirror lambda/ddb-policy.json): six DDB verbs on all four pest tables + the detections index. read-processed-armyworm:<br>s3:GetObject on processed/* |
| pest-camera-scheduler-role | AWSLambdaBasicExecutionRole                                      | pest-camera-scheduler-policy: rekognition<br>Start/Stop/DescribeProjectVersions; DDB<br>GetItem/UpdateItem on cameras                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| kvs-hls-handler-role       | AWSLambdaBasicExecutionRole                                      | customer-managed kvs-hls-handler-rolePolicy:<br>kinesisvideo:GetDataEndpoint +<br>GetHLSStreamingSessionURL only                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| pest-model-watchdog-role   | AWSLambdaBasicExecutionRole (generated scoped-logs variant live) | Rekognition-DB-handler:<br>DDB Scan/UpdateItem on cameras; rekognition<br>DescribeProjects/DescribeProjectVersions/StopProjectV                                                                                                                                                                                                                                                                                                                                                                                                                                     |

| Role | Managed policies | Inline / customer-managed policies (content)<br>ersion |
|------|------------------|--------------------------------------------------------|
|------|------------------|--------------------------------------------------------|

Plus the EventBridge Scheduler invocation role (2.15). Known deviations, recorded in `deployer/STACK_MANIFEST.md`: the `api-policy`'s `iam:PassRole` points at a role name that does not exist live (latent and harmless — per-camera schedules use classic rules with the Lambda resource policy, not `PassRole`); the `api-policy` still grants `InvokeFunction` on the dead Gen-1 `kvs-hls-url-generator` (drop on recreate); `dynamodb:*` and `AmazonSESEFullAccess` are broader than needed but functional.

Apply an inline policy (the pattern used for every one of these): - Console: IAM → Roles → role name → Add permissions → Create inline policy → JSON tab → paste → name → Create. - CLI: `aws iam put-role-policy --role-name pest-detection-processor-role --policy-name bedrock-verify --policy-document file://lambda/bedrock-policy.json --profile prod`

Bedrock model access is separate from IAM: the model must ALSO be enabled under Bedrock → Model access in the console for the region (Modify model access → tick the model → Save), or every call is `AccessDeniedException` even with perfect IAM. Claude Haiku 4.5 was confirmed `AUTHORIZED` on the development account 2026-07-22 after the Anthropic use-case form (Sonnet 4.6 followed). On the production account both Sonnet 4.6 and Haiku 4.5 were found already `AUTHORIZED` at the 2026-08-10 audit — no use-case-form wait. Budget for that approval lag on any other fresh account; the LLM gate is the pipeline's spine.

## 2.18 Rekognition Custom Labels (backend view)

The models themselves are the subject of the detection chapter; the backend facts: - Every camera row stores exactly one `custom_model_arn`; all Lambdas resolve it at call time. Model switch or rollback = one DynamoDB write of the ARN (2.4.1 quoting gotcha). - The endpoint must be `RUNNING` to serve `detect_custom_labels`. Start/stop via the dashboard, the schedule pair, or CLI: `aws rekognition start-project-version --project-version-arn **MODEL_ARN** --min-inference-units 1 --profile prod / aws rekognition stop-project-version --project-version-arn **MODEL_ARN** --profile prod`. Status: `aws rekognition describe-project-versions --project-arn **PROJECT_ARN** --profile prod`. Console: Rekognition → Custom Labels → Projects. - Running endpoints bill per hour, and TWO can bill at once (armyworm + moth). The watchdog (2.9) is the safety net; the armyworm endpoint being `STOPPED` outside test windows is the watchdog working, not a fault. - Live on `worm_cam` (dev account) since 2026-08-07: the v9 retrain v9-20260805-0713, `arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection-v9/version/v9-20260805-0713/1785913987295`, F1 0.599. The v9 family's training recipe adds data augmentation (flips, rotations, exposure jitter — the documented 13x build). The low F1 is by design in the current architecture: the detector is a high-recall front end gathering boxes down to

TILE\_MIN\_CONFIDENCE=8, and the LLM denoiser plus post-gate cleanup do the rejecting. Do not read 0.599 as pipeline accuracy. Fallback ARNs (v5 F1 0.852 etc.) are listed in docs/aws.md. - Models are ACCOUNT-BOUND. No export, import, copy, or cross-account API exists — a fresh account must create a project, load labeled data, and train-project-version (hours, billable), then write the resulting version ARN into the camera row. This was executed for real on the production account (2026-08-10/11): the training set (36,641 objects, 4.27 GB under training-data/v9/) was copied server-side by migration/copy\_training\_data.py via a temporary prefix-scoped cross-account bucket policy (revoke with --revoke after sign-off), every manifest source-ref repointed at argus-frames-506868652945, and armyworm version v9r-prod-20260810 submitted in project argus-detection. The moth detector was fully rebuilt from data alone (migration/migrate\_moth.py): Wilbur's SmartPestProject model could not move, but its 116 TRAIN + 29 TEST labelled images survived in the old account's Rekognition console bucket; the rebuild moth-prod-20260811 in project argus-moth-detection trained to F1 0.991 (original: 0.988 — same data, treat as equivalent). - THE MANIFEST TRAP (found 2026-08-10; generic lesson worth the box it is printed in): boxes drawn or corrected in the Rekognition console live in the project's DATASET, not in the S3 manifest the dataset was originally created from. After any console labelling, the manifest is stale; rebuilding a dataset from it silently loses the console work. Export the labelled truth with ListDatasetEntries (scripted in migration/train\_v9r\_on\_prod.py), repoint the source-refs, and create the new dataset from that export. **The dataset — not the S3 manifest — is the source of truth.**

## 2.19 Operations / reproduction

Daily operations quick list: - Watch the pipeline: CloudWatch → Log groups → /aws/lambda/pest-detection-processor (also pest-monitoring-api, pest-model-watchdog, kvs-hls-handler). CLI: `aws logs tail /aws/lambda/pest-detection-processor --follow --profile prod`. Log groups auto-create on first invoke; retention is Never-Expire; no alarms exist. - Check endpoint state before touching it: dashboard GET /model/status, or the describe CLI in 2.18. - Push a test image through the full pipeline: dashboard Settings → Test upload. This is also how live detection is demonstrated (there are no worms at Jewel to stage). The upload key can carry the \_\_conf<N> and \_\_llm-<alias> overrides (2.5.1). - Toggle the LLM gate per camera: flip llm\_verify\_enabled via boto3, or POST /settings with body {"camera\_id": "worm\_cam", "fields": {"llm\_verify\_enabled": true}}. The dashboard's threshold control edits post\_verify\_floor (never min\_confidence), and its model picker edits llm\_model\_id (2.6). - Redeploy the dashboard: the s3 sync command in 2.3.3. Redeploy a Lambda: the update-function-code command in 2.5.10. - After a training run, push the holdout through the live pipeline so results appear on the dashboard (project standing order), and check which endpoints are RUNNING before switching models.

Full reproduction on a fresh account (condensed; deployer/STACK\_MANIFEST.md is the recreate-level bill of materials and the ARGUS deployer deployer/deploy.py automates this order — proven for real 2026-08-10 on account 506868652945: all 15 stages in 103

seconds, post-deploy verification passed, including the tuned processor sizing/env and camera-row seeds): 1. IAM: five execution roles + one scheduler invocation role from the audit policy JSONs, account id and bucket names parameterized. Wait out IAM eventual consistency before attaching anything to them. 2. DynamoDB: four tables (detections with the by-pest-time GSI). 3. S3: the buckets (us-east-1: no LocationConstraint). Dashboard bucket order: PAB false → website → policy. Defer the frames notification. 4. Lambda layer: build + publish fyp-pillow (2.10); capture the version. 5. Lambda functions: all five (rename mirrors to `lambda_function.py` inside the zips; `kvs-hls-handler` comes from `deployer/audit/kvs-hls-handler_src/`; the watchdog MUST come from the repo mirror `lambda/pest-model-watchdog.py` — the audit \*\_src copy is pre-v6.2 and lacks the per-camera `max_runtime_min`, a real deployer bug fixed 2026-08-11). Attach fyp-pillow to the processor. Two-phase env: create pest-camera-scheduler first, then set `SCHEDULE_EXECUTOR_ARN` on the api. 6. Frames-bucket notification: `lambda add-permission FIRST`, then `put-bucket-notification-configuration` with prefix `frames/` (2.3.1). 7. Cognito: pool, then client; capture both ids (2.12). 8. API Gateway: `api` → `authorizer` → two integrations → 21 routes → `$default` stage → two invoke permissions (2.11). 9. CloudFront: after the dashboard bucket is public and website-enabled (2.13). 10. Config write-back: new API base URL + Cognito region + client id into `web/dashboard_v4/js/config.js`; new CloudFront domain into the frames CORS; sync the dashboard. 11. KVS streams (2.14) and the EventBridge Scheduler watchdog schedule (2.15). 12. Rekognition: create the project, train, write the version ARN into the camera row (2.18). 13. Seed DynamoDB: the system-config row and the camera rows (model ARNs empty until step 12 resolves them). 14. SES: verify sender + recipient (2.16).

Production migration status (2026-08-11) — what remains before the production account fully replaces the development one: - Armyworm model `v9r-prod-20260810`: training DONE 2026-08-12 (F1 0.613) and the version ARN is wired into the camera rows.

Remaining: repeat the holdout validation re-push through the live pipeline and compare against the development account's equivalent dashboard zones — the first 2026-08-12 attempt was invalidated by the processor-role IAM gap (see 9.3.14) and must be run again. The migration is only proven when the same images give the same detections. At sign-off, restore `worm_cam.max_runtime_min` from the temporary 240 back to 45. - Repoint the edge devices (Orin / mini-PC): upload bucket → `argus-frames-506868652945`, API base → the new API Gateway URL, and the production camera id. - Revoke the temporary cross-account S3 read grants (`migration/copy_training_data.py --revoke`, `migration/migrate_moth.py --revoke`, and the Jewel-frames grant) after model sign-off. - KVS/live view: not migrated; create streams only if the demo needs the Live tab (2.14). - The development account stays untouched until the production one is proven end to end; its dashboard zones also hold the threshold-study evidence the final report cites. Historical note for the handover: 36 Jewel on-site records (frames + rows, zones `zone1/zone2/zone3/field_worm`, 2026-07-29..31) were migrated to production by `migration/migrate_jewel_records.py` with S3 keys byte-identical and `model_arn` left pointing at the old account's model on purpose — it is provenance, and rewriting it would falsify history.

Security cautions for reproducers: - No AWS secret access keys, passwords, or tokens appear in the committed cloud files or in this manual, and none may ever be added. Credentials belong in the CLI profile (`aws configure --profile **PROFILE_NAME**`), never in code, docs, or chat. - One known exception exists elsewhere in the repo: an old script at `datasets/archive/experiments/pre_v3_abandoned/download.py` contains an inline Roboflow API key (credential stored there, not reproduced here). Do not copy that pattern, and do not redistribute that file. - Non-secret identifiers (account id, ARNs, bucket names, API/CloudFront/Cognito ids, IPs) are intentionally printed throughout this chapter; they are safe to share.

## 2.20 Cross-references

- Chapter 1 — system overview: where this backend sits in the full capture-to-dashboard chain.
- Chapter 4 (dashboard frontend) — the frontend that consumes `pest-monitoring-api`: module layout of `web/dashboard_v4/`, Cognito login flow, the Test upload panel, gallery delete.
- Chapter 3 (models and training) — Rekognition Custom Labels training history (v1-v9), the negatives limitation, and the evidence base behind tiling and the LLM gates (`docs/detection.md`).
- Chapter 5 (edge: Go2/Orin) — Jetson Orin, SIYI A8 Mini, the KVS producer, the `kvs_controller` polling loop, and the patrol completion gate that polls `pest-monitoring-detections`. (The mini-PC moth node is Chapter 6.)
- Chapter 7 (deployer, ARGUS) — `deployer/STACK_MANIFEST.md`, `deploy.py`, and the rehearsal plan that automates section 2.19 end to end.

## Chapter 3 — Detection models and the training pipeline

This chapter covers the two AWS Rekognition Custom Labels models (moth and armyworm), the armyworm version ladder, the CAG holdout discipline, every dataset-tooling script, the manifest format, the full train / evaluate / wire / rollback procedure, and the August 2026 retraining of both models on the NP production account. *As of 2026-08-14.* (Facts with a different date are marked with that date.)

### 3.1 Role in the system

ARGUS detects pests with AWS Rekognition Custom Labels, a managed object-detection service. We do not train neural networks ourselves. We prepare labeled images, hand Rekognition a manifest, and it trains and hosts a model. The pest-detection-processor Lambda (Chapter 2) calls that hosted model on every frame that lands in S3, whether the frame came from the SIYI A8 Mini on the Unitree Go2, from a scheduled camera, or from the dashboard's Test upload panel.

Two models exist. The moth model was inherited from the predecessor project and is stable. The armyworm model is this project's own work and went through fourteen training iterations (v0 through the v9 retrain in the ladder below). On this platform the model is a black box: the only levers are the training data, the split, and the class list, so every improvement attempt was a data experiment. This chapter records what each iteration was, what it was measured on, and what the measurements actually mean. The short version: the model is strong inside its training domain; the long-standing gap to Jewel Changi imagery was narrowed in the v9 era by a role change (the detector became a high-recall front end with an LLM verification gate behind it) plus added data augmentation, and the residual limit is precision on field scenes, not recall.

The system now runs on the NP production account **506868652945** (IAM user Student\_QianRunzhe, CLI profile prod), region **us-east-1**. The stack was deployed there 2026-08-10 by the ARGUS deployer (Chapter 7). The system was built and validated in the development account **366356442579** (CLI profile nbk2); that account is retired history, and every dev-account name or ARN in this chapter is kept only as the record of how each iteration was built. Models are account-bound: they cannot be exported or migrated, so both detectors were retrained on the production account from migrated data — the moth model as `argus-moth-detection / moth-prod-20260811` (3.4), the armyworm model as `argus-detection / v9r-prod-20260810` (Rekognition test-split F1 0.613), both wired into the production camera rows and verified live end to end 2026-08-12 (3.17). Copying an ARN across accounts is never an option. The handover snapshot `argus-repo-snapshot-20260813.zip` was published 2026-08-13.

## 3.2 Inventory

The two (prod) rows are the operative models. Every row naming account 366356442579 or a nbk2-era project is development history on the retired dev account, kept for the iteration record only.

| Component                                             | Location                                                                                                         | Purpose                                                                                                                                                                                                                                        |
|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Moth model (prod, operative) moth-prod-20260811       | arn:aws:rekognition:us-east-1:506868652945:project/argus-moth-detection/version/moth-prod-20260811/1786412382515 | Rebuilt on the production account from the recovered moth training set, F1 0.991 (3.4). Wired into the production moth_cam row.                                                                                                                |
| Armyworm model (prod, operative) v9r-prod-20260810    | arn:aws:rekognition:us-east-1:506868652945:project/argus-detection/version/v9r-prod-20260810/1786377372187       | The v9-retrain recipe retrained on the production account. Trained (Rekognition test-split F1 0.613) and wired into the production worm_cam + manual_upload rows; verified live 2026-08-12 (3.17).                                             |
| Moth model (dev, history) SmartPestProject.2026-02-15 | Rekognition, retired dev account 366356442579                                                                    | Adult moth detection, label Moths, F1 0.988. Inherited from Wilbur Teo.                                                                                                                                                                        |
| Armyworm v9 retrain (dev, history)                    | arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection-v9/version/v9-20260805-0713/1785913987295  | The settled model on the dev account's worm_cam from 2026-08-07 until the migration, label armyworm-larva. F1 0.599 — a front-end figure, not pipeline accuracy (3.5). Its recipe and data are what v9r-prod-20260810 retrained on production. |
| Armyworm v5                                           | arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection/version/v5-2026-07-07/1783394123547        | Production model 2026-07-07 → 2026-07-29, F1 0.852 in-domain (health warning in 3.5). Endpoint STOPPED.                                                                                                                                        |
| Armyworm v4 (rollback)                                | ...project/armyworm-detection/version/armyworm-detection.2026-05-                                                | Older rollback target, F1 0.719.                                                                                                                                                                                                               |



| Component                        | Location                                                                                                                                                         | Purpose                                                                          |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
|                                  | 21T12.46.19/1779338780450                                                                                                                                        |                                                                                  |
| Armyworm v6 (rejected)           | ...project/armyworm-detection/version/v6-2026-07-14/1784011732429                                                                                                | Kept for the record only. Endpoint stopped.                                      |
| Armyworm v7.1 (rejected)         | project armyworm-detection-v7, version v7-1-20260720-0604                                                                                                        | Light-worm experiment. Rejected.                                                 |
| Armyworm v7.2 (rejected)         | project armyworm-detection-v7-2, version v7-2-20260721-0301                                                                                                      | Black-set experiment. Rejected.                                                  |
| Armyworm v8, v9 first cut        | projects armyworm-detection-v8, armyworm-detection-v9                                                                                                            | The augmentation era before the v9 retrain (3.5). Superseded; endpoints stopped. |
| CAG holdout                      | datasets/holdout/cag/(batch_1/, batch_2/)                                                                                                                        | The only real CAG-domain evaluation signal. batch_2 is never trained on.         |
| Training sources                 | datasets/sources/corn_leaf/,<br>datasets/sources/moth_z1dog/, plus the vendor-licensed offline image set obtained from the project owner (kept outside the repo) | Raw labeled image supply.                                                        |
| convert_to_manifest.py           | datasets/current/                                                                                                                                                | YOLO (Roboflow maize set) to Ground Truth manifest.                              |
| convert_other_caterpillar_new.py | datasets/current/                                                                                                                                                | YOLO (the vendor-licensed offline set) to manifest, relabeled.                   |
| merge_manifests.py               | datasets/current/                                                                                                                                                | Concatenates per-source manifests into the v5 combined set.                      |
| upload_images.py                 | datasets/current/                                                                                                                                                | Uploads only manifest-referenced images to S3, idempotent.                       |
| rank_purchased.py                | datasets/current/                                                                                                                                                | Quality-ranks the purchased set, writes purchased_ranked.csv.                    |
| build_v7_2.py                    | datasets/current/                                                                                                                                                | Assembles the v7.2 set with                                                      |

| Component                                         | Location                                      | Purpose                                                                                                                               |
|---------------------------------------------------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
|                                                   |                                               | a strict leak-proof 80:20 split.                                                                                                      |
| build_v7_4.py                                     | datasets/current/                             | Assembles the fair v7.4 composition (the v8/v9 base) with the batch_2 MD5 abort guard (3.11).                                         |
| augment_build_v8.py                               | datasets/current/                             | The 13x TRAIN augmentation build (v8): exact geometric box transforms + photometric jitter + arbitrary rotations, self-tested (3.11). |
| build_v9_91.py                                    | datasets/current/                             | Re-splits the v7.4 pool 90:10 by unique source stem and augments BOTH splits 13x (v9) (3.11).                                         |
| build_answer_key.py                               | datasets/current/                             | Renders the adjudicated ground-truth answer key + confirmation sheets for the 2026-08-11 controlled evaluation (3.12).                |
| train_v7_2.py                                     | datasets/current/                             | boto3 orchestrator: project + datasets + training + watcher.                                                                          |
| evaluate_v7_1_vs_v5.py,<br>evaluate_v7_2_vs_v5.py | datasets/current/                             | Whole-image CAG-holdout arbitration between two models.                                                                               |
| evaluate_batch2_v5_vs_v4.py                       | datasets/archive/experiments/one_off_scripts/ | The original batch_2 arbitration (archived; points at pre-reorg paths).                                                               |
| migration/copy_training_data.py                   | migration/                                    | Server-side cross-account copy of the training data + manifest repoint (3.17).                                                        |
| migration/train_v9r_on_prod.py                    | migration/                                    | Retrains the armyworm model on the production account from the live dataset entries (3.17).                                           |
| migration/poll_training.py                        | migration/                                    | Network-fault-tolerant training watcher (3.17).                                                                                       |
| migration/migrate_moth.py                         | migration/                                    | Rebuilds the moth detector on the production account                                                                                  |

| Component           | Location                                                                                                                                    | Purpose                                                     |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| Manifest backups    | datasets/current/manifests_v5_* through manifests_v9/                                                                                       | (3.4).<br>Frozen manifest sets per version.                 |
| Model wiring        | DynamoDB pest-monitoring-cameras, attribute custom_model_arn                                                                                | The single runtime pointer to the live model (per account). |
| Training data on S3 | s3://argus-frames-506868652945/training-data/... (production, operative); s3://frames-armyworm-366356442579/training-data/... (retired dev) | Images + manifests<br>Rekognition trains from.              |

### 3.3 Rekognition Custom Labels primer

Skip this section if you already run Custom Labels. Everything later depends on these facts.

**Object model.** A **project** is a named container. Inside it live **datasets** (one TRAIN, one TEST) and **project versions**. A project version is one trained model: immutable, with its own ARN, its own F1 score, and its own hosting endpoint. “Retraining” always means creating a new version. Old versions stay and can be re-hosted at any time, so rollback is trivial.

**Lifecycle.** create-project-version starts training (TRAINING\_IN\_PROGRESS → TRAINING\_COMPLETED, typically 1–4 hours here). A completed model serves nothing until you host it: start-project-version (STARTING → RUNNING, 10–20 minutes). Only a RUNNING model answers detect-custom-labels. stop-project-version shuts the endpoint down.

**Cost.** Hosting bills about **\$4/hour per inference unit while RUNNING**, whether or not you call it. Training bills separately by BillableTrainingTimeInSeconds. Standing discipline in this project: start, evaluate, STOP, and verify the stop. Note two endpoints bill at once when both the moth model and an armyworm model are up.

**The ARN colon rule.** A version ARN looks like: arn:aws:rekognition:us-east-1:506868652945:project/argus-detection/version/v9r-prod-20260810/1786377372187 It contains colons. A raw aws dynamodb update-item typed in PowerShell mangles the quoting around it. When writing an ARN into DynamoDB, use boto3 (see datasets/archive/experiments/one\_off\_scripts/migrate\_camera\_ids.py for the style; the 2026-07-20 datasets reorg moved it there) or pass the item as a file:// JSON document. Never inline the ARN in a PowerShell command string.

**Account-bound.** Models cannot leave the account they were trained in. This stopped being hypothetical in August 2026: standing the system up on the NP production account required retraining both models there and updating every hardcoded account id, bucket name, and ARN. 3.17 records how it was actually done.

**One structural limit that shapes the whole system.** Custom Labels object detection cannot train on negative (zero-box) images. Every training image must carry at least one bounding box. The consequences are in 3.16.

**Console entry point:** AWS console → Amazon Rekognition → Use Custom Labels (left menu) → Projects. **CLI:** all commands are `aws rekognition ...` with `--profile prod --region us-east-1` on the NP production account. (The retired development account used `--profile nbk2`; that profile appears below only in historical records and in scripts that were run against the dev account.)

### 3.4 The moth model (inherited)

SmartPestProject.2026-02-15. **F1 0.988**, single label **Moths**. This model is the work of the predecessor FYP student, **Wilbur Teo**, whose moth-detection v1 this project inherited and extended. It is native in the development account (366356442579). No retraining was done on it during the main project; the production-account rebuild below recreated it from its own data.

The `moth_cam` camera row is served today by the production rebuild `moth-prod-20260811` (below); on the retired dev account the inherited model played the same role. Either way the model's ARN sits in that row's `custom_model1_arn` like any other model (3.15). It participates in the same processor pipeline, except that the LLM crop-verification gate never runs for it: the verify prompt asks about larvae and `moth_cam` targets adult moths, so `llm_verify_enabled` is false on that camera (Chapter 2).

Nothing in the rest of this chapter's ladder or tooling applies to the moth model. It is a finished, stable, inherited asset.

**Production-account rebuild (2026-08-11).** The model itself is account-bound and could not follow the migration, but its data survived intact: **116 TRAIN + 29 TEST labelled images** (537 + 127 boxes) in the old account's Rekognition console bucket `custom-labels-console-us-east-1-d1abc2aed2.migration/migrate_moth.py` recovered them end to end: merge a scoped cross-account read grant into the console bucket's policy, copy the images server-side into `argus-frames-506868652945/training-data/moth/`, export the labelled dataset entries with `ListDatasetEntries` (the dataset, not any S3 manifest, is the source of truth for labels — the same lesson as 3.17), repoint every source-ref, then create project **argus-moth-detection**, both datasets (TRAIN 116/116, TEST 29/29, `CREATE_COMPLETE`), and version **moth-prod-20260811**. It trained the same day at **F1 0.991** — Wilbur's original scored 0.988 on the same data, so treat the two as equivalent, not as an improvement. The new ARN is wired into the production `moth_cam` row, and **`llm_verify_enabled` is false on that row on purpose**: the verify prompt targets larvae and

this camera's target is adult moths. Revoke the console-bucket read grant with python migration/migrate\_moth.py --revoke once the migration is signed off.

### 3.5 The armyworm model ladder

Read the health warning after the table before quoting any number from it.

| Ver | Date       | Training composition                                                                | F1                        | F1 measured on               | Verdict                                                                                                    |
|-----|------------|-------------------------------------------------------------------------------------|---------------------------|------------------------------|------------------------------------------------------------------------------------------------------------|
| v0  | 2026-04-09 | 25 images (first practice run, old account 396278862184)                            | 0.818                     | its own tiny split           | Garbage / overfit. Discarded, never deployed.                                                              |
| v1  | 2026-04-28 | 103 train Roboflow maize-fallarmyworm, single class                                 | 0.749 (P 0.833 / R 0.680) | 21-image Roboflow test split | First real baseline. Recall was the weak axis.                                                             |
| v2  | 2026-05-08 | v1 + paid caterpillar set as a second class ("Route 2")                             | —                         | —                            | FAILED. Shortcut learning: the model discriminated by photo source, not species. Reverted to single class. |
| v3  | May 2026   | 248 images (108 Roboflow + 140 paid + 11 anchor images + 2 mislabelled "negatives") | 0.719                     | its own test split           | Consolidated single-class. Bud false positives surfaced.                                                   |
| v4  | 2026-05-21 | 261 train / 65 test, retrained after                                                | 0.719                     | its own 65-image test split  | Long-running deployed baseline. Still the older                                                            |

| Ver  | Date       | Training composition                                                                      | F1           | F1 measured on                      | Verdict                                                                                                                     |
|------|------------|-------------------------------------------------------------------------------------------|--------------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
|      |            | migration to nbk2                                                                         |              |                                     | rollback target.                                                                                                            |
| v5   | 2026-07-07 | 1160 train (108 Roboflow + 1040 purchased full set + 12 anchor images)                    | <b>0.852</b> | purchased-domain TEST 133           | Production model 2026-07-07 → 2026-07-29. See the health warning below.                                                     |
| v6   | 2026-07-14 | v5 + 352 close-ups (152 idle valid-split + 200 synthetic worm-centred crops) = 1512 train | 0.839        | same TEST 133 (held identical)      | REJECTED, rolled back to v5 next day. Close-up crops added no domain information; lost two previously-detected light worms. |
| v7.1 | 2026-07-20 | 1503 images total (1103 corn DST1105 FAW + 400 moth-zldog larva); no black set            | 0.744        | its own corn+moth test split        | REJECTED, keep v5. Light-colour hypothesis did not transfer; batch_2 tie.                                                   |
| v7.2 | 2026-07-21 | v7.1 sources + top-500 quality-ranked purchased black caterpillars                        | 0.794        | its own corn+moth+ black test split | REJECTED. batch_2 got worse (2/7 vs 3/7 @50%, original 7-image set); foliage false positives strengthened.                  |

| Ver  | Date       | Training composition                                                                                                                                                | F1    | F1 measured on    | Verdict                                                                                                  |
|------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|-------------------|----------------------------------------------------------------------------------------------------------|
| v7.3 | 2026-07-22 | v7.1 corn+moth base + the full ~1300 purchased black set (project armyworm-detection-v7-3, version v7-3-20260722-0610)                                              | —     | its own split     | REJECTED. Confirmed the black set is net negative at full volume, not just at top-500.                   |
| v7.4 | 2026-07-23 | the consolidated “fair” composition: corn + moth-zldog + maize supply + the established anchor images (project armyworm-detection-v7-4, version v7-4-20260723-0259) | —     | its own split     | Pulled level with v5 on the fair holdout. Kept as the fair yardstick and the base composition for v8/v9. |
| v8   | 2026-07-23 | v7.4 composition with TRAIN augmented ~13x (flips, rotations, exposure jitter; 2251 → ~29k images); TEST held at                                                    | 0.739 | v7.4’s test split | Augmentation build. Superseded by v9 in the same test cycle.                                             |

| Ver              | Date       | Training composition                                                                                                    | F1    | F1 measured on               | Verdict                                                                                                                                                                                                                                          |
|------------------|------------|-------------------------------------------------------------------------------------------------------------------------|-------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  |            | v7.4's un-augmented 567                                                                                                 |       |                              |                                                                                                                                                                                                                                                  |
| v9               | 2026-07-25 | same supply, re-split 9:1 by unique source stem, both sides augmented 13x (train 32,981 / test 3,653)                   | 0.591 | its own augmented test split | Went LIVE 2026-07-29 together with the role change below: high-recall front end + LLM verification gate.                                                                                                                                         |
| v9r (v9 retrain) | 2026-08-05 | v9 with added data augmentation (the documented 13x build: flips, rotations, exposure jitter). Version v9-20260805-0713 | 0.599 | its own test split           | <b>The settled model.</b> Live on the dev account's worm_cam from 2026-08-07; retrained on production as v9r-prod-20260810 (F1 0.613), the operative model since 2026-08-12 (3.17). Best of the ladder on the never-trained holdout (see below). |

**Standing health warning — do not quote 0.852 as accuracy.** Each F1 in the table was measured on a DIFFERENT test set that grew and changed domain with the training set. This is a record of engineering iterations, not a monotonic climb on one benchmark. Specifically, v5's 0.852 is **in-domain only**: it was earned on the purchased commercial TEST 133 and **did not transfer to CAG imagery**. On the batch\_2 CAG holdout, v5 ties v4 at the operational threshold (both 3/7 @50%, arbitrated 2026-07-13; scored on the original 7-



image batch\_2, 101-107 — the historical arbitration set, not the fixed 102-109 eval set; see 3.6). Symmetrically, do not read v9's 0.591 / v9r's 0.599 as a regression: from processor v6.x onward the detector is deliberately run as a high-recall front end (tiled, candidates gathered down to `TILE_MIN_CONFIDENCE = 8`) with the Sonnet 4.6 verification gate supplying precision, so the detector's stand-alone F1 no longer describes pipeline behaviour.

**Domain, not colour, is the binding constraint.** Three controlled follow-ups confirmed it: 1. **v6 (scale test):** the same worms at a tighter scale bought nothing — zero misses converted to hits, confidence merely deepened where the model was already right (cag\_armyworm\_104: 75.9% → 97.0%). 2. **v7.1 (colour test):** real light-coloured Spodoptera positives from corn and rice domains did not teach the model CAG light worms. The two thesis images the pivot targeted, cag\_armyworm\_003/004, scored 9.3% and 24.2% under v7.1 (internal batch\_1 spot-check numbers — not part of the outward-quotable eval set). 3. **v7.2 (volume test):** re-adding 500 black-caterpillar positives made batch\_2 worse (2/7 vs 3/7 @50%). The bud\_002 confidence rise (91% → 97.4%) (internal batch\_1 numbers) was recorded at the time as false-positive evidence, but a later ground-truth correction (see 3.6) found bud\_002 contains a real larva, so that delta is not evidence of a false-positive prior.

Locked lesson from the v5–v7.3 experiments: out-of-CAG-domain positives do not close the gap regardless of colour or volume. In-domain generalisation is strong (10/10 on never-trained purchased TEST images, most at 99%+, proven 2026-07-13). The levers that remained were deployment geometry (close-range capture is the model's working domain) and the pipeline changes of the v9 era below. v7.1 and v7.2 are kept as clean, report-grade negative results.

**The v8 → v9 → v9r era (2026-07-23 to 2026-08-07).** After v7.2/v7.3 the direction changed from sourcing new positives to (a) data augmentation on the consolidated supply and (b) changing the detector's JOB. v8 was the first 13x augmentation build; v9 superseded it in the same test cycle. v9 went live 2026-07-29 together with the architecture change: the detector became a high-recall front end (tiling on, candidates gathered down to `TILE_MIN_CONFIDENCE = 8`) and Sonnet 4.6 judges every box (Chapter 2). The measurement behind the change: raw tiled candidate coverage found every batch\_2 worm — the detector was never the recall bottleneck; the worms sat at 10–17% confidence, below the old operating floors. **v9r**, trained 2026-08-05 with added data augmentation, replaced v9 on worm\_cam on 2026-08-07 and is the settled model. On the never-trained holdout benchmark (batch\_2 102–109; see 3.6) the full v9r pipeline scores **7/8 at the 34% zero-worm-loss floor and 5/8 at floor 49** (the 2026-08-10 decision; the live production floor was later refitted to 33 — next paragraph), and cag\_armyworm\_103 — missed by every earlier model and configuration — was detected for the first time (34.9%). Still unsolved: cag\_armyworm\_107 has never been detected by any model or configuration.

**The threshold ladder and the floor-49 decision (2026-08-07 → 2026-08-10).** The full 26-image holdout was pushed through the live pipeline twice: once at post-verify floor 34 / area cap 10% (v9r\_\* dashboard zones) and once at floor 49 / area cap 5% with the

tightened verify prompt (v9r49\_\* zones). The two runs mapped the confidence axis: Jewel junk boxes cluster at 40–48%, while the newly-reachable worm hits sit at 34.9–39.4%. **No single floor separates them.** Floor 34 is the zero-worm-loss setting; floor 49 is the maximum-cleanliness setting at the cost of those low-confidence hits (batch\_2 eval 5/8 instead of 7/8; Jewel patrol frames 3+3 boxes instead of 10+8; 41 total boxes across the 26 images instead of 70). **Decision (Runzhe, 2026-08-10): floor = 49, precision-first.** Flipped live the same day — the camera row's post\_verify\_floor and the processor env POST\_VERIFY\_FLOOR both read back 49. Both runs stay on the dashboard as the threshold-study evidence. **The floor is per-build, not a constant of the system: the current live value on production is 33 (refitted 2026-08-13).** The production retrain v9r-prod-20260810 scores some of the same worms lower than the dev-account v9r did (one holdout image's top box fell from 77.2% to 14.3% across the two trainings), and the curated gallery batch was produced at floor 33, so on 2026-08-13 the live floor was set to 33 (camera row + Lambda env) so that a live Test upload draws the same boxes the gallery shows. Verified against the production worm\_cam row 2026-08-14: post\_verify\_floor 33, min\_confidence 10, tiling\_enabled true, llm\_verify\_enabled true, max\_runtime\_min 45. The floor-49 story above stays as the recorded threshold study and the report-quoted configuration. One boundary the study fixed permanently: a persistent false positive survives at 81.5% confidence, ABOVE any usable floor — thresholds cannot remove that class of error; only the verify prompt/gate can. Companion config changes, live since 2026-08-07: POST\_MAX\_BOX\_AREA 0.10 → **0.05** (the largest real worm measured in the holdout imagery is 4.39% of frame, so the headroom is only 1.14x — a close-up worm exceeding 5% of frame would be silently suppressed; first suspect if a big obvious worm has no box), and the verify prompt was tightened to describe the larva as elongated and soft-bodied with clear segmentation, typically carrying yellow-and-black stripes (widened again on 2026-08-12 for pale young larvae — 3.16). Processor v6.3 (dead experimental code stripped, 3266 → 1545 lines) deployed the same day.

**Field-realistic qualitative check (adjudicated 2026-08-11).** The 4 field-realistic images (3.6) were adjudicated box by box against the floor-49 records: all 4 worms are found (true boxes at 49.9–88.2%), alongside 5 false positives at 62–84% — above the real worm's confidence in 3 of the 4 images. Confidence does not rank truth. The false positives trace to generic-domain training data rather than site-captured data; that is the delivered system's precision ceiling, and it is stated as a limitation, not hidden. These 4 images are presented as qualitative demonstration only, never as a recall statistic.

Traceability: F1 and billable training time per version come from describe-project-versions and deployer/audit/rekognition\_\_armyworm-detection\_\_project-versions.json. Arbitration raws: datasets/holdout/cag/batch2\_arbitration\_20260713\_105204.json, v7\_1\_vs\_v5\_20260721\_021123.json, v7\_2\_vs\_v5\_20260721\_054531.json. The 2026-08-07 threshold-study runs live on the dashboard zones v9r\_\* and v9r49\_\*.

## 3.6 The CAG holdout discipline

datasets/holdout/cag/ holds the only real CAG-domain photographs in existence. They are the arbiter for every model-version decision.

- **batch\_1/** — 13 images: cag\_armyworm\_001.jpg ... cag\_armyworm\_011.jpg plus cag\_bud\_001.jpg and cag\_bud\_002.jpg. batch\_1 is used for internal spot checks only — outward-facing evaluation numbers are quoted solely from never-trained images (batch\_2 102-109, Jewel\_1/2, the 4 field-realistic photos; see the subsets below). cag\_bud\_002 carries a hand-drawn blue circle (drawn markings make an image unusable for dataset work). **Ground-truth correction (recorded in docs/state.md):** both cag\_bud\_001 and cag\_bud\_002 contain a clearly visible striped larva (bud\_002's is the one circled in blue). Both are POSITIVES, not false-positive probes. Consequence: the CAG holdout contains NO true negatives at all, so image-level false-positive rate is not measurable on it, and any past or future claim of a false-positive rate on this holdout is unsupported. Do not cite bud\_00x confidence rises as false-positive evidence.
- **batch\_2/** — the **sacred holdout. Never train on it.** The original benchmark set was cag\_armyworm\_101.jpg ... cag\_armyworm\_107.jpg (7 images, Jewel wide indoor scenes). A 2026-07-17 re-audit found 5 of those 7 carry hand-drawn circles from CAG staff (101 blue, 102 pink, 103 green with the “416” jeep in frame, 106 red, 107 blue); only 104 and 105 are clean. So treat the original batch\_2 as a 2-clean-image benchmark with 5 contaminated companions, not a pristine 7.

The batch\_2/ folder had grown to 11 files (cag\_armyworm\_101–111) by 2026-07-28. All arbitrations recorded up to 2026-07-21 used the original 7 (101–107). Provenance of the additions (per docs/state.md): cag\_armyworm\_108 and 109 arrived 2026-07-23 — both CLEAN (no hand-drawn circles) and real Jewel domain, 108 a clear patterned armyworm on granite plus leaf, 109 two small dark low-contrast worms. cag\_armyworm\_110 and 111 are Runzhe's own phone photos taken 2026-07-28 (source files IMG\_1574 and IMG\_1577), stored EXIF-upright at 3024x4032 with Orientation stripped so the pipeline and browser see the same pixels.

**The benchmark subsets behind the 3.5 numbers (fixed early August 2026, before the 2026-08-11 re-curation below):** - **Quantitative recall: cag\_armyworm\_102–109 (8 images) — the “batch\_2 eval” set.** All 2026-08 model comparisons and the threshold study (3.5) score this subset. - **Qualitative field demonstration: the 4 field-realistic images** — the two Jewel patrol frames CAG\_Jewel\_1/2 (captured by the Go2 on patrol) plus cag\_armyworm\_110/111 (shot at the Go2's capture angle and resolution). Adjudicated box by box (3.5); presented as demonstration, never as a recall statistic. - Evaluation claims in any outward-facing document are quoted ONLY from these images.

**Re-curation (2026-08-11) — the current evaluation set.** For the controlled model-ladder evaluation, Runzhe re-curated the holdout. The evaluation set is now **22 images: batch\_1 (13, unchanged) + batch\_2 101–105 + batch\_Jewel/Jewel\_armyworm\_1–4**, with the

adjudicated ground truth in `datasets/current/answer_key/answer_key.json` (33 confirmed worms; the build script is covered in 3.12). File identity was MD5-checked against S3 ETags: `Jewel_armyworm_1/2` are the former `cag_armyworm_110/111`, `Jewel_armyworm_3/4` are the former `CAG_Jewel_1/2`, and **the current batch\_2 102–105 are DIFFERENT photographs from the files the earlier arbitrations and the 2026-08-07 threshold study scored** — the 2026-07-28 hand labels for 104/105 are retired. So none of the recorded numbers above can be reproduced by re-running on today's tree; each is tied to the files as they stood on its run date. On disk, `batch_2/` still holds 13 files (101–111 plus the two Jewel patrol frames), `clean/` holds circle-removed copies, and `_pre_rebuild_backup/` / `_corrupted_20260811/` are recovery snapshots. The 22-image answer-key set, not the raw folder contents, is the operative benchmark.

The evaluate scripts `glob batch_1/*.jpg + batch_2/*.jpg`, so a naive re-run today would score 26 files, not the 20 the recorded arbitrations used — and four of today's `batch_2` files are different photographs (above). Never compare such a run against the recorded numbers.

**Whole-image evaluation protocol.** All holdout arbitration uses one `detect_custom_labels` call per image on the whole, untiled, unscaled frame, with `MinConfidence=0` so the full confidence distribution is captured, and thresholds (30 / 50 / 70, headline 50) applied locally afterwards. The production pipeline historically had a tiled path; numbers from the two paths must never be compared (see `docs/detection.md`, “Two detection paths”). Any A-vs-B model comparison must push both models through this same script path.

**Leak prevention is mechanical, not honor-system.** `build_v7_2.py` computes the MD5 of every holdout image and drops any training candidate whose hash matches (3.11). Keep that check in every future build script.

**The holdout tree has no backup but S3.** On 2026-08-12 nine holdout files (106–111, the two Jewel patrol frames, and one cleaned copy) were found missing from the local tree — deleted at some point after 2026-08-07, not moved. All nine were recovered from the old account's S3 push history (`migration/restore_holdout.py` scans every `frames/worm_cam/*` zone and pulls the newest copy; integrity was verified by image dimensions, not assumed). The lesson for any successor: the S3 push history is the only backup the holdout has, and the local tree is not versioned — do not rely on either alone.

### 3.7 `convert_to_manifest.py` — YOLO to Ground Truth manifest

**Purpose.** Converts the Roboflow maize-fallarmyworm-1 YOLOv8 export into SageMaker Ground Truth object-detection manifests, one per split, in the format Rekognition Custom Labels imports. Output: `manifests_v5_roboflow/{train,valid,test}.manifest`.

**Configuration (constants at top of file).** `DATASET_ROOT = datasets/current/maize-fallarmyworm-1`; `S3_BUCKET = frames-armyworm-366356442579`; `S3_PREFIX = training-data/armyworm`; `LARVA_CLASS_ID = 2` (the fall-armyworm-larva class in the source's class

map); OUTPUT\_CLASS\_NAME = armyworm-larva. All paths are anchored to the script's own folder (Path(\_\_file\_\_).resolve().parent), so the working directory does not matter. As checked in, S3\_BUCKET still names the retired dev bucket; any production re-run must set it to argus-frames-506868652945 first (3.17), or every emitted source-ref points at a dead bucket.

**Functions and algorithm.** - parse\_yolo\_line(line) — parses one YOLO label line. Two formats are accepted. Detection format(class\_id cx cy w h, 5 numbers) is taken as-is. Segmentation format(class\_id x1 y1 x2 y2 ..., 1 + 2N numbers,  $N \geq 3$ ) is reduced to its axis-aligned bounding box: split the numbers into xs (even indices) and ys (odd indices), take min/max of each, then  $cx = (x_{\min} + x_{\max})/2$ ,  $cy = (y_{\min} + y_{\max})/2$ ,  $w = x_{\max} - x_{\min}$ ,  $h = y_{\max} - y_{\min}$ . Anything else (odd count, non-numeric) returns None and is skipped silently. - yolo\_to\_sagemaker\_bbox(cx, cy, w, h, img\_w, img\_h) — converts normalized YOLO centre-format to absolute-pixel corner-format:  $left = (cx - w/2) * img\_w$ ,  $top = (cy - h/2) * img\_h$ ,  $width = w * img\_w$ ,  $height = h * img\_h$ , each rounded to int, with left/top clamped at 0. class\_id is always emitted as 0 (single output class). - process\_split(split\_name) — for each .txt label in the split's labels/ dir: find the matching image (.jpg/.jpeg/.png), parse all lines, keep only class LARVA\_CLASS\_ID boxes, read the true image dimensions with PIL, build the annotations list, and write one manifest JSON line pointing at s3://<bucket>/<prefix>/<split>/images/<filename>. **Images with zero kept boxes are skipped entirely** — they never enter the manifest. A label with no matching image prints a warning and is skipped.

The conversion function is small enough to quote whole — this is the exact math every v5-era manifest box went through:

```
def yolo_to_sagemaker_bbox(cx, cy, w, h, img_w, img_h):
    """Convert normalized YOLO bbox to absolute-pixel SageMaker bbox."""
    abs_w = w * img_w
    abs_h = h * img_h
    left = (cx - w / 2) * img_w
    top = (cy - h / 2) * img_h
    return {
        "class_id": 0,
        "top": max(0, int(round(top))),
        "left": max(0, int(round(left))),
        "height": int(round(abs_h)),
        "width": int(round(abs_w)),
    }
```

Note the asymmetry: only top/left are clamped (at 0); width/height are trusted as given, so a box overrunning the right or bottom edge survives unclamped. The later build scripts (3.11) clamp both corners into [0,1] before scaling.

**Data in/out.** In: YOLO labels + images under

DATASET\_ROOT/{train,valid,test}/{labels,images}. Out: one JSONL manifest per split plus a printed kept/skipped/box-count summary.

**Gotchas.** - The zero-box skip is deliberate but has a system-wide consequence: **the training set contains no true negatives at all**. That, combined with the platform limit in 3.16, is the root cause of the foliage false-positive prior. - The image size in the manifest must be the real pixel size — the script reads it from the file, never from the label. Keep it that way; a wrong `image_size` silently corrupts every box. - The class filter is hardcoded; if the source dataset's class map changes, `LARVA_CLASS_ID` must be re-verified against its `data.yaml`.

### 3.8 `convert_other_caterpillar_new.py` — the purchased-set variant

Same algorithm as 3.7 (identical `parse_yolo_line` and `yolo_to_sagemaker_bbox`), modified for the purchased set's layout. Differences only: - `DATASET_ROOT` points at the vendor-licensed offline image set obtained from the project owner — the purchased master copy deliberately lives outside the repo. - YOLOv8 default folder shape: `images/{split}/` and `labels/{split}/`, and the middle split is named `val`, not `valid`. - `SOURCE_CLASS_ID` = 0 (the set's only class) and it is **relabeled to armyworm-larva** on output. This is the v5 single-class doctrine: after the v2 multi-class failure, all positives merge into one class. - `S3_PREFIX` = `training-data/other-caterpillar`; output to `manifests_v5_purchased/`.

Gotcha carried from the 2026-07-17 audit: this purchased set (about 1300 images, 89% of v5's TRAIN) is largely **not armyworm** — black spiny temperate larvae plus tussock-moth larvae, some blurry, some sky-shot, some boxes with no visible worm. The converter faithfully converts whatever it is given. Quality control lives in `rank_purchased.py` (3.10), not here.

### 3.9 `merge_manifests.py` and `upload_images.py` — combine and ship

`merge_manifests.py`

**Purpose.** Concatenates the Roboflow and purchased manifests into `manifests_v5_combined/{train,valid,test}.manifest`, the set v5 actually trained from.

**Algorithm.** For each output split, read each source manifest, drop blank lines, `json.loads` every line as a validation gate (a malformed line raises and aborts the merge), concatenate, and write. Then a per-class census of the merged train manifest is printed from each entry's class-map.

**Configuration.** The MERGES list pairs each output name with its ordered source list. Note the split-name mapping: Roboflow `valid.manifest` merges with purchased `val.manifest`. The frozen v3 multi-class merge config is kept in comments as the historical record.

**Gotcha.** The anchor images are intentionally NOT merged here — they were appended directly to the Rekognition dataset by hand. So the combined manifest counts do not equal the trained dataset's `LabeledEntries`. Do not “fix” the discrepancy.

## upload\_images.py

**Purpose.** Uploads exactly the images the merged manifests reference — nothing else. There is no bucket constant: the destination bucket is parsed from each manifest line's source-ref, so uploads follow whatever bucket the converters wrote (the retired dev bucket in the checked-in v5 manifests; argus-frames-506868652945 once manifests are regenerated for production). Idempotent: safe to re-run after a partial failure.

**Functions and algorithm.** - `parse_s3_uri(uri)` — splits `s3://bucket/key` into bucket and key. - `s3_object_exists(bucket, key)` — `head_object`, treating 404/NoSuchKey/NotFound as absent, re-raising anything else. - `upload_from_manifest(manifest_path)` — per manifest line: take source-ref, split off the filename, map the S3 directory to a local directory via the `S3_TO_LOCAL` dict, then skip-if-missing-locally, skip-if-already-on-S3, else `upload_file`. Counters: uploaded / skipped\_exists / skipped\_missing / failed.

**Configuration.** `S3_TO_LOCAL` maps the six S3 prefixes (training-data/armyworm/... and training-data/other-caterpillar/...) to the Roboflow folder inside the repo and the vendor-licensed offline set's folders outside it. An unmapped S3 directory logs a warning and counts as failed — that is the signal the map is stale after a layout change.

**Gotcha.** The HEAD check makes re-runs cheap, but it also means a corrupted earlier upload is never repaired automatically; delete the S3 object first to force a re-upload.

## 3.10 rank\_purchased.py — quality-rank the purchased set

**Purpose.** Scores every image in the purchased set on objective proxies for the exact defects the 2026-07-17 audit found (blur, featureless boxes, few-pixel specks, sky shots) and writes `purchased_ranked.csv`, ranked best-first. Downstream builds take the top N instead of sampling randomly (Runzhe's ruling, 2026-07-17).

**Design point:** all scores are computed on the **labelled box region**, not the whole frame — what matters is whether the WORM is usable.

**Functions and algorithm.** - `laplacian_var(gray)` — variance of the Laplacian over the box crop, the standard blur proxy, implemented with a numpy sliding window (no scipy dependency). - `sky_fraction(rgb)` — fraction of pixels that look like sky (blue-dominant bright, or blown-out white), computed on the upper half of the frame only. - `parse_boxes(lbl_path)` — same YOLO detection/segmentation parsing as the converters. - Per image, the **best** box (highest sharpness) supplies `box_sharp`, `box_contrast` (std-dev inside the box), and `box_frac` (box area / image area). - Combination: percentile-rank sharpness and contrast across the whole set; normalise `box_frac` against a saturation point of 8% of frame (bigger boxes score higher up to 8%; beyond that there is no extra reward); turn `sky_frac` into a penalty saturating at 25%. Final score =  $0.40 * \text{sharp\_rank} + 0.25 * \text{contrast\_rank} + 0.25 * \text{frac\_norm} - 0.30 * \text{sky\_penalty}$ .



**Edge behaviour.** Only label rows of class 0 count (`CLASS_ID = "0"`). A box whose pixel crop is under 4 px on either side is ignored; if every box in an image is ignored, or PIL cannot open the image, the image is skipped with a log line and never ranked. `laplacian_var` returns 0.0 for crops smaller than 3x3 instead of raising. `sky_fraction` is evaluated on the upper half only, so a bright floor in the lower half cannot trigger the sky penalty.

**Data out.** `purchased_ranked.csv` with rank, score, split, stem, per-axis values, and the absolute image path, plus printed distribution stats (sky-heavy count, speck count, soft count).

### 3.11 The build scripts — leak-proof dataset assembly (v7.2 → v7.4 → v8 → v9)

Four build scripts assembled the v7.2 → v9 datasets. They share DNA: MD5 leak guards against the holdout, the unique-source-stem split rule, deterministic hash-ordered splits, and clamped pixel-annotation manifest emission. All four are local-only (nothing uploaded, nothing trained) and, as checked in, write source-refs against the retired dev bucket `frames-armyworm-366356442579` — retarget `S3_PREFIX` to `argus-frames-506868652945` before any production rebuild (3.17).

#### `build_v7_2.py` — the leak-proof template (v7.2)

**Purpose.** Assembles the v7.2 dataset: the cleaned v7.1 corn + moth supply plus the top-500 ranked black images, single class `armyworm-larva`, with a strict leak-proof 80:20 train/test split. Local only — it uploads and trains nothing. It is also the template for any future build: the holdout guard and the stem rule below must survive into every successor.

**Configuration.** `V71 = datasets/v7_1_worm` (source images + `provenance.csv`); `RANKED = purchased_ranked.csv`; `HOLDOUT = datasets/holdout/cag`; `OUT = datasets/v7_2_worm`; `MANI = manifests_v7_2`; `S3_PREFIX = s3://frames-armyworm-366356442579/training-data/v7_2/armyworm`; `N_BLACK = 500`; `TEST_FRAC = 0.20`; `CLASS_NAME = "armyworm-larva"`.

**Algorithm, in order.** 1. **Gather candidates.** Corn and moth images come from `v7_1_worm/provenance.csv` (source, filename, split already known; labels already YOLO class 0). Black images are the top-500 rows of the ranked CSV, output filenames prefixed `black_` to avoid collisions. 2. **Hash the holdout.** MD5 every image under `holdout/cag/`. 3. **Filter and group.** Per candidate: drop if the image file is missing; drop boxes that are full-frame junk (`keep_boxes` removes any box with both  $w \geq 0.98$  and  $h \geq 0.98$  normalized) and drop the image if no box survives; **drop if its MD5 matches a holdout image** (the hard leak guard); drop MD5 duplicates within the build. Survivors group by (`source`, `stem`). 4. **The stem rule — Roboflow-augmentation leak prevention.** `stem_of()` strips Roboflow's augmentation suffix `_jpg.rf.<hex-hash>` from corn/moth filenames, so every augmented copy of one photograph shares a single stem. The split is then decided **per unique source stem, not per file**: a rotated or recoloured twin can never land on the other side of the train/test line. This is the difference between a real 80:20 split and a self-graded one. 5.



**Stratified deterministic split.** Within each source, stems are sorted by the MD5 of source|stem (a stable pseudo-random order, no RNG state), the first 20% become test, the rest train. Stratifying per source keeps the test set's source mix equal to the train set's.

**6. Materialise.** Copy each image into v7\_2\_worm/{split}/images/, convert YOLO boxes to pixel annotations (clamped to [0,1] before scaling, boxes under 1 px discarded), write YOLO labels alongside, append the manifest entry, and record a provenance row. 7.

**Report.** Writes ASSEMBLY\_REPORT.md (drop counts, per-split per-source image/box table, achieved test fraction) and provenance.csv. Holdout collisions are reported explicitly; the report proves the count.

**Gotchas.** The black images' labels are found by string-replacing the images path segment with labels in the ranked CSV's path — this only works while the vendor-licensed offline set keeps the YOLOv8 layout. The creation-date in the manifest metadata is hardcoded; cosmetic only.

### build\_v7\_4.py — the fair composition (the v8/v9 base)

**Purpose.** Assembles v7.4, the consolidated “fair” composition every later version trains from: the cleaned v7.1 corn + moth-zldog base, plus the pHash-deduped Roboflow maize-FAW larva images (real localized boxes only), plus CAG batch\_1 — so v7.4 holds the same CAG-domain data v5 had and the two can be compared fairly. Single class armyworm-larva, strict 80:20 split by unique source stem, TEST\_FRAC = 0.20. Local only.

**Algorithm, where it differs from build\_v7\_2.py. - Three candidate sources.** Corn + moth come from v7\_1\_worm/provenance.csv as before. The new larva come from v7\_4\_new\_larva.txt, a tab-separated list emitted by dedup\_roboflow\_larva.py (pHash-deduped against corn/moth and the CAG holdout); only their class-2 boxes are kept (read\_yolo(lbl, keep\_class=2)) and filenames are prefixed rf\_. batch\_1 comes from the frozen v5 manifest manifests\_v5\_combined/cag\_batch1\_from\_v4.manifest: its pixel boxes are converted back to normalized YOLO (centre = corner + size/2, divided by image size), filenames are prefixed cag\_, and bud\_002 is excluded outright as the inked contaminant. - **Full-frame junk drop inside the parser.** read\_yolo drops any box with  $w \geq 0.98$  AND  $h \geq 0.98$  normalized — this is what reduces the full-frame-labelled “fall-armyworm” Roboflow set to its 3 real-box images. - **The leak gate is an ABORT, not a drop.** build\_v7\_2.py silently drops holdout collisions and reports the count. v7.4 hashes only batch\_2/ (batch\_1 is deliberately IN this build) and refuses to continue on any hit:

```
if dropped["batch2_hit"]:
    raise SystemExit(f"ABORT: {dropped['batch2_hit']} training images collide
    "
                    f"with the sacred batch_2 holdout")
```

A collision means the candidate list itself is contaminated; a silent drop would hide that. Expected output is the report line “Sacred batch\_2 collisions: 0 (asserted).” - **Name-collision rename.** Files from different sources can share a filename; a second use of a name becomes {stem}\_{md5[:8]}\_{name} instead of overwriting. - Everything else follows

the template: the `_jpg.rf.<hex>` stem rule for corn/moth, the per-source deterministic MD5-ordered 80:20 split, MD5 dedup within the build, clamped pixel-annotation emission (sub-1-px boxes discarded; an image whose every box collapses is deleted). Outputs: `datasets/v7_4_worm/{train,test}`, `manifests_v7_4/`, `ASSEMBLY_REPORT.md`, and `provenance.csv` — which both v8 and v9 consume.

### `augment_build_v8.py` — the 13x augmentation build (v8)

**Purpose.** Turns v7.4's TRAIN split into the v8 training set by augmenting every image ~13x (2251 → ~29k). Only TRAIN is augmented; TEST reuses v7.4's un-augmented test.manifest unchanged (augmenting the test set would inflate the reported F1). `batch_2` is never read. This script's transform primitives are also the engine of the v9 build below — and the concrete meaning of “added data augmentation” wherever that phrase appears in this project: flips, rotations, exposure jitter.

**The exact geometric transforms (the GEOMS table).** Eight orientation ops. Each pairs a PIL image transpose with a closed-form transform of the normalized YOLO box (`cx`, `cy`, `w`, `h`) — image op and box op MUST agree, so both live in one table row. Two examples show the style:

```
def t_hflip(b):      cx, cy, w, h = b; return (1 - cx, cy, w, h)
def t_rot90(b):      cx, cy, w, h = b; return (1 - cy, cx, h, w)    # 90deg CW
```

The eight are identity, hflip, vflip, rot90, rot180, rot270, transpose, transverse — the last two composed as `rot90(hflip)` / `rot270(hflip)`. These are exact: no resampling error ever reaches the labels.

**photometric(im)** — random brightness (0.72–1.28), contrast (0.75–1.30), colour (0.80–1.25) jitter. The box is untouched.

**rotate\_arbitrary(im, boxes, deg)** — rotates on an expanded canvas, then recomputes each box as the axis-aligned bounds of its four rotated corners (rotate each corner about the old image centre, re-origin on the new centre, take min/max, clamp to the canvas). A box is dropped if its new centre leaves the frame or either side falls below 0.002 normalized. The axis-aligned re-fit makes rotated boxes slightly loose — accepted, since the labels were loose to begin with.

**variants\_for(im, boxes)** — the per-image plan, greedy: all 8 exact orientations first; then photometric jitter on randomly chosen orientations until 11 variants exist; then two arbitrary rotations at random  $\pm\{15, 20, 25\}$  degrees (also photometric-jittered); truncate to `VARIANTS_PER_IMAGE = 13`. A variant whose boxes all vanish is skipped.

**Safety rails.** `selftest()` runs before EVERY build, not only under `--selftest`: it asserts the transform algebra (hflip is an involution, rot90 twice equals rot180, rot90 then rot270 is identity, a known corner case maps exactly, an arbitrary rotation keeps a centred box centred). A silent box-math bug would corrupt ~30k labels, so the asserts come first. `cap_size` resizes to a 1280 px long edge (LANCZOS) — avoids Rekognition's

ERROR\_INVALID\_IMAGE\_DIMENSION rejection of >4096 px images and keeps the upload sane. clamp\_boxes clamps centres into [0,1] and drops sub-0.001 slivers. manifest\_entry converts to clamped pixel corners and returns None (variant skipped) if no box survives. random.seed(1105) makes the whole run deterministic; JPEG quality 88. Unreadable images and zero-box label files are skipped with a log line.

**Data in/out.** In: datasets/v7\_4\_worm train split, driven by its provenance.csv. Out: datasets/v8\_worm/train/{images,labels} (each variant named {stem}\_\_{suffix}.jpg) + manifests\_v8/train.manifest.

### build\_v9\_91.py — the 90:10 re-split, both sides augmented (v9)

**Purpose.** v8's train:test ratio came out 29,263 : 567 (98:2). Runzhe's 2026-07-26 call: make it a conventional 9:1 with the TEST side augmented too. v9 therefore pools ALL v7.4 source images (2,818), re-splits 90:10 by unique source stem (TEST\_FRAC = 0.10), and augments BOTH splits 13x — train ~32,981 / test ~3,653, the counts in the 3.5 ladder.

**How it works.** - **Reuses v8's proven primitives instead of copying them:** it compiles augment\_build\_v8.py up to (but excluding) def main() into a module object, so selftest, GEOMS, variants\_for, cap\_size, and read\_yolo are shared, not forked. selftest() still runs before anything is written. - **The MD5 split.** Per source, unique stems are ordered by the MD5 of "{source}|{stem}" and the first 10% become test:

```
ordered = sorted(stems, key=lambda s: hashlib.md5((src + "|" + s).encode()).hexdigest())
n_test = round(len(ordered) * TEST_FRAC)
test_stems = set(ordered[:n_test])
```

MD5 of a stable string is a deterministic pseudo-random shuffle: no RNG state, the same split on every run, stratified per source — and because the unit is the SOURCE STEM, all 13 augmented variants of one photograph land on the same side of the line. - **The closing leakage assertion.** After writing everything, the script re-checks that no (source, stem) pair was actually written to both splits and exits with SystemExit if any was — a belt-and-braces check on the assignment logic itself, printed as “stem overlap train/test = 0 (asserted, no leakage)”. - Same per-split manifest emission as v8 (job-name v9\_aug\_91), same 1280 px cap, same seed 1105.

**Honest caveat, from the script's own docstring:** an augmented test set measures “handles rotated/flipped variants of held-out photos”, which reads slightly higher than a raw held-out set would. The real verdict for this project has always been the CAG holdout through the live pipeline, not this in-domain F1 — one more reason 3.5's health warning treats v9's 0.591 / v9r's 0.599 as front-end figures.

## 3.12 The evaluate scripts — whole-image holdout arbitration

evaluate\_v7\_1\_vs\_v5.py and evaluate\_v7\_2\_vs\_v5.py are the same harness with different MODELS dicts. They are the reference implementation of the arbitration protocol;

any future A-vs-B comparison should copy this file and change only the ARNs — and, on the production account, set `PROFILE = "prod"` (the checked-in scripts ran against the retired dev account).

**Purpose.** Run TWO models over the full CAG holdout (batch\_1 + batch\_2) through the identical whole-image path so the numbers are directly comparable, then print a per-image verdict table and image-level recall at 30/50/70%.

**Configuration.** `PROFILE = "nbk2"`, `REGION = "us-east-1"`. `MODELS` maps a short name to its **project ARN and version ARN** — the compared models live in different projects, so both ARNs are required per model. `THRESHOLDS = (30, 50, 70)`, `HEADLINE = 50.0`, `CLASS = "armyworm-larva"`, `START_TIMEOUT_MIN = 25`.

**Functions and the endpoint discipline.** - `status(name)` — `describe_project_versions` filtered to the version name (parsed off the ARN), returns its Status. - `start_both()` — idempotent start: skips a model already RUNNING/STARTING, calls `start_project_version(MinInferenceUnits=1)` on STOPPED/TRAINING\_COMPLETED, raises on anything else. Then polls every 20 s until both are RUNNING or the timeout hits. - `stop_both()` — requests stop on both, printing “VERIFY MANUALLY IN CONSOLE” on any error. - `detect(name, path)` — one `detect_custom_labels` call on the raw image bytes with `MinConfidence=0`, filtered to the target class. Whole image, no tiling, no upscale. - `main()` — globs `batch_1/*.jpg + batch_2/*.jpg`, runs every image through both models **inside a try/finally whose finally is stop\_both()** — the models are stopped even if the run crashes mid-way. Saves the full raw label lists plus metadata to `holdout/cag/<pair>_<timestamp>.json`, then prints: per image, each model’s max confidence and a verdict (both / A only / B only / both miss at the 50% headline); then image-level recall at each threshold.

**Cost warning, baked into the script and repeated here:** both endpoints bill roughly \$4/hr each while RUNNING. If the script dies in a way that skips the finally block (power loss, kill), verify both models are STOPPED in the Rekognition console before walking away.

**Protocol rules the harness encodes.** Capture at `MinConfidence=0` and threshold locally, so one run yields every operating point. Judge image-level recall on the max-confidence box (at the time these scripts ran, no CAG ground-truth boxes existed, so box-level precision was not scored; the 2026-08-11 answer key below changed that). Never mix these numbers with tiled-pipeline numbers. The live model stays live regardless of the outcome; the script only reads. The archived original of this pattern is `datasets/archive/experiments/one_off_scripts/evaluate_batch2_v5_vs_v4.py` (it points at pre-2026-07-20 paths; read it, do not run it).

[build\\_answer\\_key.py](#) — the adjudicated ground truth behind the 2026-08-11 controlled evaluation

**Purpose.** Renders the canonical adjudicated ground truth for the re-curated 22-image evaluation set (3.6), the confirmation sheets Runzhe adjudicated from, and the machine-

readable key datasets/current/answer\_key/answer\_key.json. Every controlled-evaluation number from 2026-08-11 onward is scored against this key. Final state: **22 images / 33 confirmed worms / 0 open proposals** — Runzhe ruled every proposed box confirmed except 103's leaf-axil candidate, which was deleted as a false positive.

**Where the boxes come from — three provenance classes, kept distinct in the code.** - confirmed(key) — Runzhe's 2026-07-28 hand labels, read from datasets/current/cag\_ground\_truth.json. - The ADJ dict — Runzhe's 2026-08-11 per-box adjudications for the four Jewel\_armyworm\_\* images, with normalized L/T/W/H coordinates recovered from the v9r49 DynamoDB records. - proposed(\*boxes, note=...) — boxes proposed by visual inspection on 2026-08-11 for the NEW batch\_2 photos (102–105) and the cleaned bud\_002; each carries a descriptive note plus the ruling stamp. The old ground truth for the new 102/103/104/105 files is retired (3.6).

**Functions.** - render\_overlay(name, path, boxes) — resizes to an 1800 px long edge, draws each box (green = confirmed, orange = proposed) with a worm k / worm k? tag, adds a black banner stating the counts, and writes overlay\_<name>.jpg. - contact\_sheet(images, cols, tile\_w, path) — fixed-width tiles with per-row heights on a dark background; produces SHEET\_A\_batch1.jpg (4 columns) and SHEET\_B\_batch2\_jewel.jpg (3 columns). - zoom\_crops() — for every box, crops a square 4.5x the box's larger side (minimum 240 px), resizes to a 460 px tile, re-draws the box in crop coordinates, labels it, and grids all crops into SHEET\_C\_zoom\_crops.jpg. The zoom sheet is what made per-box adjudication practical on 3024x4032 phone photos.

**Edge points.** Coordinates are normalized fractions throughout; only rendering multiplies by pixel size. bud\_002 deliberately uses the circle-removed file holdout/cag/clean/cag\_bud\_002\_clean.jpg, not batch\_1's inked original. The verdicts live in the script and in cag\_ground\_truth.json, so re-running redraws the artifacts without changing any ruling.

### 3.13 The manifest line format

A Rekognition Custom Labels object-detection dataset imports a **SageMaker Ground Truth manifest**: a JSONL file, one JSON object per line, one line per image. The exact shape our converters emit:

```
{
  "source-ref": "s3://argus-frames-506868652945/training-data/armyworm/train/
images/IMG_0042.jpg",
  "bounding-box": {
    "image_size": [{"width": 1920, "height": 1080, "depth": 3}],
    "annotations": [
      {"class_id": 0, "top": 412, "left": 806, "height": 63, "width": 118}
    ]
  },
  "bounding-box-metadata": {
    "objects": [{"confidence": 1.0}],
  }
```

```

    "class-map": {"0": "armyworm-larva"},
    "type": "groundtruth/object-detection",
    "human-annotated": "yes",
    "creation-date": "2026-07-07T12:00:00",
    "job-name": "armyworm-labeling"
  }
}

```

Field rules: - `source-ref` — the S3 URI of the image. The image must exist there before dataset creation (hence `upload_images.py` runs first). - The label-attribute pair — here `bounding-box` and `bounding-box-metadata` — must share the same base name; Rekognition matches them by that convention. - `image_size` — the real pixel dimensions, a single-element list. Boxes are interpreted against these numbers. - `annotations` — one object per box: integer pixel top/left/height/width, and `class_id` indexing into `class-map`. **At least one annotation is required per line** — an empty list makes the entry unlabeled, and Custom Labels excludes it from training (3.16). - `objects` — one `{"confidence": 1.0}` per annotation, parallel arrays. - `type` must be `groundtruth/object-detection`; `human-annotated` is "yes"; `creation-date` and `job-name` are free-form provenance.

One malformed line fails only that entry (it shows in the dataset's `ErrorEntries`), but a wrong-but-parseable line (bad `image_size`, swapped width/height) trains silently on garbage. Validate with `merge_manifests.py`'s parse gate and spot-check boxes visually before training.

### 3.14 Training procedure, end to end

This is the full recipe. On the NP production account — the operative target for every future training run — the values are: profile `prod`, region `us-east-1`, bucket `argus-frames-506868652945`, projects `argus-detection` and `argus-moth-detection`. (The retired development account used profile `nbk2` and bucket `frames-armyworm-366356442579`; those values survive only in the historical records and the checked-in dev-era scripts.) On a fresh account, substitute your own. `train_v7_2.py` automates steps 1–5 with `boto3` (`get_or_create_project` — which calls `create_project` when the project is absent, covering step 1 — `→ ensure_datasets → start_training`, then `--watch` runs `watch()`, which polls every 60 s and writes status, F1, and billable seconds into `v7_2_train_state.json`). The manual steps below are what it does, for when you need them one at a time.

**0. Prepare data.** Run the pipeline in order: `converter(s) → merge_manifests.py` (or a `build_*.py` that emits manifests directly) `→ upload_images.py`. Then upload the manifests themselves to the bucket, e.g. under `training-data/v7_2/armyworm/manifests/`. Rekognition must be able to read the bucket; in the same account this is normally automatic, and the console offers to attach a bucket policy if not.

**1. Create the project.** - Console: Amazon Rekognition `→ Use Custom Labels → Projects → Create project → name it → Create`. - CLI: `aws rekognition create-project --project-`



name **your-project-name** --profile **your-profile** --region us-east-1 Record the returned project ARN.

**2. Create the TRAIN dataset from the manifest.** - Console: open the project → Create dataset → “Start with an existing dataset” → “Import images labeled by SageMaker Ground Truth” → point at the manifest’s S3 location → choose training dataset → Create. - CLI: `aws rekognition create-dataset --project-arn project-arn --dataset-type TRAIN --dataset-source file://ds-train.json --profile your-profile --region us-east-1`, where **ds-train.json** contains `{"GroundTruthManifest":{"S3Object":{"Bucket":"your-bucket", "Name":"path/to/train.manifest"}}}`. (Inlining that JSON in PowerShell is fragile — always use the file form.)

**3. Create the TEST dataset** the same way with --dataset-type TEST and the test manifest. Wait for both to reach CREATE\_COMPLETE: - Console: the project’s Datasets pages show status and image counts. - CLI: `aws rekognition describe-dataset --dataset-arn dataset-arn --profile your-profile --region us-east-1` Check DatasetStats.LabeledEntries against your manifest line count and that ErrorEntries is 0. A shortfall means unlabeled or malformed entries.

**4. Train.** - Console: project page → Train model. - CLI: `aws rekognition create-project-version --project-arn project-arn --version-name v-name --output-config '{"S3Bucket":"your-bucket", "S3KeyPrefix":"training-output/prefix"}' --profile your-profile --region us-east-1` Training here has taken roughly 1–4 hours (v6: 13,828 billable seconds).

**5. Watch and read the result.** - Console: the project page shows status; the model’s Evaluate tab shows F1, precision, recall once done. - CLI: `aws rekognition describe-project-versions --project-arn project-arn --profile your-profile --region us-east-1` Look for Status: TRAINING\_COMPLETED, EvaluationResult.F1Score, and BillableTrainingTimeInSeconds.

**6. Host it to test.** - Console: model page → Use model tab → Start. - CLI: `aws rekognition start-project-version --project-version-arn version-arn --min-inference-units 1 --profile your-profile --region us-east-1` Wait for RUNNING (10–20 min). About \$4/hr from now until stopped.

**7. Judge on the CAG holdout** with an evaluate-script copy (3.12), never only on the model’s own test F1. The ladder in 3.5 is the proof of why: three versions raised or held in-domain F1 and still failed the holdout.

**8. STOP the endpoint.** - Console: Use model tab → Stop. - CLI: `aws rekognition stop-project-version --project-version-arn version-arn --profile your-profile --region us-east-1` Then re-run describe-project-versions and confirm STOPPED. Do this even after a crashed run.

**9. Wire it (only if it won)** — next section.

### 3.15 Model wiring and rollback

The live model ARN lives in exactly ONE runtime place per account: the `custom_model_arn` attribute on the camera's row in the DynamoDB `pest-monitoring-cameras` table (`worm_cam` for armyworm, `moth_cam` for the moth model). All three Lambdas resolve it from there at call time. Nothing hardcodes a model ARN. Current values: on the development account, `worm_cam` points at the v9 retrain `v9-20260805-0713` (since 2026-08-07). On the production account the camera rows are `worm_cam`, `manual_upload`, and `moth_cam` (the deployer seeds the template row as `camera-1`; it was re-keyed to `worm_cam` in the 2026-08-11 reconciliation); `moth_cam.custom_model_arn` already carries `moth-prod-20260811`, and the armyworm ARN writeback (`v9r-prod-20260810`) targets `worm_cam` and `manual_upload` (3.17).

Consequences: - **Deploying a new model = one attribute write**. No code deploy, no Lambda change. - **Rollback = the same write with the old ARN**. The v6 rollback on 2026-07-15 was exactly this.

How to write it: - Console: DynamoDB → Tables → `pest-monitoring-cameras` → Explore table items → open **worm\_cam** → edit `custom_model_arn` → paste the version ARN → Save. - CLI/script: the ARN colon rule (3.3) applies. From PowerShell, do NOT inline the ARN into `aws dynamodb update-item`. Either use `boto3`:

```
import boto3
t = boto3.Session(profile_name="nbk2").resource("dynamodb", region_name="us-east-1") \
    .Table("pest-monitoring-cameras")
t.update_item(
    Key={"camera_id": "worm_cam"},
    UpdateExpression="SET custom_model_arn = :a",
    ExpressionAttributeValues={"a": "<the-version-arn>"},
)
```

or pass the values as file:// JSON documents: `aws dynamodb update-item --table-name pest-monitoring-cameras --key file://key.json --update-expression "SET custom_model_arn = :a" --expression-attribute-values file://vals.json --profile your-profile --region us-east-1`.

Remember the endpoint half: the ARN you point at must be a **RUNNING** version, and the version you just replaced should be **STOPPED** once traffic is confirmed on the new one, or you pay for two armyworm endpoints.

### 3.16 Inference-side algorithms — where they live and why they exist

The algorithms below run in the `pest-detection-processor` Lambda and are documented operationally in **Chapter 2**. This section records only the model-side facts that explain WHY each stage exists.



**Tiling (small-target recall — back ON in production).** Small worms have too few pixels for reliable detection; 4x crop+upscale was found to be the recall sweet spot (1x about 33% recall, 4x peak, 6x no better). The processor's tiled mode (4x4 grid + 15% overlap + full-frame pass = 17 calls, NMS merge) automated that finding. But tiling also amplifies the false-positive prior — 17 chances to fire on foliage at the close-up scale the model was trained on — and the W15 ruling set `worm_cam.tiling_enabled = false`. That ruling was reversed on Runzhe's own call with new evidence: the 2026-07-23 deploy (`docs/state.md`) set `worm_cam.tiling_enabled = true` with `LLM_VERIFY_ALL_BOXES = true` — Rekognition finds at high recall, the LLM verifies every box. The deployed config (verified live through 2026-08-10) records `TILE_MIN_CONFIDENCE = 8`, which is what feeds the denoiser its volume. Never compare tiled-path numbers with whole-image numbers.

**Application-layer FP suppression (v4.3) — because Custom Labels cannot train on negatives.** This is a platform limit, proven three ways on 2026-07-13: (a) empirically — 410 clean zero-box entries appended to the TRAIN dataset registered `cl-metadata.is_labeled:false`, `LabeledEntries` stayed at 1160, and an explicit `is_labeled:true` in the manifest was ignored on import; (b) the AWS OD manifest spec makes annotations mandatory per object, with no negative form; (c) AWS re:Post confirms true negatives must be removed. So the model's learned prior is “plant close-up means armyworm” (its training set holds zero true negatives, see 3.7) and no retrain on this platform can teach it “not a worm”. At `MinConfidence=0` both v4 and v5 spray 120–300 raw boxes per image, and some surviving FPs are 90%+ confident. Suppression therefore happens OUTSIDE the model: a `DetectLabels` pass drops worm boxes sitting on hard objects (person / vehicle / furniture / machinery). This killed the jeep-tire FP on `cag_armyworm_103`; it cannot touch plant-on-plant FPs. The dead-end artifacts are kept for the record in `datasets/archive/experiments/v6_experiment/`: `build_negatives_v6.py`, `upload_negatives_v6.py`, `append_negatives_v6.py`, `census_negatives.py`, and 410 harmless unlabeled entries still sitting in the v5 TRAIN dataset (removing them would need a dataset recreate; Custom Labels ignores them).

**LLM crop verification (v4.5 hybrid gate, Bedrock).** Rationale: with negatives impossible, a second, different judge earns its keep on the marginal boxes. The order is fixed: **Rekognition finds, the LLM verifies — never the reverse.** The 2026-07-21 A/B (`datasets/verify_llm_crop.py`, 20 CAG images, pre-registered thresholds — a methodology A/B of the verifier, not a model benchmark; do not quote these fractions as detection recall) showed why: whole-image LLM screening recalled 6/14 clean positives vs 13/14 for per-box crop+upscale, because the verifier ran on Bedrock's standard 1568px resolution tier and a sub-0.5%-of-frame worm survives the downscale as a couple of visual patches. No model swap fixes that; it is an input-resolution limit. The v4.5 gate as first deployed (2026-07-22) used Claude Haiku 4.5 and trusted any box at or above the camera's `min_confidence` outright (no LLM call); only a sub-threshold box was cropped (0.6 pad), upscaled, and adjudicated — an explicit reject drops it, anything else keeps it, fail-open. **Deployed config has since evolved (verified live 2026-07-29 and 2026-08-10, `docs/aws.md`):** `LLM_VERIFY_MODEL_ID` is `us.anthropic.claude-sonnet-4-6` (not Haiku 4.5); `LLM_VERIFY_ALL_BOXES = true`, so the gate is now a denoiser that judges EVERY box —

including boxes above `min_confidence`, which can now be dropped;  
`LLM_VERIFY_MAX_BOXES` = 120 (the old value 5 is historical); Lambda timeout 600 s at 1024 MB. Since 2026-08-07 the gate is followed by post-gate cleanup: NMS/containment merge, an area cap (`POST_MAX_BOX_AREA` = 0.05), and the post-verify floor (`POST_VERIFY_FLOOR` = 49 since 2026-08-10 — the threshold decision in 3.5), with the verify prompt tightened to the yellow-and-black-stripes wording. Processor v6.3 (2026-08-07) stripped the dead experimental paths (3266 → 1545 lines); the pre-strip source is archived at `lambda/archive/pest-detection-processor_v6.2_full.py`. Rekognition still only ever sees the whole, unscaled frame; the crop exists only as Bedrock's input. **Known limitation, carried into every claim: light-coloured worms are false-rejected.** Haiku rejected `cag_armyworm_003`'s pale grey larva ("a moth or insect on orange flower, not a larva"). The verifier inherits the training set's oldest blind spot — the light-worm class that v7.1 targeted and missed — and does not cure it.

### 3.17 Operations / reproduction

**Routine operations checklist.** - Before any model test: check endpoint states with `aws rekognition describe-project-versions --project-arn project-arn --profile prod --region us-east-1` (console: each model's Use model tab). Know what is RUNNING before you add to it. - After any test: STOP what you started and confirm STOPPED. Two RUNNING endpoints (moth + one armyworm) is the normal ceiling. - Every new model version is judged on the CAG holdout via a copy of the evaluate harness (3.12) before any wiring decision. Its own test F1 alone decides nothing. - Never train on `holdout/cag/batch_2`. Keep the MD5 holdout check from `build_v7_2.py` in every future build script. - Model switch and rollback are single DynamoDB writes (3.15). Mind the colon rule. - After a training round, push the holdout through the live pipeline so results show on the dashboard, not only in a terminal table.

**Reproducing on a fresh AWS account — executed for real, 2026-08-10/11.** Models are account-bound; there is no shortcut. This section was first written as a hypothetical recipe; it has now been run for real against the NP production account **506868652945**, and the record of that run follows the generic recipe.

Generic recipe (any fresh account): 1. Create an S3 bucket for training data. Update **S3\_BUCKET** in both converters, the bucket and **S3\_TO\_LOCAL** in `upload_images.py`, **BUCKET/S3\_PREFIX** in the build and train scripts, and the MODELS ARNs in any evaluate copy. 2. Configure a CLI profile for the new account and substitute it everywhere. 3. Run: `converters` → `merge` (or `build`) → `upload images` → `upload manifests` → 3.14 steps 1–8. 4. Wire the winning ARN per 3.15 (the rest of the stack must exist first — Chapter 2 and later). 5. Expect the same numbers only if you train from the same manifests; the frozen sets are in `datasets/current/manifests_v5_*` and `manifests_v7_2/`.

**The actual production retraining (2026-08-10/11).** The ARGUS deployer (Chapter 7) had already stood up the stack on the production account, including the empty Rekognition project `argus-detection` and the bucket `argus-frames-506868652945`. Because the full

training set already lived on S3 in the old account, the converters were not re-run; three migration scripts did the model side instead:

1. **migration/copy\_training\_data.py** — server-side copy of the entire armyworm training set (**36,641 objects / 4.27 GB** under training-data/v9/) from the old bucket to the new one; the bytes never touch the operator's machine. Method: merge a temporary, prefix-scoped read-grant statement into the OLD bucket's policy (existing statements preserved — the statement is merged in by Sid, not overwritten; --revoke removes it at sign-off), copy every key with copy\_object across 32 workers, rewrite every manifest source-ref to point at the new bucket, and verify object counts. Result: 36,641/36,641 copied, 0 failed, counts matched, 36,634 manifest source-refs repointed.
2. **migration/train\_v9r\_on\_prod.py** — creates the TRAIN/TEST datasets on the new account and submits training (version v9r-prod-20260810). Its method carries the migration's most important generic lesson: **after any labelling or label-editing in the Rekognition console, the S3 manifest is stale — the DATASET is the source of truth**. Console edits never write back to S3, so rebuilding a dataset from a copied manifest silently loses every console edit, and on a set this size that mistake costs a multi-hour training run before it is even visible. The correct method, scripted here: export the live entries from the old account's dataset with ListDatasetEntries (paginated, HasErrors=False), repoint each entry's source-ref at the new bucket, upload the result as NEW manifests (training-data/v9/armyworm/manifests/{train,test}\_v9r\_prod.manifest), and create the datasets from those. Both datasets reached CREATE\_COMPLETE: TRAIN 32,986 labelled / 32,986 total, TEST 3,653 / 3,653. Training was submitted 2026-08-10 23:56 and completed 2026-08-12 10:04 SGT at **F1 0.613** — the old account's v9r scored 0.599 on the same recipe and data, so treat the two as equivalent, not as an improvement.
3. **migration/poll\_training.py** — the watcher. Deliberately network-fault tolerant: the first poller died on a DNS error when the laptop dropped its connection, which looked like a training failure in the log but was not. This version retries transient errors indefinitely (logging at the 1st/10th/60th consecutive failure), and only a terminal training status (TRAINING\_COMPLETED, TRAINING\_FAILED, FAILED, DELETING) ends the loop; it then prints the F1 and the version ARN. Poll interval 180 s.

The moth rebuild (migration/migrate\_moth.py, 3.4) is the same grant → copy → ListDatasetEntries → train pattern against the old account's console bucket, and completed same-day. Both version ARNs are now written into their camera rows per 3.15. The remaining proof step, in progress at this chapter's stamp, is the validation re-push: start the endpoint and push the holdout through the new pipeline — the migration is only proven when the same images give the same detections. Then revoke the cross-account read grants (--revoke on both migration scripts).

**Local supply caveats.** The vendor-licensed offline image set obtained from the project owner lives outside the repo — copy it onto any new machine before the purchased-set

converter or ranker can run. One credential caution: `datasets/archive/experiments/pre_v3_abandoned/download.py` contains a live Roboflow API key inline (credential stored there, not reproduced here). Do not publish or copy that file anywhere public.

### 3.18 Cross-references

- **Chapter 2** — the pest-detection-processor Lambda: tiling implementation, v4.3 suppression config and env vars, the v4.5 LLM gate architecture, `llm_verify_enabled / tiling_enabled` camera flags, and the DynamoDB record shape the models' outputs land in.
- **Chapter 4 (dashboard frontend)** — the Test upload panel (the live-detection demo path) and how holdout pushes surface in the gallery.
- **Chapter 5 (edge: Go2 + Jetson Orin)** — the SIYI A8 Mini / Unitree Go2 / Jetson Orin capture chain that produces the frames these models consume.
- **Chapter 7 (deployer)** — the ARGUS deployer that created the production-account stack (Rekognition project, buckets, tables, Lambdas) the retraining in 3.17 targeted.
- **Chapter 8 (reproduction runbook)** — the account-wide reproduction sequence that 3.17's model-side steps slot into.
- Source docs behind this chapter: `docs/detection.md` (facts and learnings), `docs/aws.md` (ARNs, accounts, Bedrock), `docs/state.md` (the 2026-08 migration record), `datasets/README.md` (folder map), and the scripts under `migration/`.

## Chapter 4 — ARGUS dashboard (web frontend)

This chapter documents the ARGUS web dashboard: its module architecture, every JS module, the liquid-glass design system, the deploy runbook, and Cognito user management. As of 2026-08-15.

The dashboard runs on the NP production account 506868652945. The ARGUS deployer stood up the full stack there on 2026-08-10 (all 15 stages, 103 seconds), both detection models were retrained on that account (moth 2026-08-11, armyworm 2026-08-12), and the handover snapshot `argus-repo-snapshot-20260813.zip` was published 2026-08-13. Account 366356442579 is the retired development account where the system was built and validated; it stays reachable only as the historical record behind the report's evidence. Both deployments run the same frontend code from `web/dashboard_v4/`; only `js/config.js` differs, and the deployer templates that file at deploy time (section 4.4).

### 4.1 Role in the system

ARGUS is the operator dashboard for the Smart Pest Monitoring System. It is the surface CAG staff and NP assessors see. It shows live camera streams (Kinesis Video Streams HLS), the detection gallery (frames captured by the SIYI A8 Mini on the Unitree Go2, or by fixed cameras, and processed by AWS Rekognition Custom Labels), analytics (zone and trend views), and settings (camera thresholds, model start/stop, schedules, alert subscribers, test uploads).

The dashboard is a pure static frontend. It holds no business logic authority. Every data operation goes over HTTPS to the `pest-monitoring-api` Lambda behind API Gateway HTTP API `vzf17s6z00` (Chapter 2), except live-video playback URLs, which come from the `kvs-hls-handler` Lambda. Authentication is Amazon Cognito. The JWT authorizer on API Gateway is the real enforcement boundary; the login screen in the browser is UX only.

It is served as plain files from an S3 website bucket behind CloudFront.

- **Production account 506868652945** — the live deployment. Stood up by the ARGUS deployer 2026-08-10, verified serving `<title>ARGUS</title>` over HTTP 200 on 2026-08-11: bucket `argus-dashboard-506868652945`, CloudFront `E1YADURLSAVNFA`, URL <https://d1dtoxef7qmugl.cloudfront.net>, API `vzf17s6z00`, Cognito pool `us-east-1_9se1FDHpc`.
- **Retired development account 366356442579** — where the system was built and validated; historical record only: bucket `pest-dashboard-366356442579`, CloudFront `E1423RGLAXWNSI`, URL <https://d1twcdquexdgi8.cloudfront.net>, API `zwpbcivmsj`, Cognito pool `us-east-1_ea0aJdus1`.

The raw S3 website URLs still work but are HTTP-only. Do not hand them out; passwords must not travel over plain HTTP. Where this chapter cites a concrete resource id without naming the account, it is the production account's; development-era ids appear only in passages clearly marked as history.

## 4.2 Inventory

All paths are relative to the project repository root, under `web/dashboard_v4/` unless noted. Every file below was read in full for this chapter; all described behavior is verified in code except where an inline OPEN ITEM note says otherwise.

| Name                      | Location                                   | Purpose                                                                                                                                                                                                                    |
|---------------------------|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>index.html</code>   | <code>web/dashboard_v4/index.html</code>   | Markup and mount points: topbar, tab bar, page content, image modal, login overlay, toast stack. Loads <code>Chart.js</code> and <code>hls.js</code> from CDN, then <code>js/main.js</code> as the single ES-module entry. |
| <code>styles.css</code>   | <code>web/dashboard_v4/styles.css</code>   | Base light theme (design tokens) plus the ARGUS v5.2 liquid-glass theme block appended at the end, plus v5.3 pagination CSS. 1736 lines.                                                                                   |
| <code>js/config.js</code> | <code>web/dashboard_v4/js/config.js</code> | The one per-host/env constants file: API base URL, Cognito region and client id, cache sizing, poll interval.                                                                                                              |
| <code>js/auth.js</code>   | <code>web/dashboard_v4/js/auth.js</code>   | Cognito sign-in via raw <code>InitiateAuth</code> fetch. Token storage, refresh, sign-out, login overlay wiring.                                                                                                           |
| <code>js/api.js</code>    | <code>web/dashboard_v4/js/api.js</code>    | Every backend call. Attaches <code>Authorization: Bearer &lt;IdToken&gt;</code> ; any 401 reopens the login overlay.                                                                                                       |
| <code>js/state.js</code>  | <code>web/dashboard_v4/js/state.js</code>  | The single shared mutable state object and the IndexedDB image cache ( <code>imageCache</code> ).                                                                                                                          |
| <code>js/utils.js</code>  | <code>web/dashboard_v4/js/utils.js</code>  | Toasts, time formatting (UTC to Singapore), <code>escapeHtml</code> , <code>camDisplayName</code> , camera classifiers, <code>convertToJpg</code> , <code>Chart.js</code> option factory.                                  |
| <code>js/bbox.js</code>   | <code>web/dashboard_v4/js/bbox</code>      | Bounding-box extraction,                                                                                                                                                                                                   |

| Name            | Location                         | Purpose                                                                                                                                                      |
|-----------------|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 | .js                              | overlay rendering, coordinate scaling, per-box verify (TP/FP) logic, image zoom/pan.                                                                         |
| js/gallery.js   | web/dashboard_v4/js/gallery.js   | Gallery tab: filters, card grid, client-side pagination (50/page), throttled thumbnail loading, delete flow.                                                 |
| js/modal.js     | web/dashboard_v4/js/modal.js     | Image detail modal: meta pane (including the LLM Verifier row), review block, overlay + zoom wiring, delete button.                                          |
| js/live.js      | web/dashboard_v4/js/live.js      | Live tab: camera cards, KVS stream on/off toggles, HLS playback via hls.js.                                                                                  |
| js/analytics.js | web/dashboard_v4/js/analytics.js | Analytics tab: stat cards, by-zone chart, zone x day heatmap, daily trend, camera health, by-camera split.                                                   |
| js/settings.js  | web/dashboard_v4/js/settings.js  | Settings tab and its four sub-tabs: Cameras, Test upload, Schedules, Alerts. Model status polling lives here.                                                |
| js/costs.js     | web/dashboard_v4/js/costs.js     | Dormant Costs tab. Not in the tab list (the shared IAM user has no Cost Explorer access). Kept so it can be re-enabled by adding it back to TABS in main.js. |
| js/main.js      | web/dashboard_v4/js/main.js      | Bootstrap, tab router, topbar notifications, and the window bridge for inline onclick handlers.                                                              |
| Local fallback  | web/_archive/dashboard_v3_9.html | Single-file pre-auth predecessor, kept untouched for emergencies. NOTE: doc lag. docs/dashboard.md says                                                      |

| Name              | Location                                            | Purpose                                                                                           |
|-------------------|-----------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                   |                                                     | web/dashboard_v3_9.html; the file has since moved into web/_archive/ (location verified on disk). |
| Pre-reskin backup | web/_archive/dashboard_v4_v42_pre-argus_2026-07-10/ | Full copy of the dashboard before the ARGUS liquid-glass reskin.                                  |

Cloud resources this chapter touches, production account 506868652945 (all created by the ARGUS deployer, Chapter 7): S3 argus-dashboard-506868652945 (static website, us-east-1), CloudFront E1YADURLSAVNFA, Cognito User Pool us-east-1\_9se1FDHpc (pool name pest-dashboard-users) with app client 6vebotf45bp8u46cnraddiaplv (dashboard-web), API Gateway HTTP API vzf17s6z00 (21 routes, JWT on all but GET /stream/status), and the frames bucket argus-frames-506868652945 (the deployer writes the CORS rule for the website and CloudFront origins automatically — Test upload needs it).

Development-era equivalents on the retired account 366356442579 (history; kept because older records and the report’s evidence reference them): S3 pest-dashboard-366356442579, CloudFront E1423RGLAXWNSI, Cognito pool us-east-1\_ea0aJdusl with app client 4husu6afr835e235eu9dqp8av6, API Gateway HTTP API zwpcbivmsj with JWT authorizer cognito-dashboard (id enxa26), frames bucket frames-armyworm-366356442579.

### 4.3 Architecture: vanilla JS ES modules, no framework

The app is deliberately framework-free. It is plain ES modules (`<script type="module">` with `import/export`), no bundler, no build step. Any static HTTP server can serve it, which is what makes the S3 + CloudFront hosting trivially reproducible. This is a design decision, not a shortcut: the whole deploy is one `aws s3 sync`.

Rendering model: each tab render function writes an HTML string into `#page-content` via `innerHTML`. Interactive elements use inline handlers (`onclick="switchTab('live')"`). Inline handlers resolve names on `window`, and module scope is not `window` scope, so `main.js` ends with one `Object.assign(window, {...})` block that exposes exactly the inline-handler surface. Everything else stays module-private. If a new inline `onclick` is added anywhere, its function must be added to that block too, or the click throws “X is not defined”. This is the module split’s contract; do not regress to flat `<script src>` tags, which would put everything back in global scope.

Two CDN globals load before the module graph: `window.Chart` (chart.js 4.4.1) and `window.Hls` (hls.js 1.4.12), both from `jsdelivr` in `index.html`.

Boot sequence (`main.js init()`, verified in code): 1. `renderTabs()` paints the tab bar. 2. `imageCache.open()` opens `IndexedDB` early so the first gallery render is instant. 3. `initAuthUi(startApp)` wires the login form. 4. If `isLoggedIn()`, `startApp()` runs



immediately; otherwise `requireLogin()` opens the login overlay. 5. `startApp()` fetches `/settings`, `/model/status`, and `/stream/status`, then `switchTab(state.tab)` (default tab: `live`).

`switchTab(id)` also does teardown: destroys the HLS player when leaving Live, destroys Chart.js instances when leaving Analytics, and stops model-status polling when leaving Settings.

NOTE: doc lag. The module list in `docs/dashboard.md` names a `js/cameras.js`. No such file exists. Camera cards live in `js/settings.js` (Settings → Cameras sub-tab) and the live-view camera cards live in `js/live.js`.

## 4.4 config.js

One exported object, `CONFIG`. This is the only file to touch when moving hosts or accounts. The repository copy still carries the development-era values; the deployer templates the production values in at deploy time (explained below the table). Verified in code:

| Key                                 | Value                                                               | Meaning                                                                                      |
|-------------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>HTTP_API</code>               | <code>https://zwpcbivmsj.execute-api.us-east-1.amazonaws.com</code> | API Gateway HTTP API base URL.                                                               |
| <code>COGNITO_REGION</code>         | <code>us-east-1</code>                                              | Region of the user pool.                                                                     |
| <code>COGNITO_CLIENT_ID</code>      | <code>4husu6afr835e235eu9dqp8av6</code>                             | App client id. This is a public identifier, not a secret.                                    |
| <code>SG_OFFSET_HOURS</code>        | <code>8</code>                                                      | Singapore is UTC+8. Backend stores UTC; all UI shows SGT.                                    |
| <code>MODEL_POLL_INTERVAL_MS</code> | <code>5000</code>                                                   | Custom-model status poll period while Settings → Cameras is open.                            |
| <code>CACHE_DB / CACHE_STORE</code> | <code>pest-monitor-cache / images</code>                            | IndexedDB database and object store names.                                                   |
| <code>CACHE_MAX_BYTES</code>        | <code>400 MB</code>                                                 | Image cache ceiling (bumped from 200 MB in v5.3 when the gallery fetch went to 500 records). |

A `WS_URL` used to exist; WebSocket was removed in v3.7 and the dashboard polls instead.

To reproduce on a new AWS account: change `HTTP_API`, `COGNITO_REGION`, and `COGNITO_CLIENT_ID` here and nothing else in the frontend.

The values in the table above are the development account's; the repository copy of `config.js` keeps them. On the production account the ARGUS deployer templates the file

at deploy time: `deployer/deploy.py` (stage “writeback”) regex-replaces `HTTP_API`, `COGNITO_REGION`, and `COGNITO_CLIENT_ID` in memory and uploads the rewritten copy to `argus-dashboard-506868652945`, leaving the repo file untouched. Verified through the CDN 2026-08-11: the deployed production `config.js` carries `HTTP_API`: `'https://vzf17s6z00.execute-api.us-east-1.amazonaws.com'` and `COGNITO_CLIENT_ID`: `'6vebotf45bp8u46cnraddiaplv'`. Cosmetic stale comment: the templating rewrites only the values, so the comment above those lines still names the old pool id `us-east-1_ea0aJdus1`. The values are correct; do not let the comment mislead you about which pool the production deployment uses (`us-east-1_9selFDHpc`).

## 4.5 auth.js — Cognito sign-in

Purpose: sign-in against the Cognito IDP endpoint with raw fetch, no AWS SDK. This fits the bundler-free setup. Tokens live in `localStorage` under the key `pest-monitor-auth`. Verified in code end to end.

Key functions:

- `idpCall(target, body)`: POST to `https://cognito-idp.us-east-1.amazonaws.com/` with header `X-Amz-Target: AWSCognitoIdentityProviderService.<target>` and content type `application/x-amz-json-1.1`. Non-OK responses throw an `Error` carrying the Cognito `__type` as `err.code`.
- `login(email, password)`: `InitiateAuth` with `AuthFlow: USER_PASSWORD_AUTH`. If Cognito returns any `ChallengeName`, the function throws “Account needs a password reset (<ChallengeName>) — ask the admin.”, with the actual challenge name interpolated. This is deliberate: accounts are admin-created with permanent passwords (`admin-set-user-password --permanent`), so the login UI intentionally does not implement the `FORCE_CHANGE_PASSWORD` challenge flow.
- `storeAuthResult(r, email)`: saves `{idToken, refreshToken, expiresAt, email}`. Cognito omits `RefreshToken` on refresh responses, so the previous refresh token is kept.
- `refreshSession()`: `InitiateAuth` with `AuthFlow: REFRESH_TOKEN_AUTH`. Returns `false` on any failure.
- `isLoggedIn()`: `true` if a refresh token exists, or an unexpired ID token exists.
- `getIdToken()`: returns a valid ID token or `null`. It refreshes proactively when the token is within 5 minutes of expiry. Every API call goes through this, so refresh is automatic and invisible.
- `logout()`: clears the stored tokens (removes only the `pest-monitor-auth` key; the IndexedDB image cache survives) and reopens the login overlay. Wired to the topbar “Sign out” button as `logoutClick`.
- `requireLogin()` / `hideLogin()`: toggle the `#login-screen` overlay. `requireLogin` is also called by `api.js` on any 401, so an expired or revoked session re-gates the app mid-use.

- `initAuthUi(onLogin)`: wires the login form submit. On success it blanks the password field, hides the overlay, and calls `onLogin` (which is `startApp`). Cognito error messages show inline in `#login-error`.

Token facts, production pool `us-east-1_9selfDHpc` (created by the ARGUS deployer; pool name `pest-dashboard-users`, the same name as the retired development pool it reproduces): 12-hour ID tokens, 30-day refresh tokens, sign-in by email, self-signup disabled, admin-create-only, app client has no secret. Verified in `deployer/deploy.py` stage 7: `IdTokenValidity=12 hours`, `RefreshTokenValidity=30 days`, `AllowAdminCreateUserOnly`, `GenerateSecret=False`, auth flows `USER_PASSWORD_AUTH` + refresh. Passwords are set permanent via `admin-set-user-password --permanent`; the first production sign-in user was created and confirmed 2026-08-11.

Security note: no password, secret key, or token appears anywhere in the repository for this flow. The client id and pool id are public identifiers by design.

## 4.6 `api.js` — the API client

Every backend call goes through `api._fetch(path, opts)`. Verified in code:

1. Builds headers with `Content-Type: application/json`.
2. `await getIdToken()`; if a token comes back, adds `Authorization: Bearer <IdToken>`.
3. Fetches `CONFIG.HTTP_API + path`. Parses the body as JSON, falling back to `{raw: text}`.
4. If status is 401: calls `requireLogin()` and throws “Signed out — please sign in again”.
5. Any other non-OK status throws `data.error` or `HTTP <status>`.

The full endpoint surface (all verified in code):

| Method call                                                               | HTTP request                                                                                               |
|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>getSettings()</code>                                                | GET /settings                                                                                              |
| <code>postSettings(body)</code>                                           | POST /settings                                                                                             |
| <code>getHistory(filters)</code>                                          | GET /history?... (filters become query params; empty values dropped)                                       |
| <code>getPresignedUrl(key)</code>                                         | GET /presigned-url?key=...&method=GET                                                                      |
| <code>getPresignedUploadUrl(key)</code>                                   | GET /presigned-url?key=...&method=PUT<br>(backend actually returns a presigned POST; see 4.11 Test upload) |
| <code>getModelStatus()</code>                                             | GET /model/status                                                                                          |
| <code>startModel(camera_id) /</code><br><code>stopModel(camera_id)</code> | POST /model/start / POST /model/stop                                                                       |
| <code>getCost(days)</code>                                                | GET /cost?days=N (dormant Costs tab)                                                                       |

| Method call                                                               | HTTP request                                                                                                                     |
|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>getIdentities() / addIdentity(email) / removeIdentity(email)</code> | GET / POST / DELETE /identities                                                                                                  |
| <code>getSchedule() / postSchedule(body) / deleteSchedule(camera)</code>  | GET / POST / DELETE /schedule                                                                                                    |
| <code>getScheduleLogs(limit)</code>                                       | GET /schedule-logs?limit=N                                                                                                       |
| <code>getVideoPlayback(stream)</code>                                     | GET /video-playback?stream=<kvs_stream_name>                                                                                     |
| <code>getStreamStatus(camera)</code>                                      | GET /stream/status (the one route with no JWT authorizer; the Jetson Orin and mini-PC kvs_controller.py poll it unauthenticated) |
| <code>startStream(camera_id) / stopStream(camera_id)</code>               | POST /stream/start / POST /stream/stop                                                                                           |
| <code>verifyDetection(image_id, bbox_index, verdict)</code>               | POST /detection/verify with verdict of "TP", "FP", or null (null clears that box's verdict)                                      |
| <code>deleteDetection(image_id)</code>                                    | DELETE /detection?image_id=... (route glwyqo0, JWT-gated, stays gated even during the emergency rollback)                        |

## 4.7 state.js — shared state and the IndexedDB image cache

state is one exported mutable object; modules import and mutate it directly. Notable fields (all verified in code): tab, settingsSub, cameras, schedules, cameraOps, modelStatuses, streamStatuses, galleryFilters (default limit 500), galleryItems, galleryPage, notifs (capped at 50), hls, liveCam, chart handles, modelPollTimer, db, uploadFile, uploadCam, and verifyMap ({image\_id: {bbox\_index: 'TP' | 'FP'}}), the per-box verdict store).

imageCache is a persistent thumbnail cache in IndexedDB so a refresh does not re-fetch every presigned URL. Key = the S3 object key. Value = {key, blob, mime, fetched\_at, size}. Verified per function:

- `open()`: opens database pest-monitor-cache version 1. On first open (onupgradeneeded) it creates the images object store keyed by key with a fetched\_at index. The handle is memoized on state.db, so repeat calls cost nothing.
- `get(key) / put(key, blob) / remove(key) / clear()`: one-transaction wrappers. Each wraps the IndexedDB work in try/catch and resolves null or false on failure instead of rejecting, so a broken cache (private browsing, quota exhaustion) can never take the UI down.
- `fetchAndCache(key, presignedUrl)`: cache-first. On a hit it returns an object URL for the stored blob. On a miss it fetches the presigned URL, stores the blob, and

returns an object URL for it. On any fetch failure it returns the presigned URL itself, so the image still displays, just uncached.

- `evictIfFull()`: reads all entries and sums size. Under `CACHE_MAX_BYTES` (400 MB) it returns. Over the cap it sorts by `fetch_at` ascending (LRU) and deletes oldest-first until the freed bytes exceed `total - 0.7 * cap`, which lands usage at about 70% of the cap. It runs once per gallery page, after that page's thumbnails finish loading (end of `loadThumbsForIndices`), not on every put.
- `size()`: returns `{count, bytes}`; feeds the summary bar's cache line.

All cache operations are best-effort; every failure path falls back to direct presigned URLs.

## 4.8 utils.js — helpers

All verified in code:

- `toast(msg, kind, ms)` and `refreshToastFan()`: see 4.14.
- Time helpers: `_parseUtc` treats DynamoDB's "YYYY-MM-DD HH:MM:SS" strings as UTC (appends Z). `fmtTime`, `fmtTimeShort`, `fmtDate` all render with `timeZone: 'Asia/Singapore'`. `todayYMD` and `daysAgoYMD` compute Singapore dates for filter defaults.
- `escapeHtml(s)`: used on every interpolated string in every module's HTML templates.
- `convertToJpg(file, quality=0.92)`: canvas-based conversion of any browser-decodable image (PNG, HEIC, WebP, BMP, GIF) to JPG, transparency flattened onto white, filename rewritten to `.jpg`. Pass-through for JPEGs. Used by Test upload.
- Camera classifiers: `isManualCamera` (`manual_upload`), `isTestCamera` (`person_cam`), `needsKvs`, `needsSchedule` (only custom-model cameras; their Rekognition Custom Labels endpoints cost about \$4/hour), `isWormCam` (id starts with `worm` or `armyworm`), `camAvatarClass`, `camInitial`.
- `statusBadge(status)`: maps model states to badges. `RUNNING` shows "Live", `STOPPED` shows "Standby", `TRAINING_COMPLETED` shows "Ready", `NOT_CONFIGURED` shows "Free mode", `NOT_FOUND` shows "Setup incomplete".
- `camDisplayName(id)`: the static map that hides internal camera ids from users. Verified mapping: `worm_cam` → "Worm Cam", `moth_cam` → "Moth Cam", `armyworm_go2_a8mini` → "Worm Cam" (pre-migration id), `moth_cam_01` → "Moth Cam" (pre-migration id), `wilbur-fyp-project` → "Moth Cam" (Wilbur-era legacy records), `manual_upload` → "Test upload". Unknown ids fall back to the raw id. The map is static on purpose: it must cover legacy record ids that no longer exist in the cameras table (the `camera_id` migration rewrote camera rows but left old S3 keys and legacy rows in place).
- `cleanLabel(label)`: strips a trailing parenthetical from camera labels for display only.

- `chartOpts(xLabel, yLabel)`: the shared Chart.js options factory used by analytics and costs.

## 4.9 Gallery (gallery.js + modal.js)

### Page layout and filters

`renderGalleryPage()` writes: summary bar, card grid, pagination bar, then the filter bar below the grid (v5.1 layout decision: images first). Filters: date range preset (Today / 3 / 7 / 14 / 30 days / All time), pest type (from each camera's `target_label`), free-text zone, camera, and detected-only/clear-only. `applyGalleryFilters()` converts the range preset into `date_from/date_to` (Singapore dates) and reloads. All filtering is server-side via GET /history; the fetch limit is 500 records.

### Load path

`loadGallery()` fetches, stores resp. items in `state.galleryItems`, resets to page 1, then hydrates `state.verifyMap` from every fetched record before drawing. That last step is the fix for the historical X-dismiss bug: v3\_9 only hydrated verdicts on the analytics/settings paths, so a fresh load straight into Gallery had an empty `verifyMap` and every dismissed false positive reappeared. Verified present in code (`for (const it of state.galleryItems) hydrateVerifyMap(it);`).

`updateGallerySummary(resp)` builds the summary bar. Per-pest chips count verifiable boxes, not photos: `getVerifiableBoxes` per record, summed by `target_label` (the v3.6.2 counting rule). The record count carries a “most recent 500 — narrow the date range for older” hint whenever the fetch limit was hit. The cache line shows `imageCache.size()` as count + MB with a Clear button; `clearImageCache()` asks for a native confirm, clears the store, and reloads the gallery. After a delete, the bar is recomputed locally from the module-level `lastScanned` counter instead of re-fetching.

### Pagination (v5.3)

Client-side, 50 cards per page (`PAGE_SIZE = 50`). The backend returns up to 500 most-recent matching records already sorted; `renderGalleryGrid()` slices `state.galleryItems` by `state.galleryPage`. Cards keep their global index (`start + i`) so `openImg`, `deleteGalleryItem`, and thumbnail loaders always resolve against `state.galleryItems` no matter which page shows. `renderPagination()` builds prev/next, windowed page numbers (first, last, current's neighbors, with ellipsis gaps), and a jump-to-page input. `gotoGalleryPage(p)` clamps, re-renders, and scrolls the grid into view. Records older than the most-recent 500 are reached by narrowing the date filter (server-side).

### Thumbnail loading

`loadThumbsForIndices(indices)` loads only the current page's thumbnails, 4 at a time (a worker pool over a shared cursor). Per record, key selection: if the record has drawable

boxes, load the original frame (`original_image_key || image_id`) and draw boxes client-side; legacy records without bboxes fall back to the baked `processed_image_key`.

`loadOneThumb` order of attempts, verified in code:

1. IndexedDB cache hit: instant object URL.
2. Cache miss: GET `/presigned-url`, then fetch the blob once over CORS, put it in the cache, and display from that same blob. A single CORS fetch (not `<img src>` plus a second fetch) avoids the browser caching an opaque no-cors response that the cache fetch then cannot read.
3. 403/404 on the object: switch to the processed-image fallback with box drawing disabled (boxes are already baked into that image; drawing again would double them).
4. Other failures: up to 3 attempts with escalating 800 ms backoff (800 ms x attempt number, so 800 ms then 1600 ms), then a “Retry” button on the card (`showRetryButton / retryThumb`).

Stale-closure guard (v4.2, verified): a delete re-renders the grid and shifts indices, so an in-flight retry’s index can point at a different record. `loadOneThumb` proceeds only if the record at its index still owns the key it was started for.

## Gallery cards and badges

`galleryCard(it, idx)` shows the thumb, a bbox overlay div, a pest badge (“Armyworm-larva · 3” style, count appended when more than one box), zone + time footer, the camera display name (via `camDisplayName`), and a Delete button. Cards show no confidence percentage: every surviving box already passed the processor’s server-side gate (v4.5 decision).

Verification badges: `aggregateVerdict(it)` rolls per-box verdicts into one corner badge. States: `n/total` (partially reviewed), `✓` (all confirmed), `X` (all false positives), `n✓/mX` (mixed). Unreviewed records show no badge.

LLM verifier display (v5.3, verified in code): the detection modal’s meta pane has a “Verifier” row showing which multimodal model judged the boxes on that frame. `modal.js llmVerifierName(rec)` reads the record field `llm_verify_model` (written by pest-detection-processor on every row so a model A/B stays readable later), then strips the region prefix (`us. / eu. / apac. / global.`), the trailing `-vN:M` version suffix, and the vendor prefix (`anthropic. / amazon. / meta. / mistral. / google. / qwen.`) for display. Records without the field (legacy, or cameras with LLM verify off) show a dash. That row is the whole LLM display surface: there are no per-box LLM verdict badges in the gallery, overlay, or review list (the v4.5 UI simplification removed them; a box present in the record is already a confirmed detection). No field named `verified_by_llm` is read anywhere in the frontend. The backend does write more: pest-detection-processor adds per-box `verified_by_llm` (“true”/“false”) and `llm_reason` fields on boxes the LLM verifier judged, alongside the



per-record `llm_verify_model`. The UI ignores both per-box fields; only `llm_verify_model` is read.

## The detection modal (`modal.js`)

`openImg(idx)` opens `#image-modal` with two panes. The meta pane shows Camera (display name), Zone, Target, Detected, Model (`model_type`), Verifier (see above), Labels count, Time, and Model version (a testing aid, not a final feature: the version segment regex-parsed out of the stored `model_arn`; legacy records show a dash). Below that: the review block and a Delete button.

Image pane: same cache-first, blob-once load path as thumbnails, then an overlay div (`.bbox-overlay`) tagged with the record's `image_id`. `positionOverlay()` sizes the overlay to exactly the rendered image rect using `offsetLeft/Top/Width/Height` (these ignore CSS transforms, so it is the untransformed base box); a window resize listener re-runs it, and `closeImageModal` removes that listener together with the zoom listeners through the stored state `._imgZoomCleanup`. If the original frame is missing, an error handler swaps to the processed image and blanks the overlay (using `Fallback`), since that image has baked boxes; the handler also fires immediately when a cached `<img>` has already failed (`imgEl.complete && imgEl.naturalWidth === 0`). A zoom hint (“scroll · pinch · dbl-click to zoom”) shows for 2.5 s.

Review block (`renderReviewBlock`): one row per drawable box, sorted by confidence descending, so row order matches the visual prominence of the boxes and the index base matches `verifyMap`. Each row shows #n, species name, confidence to one decimal, and one X flag button; a “flagged” counter sits in the block header. The counting model is opt-out: a detection counts unless a human marks it FP. CAG staff only flag false positives; they never have to confirm true ones. `refreshOpenModalReview(image_id)` re-renders just the review block and the overlay on a verdict change, without re-fetching the image.

Close paths: the X button, clicking the backdrop, or Escape. All route through `closeImageModal()`, which also detaches the window-level zoom listeners.

## Two-step delete flow (v4.2)

Step one is the Delete button (on the card, with `event.stopPropagation()`, or in the modal). Step two is a native `confirm()` dialog that states exactly what happens: removes the DynamoDB record, removes the stored S3 image, cannot be undone. Only then does `deleteGalleryItem(idx)` call `api.deleteDetection(it.image_id)` (DELETE `/detection?image_id=`).

On success, the frontend evicts every cached image variant from IndexedDB, drops every card sharing that `image_id` (the API deletes all rows for the id, and duplicates share the PK), re-renders the grid, and recomputes the summary bar locally from `lastScanned`. The modal closes only if the delete succeeded. Server-side safety (from `docs/aws.md` and `docs/dashboard.md`, E2E-tested 2026-07-07): 404 with nothing touched if no record exists;



frame deletes gated to the frames/ prefix in code and IAM; any S3 failure returns 500 with rows kept, so the operation is retry-safe.

## 4.10 Bounding boxes and verification (bbox.js)

NOTE: doc lag. docs/dashboard.md calls this “drawing boxes on a canvas overlay”. The implementation is not a `<canvas>`; it is absolutely-positioned `<div>` elements inside an overlay container, with box geometry expressed as percentages. The only real canvases are Chart.js charts and the `convertToJpg` helper. The behavior described in the docs is otherwise accurate.

### Box extraction (two sources, verified)

- `getVerifiableBoxes(it)`: parses `it.labels` (JSON), keeps entries whose name equals the record’s `target_label`, sorts by confidence descending. This is the counting basis for the summary chips, analytics, and `verifyMap` indices (`bbox_0` = highest confidence).
- `getDrawableBoxes(it)`: parses `it.bboxes` (the structured coordinates the processor writes: `left`, `top`, `width`, `height`, all normalized 0-1), drops zero-area boxes, sorts by confidence descending. This is the geometry source for the modal overlay, card overlays, and the review block.

Standing rule (introduced in v4.5, still true under the v6.3 processor): neither function re-filters by any camera threshold. The processor’s server-side gate already decided which boxes survive before the record was written — in the current pipeline, Rekognition proposes candidates at a low floor (`min_confidence` 10), the LLM verifier judges every box, and only boxes that pass verification and the post-verify floor (`post_verify_floor`; live production value 33, verified against the production camera row 2026-08-14) are written. A box present in the record is a confirmed detection. Re-filtering client-side would wrongly hide a legitimate detection the gate confirmed, and raising the threshold later would retroactively hide old confirmed records.

### Coordinate scaling

Modal overlay (`renderOverlayBoxes` + `modal.js` positioning): the overlay div is sized to the rendered image’s untransformed rect. Each box is a child div at `left: left*100%; top: top*100%; width: width*100%; height: height*100%`. Because the geometry is percentage-based, boxes stay correct at any modal size. During zoom/pan, `attachImageZoom` applies the identical CSS transform string to the image and the overlay, so boxes track the image exactly. Since v5.4 it also writes the live scale onto the overlay as the custom property `--z` (`overlay.style.setProperty('--z', scale)`), which is what keeps the box chrome from growing with the zoom — see below.

Card overlay (`paintCardBoxes`): the thumb uses `object-fit: cover` on a 4:3 box, so the frame is scaled to fill and center-cropped. The algorithm reconstructs that cover rect: `scale = max(thumbW/naturalW, thumbH/naturalH)`, display size `dw = naturalW*scale`,

`dh = naturalH*scale, overlay offset ((thumbW-dw)/2, (thumbH-dh)/2)`. The overlay may overflow the thumb; the thumb's overflow: hidden clips it. Boxes are again percentage-positioned children, with no labels on cards. Boxes flagged FP are skipped so the card matches the modal.

Each modal overlay box carries a label with the species name and the confidence rounded to a whole percent, plus a  $\times$  button ("Flag false positive — removes this box"). NOTE: code-comment lag. The v4.5 comment above `renderOverlayBoxes` says "plain box + species name only — no confidence"; the rendered template actually includes `${b.confVal.toFixed(0)}%`. The rendered output is the ground truth here.

## Verification logic

- `verifyClick(image_id, bbox_index, verdict)`: toggle semantics (clicking the active verdict clears it, sending null). Optimistic update of state.`verifyMap`, immediate re-render of grid and modal review block, then `POST /detection/verify`. On backend failure the previous verdict state is restored and an error toast shows. Verified in code.
- `hydrateVerifyMap(it)`: reads the record's verifications map (object or stringified JSON) into `verifyMap`. Legacy records with a per-image verified boolean are migrated: true projects TP onto every box, false projects FP. Local in-memory edits always win; hydrate never overwrites an existing entry, so unsaved clicks are not clobbered by a refetch.
- `getCountedBoxes(it)`: the opt-out counting rule used by every analytics widget: a box counts unless flagged FP. TP and unreviewed both count.
- `redrawModalOverlay(image_id)`: swaps only the overlay's innerHTML, leaving the image element and its zoom transform untouched.

## Image zoom (`attachImageZoom`)

Scale range 1x to 6x. Wheel zooms toward the cursor (factors 0.85 out, 1.18 in); `zoomToward` keeps the screen point under the cursor fixed by adjusting the translation by the scale ratio. Double-click toggles 1x and 2.5x. Drag pans when zoomed, clamped to image bounds (`max offset = baseSize*(scale-1)/2`). Touch: two-finger pinch toward the pinch center, one-finger pan. Returns a cleanup function that removes the window-level `mousemove`/`mouseup` listeners; element listeners die with the element.

**Zoom-aware box chrome (v5.4, 2026-08-15).** The overlay carries the same CSS transform as the image, so before this change every decoration scaled with the zoom: a 4 px border rendered as 24 px at 6x, and the per-box flag button grew six-fold. On a small larva the chrome covered the animal the operator had zoomed in to inspect. The fix keeps the geometry in percentages, as before, but divides every decoration by the published scale. In `styles.css`:

```
.bbox-box { border: calc(2.4px / var(--z)) solid rgba(255, 60, 60, 0.95); }
.bbox-label { font-size: calc(15px / var(--z)); padding: calc(3px / var(--z)) }
```

```

    calc(7px / var(--z)); }
.bbox-x { width: calc(20px / var(--z)); height: calc(20px / var(--z));
          top: calc(-11px / var(--z)); right: calc(-11px / var(--z)); }

```

.bbox-overlay declares --z: 1 as the fallback, so an overlay that is never zoomed still renders correctly. The result is that the outline and the flag button hold a constant on-screen size at any zoom while the target keeps growing. The base border was also thinned from 4 px to 2.4 px in the same change.

## 4.11 Cameras and settings (settings.js)

renderSettingsPage() loads settings, model statuses, schedules, and the last 500 history records (to compute per-camera “last detection” and “today count”, counted in boxes, not photos), then renders four sub-tabs: Cameras, Test upload, Schedules, Alerts. The former Global sub-tab was removed; a remembered state.settingsSub of 'global' redirects to Alerts (verified in code). saveGlobal (email alerts on/off plus primary recipient) is still exported but no rendered markup produces its input elements any more; it is vestigial.

### Cameras sub-tab

One card per deployed camera (manual\_upload is excluded; it has its own Test upload sub-tab). Card order: custom-model worm cameras first, other custom cameras, then free-mode cameras. Each card (all verified in code):

- Header: avatar, display label, status badge and Start/Stop buttons (cameraStatusActionsHtml). Start/Stop only render for custom-model cameras whose custom\_model\_arn is set and not a REPLACE\_ placeholder. Buttons disable during STARTING/STOPPING.
- Status hint line (cameraStatusHintHtml): “Starting up · ready in ~5 min”, “Detection live”, “Idle”, etc.
- Operational tier: Last detection time, Today’s box count, and an Auto-schedule quick toggle. The toggle locks during model state transitions and is only enabled for cameras where needsSchedule is true (custom model, since endpoints cost about \$4/hour).
- Detection settings (collapsible):
  - AI filter threshold: a number input (0-100, step 1) bound to the camera field post\_verify\_floor — the confidence floor applied after the LLM verification step decides which boxes are written. It auto-saves through debouncedSaveCamera (500 ms debounce) into POST /settings; after saving, the code re-fetches settings and verifies each field actually changed (the verify loop compares post\_verify\_floor numerically), showing a mismatch toast if not. The input’s markup fallback is 33 (cam.post\_verify\_floor ?? 33). Live production value: **33**. History: 49 was the value decided and flipped live 2026-08-10; the retrained production model scores the same images lower, so the floor was refitted per-build to 33 on 2026-08-13 (camera row +

Lambda env together). The floor is a per-build fit, not a universal constant. The low Rekognition candidate floor (`min_confidence`, 10 in production) is no longer exposed in this UI; the user-facing threshold is the post-verify floor.

- AI model: a per-camera selector bound to `llm_model_id` — Claude Sonnet 4.6 (\$20 / 1M tokens) or Claude Haiku 4.5 (\$5 / 1M tokens) — choosing which model verifies every detection on this camera (a v6.3 processor feature). Rendered only for custom-model cameras; saved through the same debounced path.
- Zoom scan toggle: only rendered for custom-model cameras. The UI label is “Zoom scan — Split & zoom in to catch small pests”; the persisted field is `tiling_enabled` (live production value: true, alongside `llm_verify_enabled` true on the same row). `toggleTiling(id, enabled)` saves it via `POST /settings` and updates local state; on failure the checkbox reverts. Cloud-side, the flag makes pest-detection-processor tile that camera’s frames (grid split + per-tile upscale + per-tile detect + NMS). It is the same camera, just a switch, not a separate camera. Verified in code.

The write path behind those inputs (`saveCameraSettings(id)`, verified in code): it harvests whichever inputs exist in the DOM by id — `cam-{id}-label`, `-target`, `-conf` (parsed to int as `post_verify_floor`), `-llm`, `-waypoint`, `-kvs`. An absent input leaves that field out of the POST entirely, so the one function serves every card variant. It then sends `POST /settings` with `{camera_id, fields}`, re-fetches `GET /settings`, and compares each sent field against the fresh copy: numeric compare for `min_confidence` / `post_verify_floor`, string compare for the rest; empty and null sent values are skipped. On a full match the save indicator flashes “ok” for 2 s and a toast counts the saved fields. On a mismatch the indicator shows the error state and a toast names the mismatched fields. Any thrown error (network, 401) lands in the same error state. The debounce wrapper keeps one timer per camera id (`_saveDebounceTimers`), so edits on two cameras cannot cancel each other’s saves.

Model start/stop: `modelStart(id)` first shows a confirm dialog stating the roughly \$4/hour cost and the 5-10 minute STARTING window, then `POST /model/start`, optimistic status patch, and polling resumes. `modelStop(id)` mirrors it.

Model status polling: `startModelPolling()` runs only while Settings → Cameras is open and at least one camera has a real custom model ARN. Every 5 s it fetches `/model/status`; on change it calls `patchCameraStatusUI()`, a surgical DOM update that only touches the status badge, buttons, hint, and schedule-toggle lock. User-edited inputs are never re-rendered mid-edit. Transition toasts fire on RUNNING, STOPPED, and FAILED. `stopModelPolling()` is called on any tab or sub-tab change.

## Test upload sub-tab

This is the panel that demonstrates live detection at the final presentation (there are no worms at Jewel to stage). Drag-drop or browse an image, pick which camera config (and so

which detection model) to run it with, pick which AI verification model judges the boxes for this run (Claude Sonnet 4.6 or Haiku 4.5, with per-model cost shown), then submit. `uploadSubMarkup` self-heals a stale remembered camera: if `state.uploadCam` no longer exists (for example after a `camera_id` migration), it falls back to the first deployed camera. The pipeline, verified in code:

1. Non-JPEG files convert to JPG in the browser (`convertToJpg`). 20 MB pre-conversion cap; non-image MIME types rejected.
2. The Run button gates on model readiness (`uploadModelReady`): custom-model cameras require RUNNING status; free-detection cameras are always ready. The hint line re-fetches status so the gate is accurate even if the Cameras tab was never opened this session.
3. Key construction: `frames/{camera_id}/{waypoint}/{timestamp}_{safeName}`, where the waypoint segment is `manual_test__conf10__11m-{model}`. The extension comes from the (possibly converted) filename: `convertToJpg` rewrites non-JPEG names to `.jpg`, while pass-through JPEGs keep their original extension. `conf10` pins the Rekognition candidate floor at 10 (the validated production candidate floor) for this run; `__11m-sonnet46` or `__11m-haiku45` picks the verification model for this run only. The S3-triggered processor Lambda parses those suffixes and applies the overrides for this one detection run, with no DynamoDB mutation. The earlier user-facing min-confidence override input is gone; the per-run choice the operator makes now is the verification model. NOTE: code-comment lag. The comment in `settings.js` calls that Lambda `image-detection-handler`; the deployed function is `pest-detection-processor` (`docs/aws.md`).
4. `GET /presigned-url?method=PUT` returns a presigned POST (`{url, fields, method: "POST"}`); the file uploads direct to S3 via `FormData` with the file as the last field. A legacy presigned PUT path is kept as fallback.
5. `waitForDetection(key)` polls `GET /history?limit=10` every 3 s (first poll after 2 s) for up to 60 s until a record with that exact `image_id` appears, then rejects with a timeout error pointing at CloudWatch. On a match, `renderUploadResult` draws the result inline: the original frame with drawn boxes (minus any boxes already flagged FP in `verifyMap`; the baked processed image for legacy records without box geometry), a detected/clear badge, the raw confidence, and the returned label count.

Gotcha (hit and fixed 2026-07-07, on the development stack): the browser posts the file straight to S3, so the frames bucket must list the dashboard origin in its CORS AllowedOrigins. On production the deployer's writeback stage already set the rule on `argus-frames-506868652945` to the website and CloudFront origins. "Failed to fetch" on Test upload from any new origin (for example a localhost dev server) means: add that origin. Console: S3 → Buckets → `argus-frames-506868652945` → Permissions → Cross-origin resource sharing (CORS) → Edit. CLI: `aws s3api put-bucket-cors --bucket argus-frames-506868652945 --cors-configuration file://cors.json --profile prod`.

## Schedules sub-tab

One schedule card per custom-model camera: enable toggle, start time, active-day chips with “Every day” / “Weekdays” presets, Save and Delete. Data-source nuance, verified in code: the schedule card reads `cam.schedule` (the copy embedded in the camera row by `GET /settings`), while the quick toggle on the camera card reads `state.schedules` from `GET /schedule`; both write through `POST /schedule`. There is no stop time any more: the card’s helper text states the contract — the schedule starts the model, one detection round runs, and the model is shut down automatically. The auto-close is not the schedule’s doing: the pest-model-watchdog Lambda (the v6.2 per-camera build, which reads each camera’s `max_runtime_min` — 45 for `worm_cam` — instead of one global cap) runs on a 15-minute EventBridge schedule and stops any endpoint past its budget. The ARGUS deployer installed exactly this build on production. `saveSchedule` posts `{camera_id, enabled, start_time, days}` to `POST /schedule`. Below the cards, an execution log table shows the last 30 automated start/stop events from `GET /schedule-logs`. The quick toggle (`toggleScheduleQuick`) reuses an existing schedule’s values or defaults to 05:40 every day (matching the production camera’s schedule, stored at 05:40 daily but currently disabled); on a failed POST it flips the checkbox back.

## Alerts sub-tab

Email subscriber management over SES identities: list with verification status badges, Add (sends the SES verification email), Resend for pending addresses (same idempotent backend call), Remove (blocked for the primary address), and a Refresh status button. An info panel explains the SES sandbox verification flow: production SES access removes the recipient-verification requirement. SES on the production account 506868652945 is in sandbox mode, so every recipient must be individually verified. The primary alert address was verified on 2026-08-11, so alerts reach that one address; any further subscriber must verify through this same sub-tab. (The retired development account was sandboxed too — not a migration regression.)

## Removed: the Global sub-tab

Earlier builds had a fifth Global sub-tab (email alerts on/off, primary recipient, auto-capture, capture interval). It was removed; its `state.settingsSub` value now redirects to Alerts. The exported `saveGlobal` function survives in `settings.js` but nothing renders its inputs, so it is dead UI code. Global settings still exist backend-side (`POST /settings` with a `global` body); they are just no longer editable from this dashboard build.

## 4.12 Live view (live.js)

`renderLivePage()` lists every camera that has a `kvs_stream_name`, defaulting the selection to the worm camera (its KVS stream `armyworm-cam-stream` is the primary one; `moth-cam-stream` is parked — stream names are unchanged across the account migration). The stream-status fetch inside it is best-effort: a failure only logs a warning and the page still renders. `selectLiveCam(id, ev)` ignores clicks that originate on the KVS toggle

(`ev.target.closest('.toggle')`), sets `state.liveCam`, and re-renders the page. Each camera card has a KVS push toggle: `toggleStream` calls `POST /stream/start` or `/stream/stop`, which sets the desired state; the GStreamer producer on the device (Jetson Orin or mini PC) picks it up on its next poll of `GET /stream/status`. On success `toggleStream` patches `state.streamStatuses` and re-renders; on failure it toasts and re-renders too, which redraws the checkbox from the last known state — the failed flip reverts itself. The status route is the only unauthenticated route in the API, precisely so the devices need no tokens.

Playback path, verified in code: `loadStream()` calls `api.getVideoPlayback(kvs_stream_name)` (`GET /video-playback?stream=...`), which returns `{hls_url}`, a Kinesis Video Streams HLS session URL. That route is served by the `kvs-hls-handler` Lambda (see Chapter 2; its existence is confirmed by the deployer audit `deployer/audit/lambda__kvs-hls-handler.json`). `playHls` destroys any prior player, then either attaches `hls.js` (new `Hls({lowLatencyMode: true})`) or, on Safari-class browsers with native HLS, sets `video.src` directly. Errors render an overlay telling the operator to toggle the stream on and check the producer. NOTE: doc lag. The Lambda list in `docs/aws.md` does not mention `kvs-hls-handler`.

Production-account status: the 2026-08-10 deploy ran without `--live-view`, so the deployer created no KVS stream resources. The production `worm_cam` row does carry `kvs_stream_name` `armyworm-cam-stream` (verified against the production DynamoDB row 2026-08-14) with `stream_enabled` `false`, so the Live tab lists the camera with its toggle off. Live playback stays dark until the stream resource exists on the production account and the device-side producers run with production credentials: the repo mirrors of `robot/kvs_controller.py` and `minipc/kvs_controller.py` already default to the `vzf17s6z00` API (repointed 2026-08-13), but the Jetson Orin and mini PC still carry development-account keys, and the on-device sync is pending the hardware's return. Known open item of the migration, not a defect.

## 4.13 Analytics (`analytics.js`)

All widgets count bounding boxes, not photos (v3.6.2 decision: one photo with 5 worms contributes 5, matching what CAG wants: “where are the pests”). All use `getCountedBoxes`, so the opt-out FP rule applies uniformly. Data source: one `GET /history` fetch for the chosen date range (default last 30 days, limit 500), with `hydrateVerifyMap` run over every record first. Verified in code:

- Stat cards: Today, Range total (with photo count), Flagged FP rate over the last 7 days (denominator is all detections in the window, not just reviewed ones), Peak day, Avg/day.
- By zone: horizontal bar chart of the top 5 waypoints, last 7 days. Note that this widget is scoped to 7 days while the rest of the tab follows the date picker, so on a quiet site it can read “No detections in the last 7 days” beside a non-zero Range total. That is correct behaviour, not a fault.



- Zone heatmap (`renderZoneHeatmap`, added in the v5.1 ARGUS reskin, per `docs/dashboard.md`; the code comment in `analytics.js` labels it v5.0 — same change, inconsistent label): zone x day grid, last 14 days, top 6 zones ranked by total. Cell shade is `rgba(10,132,255, 0.16 + 0.72*(n/max))`; zero cells get a faint neutral. Tooltip per cell: zone, date, count. A legend shows the shade ramp and the peak per-day count. Built as a CSS grid of divs, columns auto repeat(14, 1fr); the wrap scrolls horizontally on narrow screens.
- Daily trend: combined bar + line chart over the selected range (both datasets carry the same daily-bucket numbers; the line is a styled duplicate for readability, not a computed trend).
- Camera health: days since last detection per camera. Green up to 3 days, warning to 7, red “stale” beyond 7, “No data” otherwise. This answers “is the camera alive”, not “are there pests”.
- By camera: box counts with percentage bars, using `camDisplayName`.

Counting helpers, verified in code: `zoneBuckets7d(items)` keeps only records whose Singapore-converted `detection_time` falls in the last 7 x 24 h and sums `getCountedBoxes` per `waypoint_id`; `dailyBuckets(items)` does the same keyed by Singapore date over the whole fetched range. `renderStats` derives everything from `dailyBuckets`: Today = the bucket for today’s Singapore date, Range total = sum of all buckets, Peak day = the max bucket, and Avg/day divides by active days (days with at least one counted box), not calendar days. The FP-rate algorithm: over the last 7 days, denominator = every verifiable box in the window, numerator = every FP verdict in `verifyMap` for those records; a window with no detections renders a dash.

FIXED 2026-08-16 (v5.5, deployed and verified live). This section previously carried the defect as open; it is recorded here because the failure mode is worth knowing and because the same shape can recur.

`renderDailyChart` in `web/dashboard_v4/js/analytics.js` read an undeclared variable — `if (sub) sub.textContent = ...` — left dangling when the card’s subtitle line was removed in the 2026-08-05 small-caption cleanup. ES modules run in strict mode, so that read threw a `ReferenceError`, and the throw landed in `loadAnalytics`’s catch. The damage was larger than one chart: `loadAnalytics` calls its six render functions in sequence, so **the two calls after the throw, `renderCamHealth` and `renderByCam`, never executed at all**. Stat cards, By zone and the heatmap had already rendered, which is why the page read as “partly empty” rather than broken, and why the fault survived so long. A red toast showed “Analytics: sub is not defined”.

The fix reinstates the subtitle element (`<div class="card-sub" id="daily-sub">` in the Daily trend card head) and declares `const sub = document.getElementById('daily-sub')`. Deleting the line would also have worked, but the subtitle carries useful information (“30 days · 38 pests”), so it was restored rather than dropped.



A second defect was fixed in the same pass. The By-zone chart sets `indexAxis: 'y'`, which makes the bar horizontal: the x axis then carries the counts and the y axis carries the zone names. The titles were passed as `chartOpts( 'Zone', 'Detections' )`, exactly reversed, so the rendered chart printed “Zone” beneath a 0-40 count axis. Now `chartOpts( 'Detections', 'Zone' )`.

Testing note. The browser caches these ES modules hard. After deploying, a plain reload kept executing the old `analytics.js` even though CloudFront was already serving the new file. Force a cache refill first, then reload:

```
for (const u of ['/js/analytics.js', '/js/bbox.js', '/js/config.js']) {  
  await fetch(u, { cache: 'reload' });  
}
```

Chart instances are stored on state and destroyed on tab exit and re-render (Chart.js requirement). A tab-switch guard (`if (state.tab !== 'analytics') return`) prevents a slow fetch from painting over another tab.

## 4.14 Notifications and toasts

Topbar notifications (`main.js`): a bell button with a count badge and a dropdown. `addNotif(e)` (defined and exported in `main.js`) is written to push detection events (`kind: 'detected'`) and schedule events (`kind: 'schedule'`) onto `state.notifs`, capped at 50, and re-render the dropdown. In the current code, however, the feed is dormant: `addNotif` has zero callers anywhere in `web/dashboard_v4` and is not exposed on the window bridge, and no polling loop feeds it. Its producer was the WebSocket removed in v3.7 and was never re-wired to polling. The bell, count badge, and dropdown all render, but no event ever populates them. Clicking outside the dropdown closes it; a Clear button empties the list.

Toasts (`utils.js`): `toast(msg, kind, ms)` prepends a toast into `#toast-stack` (newest on top), auto-dismisses after `ms` (default 3000) with a slide-out animation, and supports a manual X. `refreshToastFan()` recomputes each toast's --fan-offset (cumulative heights + 8 px gaps) so stacked toasts fan out without overlapping on hover. The X handler is inline HTML, which is why `refreshToastFan` must be on window.

## 4.15 Dormant module: costs.js

A complete Costs tab (range pills 7 to 365 days, spend hero, month-to-date, per-service breakdown with usage units, per-day table, spend chart) that is not registered in TABS. Reason, verified in the code comment: the shared nbk2 IAM user has no Cost Explorer access; `ce:GetCostAndUsage` requires the account root to enable IAM access to billing, which is not available on shared nbk2. To re-enable: add the entry back to TABS in `main.js` and grant the API's Lambda role `ce:GetCostAndUsage`. The window bridge already exposes `renderCostsPage` and `changeCostRange`.

## 4.16 Design system: liquid glass (styles.css v5.2)

styles.css is layered: the original warm-neutral light theme (design tokens at the top: cream --bg: #faf8f2, teal accent, DM Sans + JetBrains Mono) stays untouched, and the “ARGUS THEME (v5.2)” block appended at the end overrides it. Revert = delete that block. Verified in the file (the block starts at line 1402).

Why v5.2 exists: v5.1’s base was near-white, so the glass had nothing to transmit and read as flat. v5.2 adds a visible pastel color field behind everything, then makes every surface actually translucent.

The exact tokens (v5.2 :root overrides, verified):

- Base: --bg: #eef1f5, --surface: rgba(255,255,255,0.35), ink #1d1d1f, accent moved to iOS blue --teal: #0a84ff (the variable keeps its old name).
- Pastel color field: body::before is a fixed, inset: -12% layer of five radial gradients (blue 0.32, cyan 0.30, purple 0.22, green 0.18, yellow 0.10) that drifts slowly (argus-drift, 48 s alternate) unless the user prefers reduced motion. The login screen has its own four-gradient variant.
- Glass recipe, panels: --lg-bg: rgba(255,255,255,0.22) + --lg-blur: blur(10px) saturate(190%) applied as backdrop-filter, a 1px double rgba(30,40,60,0.10) border, and --lg-rims.
- Rims (the inset shadow stack that simulates glass thickness): two 2 px inner white highlights at opposite corners at full opacity, two softer 8 px white spreads, and a hair-thin dark inner line (inset 0 0 2px rgba(30,40,60,0.22)).
- Specular: every panel (.card, .stat-card, .filter-bar, .modal-content, .login-card, .image-card) gets an ::after overlay: a 45° linear gradient with white at 0.65 alpha at both extremes, transparent through the middle, blurred 3 px. Content sits above it via explicit z-index: 1 rules.
- Controls variant: --lg-bg-ctl: rgba(255,255,255,0.30), --lg-blur-ctl: blur(6px) saturate(180%), --lg-rims-ctl (1.5 px highlights + 1.5 px dark line). Buttons, tabs, chips, badges, and inputs are all glass. Tabs became glass pills (border-radius: 980px) and the old active-tab underline is removed. .btn-primary is translucent blue (0.55) with a text shadow and a blue glow; inputs are “glass wells” with an inner-shadow stack and a blue focus ring.
- Brand: the nine-line aperture SVG (same mark as the deployer wordmark) inside a circular .brand-logo with a blue gradient. Rebranded from “Pest Monitor” to ARGUS 2026-07-10.
- Tuning knobs: adjust --lg-bg, --lg-blur, --lg-rims (panels) and the -ctl variants (controls) in :root. Nothing else needs touching to retune the glass.

The v5.3 pagination CSS (.gallery-pagination, .pager-\*) is appended after the theme block and is token-based, so it works in both themes.

## 4.17 Operations / reproduction

### Serve locally (development)

Any static server works. From the repository root:

```
python -m http.server 5501 --directory web/dashboard_v4
```

Then open `http://localhost:5501/`. VS Code Live Server on `web/dashboard_v4/index.html` also works. The app talks to the live API, so a Cognito sign-in is still required.

### Deploy (the runbook, development account)

Console path: S3 → Buckets → `argus-dashboard-506868652945` → Upload (drag the contents of `web/dashboard_v4/`); then verify the bucket's Properties → Static website hosting shows `index.html` as both index and error document. But the CLI is the intended path:

```
aws s3 sync web/dashboard_v4 s3://argus-dashboard-506868652945 --delete --cache-control "no-cache, must-revalidate" --exclude "*.md" --exclude ".claude/*" --profile prod
```

Replace **nbk2** with your own CLI profile, and the bucket name with **your bucket** on a fresh account.

The `--cache-control` flag is required. Without it the uploaded files carry no Cache-Control header, browsers heuristically cache the JS, and users keep seeing the old build until a hard refresh. This bit the project on 2026-07-14; the flag was added the same day.

CloudFront behavior: distribution `E1YADURLSAVNFA` uses the managed `CachingDisabled` policy, so every request hits S3. A plain `s3 sync` is therefore a full redeploy with no invalidation step. Console check: CloudFront → Distributions → `E1YADURLSAVNFA` → Behaviors → default behavior → Cache policy = `CachingDisabled`. CLI check:

```
aws cloudfront get-distribution-config --id E1YADURLSAVNFA --profile prod --query "DistributionConfig.DefaultCacheBehavior.CachePolicyId"
```

(`CachingDisabled`'s managed policy id is `4135ea2d-6df8-44a3-9df3-4b5a84be39ad`.)

Verify a deploy: `curl -s https://d1dtoxef7qmugl.cloudfront.net | findstr "<title>"` should show ARGUS, and a hard-refreshed browser session should log in and load the gallery with zero console errors.

### Deploy on the production account

The production dashboard was deployed by the ARGUS deployer (Chapter 7), not by a manual sync: `deployer/deploy.py` uploads every file of `web/dashboard_v4/` to `argus-dashboard-506868652945`, templating `js/config.js` in flight (section 4.4), and its CloudFront distribution `E1YADURLSAVNFA` also uses the `CachingDisabled` policy.

Note on frontend-only updates. The repository copy of `config.js` carries the live production values (API `vzf17s6z00`, Cognito pool `us-east-1_9selFDHpc`, client `6vebotf45bp8u46cnnraddiaplv`), corrected on 2026-08-15, so a plain `aws s3 sync web/dashboard_v4 s3://argus-dashboard-506868652945 --profile prod` is safe on this deployment. It was NOT safe before that date: the file still held the development account's values, and a hand-run sync would have overwritten the deployed, correctly templated `config.js` and silently pointed the production dashboard at a retired backend. The rule to keep: `config.js` is the one host-specific file, so before any hand-run sync confirm its three values match the account you are deploying to, or let the deployer's writeback stage template them (7.4).

Verify: `curl -s https://d1dtoxef7qmugl.cloudfront.net | findstr "<title>"` shows ARGUS, and the served `js/config.js` must contain `vzf17s6z00` and `6vebotf45bp8u46cnnraddiaplv` (checked good 2026-08-11).

## Cognito user management

Console path: Amazon Cognito → User pools → `pest-dashboard-users` (`us-east-1_ea0aJdus1`) → Users → Create user / select user → Delete. Self-signup is disabled; accounts are admin-created only. The commands below target the development pool with profile `nbk2`; for the production account substitute pool id `us-east-1_9selFDHpc` and profile `prod` — the flow is identical, and the first production user was created this way on 2026-08-11.

Create a user (replace **EMAIL** and **nbk2**):

```
aws cognito-idp admin-create-user --user-pool-id us-east-1_9selFDHpc --username EMAIL --message-action SUPPRESS --profile prod
```

Set a permanent password (policy: at least 8 characters, lowercase and number required; uppercase and symbols optional). Replace **EMAIL** and **PASSWORD**:

```
aws cognito-idp admin-set-user-password --user-pool-id us-east-1_9selFDHpc --username EMAIL --password PASSWORD --permanent --profile prod
```

--permanent matters. Without it the account is stuck in `FORCE_CHANGE_PASSWORD`, and the login UI intentionally does not handle that challenge (verified in `auth.js`: any `ChallengeName` shows “ask the admin”).

Remove a user (replace **EMAIL**):

```
aws cognito-idp admin-delete-user --user-pool-id us-east-1_9selFDHpc --username EMAIL --profile prod
```

Never write any password into a committed file or chat log.

## Emergency rollback: authorizer detach

If login misbehaves during a demo, two levers, in order.

Lever 1: detach the JWT authorizer so the API is instantly open again. Console path: API Gateway → APIs → zwpcbivmsj → Routes → select each route → Attach authorization → set to None. In practice use the PowerShell loop (route ids are stable):

```
foreach ($r in @("1xu5zhs","1zwy2vl","2dlsspt","33iv13c","58gsnz6","7wgkuzm",
"ao22ksu","b3kl8si","i8pcllp","jedpy5t","krzufjf","nqy131o","odwxz00","t4w6cz
h","tos63cu","ttif38k","w0ivphi","wqh3mvj","zbh9lli")) { aws apigatewayv2 upda
te-route --api-id vzf17s6z00 --route-id $r --authorization-type NONE --profil
e prod }
```

Route glwyqo0 (DELETE /detection, destructive) is deliberately not in this list. Leave it JWT-gated even during a rollback; nothing in the no-auth fallback uses it.

Re-attach later with the same loop using --authorization-type JWT --authorizer-id enxa26.

These route ids belong to the development API vzf17s6z00. The production API vzf17s6z00 has its own route ids; list them first with `aws apigatewayv2 get-routes --api-id vzf17s6z00 --profile prod` and apply the same pattern (leave DELETE /detection gated).

Lever 2: if the login UI itself is broken, open the local fallback `web/_archive/dashboard_v3_9.html` via a local static server. It has no auth and sends no token, so it only works after lever 1. It still has the pre-v4.0 X-dismiss bug; that is accepted for an emergency fallback.

## Reproducing on a different AWS account (for NP staff)

This is no longer theory: the ARGUS deployer (Chapter 7) performed exactly this reproduction on the NP production account 506868652945 on 2026-08-10 — all 15 stages in 103 seconds, dashboard, Cognito, API, and frames-bucket CORS included — so the automated path is `python deployer/deploy.py` and the steps below are the manual equivalent (and the explanation of what the deployer does).

1. Create the S3 website bucket, enable static website hosting (index and error document `index.html`), allow public read.
2. Create the CloudFront distribution with the S3 website endpoint as origin and the `CachingDisabled` cache policy.
3. Create a Cognito user pool (email sign-in, self-signup off) and an app client with no secret, `USER_PASSWORD_AUTH` and refresh flows enabled.
4. Deploy the backend (Chapter 2; full fresh-account order in Chapter 8), then attach a JWT authorizer for that pool to every API route except `GET /stream/status`.
5. Edit `js/config.js`: **HTTP\_API**, **COGNITO\_REGION**, **COGNITO\_CLIENT\_ID**. This is the only frontend change needed.
6. Add the new CloudFront origin to the frames bucket's CORS AllowedOrigins (needed for Test upload).

## 7. Run the deploy sync, create users, sign in.

Remember the account-bound caveat: Rekognition Custom Labels models cannot migrate across accounts. The dashboard works immediately, but detection needs the models retrained on the new account. This was done during the 2026-08-10/11 migration: the training data was copied server-side to `argus-frames-506868652945`, the armyworm model (the v9r data-augmentation build) was submitted for training as `v9r-prod-20260810` in project `argus-detection`, and the moth model was rebuilt from the surviving labelled data as `moth-prod-20260811` in project `argus-moth-detection` (F1 = 0.991). Details in Chapter 3 and the migration scripts under `migration/`.

Security caution while reproducing: one historical script, `datasets/archive/experiments/pre_v3_abandoned/download.py`, contains an inline Roboflow API key (credential stored there, not reproduced here). Do not copy that file, or any file containing a credential, into anything that gets published or synced to the dashboard bucket. The deploy command's `--exclude` flags only cover `*.md` and `.claude/*`; the real protection is that the sync source is `web/dashboard_v4` only.

## 4.18 Cross-references

- Chapter 1: system overview; where the dashboard sits in the end-to-end chain.
- Chapter 2: the cloud stack this frontend calls — `pest-monitoring-api` routes, `kvs-hls-handler`, `pest-detection-processor` and the v4.5 hybrid Rekognition/LLM gate whose output the gallery trusts, DynamoDB record shape, S3 buckets, IAM.
- Chapter 3: detection models — Rekognition Custom Labels version history, `min_confidence` semantics, tiling, holdout methodology.
- Chapter 5: Unitree Go2 + Jetson Orin — patrol, SIYI A8 Mini capture, the capture-upload gate that produces gallery records.
- Chapter 6: mini PC — the KVS controller service behind the Live tab's stream toggles.
- Chapter 7: ARGUS deployer, which recreates this whole stack from the manifest.
- Chapter 8: full fresh-account reproduction, superset of section 4.17's frontend-only steps.

## Chapter 5 — Edge platform: Unitree Go2 + Jetson Orin

This chapter covers the mobile edge platform: the Unitree Go2 quadruped, the Jetson Orin mounted on its back, the SIYI A8 Mini gimbal camera, USLAM navigation and its map profiles, the gated patrol script, the patrol scheduler, the live-stream producer, and the full operations runbook. *As of 2026-08-20.*

### 5.1 Role in the system

The Go2 is the demo and testbed carrier for the ARGUS armyworm camera. It walks a surveyed route, stops at waypoints, and lets the SIYI A8 Mini capture a frame at each stop. The Jetson Orin on its back runs everything project-side: the patrol node, the frame capture, the S3 upload, the DynamoDB detection gate, and the Kinesis Video Streams producer for the dashboard's live view. The dog's own sport MCU runs USLAM (the localization and navigation service); the Orin only talks to it over ROS 2 topics.

In the end-to-end chain, this platform is the front end of the detection pipeline: A8 frame -> S3 (argus-frames-506868652945) -> the pest-detection-processor Lambda -> AWS Rekognition Custom Labels + the Bedrock LLM verifier -> DynamoDB (pest-monitoring-detections) -> dashboard. The patrol script waits at each capture waypoint until the DynamoDB record for its uploaded frame exists, so the robot's motion is gated on the cloud pipeline having actually processed the frame.

Project framing to keep straight: the Go2 is a demo/testbed only. The production design for CAG at Jewel Changi is fixed cameras at each waypoint, no robot deployment. This chapter documents a finished milestone: on 2026-07-30 the Go2 ran three consecutive 5/5 autonomous patrols on the real Jewel site map, app-free, from both cold boot and back-to-back starts, best run 150 s with zero retries, with the full capture -> S3 -> Rekognition -> LLM -> DynamoDB -> dashboard chain live.

**Account note (updated 2026-08-13).** Everything in this chapter was built and validated on the development account 366356442579; the system now runs on the NP production account 506868652945. The ARGUS deployer stood the full cloud stack up there on 2026-08-10 (all 15 stages, 103 s; Chapter 7), both Rekognition models were retrained on it, and the handover snapshot argus-repo-snapshot-20260813.zip was published to it on 2026-08-13. Production names: frames bucket argus-frames-506868652945, API <https://vzf17s6z00.execute-api.us-east-1.amazonaws.com>. The deployer originally seeded the armyworm camera row as camera-1, but it was re-keyed to worm\_cam on 2026-08-11, so the Orin's CAMERA\_ID needs NO change. The repo mirrors of the edge scripts were repointed to these production values on 2026-08-13: S3\_FRAMES\_BUCKET in go2\_patrol\_gated.py and capture\_4k\_hdmi.py, and the API\_BASE default in kvs\_controller.py. One device pass is still open — the hardware was away when the repoint was made, so the mirrors must be synced onto the Orin and the on-device ~/.aws/credentials swapped from the dev-account cag\_user key to a production-account key; 5.12.6 has the checklist. KVS / live view was skipped at deploy, so a stream



must still be created on the production account before the live-stream producer of 5.10 has a target there.

## 5.2 Inventory

| Item                       | Location                                                      | Purpose                                                                     |
|----------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------|
| Unitree Go2 EDU            | physical; sport MCU at 192.168.123.161                        | Quadruped. USLAM runs on its MCU (DDS only, no SSH).                        |
| Jetson Orin Nano           | on the Go2's back; 192.168.123.18 (dog net); SSH user unitree | Runs patrol, capture, upload, gate, KVS producer. Ubuntu 20.04, ROS 2 Foxy. |
| SIYI A8 Mini gimbal camera | on the Go2; 192.168.144.25, UDP control 37260                 | Frame source (RTSP 1080p) + live-stream source.                             |
| Mount plate                | robot/Karlz+Livox+Mount+plate.stl                             | Printable mounting plate for the payload stack.                             |
| robot/setup_go2.sh         | mirrors Orin<br>~/setup_go2.sh                                | Per-terminal DDS environment (CycloneDDS bound to the dog link by IP).      |
| robot/go2_patrol_gated.py  | mirrors Orin<br>~/go2/go2_patrol_gated.py                     | The gated patrol node (bringup + waypoints + capture + S3 + DDB gate).      |
| robot/pose.py              | mirrors Orin<br>~/go2/pose.py                                 | One-shot pose grab for waypoint surveying.                                  |
| robot/get_map_id.py        | mirrors Orin copy                                             | Read-only: print the currently loaded USLAM map id.                         |
| robot/nav_probe.py         | mirrors Orin copy                                             | Single-goal planner probe (is the map plannable at all).                    |
| robot/map_profiles.md      | repo only                                                     | Per-map parameter sets (INITIAL_POSE + WAYPOINTS) for every recorded map.   |
| robot/tests/wp_test_1.py   | + .log                                                        | Nav-only route validation, round 1 (superseded by round 2).                 |
| robot/tests/wp_test_2.py   | + .log                                                        | Nav-only route validation on the lab PRIMARY map; the 4/4 proof.            |
| robot/tests/wp_test_3.py   | repo + Orin                                                   | Nav-only route validation on the lab DEMO map.                              |



| Item                          | Location                                  | Purpose                                                                                                                                                                                                                |
|-------------------------------|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| robot/tests/wp2_probe.py      | repo + Orin                               | Planner probe: find a plannable cell for a refused waypoint.                                                                                                                                                           |
| robot/tests/wp_survey.py      | repo + Orin                               | Hands-free waypoint survey on a new map (odom motion/silence detector).                                                                                                                                                |
| robot/tools/pose_logger.py    | Orin /tmp/pose_logger.py during surveys   | Continuous pose logger to /tmp/pose_log.txt; used for the Jewel walk-the-route survey.                                                                                                                                 |
| robot/tools/localize_only.py  | repo + Orin                               | Localize without nudging, report the settled pose (for surveying a parking spot).                                                                                                                                      |
| robot/tools/uslam_listen.py   | repo + Orin                               | Raw listener on both /uslam/server_log and /uslam/client_command, each line annotated with the live pose, logging to /tmp/uslam_listen.log. The client_command tap is how the phone app's own seed pose was recovered. |
| robot/tools/bringup_test.py   | repo + Orin                               | Bringup-sequence-only test (stop -> seed -> start), no route.                                                                                                                                                          |
| robot/kvs_controller.py       | mirrors Orin<br>~/go2/kvs_controller.py   | Polls the dashboard stream toggle; starts/stops the KVS pipeline.                                                                                                                                                      |
| robot/run_kvs_controller.sh   | mirrors Orin<br>~/run_kvs_controller.sh   | Env + credentials wrapper the systemd service execs.                                                                                                                                                                   |
| robot/kvs-controller.service  | mirrors Orin systemd unit                 | Boot-persistent service wrapping the two files above.                                                                                                                                                                  |
| robot/patrol_scheduler.py     | mirrors Orin<br>~/go2/patrol_scheduler.py | Polls the dashboard's /schedule row; launches go2_patrol_gated.py at the scheduled time.                                                                                                                               |
| robot/run_patrol_scheduler.sh | mirrors Orin<br>~/run_patrol_scheduler.sh | Non-interactive ROS-env wrapper the systemd service execs.                                                                                                                                                             |

| Item                                                   | Location                                          | Purpose                                                                                                                                                                                                                      |
|--------------------------------------------------------|---------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| robot/patrol-scheduler.service                         | mirrors Orin systemd unit                         | Boot-persistent service wrapping the two files above.                                                                                                                                                                        |
| robot/capture_4k_hdmi.py                               | repo + Orin                                       | Experimental 4K capture over HDMI + UVC card (see 5.3; the fitted card capped at 1080p).                                                                                                                                     |
| robot/_archive/2026-07-29/, robot/_archive/2026-07-30/ | repo                                              | Field run logs: the failed first Jewel run, the 5/5 clean runs, Orin script backups.                                                                                                                                         |
| KVS stream armyworm-cam-stream                         | AWS, us-east-1                                    | Live H.264 stream from the A8 to the dashboard. The stream name is unchanged across accounts, but the 2026-08-10 production deploy skipped KVS — create the stream on 506868652945 before enabling live view there (5.12.6). |
| S3 bucket argus-frames-506868652945                    | AWS, us-east-1, production account                | Frame drop point; S3 event triggers the processor Lambda. The dev account used frames-armyworm-366356442579 (history).                                                                                                       |
| DynamoDB table pest-monitoring-detections              | AWS, us-east-1 (same table name on both accounts) | Detection records; the patrol gate polls it.                                                                                                                                                                                 |

## 5.3 Hardware and wiring

**Unitree Go2 EDU.** Two computers matter:

- **Sport MCU 192.168.123.161** — wired, DDS only, no SSH. USLAM (mapping, localization, navigation) runs HERE, not on the Orin. There is no ps visibility into it from the Orin. If the USLAM service crashes or wedges, the only recovery is a full power-cycle of the dog.
- **Jetson Orin Nano 192.168.123.18** — SSH user unitree, SSH alias go2, Ubuntu 20.04, ROS 2 Foxy. Note: the unitree\_ros2\_example binaries sit in bin/, not lib/<pkg>/ — run them directly; ros2 run fails.

**Networks.** Three distinct links touch the Orin:

- **Dog net 192.168.123.0/24** (wired): MCU .161, Orin .18, mini PC bridged NIC .99, laptop .50 (ASIX USB Ethernet adapter on the dock, manual static, gateway/DNS blank; reset to DHCP for normal networking).
- **A8 net 192.168.144.0/24** (wired, second USB Ethernet dongle): A8 at .25, Orin at .30 via the NetworkManager profile a8-link (autoconnect). a8-link is MAC-locked; on 2026-07-29 it had to be re-bound from a missing dongle (6C:1F:F7:21:52:73) to the AX88179B actually fitted (6C:1F:F7:28:CD:0C). If the A8 link ever dies after a hardware shuffle, check this MAC bind first. A second gotcha found the same day: an orphan profile Wired connection 2 was grabbing eth0 and starving a8-link; it is now set to autoconnect no.
- **WiFi (RTL8821CU dongle, wlan0)**: npwireless (NP campus, priority 100), apps-jewel (CAG's Apps@Jewel at Jewel, priority 90, hidden SSID — 802-11-wireless.hidden yes is required), iPhone Air hotspot fallback. All nmcli operations need sudo over SSH. The SSID is Apps@Jewel with an "s"; App@Jewel fails identically to being out of range. CA validation is off on both enterprise profiles on purpose: the Orin boots with a 1970 clock. Apps@Jewel has AP client isolation — the Orin gets full internet (S3 + Rekognition verified) but the laptop cannot SSH to it over that SSID; see 5.12.4.

**SIYI A8 Mini power.** The A8 is powered over an XH connector cable. The GND wire of that cable was resoldered after unrelieved tension snapped it. Before any sustained untethered operation the cable needs strain relief (a service loop) — a standing OPEN roadmap item, still not done as of 2026-08-11. The supply rail the XH cable draws from is not recorded in any repo source; confirm on the hardware before rewiring anything.

**SIYI A8 Mini video paths.** RTSP at rtsp://192.168.144.25:8554/main.264 caps at 1080p; 4K exists only on the SD card and the HDMI port. A 4K-over-HDMI detour was tried on 2026-07-29: the A8 was rewired to mini-HDMI -> USB capture card -> Orin. The fitted card turned out to be a MACROSILICON MS2109 (USB 2.0) that maxes out at 1920x1080 in both MJPG and YUYV, so the path gained nothing over RTSP, and it also killed gimbal UDP control (port 37260 rides the ethernet link HDMI does not carry). The A8 was reverted to ethernet the same day and the full chain re-verified. robot/capture\_4k\_hdmi.py remains as the experiment harness; it forces the MJPG fourcc before setting resolution and verifies the actual delivered frame size so a 1080p frame can never be mislabelled as 4K. No genuine 4K frame was ever captured through it: the only card ever fitted was the 1080p-max MS2109, the script's guard refuses to upload non-3840x2160 frames, and the test\_4k S3 zone was never populated. The card was unplugged in the same-day revert to ethernet.

Function-level detail of robot/capture\_4k\_hdmi.py (no ROS involved; constants: WARMUP = 8 discard frames, JPEG\_Q = 92, CAMERA\_ID = worm\_cam, bucket argus-frames-506868652945 since the 2026-08-13 repoint):

- open\_dev(dev\_index, w, h) — opens /dev/videoN via V4L2 and sets the MJPG fourcc BEFORE the resolution. Order matters: without MJPG most cheap UVC cards

silently cap at 1080p (USB bandwidth cannot carry raw YUY2 at 4K) and OpenCV keeps the old size without any error. Returns the capture handle or None.

- `grab(dev_index, w, h)` — reads WARMUP frames keeping the last (exposure and AWB settle), JPEG-encodes at quality 92, and returns `(jpeg_bytes, actual_w, actual_h)`. The size is read back from the delivered frame, not the requested mode — that read-back is what the mislabel guard keys on. `(None, 0, 0)` on open, decode, or encode failure.
- `upload(zone, jpeg_bytes)` — put\_object to `frames/worm_cam/<zone>/<UTC ts>.jpg`, the same key scheme as the patrol, so a test frame flows through the normal S3 -> processor -> DynamoDB -> dashboard chain and lands in the named dashboard zone.
- `probe()` — lists `/dev/video*` nodes, prints the `v4l2-ctl --list-formats-ext` command to enumerate modes (look for MJPG 3840x2160), and open-tests each node. Capture cards usually expose two nodes; only the first reads frames.
- `main()` — `--shot` grabs at 3840x2160 and REFUSES to upload unless the delivered size is exactly 3840x2160 (exit code 2): this is the guard that makes a 1080p frame impossible to mislabel as `test_4k`. `--ab` then grabs a 1080p arm of the same scene into `test_1080p`; that arm uploads even on a size mismatch, labelled with its true size, because the 1080p side is the control, not the claim. Exit code 1 = no frame decoded at all.

**Untethered rule.** Any route with forward motion must run untethered — a connected cable can drag hardware off a table or off the dog. The patrol script’s launch countdown exists to give the operator time to unplug. The remote stays in the operator’s hand as the e-stop, with the critical caveat in 5.5: pressing any button on it stops USLAM outright and ends that patrol run.

## 5.4 DDS environment: `setup_go2.sh`

`robot/setup_go2.sh` (Orin: `~/setup_go2.sh`) is a 4-line environment file sourced in every terminal that touches ROS:

- `RMW_IMPLEMENTATION=rmw_cyclonedds_cpp` — the patrol deliberately uses plain `rclpy` over CycloneDDS. The Unitree Python SDK (`unitree_sdk2_python`) is avoided: its cyclonedds Python binding segfaults on this Orin.
- `CYCLONEDDS_URI` pins `NetworkInterfaceAddress` to `192.168.123.18` — the dog link is selected by IP, so `eth0/eth1` interface renames do not matter.

Sourcing rules (these bite hard over SSH, see 5.12.4): interactive terminals answer 1 (foxy) at the `.bashrc` fishros prompt, then source `~/setup_go2.sh`. Non-interactive shells (Posh-SSH exec channels, `bash -c`) get NO ROS from `.bashrc` at all — they need the explicit triple source:

```
source /opt/ros/foxy/setup.bash && source ~/cyclonedds_ws/install/setup.bash
&& source ~/setup_go2.sh
```

## 5.5 USLAM navigation

USLAM is Unitree's on-dog SLAM/nav service. Its entire control surface is plaintext `std_msgs/String` on two topics — no Nav2, no custom message types:

- commands out: `/uslam/client_command`
- feedback in: `/uslam/server_log`

**Wire format (reverse-engineered from the app, verified W10–W12).** Two quoting rules, not interchangeable:

- Control verbs (`common/*`, `localization/*`, `navigation/start`): the String data CARRIES an inner pair of double quotes — the message data is `"common/get_map_id"` including the quotes.
- Goals (`navigation/set_goal_pose/x/y/yaw`): BARE path, no inner quotes.

Coordinates are x/y/yaw in meters and radians, in the map frame. Project yaw convention:  $\text{yaw} = 2 \cdot \text{atan2}(q_z, q_w)$ , wrapped to  $[-\pi, \pi]$ .

**Bringup sequence (app-free, as implemented in the patrol — see 5.7 for the code):**

1. `common/get_map_id` -> success + map id. Doubles as the MCU liveness check: silence means the USLAM service is down, and the only fix is a full dog power-cycle. It also establishes DDS discovery for everything that follows.
2. `localization/stop` first, IF a localization is already running. Seeding and starting on top of a live localization FAILS: verified 2026-07-30, three `localization/start` calls each returned success and then `[Localization] initialization failed!` about 6 s later. The phone app always sends stop first; the patrol now mirrors it, which makes bringup idempotent from cold, running, or failed-dirty state (14.2 s measured).
3. `localization/set_initial_pose/x/y/yaw` -> success + 6-DOF echo.
4. `localization/start` -> wait for “uslam is initialized!” / “initialization succeed!” on `/uslam/server_log`.
5. The dog **MUST** be moving for step 6 to succeed — localization only tracks while the dog moves. The patrol sends a small in-place rotate goal (the “nudge”) to start tracking.
6. `navigation/start` -> “navigation/start/success”, state WAITING.
7. `navigation/set_goal_pose/x/y/yaw` -> TRACKING -> `navigation/state_transition/REACHED` -> WAITING. After REACHED the dog holds position until the next goal; no pause logic is needed.

Failure tokens on `/uslam/server_log`: NO\_PATH, GOAL\_CANCELLED, FAILURE, GOAL\_POINT\_UNREACHABLE.

**Map lifecycle.** USLAM auto-starts on the MCU at power-on and auto-loads the LAST map. A normal cold boot needs no phone app at all — app-free bringup is verified on the Jewel

map (2026-07-30: seed -> localization/start -> success in ~6 s, pose corrected 0.083 m off the seed, whole bringup 18.5 s). The app is needed only to: record a new map, select and localize on a DIFFERENT map than the last-loaded one, and diagnose crashes. Recorded maps persist on the dog; recording a new one does not delete old ones. One field exception (2026-07-09): on a freshly recorded map the CLI cold-init did not take — localization on a brand-new map needed the app once.

**Hard-won operational facts. These are the reasons the patrol works; read them before touching anything that talks to USLAM.**

- **RAPID REPEATED CONTROL VERBS WEDGE THE MCU** (the single most important operational fact, learned 2026-07-30). Two independent wedges, same mechanism: ~46 joystick-triggered USLAM stops produced 4516 TIMEOUT\_ODOMETRY events; six navigation/start messages in 26 s drove the TIMEOUT count from 1 to over 1500 within minutes. Design rule: **send a verb ONCE, wait for its reply token, and never stack retry loops on top of a repeating sender.** The patrol's send\_verb and send\_goal defaults are now repeat=1 (they were 3, added against DDS discovery loss; discovery is already established by the get\_map\_id handshake). A wedged MCU still answers get\_map\_id normally while the robot is unusable — liveness checks look fine. Seeing TIMEOUT\_ODOMETRY and TIMEOUT\_POINTCLOUD together = power-cycle the dog; do not hunt for it on the Orin. (ros2 topic hz showing nothing on /utlidar/\* or /lowstate is a QoS artifact — sensor topics are best-effort, hz subscribes reliable; judge by publisher counts on /uslam/\* topics.)
- **Pressing a button on the remote STOPS USLAM.** /uslam/server\_log prints Joystick button is pressed! Uslam is stopped now!. Localization is torn down outright; the patrol loses tracking and navigation/start refuses. Treat the remote as e-stop only: pressing it ends that patrol run. Reposition the dog by hand BEFORE launching, never during.
- **localization/start returning success does NOT mean the dog is localized in the map.** Seed set\_initial\_pose with a pose the dog is not actually at, and USLAM still accepts it, returns success, and starts publishing /uslam/localization/odom — but it is integrating odometry from that seed, with no map match. The pose stream looks perfect (smooth, zero discontinuities over 214 s at Jewel) while being anchored in the wrong place; the error grows with distance (~5 m offset after ~10 m of walking). A whole waypoint survey done in such a frame is void — this is exactly what killed the first Jewel map on 2026-07-29.
- **The only trustworthy signal is /uslam/server\_log.** A real relocalization failure prints [Localization] initialization failed! ~5 s after start and auto-sends localization/stop; localization/get\_status answers status/0 when not localized. A genuinely successful relocalization shows a visible correction between seed and converged pose (0.44 m on the 2026-07-29 success) — that correction is the signature of a real scan match. The patrol's own "localization initialized." log line once matched on a run whose frame was ~5 m out, so it is not proof either;

cross-check the reported pose against a known physical spot before trusting a survey.

- **Localization init is intermittent, ~50-60% per attempt, cause never identified.** Identical seed, identical dog position: failed at 12:56, succeeded at 13:04 on 2026-07-30. The patrol retries the whole stop -> seed -> start sequence up to `LOCALIZE_ATTEMPTS = 4` (all-four-fail probability ~6%).
- **Mislocalization signature:** EVERY goal fails instantly with `NO_PATH/FAILURE` — even a zero-distance goal — and the dog shuffles and spins without departing (measured: 5.6 m of cumulative path for 0.07 m of net displacement and 358 degrees of yaw). Do not chase waypoint clearance for this; clearance measured in a bad frame proves nothing. Recovery = re-localize in the app, confirm via the `server_log` success token, then re-survey the waypoints in the same session. Do NOT re-record the map.
- `/uslam/localization/odom` only publishes while the dog moves. Standing still = no frames. Normal, not a fault.
- **Obstacle avoidance must be ON or odom is silent entirely** (found 2026-07-09). Avoidance OFF on the remote kills `/uslam/localization/odom`. Symptoms: `pose.py` reports “NO ODOM within 20s”; `wp_survey.py` records zero points while the dog visibly walks. Re-enable avoidance and the topic returns.
- Waypoints too near walls: the planner silently reroutes or refuses. Keep clearance; the patrol fails loudly instead of guessing. Clearance ANALYSIS is not possible on this stack: `/uslam/cloud_map` never publishes and `/uslam/localization/cloud_world` carries live lidar returns (bystanders read as obstacles). Judge waypoints by whether a patrol reaches them, not by computed clearance.
- USLAM treats every `set_goal_pose` as a new goal. A repeat arriving after TRACKING starts raises `GOAL_CHANGED` and kills the goal the script is waiting on — this is why `send_goal` must stay at `repeat=1` (with `repeat=3` waypoints failed at random).
- `ros2 topic pub -r N` (continuous publish) drops the SSH session — always -- once with 1–2 resends.
- The CycloneDDS Python binding segfaults intermittently; the first run after `sudo reboot` is clean.

## 5.6 Map profiles: `map_profiles.md`

`robot/map_profiles.md` holds the full parameter set for every recorded map: an `INITIAL_POSE` dict and a `WAYPOINTS` list, ready to paste into `go2_patrol_gated.py`. Switching maps is a three-step procedure:

1. Select the map in the app once (localize on it).
2. Paste that map’s block into `go2_patrol_gated.py` (`INITIAL_POSE` + `WAYPOINTS`).
3. Push the script to the Orin.

Nothing else changes, because the patrol script reads the live map id from `get_map_id` at bringup — there is no hardcoded map id anywhere in the code.

### The maps, newest first:

- **JEWEL SITE v2 — 1BEC7FFDF97C47AC8BD751143D3FE187 (CURRENT, the zone of record).** Re-mapped by Runzhe on site 2026-07-29 after the first Jewel map proved unusable. Surveyed by walking the route twice with the continuous pose logger and extracting the dwell points (round-to-round repeatability 0.9–2.4 cm; a 14 m loop closed to 0.17 m). Validated by three consecutive 5/5 autonomous patrols on 2026-07-30. Route: wp1 -> zone1 -> zone2 -> zone3 -> wp\_return, capture at the three zones only; wp1 and wp\_return carry "capture": False.
- **RETIRED — first Jewel map F0E056FC045649B7BE3BDFF92FC54363** (2026-07-29, unusable). Every surveyed coordinate on it is void: the survey ran in an odometry frame that never matched the map (see 5.5, the localization-success trap). Kept in `map_profiles.md` as the documented failure case, including the retracted clearance analysis.
- **PRIMARY — lab room 04114624684C4194B7008EDB3A5642D2.** Recorded W13, re-surveyed 2026-07-03, validated 4/4 REACHED (~50 s loop) by tests/wp\_test\_2.py. Historical: the W15 roadmap item “switch back to the PRIMARY map” became obsolete on 2026-07-29 when the Jewel map became the zone of record.
- **DEMO — showcase route DACB71661B5A48D48C6631AB2480C611.** Recorded 2026-07-09 for the W14 NP Robotics Centre SPF showcase. Capture at wp1 in place; ends at wp3, no return leg. wp2 was re-picked after the original spot proved to be a planner dead zone (see wp2\_probe.py, 5.9). Also historical.
- **RETIRED / dead:** client-test rescan 7853B2C397A44F8EB317D1C12D5B1F1C (discarded 2026-07-09), 7E7AABAE... (W12, unplannable), DA230BE6... (old room). Do not use.

**Two settings the Jewel route depends on** (recorded in `map_profiles.md`, present in the code): `send_goal` default `repeat=1` (see 5.5), and do not launch while driving the dog by remote — a nav goal fired into manual control stopped localization mid-run at Jewel and aborted the patrol at bringup.

NOTE: doc lag — `map_profiles.md`'s Jewel v2 block says `SKIP_LOCALIZATION_BRINGUP = True` is required and lists `INITIAL_POSE = (-5.001, -0.817, 1.249) / wp_return = (-5.013, -0.825, 1.374)`. The deployed code (synced from the Orin 2026-07-30, after the stop-first bringup fix) has `SKIP_LOCALIZATION_BRINGUP = False` — app-free bringup is now the verified default — and carries `INITIAL_POSE = wp_return = (-4.970, -0.657, 1.260)`. The code is what ran the three 5/5 validation patrols; follow the code. NOTE: doc lag — `docs/hardware.md` still names the retired 7853B2... rescan as the active map; the live map is read at bringup anyway.



## 5.7 go2\_patrol\_gated.py — the gated patrol node

**Purpose.** One rclpy node that drives the whole demo loop: USLAM bringup, waypoint navigation, gimbal settle, frame capture, S3 upload, DynamoDB detection gate, next waypoint. Runs on the Orin (~/go2/go2\_patrol1\_gated.py) under tmux or plain SSH.

**Configuration (constants at the top of the file, values as deployed):**

| Constant                                               | Value                                                           | Meaning                                                                                                                                                                                                                 |
|--------------------------------------------------------|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CMD_TOPIC / LOG_TOPIC                                  | /uslam/client_command /<br>/uslam/server_log                    | USLAM control surface. Do not change.                                                                                                                                                                                   |
| INITIAL_POSE                                           | {x: -4.970, y: -0.657,<br>yaw: 1.260}                           | Jewel-map fallback localization seed = the dog's physical parking spot at launch (= wp_return).                                                                                                                         |
| WAYPOINTS                                              | wp1, zone1, zone2, zone3,<br>wp_return                          | The Jewel route in loop order. "capture": False on wp1 and wp_return = navigate only. Waypoint names become the S3 key segment, so zone names stay traceable in the dashboard. Optional per-point cam dict (see below). |
| NUDGE_ENABLED /<br>NUDGE_DELTA_YAW /<br>NUDGE_SETTLE_S | True / 0.30 rad / 8.0 s                                         | Pre-navigation in-place rotate so localization starts tracking.                                                                                                                                                         |
| NAV_START_ATTEMPTS                                     | 2                                                               | navigation/start refusals are recoverable; each retry re-nudges harder (2x delta).                                                                                                                                      |
| SKIP_LOCALIZATION_BRINGUP                              | False                                                           | False = full app-free bringup (stop -> seed -> start). True = trust an app-established localization and leave it alone.                                                                                                 |
| A8_IP / A8_CTRL_PORT /<br>RTSP_URL                     | 192.168.144.25 / 37260 /<br>rtsp://192.168.144.25:8554/main.264 | Gimbal endpoints.                                                                                                                                                                                                       |
| AWS_REGION                                             | us-east-1                                                       |                                                                                                                                                                                                                         |
| CAMERA_ID                                              | worm_cam                                                        | Routes the S3 key to the worm camera config (renamed from                                                                                                                                                               |

| Constant                               | Value                      | Meaning                                                                                                                                                                                                                                  |
|----------------------------------------|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                        |                            | armyworm_go2_a8mini in the 2026-07 camera-id rename). The production account 506868652945 row is also worm_cam (deployer-seeded as camera-1, re-keyed 2026-08-11), so this constant needed no change in the production repoint (5.12.6). |
| S3_FRAMES_BUCKET                       | argus-frames-506868652945  | Production frames bucket (repointed 2026-08-13 from the dev-account frames - armyworm-366356442579).                                                                                                                                     |
| DDB_DETECTIONS                         | pest-monitoring-detections |                                                                                                                                                                                                                                          |
| START_COUNTDOWN_S                      | 3                          | Time to clear the area and unplug the cable.                                                                                                                                                                                             |
| GETMAP_TIMEOUT_S                       | 8                          | Silence beyond this = MCU USLAM down, abort.                                                                                                                                                                                             |
| LOCALIZE_TIMEOUT_S / LOCALIZE_ATTEMPTS | 30 / 4                     | Localization init is intermittent (~50-60% per try); retry the whole sequence rather than abort.                                                                                                                                         |
| NAV_START_TIMEOUT_S                    | 20                         | Wait for navigation/start/success.                                                                                                                                                                                                       |
| NAV_REACH_TIMEOUT_S                    | 90                         | Per-waypoint max wait for REACHED.                                                                                                                                                                                                       |
| NAV_RETRY_ONCE / ABORT_ON_NAV_FAIL     | True / False               | Retry a failed goal once; on final failure skip the point and continue (set True to abort instead).                                                                                                                                      |
| GIMBAL_SETTLE_S                        | 1.5                        | Settle after a LOCK override drove the gimbal to an absolute angle.                                                                                                                                                                      |
| FOLLOW_SETTLE_S                        | 2.0                        | FOLLOW mode lags the body ~1-2 s; wait this long after the dog stops before capturing.                                                                                                                                                   |
| CAPTURE_WARMUP                         | 30                         | RTSP frames discarded before keeping one.                                                                                                                                                                                                |

| Constant                                | Value                                                             | Meaning                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDB_GATE_TIMEOUT_S /<br>DDB_GATE_POLL_S | 150 / 1.5                                                         | Detection-gate poll budget and interval; fail-open on timeout. The 150 s was originally sized under the processor Lambda's then-180 s timeout; the processor now runs at 600 s (1024 MB), so on a pathological frame the gate fails open before the Lambda finishes — acceptable, since the measured v6.3 envelope (24–54 s per tiled frame with full LLM verification) sits well inside the budget. |
| LOC_OK_TOKENS /<br>LOC_FAIL_TOKENS      | init-succeed strings /<br>“initialization failed”                 | Without the fail tokens the script would wait out the full timeout on a failure it was already told about.                                                                                                                                                                                                                                                                                           |
| NAV_FAIL_TOKENS                         | NO_PATH,<br>GOAL_CANCELLED,<br>FAILURE,<br>GOAL_POINT_UNREACHABLE | Goal-fatal tokens.                                                                                                                                                                                                                                                                                                                                                                                   |

AWS credentials are NOT in the file — boto3 reads the Orin's environment / ~ / .aws (the cag\_user IAM user). Credential stored there, not reproduced here. Open item (5.12.6): the key on the device is still the dev-account cag\_user key, which cannot write the production bucket — the repointed script only works end to end after the pending on-device credential swap.

NOTE: doc lag inside the code — the comment block above INITIAL\_POSE still describes the PRIMARY lab-room map, but the values below it are the Jewel v2 set. The values are current; the comment is stale.

**Class USLAMClient(Node)** — the USLAM protocol driver:

- `_on_log(msg)` — subscriber callback; buffers each /uslam/server\_log line as (monotonic\_ts, text) in a 500-line deque and notifies waiters.
- `_on_odom(msg)` — subscriber on /uslam/localization/odom; keeps the latest pose (project yaw convention) plus its timestamp. This is the live pose feed the bringup seeds and nudges from.

- `live_pose(timeout=5.0, max_age=None)` — polls the stored odom pose every 0.1 s until one exists or timeout runs out; returns (x, y, yaw) or None. With `max_age` set, a pose older than that many seconds is refused (returns None): driving the dog by remote stops USLAM and freezes the last pose at where the dog USED to be, and seeding from that is worse than not seeding at all. Odom only flows while localization runs, so before bringup this correctly returns None.
- `now()` — monotonic timestamp; callers pass it as the since mark so `wait_for_any` only matches lines that arrived after the command went out.
- `wait_for_any(substrings, since, timeout)` — blocks until a buffered line at/after `since` contains any of the substrings; returns the matched substring or None on timeout. This is the entire event mechanism. Algorithm: on every wake it rescans the full 500-line deque and matches only lines stamped at/after `since` (the monotonic mark taken BEFORE the command went out, so a stale reply from an earlier command can never satisfy a new wait); the condition-variable `wait` uses the remaining time, so the deadline is exact and a matching line wakes it immediately.
- `_send_raw(data)` — the single wire writer: wraps the string in a `std_msgs/String` and publishes it once on `/uslam/client_command`. All quoting is decided by the two callers below; `_send_raw` adds nothing.
- `send_verb(verb, repeat=1, gap=0.4)` — publishes a control verb WITH the inner double quotes, repeat times with a 0.4 s gap. Default repeat is 1 (see 5.5, the MCU wedge rule; it was 3 until 2026-07-30).
- `send_goal(x, y, yaw, repeat=1, gap=0.4)` — publishes `navigation/set_goal_pose/x/y/yaw` as a BARE path. Repeat 1 for the same reason plus `GOAL_CHANGED`.

The two senders are the whole wire protocol in miniature — the quoting split and the `repeat=1` rule are both visible in the code:

```
def send_verb(self, verb, repeat=1, gap=0.4):
    log(f" -> CMD (verb) \"{verb}\"")
    for _ in range(repeat):
        self._send_raw(f'"{verb}"')          # verb rides WITH inner quotes
        time.sleep(gap)

def send_goal(self, x, y, yaw, repeat=1, gap=0.4):
    path = f"navigation/set_goal_pose/{x:.6f}/{y:.6f}/{yaw:.6f}"
    log(f" -> CMD (goal) {path}")
    for _ in range(repeat):
        self._send_raw(path)                  # goal is a BARE path, no quotes
        time.sleep(gap)
```

The repeat loop is the 2026-07-30 fix frozen into the defaults. Repeat was 3 — a hedge against a missed first publish before DDS discovery settles — and that hedge is exactly what wedged the MCU (each duplicate verb counts against the wedge budget) and randomised waypoints (a duplicate `set_goal_pose` arriving after TRACKING raises

GOAL\_CHANGED and kills the goal in flight). Discovery is already established by the get\_map\_id handshake at bringup, so one publish is enough; callers keep the repeat parameter only so a future caller can raise it deliberately, never by default. (One doc lag inside the code: send\_verb's docstring still carries the "sent a few times" DDS-discovery rationale from the repeat=3 era. The shipped default is 1; the docstring is stale.) - bringup() — the sequence of 5.5 as code: 1. get\_map\_id with the 8 s liveness timeout — on silence it logs "POWER-CYCLE the whole dog" and returns False. It logs the map id line it got back (the live map id read — no hardcode) and establishes DDS discovery. 2. Unless SKIP\_LOCALIZATION\_BRINGUP, loop up to LOCALIZE\_ATTEMPTS times: pick a seed — a FRESH live pose (max\_age=3.0) if odom is flowing, else INITIAL\_POSE (in which case the dog must be parked on it); if the seed was live, send localization/stop and wait for its success (fresh odom is the proxy for "something is running"; no odom = nothing to stop, skip the churn); set\_initial\_pose with the seed; localization/start; wait on both OK and FAIL tokens. Break on success; otherwise sleep 2 s and retry the whole stop -> seed -> start sequence. 3. If NUDGE\_ENABLED: send one goal at the current (or seed) position with yaw + 0.30 rad, sleep NUDGE\_SETTLE\_S. The nudge is deliberately NOT verified — a pose-change check was tried on 2026-07-30, reported "did not move" three times per run while navigation/start then succeeded first try, and cost 37 s per run without ever changing an outcome. 4. navigation/start, up to NAV\_START\_ATTEMPTS tries; each refusal re-nudges with double delta before retrying. Any failed stage returns False and the patrol aborts. - goto(x, y, yaw) — send one goal, wait up to 90 s for state\_transition/REACHED or a failure token; on failure or timeout, log FAIL-LOUD and retry once (NAV\_RETRY\_ONCE); returns True only on REACHED. The REACHED token and the four NAV\_FAIL\_TOKENS sit in ONE wait\_for\_any call, so a NO\_PATH returns immediately instead of burning the 90 s budget; the retry sleeps 1 s and re-sends the identical goal (a second send\_goal call, still repeat=1 — this is a new goal after the old one died, not a duplicate into a live one).

**Class Gimbal** — SIYI A8 control via the mzahana siyi\_sdk, entirely fail-soft: any gimbal error only logs, the patrol never stops for it.

- `__init__` appends `~/a8/siyi_sdk` and `~/a8` to `sys.path` (this sdk build is flat-layout: from `siyi_sdk` import `SIYISDK`, not package-style).
- `connect()` — opens the UDP link to `192.168.144.25:37260` and calls `_set_follow()`. On ANY failure it returns False and the patrol runs without gimbal control.
- `_set_follow()` — `requestFollowMode()`: gimbal yaw tracks the dog's body. FOLLOW is also the saved power-on default (5.11), so this call is defensive, not load-bearing.
- `apply_override(cam)` — the per-waypoint camera override interface. A waypoint may carry `"cam": {"pitch_down": deg 0..25, "yaw_offset": deg (+ = lens right), "zoom": absolute (1.0 = wide)}`. Absolute angles need LOCK mode, so this calls `requestLockMode()`, then `requestSetAngles(yaw_off, pitch_dn)` (pitch positive = down on this inverted mount), then `requestAbsoluteZoom` if zoom is given.

No deployed waypoint currently carries cam; the interface exists for future fine-tuning.

- `restore_follow()` — back to FOLLOW right after the capture at an override point.
- `close()` — disconnect at patrol end; one benign [Errno 9] Bad file descriptor is normal.

### Capture / upload / gate functions:

- `capture_frame()` — opens the A8 RTSP stream with OpenCV over TCP (`OPENCV_FFMPEG_CAPTURE_OPTIONS=rtsp_transport;tcp` is set before `import cv2`), reads `CAPTURE_WARMUP = 30` frames keeping the LAST good one (draining the decoder's stale buffer so the kept frame is current), JPEG-encodes at quality 92, returns bytes or None. Frames are 1920x1080 (the RTSP cap). Three failure exits, each one log line and None: stream will not open, stream opened but zero frames decoded, JPEG encode failed — the caller then skips upload and gate for that point and the patrol continues.
- `upload_frame(s3, waypoint_name, jpeg_bytes)` — `put_object` to `frames/{CAMERA_ID}/{waypoint}/{UTC timestamp}.jpg` in `argus-frames-506868652945`. The `worm_cam` key segment routes the frame to the armyworm camera config cloud-side, where tiling and the LLM gate apply. Returns the S3 key. It does not catch its own exceptions — the caller in `main()` wraps it and skips the gate on failure.
- `wait_for_detection(ddb_table, image_key)` — polls pest-monitoring-detections with a Key query on `image_id == the S3 key (Limit=1)`, every 1.5 s for up to 150 s. **The processor's `put_item` is UNCONDITIONAL — clean frames (zero detections) also write a record — so the gate cannot dead-lock on a clean waypoint.** When the record appears it logs detected, `target_confidence`, and box count, and opens the gate. A query exception inside the loop only logs and retries on the next poll — a transient DynamoDB or network error cannot abort the patrol. On timeout it proceeds anyway (fail-open) with a loud log line. Measured end-to-end on site: 10–47 s per frame depending on the detection config (13/13/10 s gates on the 2026-07-29 5/5 run; 19/20/19 s on the 2026-07-30 zero-retry run). The current cloud side — processor v6.3 (dead code stripped 2026-08-07) with the v9-family model, tiling, the Sonnet 4.6 verify gate, display floor 33 (`post_verify_floor`; the 2026-08-10 decision was 49, refitted per-build to 33 on 2026-08-13) — measures 24–54 s per tiled frame, still inside the 150 s gate budget.

**`main()` — the loop.** Build boto3 S3 + DynamoDB clients; `gimbal.connect()` once for the whole patrol; init `rcipy` and spin `USLAMClient` in a daemon thread. Print the safety banner and the countdown (this is when the operator unplugs the cable). `bringup()` or `abort`. Then for each waypoint in order: `goto()`; on failure count it and (since `ABORT_ON_NAV_FAIL=False`) skip to the next point. On REACHED: if `"capture": False`, continue (navigate-only). Otherwise apply the cam override if present; sleep `GIMBAL_SETTLE_S` (override applied) or `FOLLOW_SETTLE_S` (pure FOLLOW catching up after

the dog's final in-place rotation); `capture_frame()`; restore FOLLOW if an override was applied; upload; `wait_for_detection()`. Every scan step fails soft: a None frame or an S3 upload exception logs, skips the gate for that point, and continues the route — only bringup failure aborts the patrol. Finish with a reached/failed summary; the finally block closes the gimbal and shuts rclpy down even on Ctrl+C.

**Data in/out per scan waypoint:** USLAM goal out on `/uslam/client_command`; REACHED in on `/uslam/server_log`; one JPEG in memory from the A8 RTSP; one S3 object out; one DynamoDB record polled in. Nothing is stored on the Orin.

**Validation record (2026-07-30, Jewel).** Three consecutive 5/5 runs, app-free, cold boot and back-to-back both proven; best run 150 s with zero retries (`robot/_archive/2026-07-30/patrol_run4_clean_zero_retries.log`). Judged by what the demo needs — the three zone captures — earlier runs that day, before the `repeat=1` fix, scored 3/3, 3/3, 2/3; the failures cluster on wp1, the first goal after bringup (navigation tracking not fully warmed), and cost nothing because wp1 captures nothing. One measured caveat: at the moment REACHED fires the believed pose is within 0.013–0.041 m of the goal, yet the dog physically parks short and the pose re-converges over the next minutes to 0.17–0.46 m off — localization carries error during the run and the planner declares arrival on the wrong estimate. Re-recording `wp_return` would not fix this; the dog would stop short of the new target too.

## 5.8 pose.py — one-shot pose grab

`robot/pose.py` (Orin: `~/go2/pose.py`) is the basic survey tool. Run once at a spot; it subscribes to `/uslam/localization/odom`, keeps the FIRST frame, prints exactly one line and exits:

```
x= 0.143  y= -0.009  yaw= -1.569
```

- Class `OneShot(Node):` `_cb(m)` converts the quaternion to yaw with the project convention  $2 * \text{atan2}(q_z, q_w)$  wrapped to  $[-\pi, \pi]$ , stores the first result, and wakes main via a `threading.Event`.
- 20 s timeout. Because odom only publishes while the dog moves, “NO ODOM within 20s” usually just means the dog is standing still — nudge it with the remote and re-run. It is also the avoidance-OFF symptom (5.5). Exit code 0 on success, 1 on timeout.

Three levels of survey tooling exist, in the order they were built:

1. `pose.py` — one shot per point (nudge, run, read).
2. `tests/wp_survey.py` — hands-free: watches the odom stream for motion-then-silence (stand still ~5 s at each spot), records the last pose before each silence, and prints a paste-ready WAYPOINTS block. Sanity caps: minimum 0.25 m separation, max 12 points.

3. `tools/pose_logger.py` — continuous logger to `/tmp/pose_log.txt`; the Jewel v2 route was surveyed by walking the route twice with this running and extracting the dwell points afterwards. `tools/localize_only.py` complements it for parking spots: it does the localization half only (stop -> seed -> start, importing the patrol's own machinery) without the heading-destroying nudge, then reports the settled pose.

## 5.9 The tests harness (`robot/tests/`)

These scripts validate navigation WITHOUT touching `go2_patrol_gated.py` — pure USLAM, no gimbal, no S3/DDB. `wp_test_1` and `wp_test_2` keep their run logs next to them (the `.log` files are the data product); `wp_test_3` and `wp2_probe` have no repo logs. They predate the 2026-07-30 `repeat=1` rule, so their `send_verb/send_goal` defaults are still 2–3; run them as-is for lab regression only, and copy the patrol's `repeat=1` discipline into anything new.

**`wp_test_1.py` — round 1.** Same wire protocol as the patrol, three safety gates before any traverse: (1) `get_map_id` silent -> abort (MCU down); (2) localization does not init -> abort; (3) zero-distance sanity goal (= the seed pose itself) rejected -> abort (dog not at start / bad localization). Then the route, one retry per point. Round 1 aborted in the field: an app re-calibration session between runs reset the navigation state, so the one-shot nudge no longer activated tracking and navigation/start timed out.

**`wp_test_2.py` — round 2, the lab 4/4 proof.** Replaces the one-shot nudge with a PATIENT LOOP: for up to `TRACKING_WINDOW_S = 240` it alternates nudge goals and navigation/start attempts while the operator wiggles the dog with the remote (the script prints “>>> WIGGLE THE DOG WITH THE REMOTE NOW <<<”). Real motion -> localization tracks -> navigation/start succeeds -> break. Then the zero-distance sanity gate, then the route. Result on the PRIMARY lab map: 4/4 REACHED in ~50 s from a cold boot, zero app involvement (2026-07-03). Every abort path prints a machine-greppable verdict line (### TEST COMPLETE ### verdict=...). Note the operator-wiggle technique is now superseded for patrols by the scripted nudge, and post-2026-07-30 the wiggle needs care: only remote BUTTON presses are proven to stop USLAM (the server log line is “Joystick button is pressed! Uslam is stopped now!”); stick-only driving was used successfully as the wiggle on 2026-07-03 without stopping USLAM, but no stick-only test exists after the 2026-07-30 finding. If wiggling, use sticks only, touch no buttons, and treat even that with caution.

**`wp_test_3.py` — same skeleton for the lab DEMO map.** Route sanity -> wp2 -> wp3, ending at wp3 (no return leg). The zero-distance sanity goal doubles as a rehearsal of that route's wp1 capture-in-place behaviour.

**`wp2_probe.py` — the planner-probe methodology.** Field lesson 2026-07-09, worth stating as a rule: **“the dog can stand there” does not mean “the planner will go there.”** During the DEMO-map patrol, wp1 and wp3 REACHED but wp2 at (1.143, -0.176) was refused INSTANTLY with FAILURE twice. Instant refusal with everything else passing (zero-distance sanity OK, other goals OK) means the goal CELL is bad — edge of free space / too near an



obstacle on a quickly-recorded map — NOT mislocalization (whose signature is EVERY goal failing, 5.5). The probe sends candidate goals and stops at the first one the planner accepts; that candidate becomes the new waypoint. Round 1 probed 5 candidates around the original spot — all refused (a genuine dead zone). Round 2 probed a physically different spot ~2.5 m away, which REACHED and was committed to the patrol, `map_profiles.md`, and `wp_test_3.py`. If no candidate is accepted: pick a physically different spot, or re-record the map slowly with full coverage of that area. Consequence for all future routes: validate every new waypoint with a probe goal, not just with `pose.py`.

**nav\_probe.py — is the map plannable at all.** One short goal into floor the dog just drove across, from a verified-accurate localization. REACHED = the planner works, the waypoints were just in bad cells (fix coords). NO\_PATH = the map itself is unplannable (re-record slowly, full coverage, loop closure). Its header carries the landmine warning: a stale SEED pose makes every goal NO\_PATH and the verdict meaningless — always re-read odom first.

## 5.10 kvs-controller.service — the A8 live-stream producer

**Purpose.** Push the A8's H.264 RTSP feed to the Kinesis Video Streams stream `armyworm-cam-stream` whenever the dashboard's live-view toggle is on. Pure passthrough — no re-encode. Boot-persistent: the systemd service plus the `a8-link` autoconnect profile mean the stream capability survives an unattended Orin reboot with no operator action (verified; it also ran throughout the 2026-07-30 Jewel visit).

Three files:

- **kvs-controller.service** (systemd unit): `User=unitree`, `WorkingDirectory=/home/unitree/amazon-kinesis-video-streams-producer-sdk-cpp/build`, `ExecStart=/home/unitree/run_kvs_controller.sh`, `Restart=always / RestartSec=5`, `WantedBy=multi-user.target`.
- **run\_kvs\_controller.sh** (wrapper): exports `GST_PLUGIN_PATH` and `LD_LIBRARY_PATH` for the KVS producer SDK build; reads `aws_access_key_id` / `aws_secret_access_key` out of the standard `~/.aws/credentials` (the `cag_user` profile) with `awk` and exports them so the `kvssink` GStreamer element can authenticate — credentials stored there, not reproduced anywhere in the script or this manual; exports `CAMERA_ID="worm_cam"`; cds to the SDK build dir (`kvssink` writes `./log/kvs.log` relative to CWD) and execs the Python daemon.
- **kvs\_controller.py** (daemon): a reconcile loop.
  - `fetch_desired_state()` — GET `{API_BASE}/stream/status?camera=worm_cam` against the `pest-monitoring-api` API Gateway, once per reconcile cycle (`POLL_INTERVAL_SEC = 5`), with a 10 s HTTP timeout on each request. The code's default `API_BASE` is the production API `https://vzf17s6z00.execute-api.us-east-1.amazonaws.com` (repointed 2026-08-13; the dev account's `zwpcbivmsj` API is retired history). Plain HTTPS, no AWS credentials — this route is deliberately exempt from the dashboard's Cognito JWT authorizer so device

pollers stay untouched by auth. Returns (stream\_enabled, kvs\_stream\_name) parsed from the JSON body, or (None, None) on ANY error (URL, HTTP, JSON decode, socket — all caught), in which case the loop keeps its current state and retries next cycle.

- `build_gst_args(stream_name)` — `gst-launch-1.0 rtspsrc location=rtsp://192.168.144.25:8554/main.264 protocols=tcp latency=0 drop-on-latency=true ! rtph264depay ! h264parse ! kvssink stream-name=<name> storage-size=512 aws-region=us-east-1 fragment-duration=2 max-latency=2`.
- `is_gst_running()` — the one liveness check: True while the child-process handle exists and `poll()` reports it has not exited.
- `start_gst(stream_name)` — refuses a double-start (`is_gst_running()` guard), then spawns `gst-launch-1.0` in its OWN process group (`preexec_fn=os.setsid`) with `GST_PLUGIN_PATH / LD_LIBRARY_PATH` injected into the child's environment. `stdout/stderr` are discarded on purpose — `kvssink` writes its own `./log/kvs.log` under the SDK build dir.
- `stop_gst()` — kills the whole process group: `SIGTERM` first, wait up to 5 s, escalate to `SIGKILL` and wait 2 s more (`gst-launch` forks helpers, so group-kill matters; a process already gone is treated as stopped). Always clears the child handle.
- `main()` — the reconcile loop, every 5 s: fetch desired state; a failed fetch just sleeps and retries, changing nothing. If the child died on its own, log its return code and clear the handle. Then reconcile: desired ON and not running -> `start_gst`, UNLESS `kvs_stream_name` came back empty — then it logs an error and starts nothing (the guard for a camera row with the toggle on but no stream name). Desired OFF and running -> `stop_gst`. The (desired, running) pair is logged only when it changes, so a healthy idle loop is silent. This loop is what turns the dashboard's DynamoDB `stream_enabled` flag into real video. `graceful_exit` (the `SIGTERM/SIGINT` handler) tears the pipeline down on `systemd stop / Ctrl+C` so no orphan keeps pushing to KVS.
- NOTE: doc lag inside the file — `main()`'s docstring still says `camera=armyworm_go2_a8mini` (the code default and the exported env are `worm_cam`, the 2026-07 camera-id migration), and the module header plus the `build_gst_args` docstring still name the retired dev CLI profile `nbk2`. The operative values are the code's: `worm_cam` and the production `API_BASE`. The code wins.

The environment can override everything (`CAMERA_ID`, `API_BASE`, `RTSP_*`, `KVS_SDK_DIR`); the same daemon source drives the mini PC's `moth-cam-stream` instance with different env (Chapter 6).

Operational rules: the KVS pair is driven by the `systemd` service — do not run `run_kvs_controller.sh` by hand; restart with `sudo systemctl restart kvs-controller`.

Live streaming and patrol capture were confirmed non-contending on the A8 RTSP (both can read the stream at once). One failure signature worth knowing: because the daemon only polls `/stream/status`, it stays active (running) even when the RTSP source is unreachable — the fault only surfaces when the dashboard actually enables the stream. This is exactly what happened during the 2026-07-29 HDMI detour.

Production status (2026-08-14): the daemon's default `API_BASE` targets the production API (repo mirror repointed 2026-08-13), and the production `worm_cam` row reads `kvs_stream_name = armyworm-cam-stream, stream_enabled = false` (verified against the live row 2026-08-14). The missing piece is the stream itself: the 2026-08-10 deploy ran with live view off, so NO KVS streams exist on the production account — create `armyworm-cam-stream` there (5.12.6) before enabling the dashboard toggle, or the daemon will start a pipeline with nothing to write to. The on-device unit also still needs the 5.12.6 device pass (mirror sync + credential swap; `kvs-controller.service` may carry its own `Environment=` overrides for `API_BASE` — check both units during the pass). Whether the demo needs live view on the production account is an open call.

## 5.11 Gimbal control map (SIYI A8 Mini)

The control mapping on this unit (`mount=2`, unfixable in firmware) — needed by anyone extending the cam override interface:

- **FOLLOW mode** — gimbal yaw tracks the dog's body. This is the wanted default. It is saved as the POWER-ON default in SIYI PC Assistant (work mode -> Follow -> save) and persists across reboots, so the SDK never needs `requestFollowMode()` at startup (the patrol still calls it defensively on connect, which is harmless).
- **The LOCK boot bug and its fix.** Root cause of an early field failure: the A8 was booting in LOCK mode, so the gimbal did not track the dog's heading and captures pointed the wrong way. Fix = save FOLLOW as the power-on default in SIYI PC Assistant, as above. Fixed and closed; if captures ever point wrong again after a gimbal swap or firmware flash, re-check this setting first.
- **LOCK mode** is required for absolute angle commands (`requestSetAngles`). Yaw is 1:1 (+ = lens right). **Pitch is INVERTED: + = lens DOWN**, clamped to `[-90, +25]`, command 0 = level-forward.
- `getAttitude()` pitch reads back WRAPPED — do not use it for control.
- `[Errno 9] Bad file descriptor` on disconnect is benign.
- `FOLLOW_SETTLE_S = 2.0` in the patrol exists because FOLLOW lags the body by ~1–2 s; capture without the wait gets a mid-swing frame.
- Firmware note: `docs/hardware.md` records fw 0.2.8 for the control-mapping findings; the 2026-07-29 `a8_status.py` readout after the ethernet revert reported fw 09030073. No repo source reconciles the two ids and no flash is recorded; the control mapping has not been re-characterized against the second number. Settle it by re-reading the unit (SIYI PC Assistant or `a8_status.py`) before extending the cam override interface.

## 5.12 Operations and reproduction

### 5.12.1 Cold boot sequence

1. Power on the Go2. USLAM auto-starts on the sport MCU and auto-loads the last map (currently Jewel v2). No app needed for a normal boot.
2. The Orin boots with it. `kvs-controller.service` starts on its own; `a8-link` autoconnects to the gimbal; the WiFi profile with the highest priority that is in range associates on its own (verified for `Apps@Jewel` after a power cycle, including the 1970 -> real clock correction). Nothing to do.
3. If the loaded map is NOT the one you intend to patrol: open the Unitree app once, select the map, localize on it. This is per-map-switch, not per-boot.
4. Confirm obstacle avoidance is ON on the remote (odom is silent otherwise, 5.5).
5. First ROS run after a reboot is the clean one (the CycloneDDS binding segfaults intermittently otherwise). If a script segfaults at import, `sudo reboot` and run again.

### 5.12.2 Running a patrol

Standing procedure, exactly as validated on 2026-07-30:

1. Park the dog physically on `INITIAL_POSE = wp_return = (-4.970, -0.657, 1.260)`. Do any manual repositioning NOW — never mid-run.
2. SSH in and launch (tmux keeps the run alive if the SSH session drops):

```
ssh unitree@<orin-ip>
tmux new -s patrol
# interactive shell: answer 1 (foxy) at the fishros prompt, then:
source ~/setup_go2.sh
python3 ~/go2/go2_patrol_gated.py
```

3. During the countdown: unplug the external cable (untethered rule), clear the area, keep the remote in hand as the e-stop — and remember that pressing ANY button on it stops USLAM and ends the run (5.5). Detach tmux with `Ctrl+b d`.
4. Watch the log: `bringup` (map id -> localization attempt N -> nudge -> navigation started), then per waypoint `REACHED` -> `Captured` -> `Uploaded` -> `GATE open`.

Which `<orin-ip>` works depends on the site: at NP, the campus WiFi address; at Jewel, `Apps@Jewel` has AP client isolation, so the laptop cannot reach the Orin over that SSID — use the wired `192.168.123.18` link for setup (unplug before motion) or the iPhone Air hotspot profile for untethered supervision.

This manual launch is still how every patrol run to date has been started. 5.13 covers the systemd daemon that automates the launch step itself (not the physical safety steps, which stay manual) against the dashboard's existing Schedule panel.

Before a run where you want real detections: start the Rekognition Custom Labels model (the processor writes clean records when the model is stopped — the gate still opens, but with no boxes). On the production account you do not need to remember the stop: the

pest-model-watchdog Lambda (the v6.2 per-camera build, on a 15-minute EventBridge schedule) auto-stops the endpoint once it has run past the camera row's `max_runtime_min` — 45 minutes on `worm_cam` — so start the model shortly before the patrol and expect it to be stopped for you afterwards. Model start/stop procedures are in Chapter 3; the detection configuration the gate waits on is in Chapter 2.

### 5.12.3 Surveying and validating a new route

1. Record the map with the app (drive slowly, full coverage, loop closure). Keep clearance from walls and low planting.
2. Localize on the new map and CONFIRM it against `/uslam/server_log` — a success token plus a visible seed-to-converged correction (5.5). Do not trust `localization/start/success` alone: a survey taken in an unmatched frame is void, which is how the first Jewel map was lost.
3. Survey the points: walk the route with `tools/pose_logger.py` running and extract dwell points (best), or stand at each spot for `tests/wp_survey.py`, or take `pose.py` one-shots. Cross-check at least one point against a known physical spot.
4. Add a new map block (INITIAL\_POSE + WAYPOINTS) to `robot/map_profiles.md`.
5. Validate nav-only first (copy the `wp_test_3.py` skeleton with the new coords). Any instantly-refused point: probe alternatives with the `wp2_probe.py` pattern — a probe goal, not `pose.py`, is what proves a waypoint (5.9).
6. Only then paste the block into `go2_patrol_gated.py` and push to the Orin.

### 5.12.4 SSH conventions (from the laptop)

- Remote ops use **Posh-SSH** (PowerShell module, password auth) from the Win11 laptop.
- Non-interactive exec channels get no ROS from `.bashrc` (the fishros block prompts and bails). Prefix every ROS command with the triple source: `source /opt/ros/foxy/setup.bash && source ~/cyclonedds_ws/install/setup.bash && source ~/setup_go2.sh && <command>`.
- Long jobs: launch detached — `nohup python3 -u ~/go2/tests/wp_test_2.py > ~/go2/tests/wp_test_2.log 2>&1 &` — then poll the log file.
- Killing a detached job from an exec channel: `kill -f` must use a NON-self-matching pattern, e.g. `kill -f "[w]p_test"` — a plain pattern matches the `kill` command line itself.
- `ros2 topic pub` with `-r N` (continuous) drops the SSH session — use `--once` and resend 1–2 times. Post-2026-07-30 caveat: resends of control verbs count against the MCU wedge budget (5.5) — resend sparingly and only when the reply token did not come back.
- Writing files to the Orin over SSH: nano is installed; heredoc `cat > file << 'EOF' ... EOF` works for scripted writes.
- The AWS CLI on the Orin is v1 (parameter format differs from v2 on the laptop, notably for `kinesisvideo-archived-media`).

### 5.12.5 Failure triage

| Symptom                                                                                                                | Diagnosis                                                                 | Recovery                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| get_map_id silent (patrol logs FATAL)                                                                                  | USLAM service crashed on the MCU                                          | Full power-cycle of the dog. Nothing on the Orin can restart it.                                                   |
| TIMEOUT_ODOMETRY + TIMEOUT_POINTCLOUD flooding<br>/uslam/server_log; dog only turns in place; get_map_id still answers | MCU wedged by rapid repeated control verbs / stops                        | Power-cycle the dog. Then find and remove whatever was spamming verbs.                                             |
| EVERY goal instantly NO_PATH/FAILURE, even zero-distance; dog shuffles without departing                               | Mislocalization (frame never matched the map, or localization went stale) | App re-localize, confirm via server_log token + seed correction, re-survey same session. Do NOT re-record the map. |
| ONE goal instantly FAILURE, others fine                                                                                | Bad goal cell (dead zone)                                                 | Probe alternatives (wp2_probe.py pattern); move the waypoint.                                                      |
| localization/start succeeds then initialization failed! ~6 s later, repeatedly                                         | Started on top of an already-running localization                         | Send localization/stop first (the patrol does this since 2026-07-30).                                              |
| Localization init fails intermittently with a good seed                                                                | Known ~50-60% per-attempt flake, cause unidentified                       | Let LOCALIZE_ATTEMPTS = 4 retry; park the dog on INITIAL_POSE if odom is not flowing.                              |
| Patrol aborts at bringup right after the dog was driven by remote                                                      | A button press stopped USLAM / the frozen stale pose was refused as seed  | Re-launch; position the dog by hand only BEFORE launching.                                                         |
| pose.py “NO ODOM”; survey records nothing while the dog walks                                                          | Obstacle avoidance toggled OFF                                            | Re-enable avoidance on the remote.                                                                                 |
| navigation/start never succeeds                                                                                        | Dog not moving, so localization not tracking                              | Let the retry re-nudge (it doubles the delta); or raise NUDGE_DELTA_YAW.                                           |
| Python segfault at CycloneDDS import                                                                                   | Intermittent binding fault                                                | sudo reboot; first run after reboot is clean.                                                                      |
| Captures point the wrong way                                                                                           | Gimbal booted in LOCK                                                     | Re-save FOLLOW as power-on default in SIYI PC Assistant (5.11).                                                    |

| Symptom                                                 | Diagnosis                                                             | Recovery                                                                                                                                                                 |
|---------------------------------------------------------|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gimbal or RTSP unreachable at 192.168.144.25            | a8-link not up — MAC bind mismatch or a rogue profile holding the NIC | <code>nmcli con show</code> ; re-bind a8-link to the fitted dongle's MAC; keep Wired connection 2 autoconnect off (5.3).                                                 |
| No live stream on dashboard, service “active (running)” | Controller only polls; source or toggle is the fault                  | Check <code>GET /stream/status?camera=worm_cam</code> returns <code>stream_enabled=true</code> ; ping 192.168.144.25; <code>systemctl status kvs-controller</code> last. |

### 5.12.6 Reproducing the AWS side on a new account

This is no longer hypothetical: the procedure was executed for real on 2026-08-10/11, when the ARGUS deployer (Chapter 7) stood up the full cloud stack on the NP production account 506868652945 in 103 s (all 15 stages). That deploy fixes the names the devices must use at cutover:

- Frames bucket: `argus-frames-506868652945` (ARGUS naming, not the `frames-armyworm-<account-id>` pattern of the dev account).
- Armyworm camera id: `worm_cam`, unchanged — the deployer seeded the row as `camera-1` but it was aligned to the reference id on 2026-08-11 (plus `manual_upload`; a `moth_cam` row was seeded separately for the moth rebuild). The Orin's `CAMERA_ID` stays `worm_cam`.
- API Gateway: `https://vzf17s6z00.execute-api.us-east-1.amazonaws.com`; dashboard `https://d1dtoxef7qmugl.cloudfront.net`.
- KVS: skipped (live view off) — no streams exist on the production account, so the `kvs-controller` service has nothing to drive there yet.

**Where the cutover stands (2026-08-14).** The cloud side is done: the stack runs on production, both models are retrained there (`armyworm v9r-prod-20260810`, `moth moth-prod-20260811`), and the repo mirrors of the edge scripts were repointed to the production values on 2026-08-13. The DEVICE side is not: the hardware was away when the repoint was made, so the Orin and mini PC still run the old script copies with dev-account `cag_user` keys, and still upload to the dev account until the pending device pass (sync the mirrors onto the devices, swap `~/aws/credentials` to a production-account key). The generic procedure below remains the reference for any future fresh account (and for creating the pieces the deployer skipped, like the KVS stream and the device IAM user).



The edge scripts touch four AWS resources. On a fresh account, create/verify each (region us-east-1 throughout; full backend creation order is in Chapter 2, and the ARGUS deployer of Chapter 7 automates most of it):

- **Frames bucket.** Console: S3 -> Buckets -> Create bucket -> name it (the pattern used here is frames-armyworm-`<account-id>`). CLI: `aws s3 mb s3://**your-frames-bucket** --region **us-east-1**` Verify the patrol's uploads land — Console: S3 -> **your-frames-bucket** -> frames/worm\_cam/. CLI: `aws s3 ls s3://**your-frames-bucket**/frames/worm_cam/ --recursive --region **us-east-1**`
- **Detections table.** Console: DynamoDB -> Tables -> pest-monitoring-detections -> Explore table items -> Query, partition key image\_id = the S3 key the patrol logged. CLI: `aws dynamodb query --table-name **pest-monitoring-detections** --key-condition-expression "image_id = :k" --expression-attribute-values "{\":k\":{\":S\": \"**frames/worm_cam/zone1/<timestamp>.jpg**\"}}\" --region **us-east-1**`
- **KVS stream.** Console: Kinesis Video Streams -> Video streams -> Create video stream -> name armyworm-cam-stream. CLI: `aws kinesisanalytics create-stream --stream-name **armyworm-cam-stream** --data-retention-in-hours **24** --region **us-east-1**`
- **Device IAM user.** Console: IAM -> Users -> Create user (the pattern here is cag\_user) -> attach a policy allowing s3:PutObject on the frames bucket, dynamodb:Query on the detections table, and the KVS producer actions (kinesisvideo:DescribeStream, kinesisvideo:GetDataEndpoint, kinesisvideo:PutMedia) -> create an access key. Then on the Orin run `aws configure` as user unitree and enter the key pair there — the key lands only in `~/.aws/credentials` on the device, never in any script or repo file. CLI (policy attach): `aws iam put-user-policy --user-name **cag_user** --policy-name **edge-device-policy** --policy-document file:///**policy.json**`

Then edit the constants at the top of `go2_patrol_gated.py` (`S3_FRAMES_BUCKET`, `CAMERA_ID`, table name if changed) and the `API_BASE / CAMERA_ID` env for `kvs_controller.py` to point at the new account, push both to the Orin, and `sudo systemctl enable --now kvs-controller`. For the 2026-08 production cutover specifically, the repo-side edits are ALREADY DONE (2026-08-13): `S3_FRAMES_BUCKET = argus-frames-506868652945` in `robot/go2_patrol_gated.py` and `robot/capture_4k_hdmi.py`, `API_BASE` default = `https://vzfl7s6z00.execute-api.us-east-1.amazonaws.com` in `robot/kvs_controller.py`, and `CAMERA_ID` unchanged at `worm_cam` (the production row was re-keyed to the reference id 2026-08-11). Both production models are trained. What remains is the device pass: sync the mirrors onto the Orin (`~/go2/`), swap the on-device `~/.aws/credentials` to a production-account key, check `kvs-controller.service` for on-device `Environment=` overrides, and hold the `kvs-controller` start until a KVS stream exists on that account.



Security note for reproducers: never commit or paste AWS secret keys, device passwords, or tokens into any project file — the device `~/ .aws/credentials` file is their only home on the edge. One historical caution: a Roboflow API key sits inside `datasets/archive/experiments/pre_v3_abandoned/download.py` in the repo (credential stored there, not reproduced); treat that folder as sensitive and do not publish it.

## 5.13 patrol-scheduler.service — auto-launch on the dashboard schedule

**Purpose.** Close a gap between the dashboard and the robot: the Schedule panel's /schedule row (Chapter 4) previously only drove pest-camera-scheduler (Rekognition model start/stop, Chapter 3) — nothing turned that same schedule into an actual patrol launch, so `go2_patrol_gated.py` stayed purely manual (5.12.2), unlike the KVS pair's systemd-driven poll loop (5.10). `patrol_scheduler.py` is the missing listener, built as a second daemon on the same pattern as `kvs_controller.py`.

Three files, same shape as the KVS trio in 5.10:

- **patrol-scheduler.service** (systemd unit): `User=unitree, WorkingDirectory=/home/unitree/go2, ExecStart=/home/unitree/run_patrol_scheduler.sh, Restart=always / RestartSec=5, WantedBy=multi-user.target.`
- **run\_patrol\_scheduler.sh** (wrapper): sources the ROS 2 Foxy environment non-interactively — `/opt/ros/foxy/setup.bash -> ~/cyclonedds_ws/install/setup.bash -> ~/setup_go2.sh` — then execs the daemon. This triple-source is required because `.bashrc`'s fishros block prompts foxy/noetic interactively (5.12.4); under systemd that prompt would just hang forever with no ROS environment loaded. The daemon's subprocess launch of `go2_patrol_gated.py` inherits this same sourced environment, so the patrol script's `rclpy` import and `/uslam/*` topics work without re-sourcing per run.
- **patrol\_scheduler.py** (daemon): a reconcile loop, structurally close to `kvs_controller.py`.
  - `fetch_schedule()` — GET `{API_BASE}/schedule?camera=worm_cam` against `pest-monitoring-api`, once per `POLL_INTERVAL_SEC = 30`. Same reasoning as the KVS poller: plain HTTPS, no AWS credentials on the control path. Returns `{"enabled", "start_time", "days"}` or `None` on any error, in which case the loop just retries next cycle.
  - `schedule_matches_now()` — converts the dashboard's Singapore-local `start_time` against the current time computed as UTC+8 (fixed offset, no DST), the same conversion `_cron_expression` already does server-side for the EventBridge rule (Chapter 2); the daemon deliberately does not trust the Orin's own system timezone. An empty `days` list means every day, matching the API's own default.

- **Safety arm gate — the reason this is not a plain cron job.**  
go2\_patrol\_gated.py needs a human physically present: remote in hand as e-stop, area cleared, external cable unplugged before motion (5.12.2). A scheduled trigger cannot satisfy any of that by itself, so the daemon only launches if ~/go2/.patrol\_armed was touched within the last PATROL\_ARM\_MAX\_AGE\_MIN (default 60) minutes — the human does the pre-flight check on site and then touches that file shortly before the scheduled time. A schedule match with a stale or missing arm file is logged once per day and skipped, not forced through. PATROL\_REQUIRE\_ARM=0 disables the gate; this is not recommended for any run at a real venue.
- launch\_patrol() — refuses a double-launch if a patrol subprocess handle is already alive, then spawns go2\_patrol\_gated.py in its own process group (preexec\_fn=os.setsid, matching start\_gst()'s pattern in 5.10), with output to a fresh timestamped file under ~/go2/patrol\_logs/ — a scheduled run has nobody watching a tmux session, so the log file is the only record.
- Dedup state (~/go2/.patrol\_scheduler\_state.json, last\_fired\_date) fires at most once per SGT calendar date, so the 30 s poll can't double-launch inside the matching minute, and a service restart mid-day does not re-fire a schedule that already ran.

Deploy: copy patrol\_scheduler.py + run\_patrol\_scheduler.sh to ~/go2/ and ~/ on the Orin, patrol-scheduler.service to /etc/systemd/system/, then sudo systemctl daemon-reload && sudo systemctl enable --now patrol-scheduler.

**Status (2026-08-20): written and reviewed against the codebase, not yet deployed or dry-run on the physical Orin.** Before relying on it for anything unattended, run an on-site dry run: set a near-future schedule on the dashboard, touch ~/go2/.patrol\_armed after the physical pre-flight check, and confirm the service actually launches the patrol and writes a log.

## 5.14 Cross-references

- **Chapter 1** — where this edge platform sits in the whole ARGUS architecture, and the fixed-cameras production design the Go2 stands in for.
- **Chapter 2** — the cloud half the gate depends on: pest-detection-processor (v6.3), its unconditional put\_item, the S3 event wiring, tiling and the Bedrock LLM verification gate, API Gateway vzf17s6z00 on the production account (the dev account's zwpcbivmsj is retired history), the JWT-exempt GET /stream/status route, and the SGT->UTC cron conversion patrol\_scheduler.py mirrors client-side.
- **Chapter 3** — Rekognition Custom Labels model versions, start/stop procedures and cost, and the holdout methodology behind the detection claims.
- **Chapter 4** — the dashboard: the Schedule panel that now drives both the model watchdog and (5.13) the patrol launch, the live-view toggle that kvs\_controller.py

obeys, the Gallery that renders the patrol's zone frames, the Test upload panel used for live demos.

- **Chapter 6** — the mini PC's parallel moth-cam-stream producer (same `kvs_controller.py` source, different env) and the Hikvision moth camera.
- **Chapter 7** — the ARGUS deployer, which reproduces the cloud resources of 5.12.6 on a fresh account (executed for real on 506868652945, 2026-08-10).
- **Chapter 8** — the end-to-end reproduction runbook that sequences this chapter's procedures with the cloud-side ones.

## Chapter 6 — Edge platform: mini PC (moth camera)

This chapter covers the mini PC that drives the fixed Hikvision moth camera: its network topology, the Kinesis Video Streams producer daemon, the still-frame capture script, the systemd units, the reverse SSH tunnel that makes the machine reachable, and the operating procedures.

*As of 2026-08-15. Live config values verified against the production DynamoDB rows 2026-08-14.*

### 6.1 Role in the system

The production concept for ARGUS is fixed cameras at each waypoint. The Unitree Go2 is a demo testbed; the mini PC is the fixed-camera edge node. It sits in the lab, wired to a Hikvision IP camera (192.168.1.66) that watches for adult moths. Its job is to turn that camera into two cloud inputs: a live H.264 video stream into Kinesis Video Streams (moth-cam-stream), and, on demand, still JPEG frames into S3 that trigger the detection pipeline.

The machine is a Windows 11 host running an Ubuntu 22.04 VM under VMware. Everything project-related runs inside the VM as user wilburteo (the account name is inherited from the predecessor, Wilbur Teo). The VM's uplink is NAT'd, so no machine anywhere can open a connection into it. All remote administration goes through a self-healing reverse SSH tunnel that the VM keeps open to the Jetson Orin on the Go2.

Control is cloud-driven. The streaming daemon never decides anything locally: it polls one HTTP API route (GET /stream/status) every 5 seconds and starts or stops the GStreamer producer to match what the dashboard says. Toggling “Stream on” for the Moth Cam card on the dashboard is the only control action an operator needs.

**Account note (updated 2026-08-13).** The system runs on the NP production account 506868652945 (CLI profile prod). The full ARGUS stack deployed there 2026-08-10 (dashboard <https://d1dtoxef7qmug1.cloudfront.net>, API <https://vzf17s6z00.execute-api.us-east-1.amazonaws.com>), both models were retrained there (moth moth-prod-20260811 on 2026-08-11, armyworm v9r-prod-20260810 on 2026-08-12, Chapter 3), and the handover snapshot argus-repo-snapshot-20260813.zip was published 2026-08-13. The development account 366356442579 (CLI profile nbk2) is where this node was built and validated; it appears in this chapter only as history. The repo copies of the mini PC scripts were repointed to production 2026-08-13. Two caveats specific to this node. First, the VM itself has not had its sync pass yet: the script copies and AWS keys deployed on the machine still carry development-account values until the hardware returns (the pending pass is listed in Section 6.10). Second, the KVS live-view path has no production counterpart yet: the production deploy ran without --live-view, so the account has zero video streams. Stream names are unchanged across accounts — the production moth\_cam row carries kvs\_stream\_name = moth-cam-stream

with `stream_enabled = false` — so the daemon polls the production API and correctly starts nothing. The still-frame path (Section 6.7) works on production as-is.

## 6.2 Inventory

| Item                                             | Location                                                              | Purpose                                                                                                                                                                                                                          |
|--------------------------------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>kvs_controller.py</code>                   | repo minipc/; deployed<br>/home/wilburteo/kvs_controller.py on the VM | Polling daemon:<br>/stream/status → start/stop<br>GStreamer → moth-cam-stream                                                                                                                                                    |
| <code>run_kvs_controller.sh</code>               | repo minipc/; deployed<br>/home/wilburteo/run_kvs_controller.sh       | Wrapper the service execs;<br>exports credentials, RTSP<br>settings, and<br>CAMERA_ID=moth_cam                                                                                                                                   |
| <code>kvs-controller.service</code>              | repo minipc/; deployed<br>/etc/systemd/system/kvs-controller.service  | systemd unit;<br>User=wilburteo,<br>Restart=always                                                                                                                                                                               |
| <code>capture_and_upload_v4_armyworm.py</code>   | repo minipc/                                                          | Still-frame path: RTSP (or<br>local file) → JPEG → S3 →<br>detection pipeline                                                                                                                                                    |
| <code>capture_and_upload_v3_person_cam.py</code> | repo minipc/                                                          | Historical predecessor (W5,<br>old account). Kept for<br>history only. Never run.                                                                                                                                                |
| <code>reverse-tunnel-fyp.service</code>          | VM only,<br>/etc/systemd/system/ (no<br>repo copy)                    | Self-healing reverse SSH<br>tunnel VM → Orin port 2222                                                                                                                                                                           |
| Hikvision moth camera                            | 192.168.1.66, RTSP path<br>/Streaming/channels/101                    | The fixed moth camera;<br>RTSP source for both scripts                                                                                                                                                                           |
| Ubuntu 22.04 VM                                  | VMware on the Win11 host;<br>NAT IP 192.168.189.130                   | Runs everything above                                                                                                                                                                                                            |
| <code>moth-cam-stream</code>                     | stream name on the<br>moth_cam row; same name<br>on both accounts     | The video stream the<br>daemon produces into.<br>Streaming was built and<br>HLS-validated on the<br>development account; the<br>production account<br>506868652945 has no KVS<br>streams yet (deploy ran<br>without --live-view) |
| <code>moth_cam</code> row                        | DynamoDB pest-<br>monitoring-cameras<br>(partition key attribute)     | Per-camera config:<br>stream_enabled,<br>kvs_stream_name, model                                                                                                                                                                  |

| Item                   | Location                                                                    | Purpose                                                               |
|------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------|
| KVS Producer SDK build | camera_id)<br>/home/wilburteo/amazon-kinesis-video-streams-producer-sdk-cpp | settings<br>C++ SDK build that provides the kvssink GStreamer element |

## 6.3 Topology and networking

Layout:

```
Hikvision 192.168.1.66 —RTSP—> Ubuntu 22.04 VM (wilburteo)
                                ens33 NAT 192.168.189.130 —> internet
(egress only)
                                ens37 bridged 192.168.123.99/24 (dog wired
                                net, only when plugged)
                                VMware NAT on Win11 host (campus IP, e.g. 10.
                                1.67.21)
```

Facts that matter:

- The VM has two NICs. ens33 is VMware NAT (192.168.189.x subnet, VM at 192.168.189.130). ens37 is bridged onto the Go2 wired network as 192.168.123.99/24 and only works when the dog-net cable is plugged in.
- Because ens33 is NAT, the VM’s outbound traffic appears to the campus network as the Win11 host’s IP (for example 10.1.67.21). **No inbound path to the VM exists from anywhere.** This is why the reverse tunnel (Section 6.8) exists.
- The Hikvision camera is reached by RTSP at `rtsp://admin:<password>@192.168.1.66:554/Streaming/channels/101`. The password is never written into any active repo file; it is supplied by environment variable on the VM only. The retired v3 script is the historical exception — it hardcodes the credential inline (Section 6.7).
- Streaming to AWS needs only outbound HTTPS, so KVS production works fine through the NAT with no tunnel and no Orin. The tunnel is for administration only.

## 6.4 kvs\_controller.py — the streaming daemon

### Purpose

A single-file Python 3 daemon (v2, “API-driven”). It polls the backend for the desired streaming state of one camera and reconciles a GStreamer child process to match. One process drives one camera; a second camera means a second instance with a different CAMERA\_ID. The Orin runs its own copy of the same file for the Go2’s SIYI A8 Mini (worm\_cam, Chapter 5); this chapter’s instance drives moth\_cam. (Note: as of 2026-07-29 the Orin instance’s RTSP source no longer exists — the A8 was moved to an HDMI-USB capture card and worm\_cam streaming is broken; see Chapter 5. The service still polls /stream/status, so it looks healthy. This chapter’s moth\_cam instance is unaffected.)

## The control contract: GET /stream/status

Every POLL\_INTERVAL\_SEC (5 s) the daemon calls:

```
GET https://vzfl7s6z00.execute-api.us-east-1.amazonaws.com/stream/status?camera=moth_cam
```

The response carries stream\_enabled (bool) and kvs\_stream\_name (string), read from the camera's row in DynamoDB pest-monitoring-cameras by the pest-monitoring-api Lambda. The dashboard's "Stream on" toggle writes stream\_enabled; the daemon picks the change up within one poll cycle (~5 s).

This is deliberately **the one unauthenticated route in the whole API**. Every other route requires a Cognito JWT — a design set 2026-07-06 on the development account and reproduced exactly on the production API (21 routes, JWT on all but this one; authorizer name cognito-dashboard). GET /stream/status stays open so the edge daemons can poll it with a plain HTTPS GET and zero AWS credentials. The route is read-only and returns nothing sensitive. Consequence: the control path needs no credentials at all; only the kvssink child process needs AWS keys, inherited from the environment.

This is a v2 design change. v1 did a direct DynamoDB get\_item on the old system-config nested map (cameras.{id}.stream\_enabled). The W7-era move onto the shared development account (nbk2) put camera config into per-row pest-monitoring-cameras, so that read stopped working; v2 switched the control source to the HTTP route. The route survives the 2026-08 production migration unchanged: table names and the response shape are identical on account 506868652945.

**Production status (2026-08-13).** The code default API\_BASE in minipc/kvs\_controller.py has pointed at the production API since the 2026-08-13 repoint. The copy deployed on the VM predates that repoint and still polls the development API until the pending device sync pass (Section 6.10); the polling contract is identical on both, so the sync is a file copy plus a credential swap. One production reality to keep in mind: the account has no KVS streams yet, and the moth\_cam row reads stream\_enabled=false, so the daemon idles. If the dashboard toggle is flipped on before a moth-cam-stream stream is created there, the daemon will start the pipeline, kvssink will fail against the missing stream, and the crash-detect in main() will relaunch it every cycle — visible restart churn in journalctl. Create the stream first (Section 6.10, reproduction step 1).

## Key functions

**fetch\_desired\_state() — read the control signal.** Purpose: ask the backend whether this camera should be streaming right now. How it works: builds {API\_BASE}/stream/status?camera={CAMERA\_ID} (the camera id is URL-quoted), sends a plain HTTPS GET with an Accept: application/json header and a **10 s timeout**, parses the JSON body, and returns (bool(stream\_enabled), kvs\_stream\_name). A missing stream\_enabled field reads as False. Failure behaviour: any URLError, HTTPError, JSON

ValueError, or OSError is caught, logged as API poll failed, and the function returns (None, None). The main loop treats (None, None) as “no information” — it keeps the current state and retries next cycle. A flaky network can therefore never stop a running stream, and never start one either.

**build\_gst\_args(stream\_name) — assemble the pipeline command.** Purpose: produce the full `gst-launch-1.0` argument list (a Python list, no shell). The pipeline **transcodes**: `rtspsrc` (location = the assembled RTSP URL, latency=0, drop-on-latency=true, protocols=tcp) → `decodebin` → `videoconvert` → `x264enc` (key-int-max=45 = one keyframe every 45 frames, tune=zerolatency, pass=cbr, speed-preset=superfast, bitrate=2000000) → `h264parse` → `kvssink` (stream-name={stream\_name}, storage-size=512, aws-region from AWS\_DEFAULT\_REGION, frame-timecodes=false, fragment-duration=2, max-latency=2). Transcoding exists because the Hikvision RTSP output is not guaranteed to be clean H.264 that kvssink can pass through. Contrast with the Orin’s A8 pipeline, which is passthrough (`rtspsrc` → `rtph264depay` → `h264parse` → `kvssink`, Chapter 5).

**is\_gst\_running() — child alive check.** True when `gst_proc` is set and `gst_proc.poll()` returns None (the process has not exited). This one-liner is the “actual state” side of every reconcile decision.

**start\_gst(stream\_name) — launch the producer.** Steps: (1) if a pipeline is already running, log a warning and return — the start is idempotent; (2) copy the daemon’s environment and add `GST_PLUGIN_PATH = <sdk>/build` and `LD_LIBRARY_PATH = <sdk>/open-source/local/lib` so GStreamer can find kvssink; (3) `subprocess.Popen` with `stdout/stderr` sent to `/dev/null` and `preexec_fn=os.setsid`, which puts the pipeline in its own process group — the property `stop_gst()` relies on; (4) log the child PID. AWS keys are never passed explicitly; kvssink inherits them from the environment the run script exported.

**stop\_gst() — tear the pipeline down completely.** Steps: if nothing is running, just clear the stale handle and return. Otherwise send `SIGTERM` to the **whole process group** (`os.killpg`), wait up to 5 s; on timeout escalate to `SIGKILL` on the group and wait 2 s more; a `ProcessLookupError` (already dead) is silently ignored. Always ends by setting `gst_proc = None`. The group kill matters because `gst-launch-1.0` spawns children; killing only the parent leaks the RTSP connection to the camera.

**graceful\_exit(signum, \_frame) — clean shutdown.** Registered for `SIGTERM` and `SIGINT` in `main()`. Logs the signal, calls `stop_gst()`, exits 0. This is what makes systemd restarts clean: the pipeline dies before the daemon does, so no orphan holds the camera or the stream.

**main() — the 5-second reconcile loop.** Purpose: keep the actual GStreamer state equal to the desired state from the API, forever. Each cycle:

1. Poll: `fetch_desired_state()`.



2. **Hold state on a failed poll:** if the poll returned None, sleep POLL\_INTERVAL\_SEC and continue. Nothing is started or stopped on missing information — a dead backend freezes the current state rather than flapping it.
3. Crash detection: if gst\_proc is set but the child has exited, log GStreamer exited unexpectedly with the return code and clear the handle. The desired state is still true, so step 4 restarts it — automatic recovery one cycle (~5 s) after any GStreamer crash.
4. Reconcile desired vs actual:

```

    if desired and not running:
        if not stream_name:
            log.error(f"stream_enabled=true but kvs_stream_name is empty
for {CAMERA_ID}")
        else:
            start_gst(stream_name)
    elif not desired and running:
        stop_gst()

```

The empty-name error repeats every cycle until the camera row is fixed; nothing starts. The two remaining combinations (already matching) need no action. 5. Log only on transitions: the (desired, running) tuple is compared with the last logged tuple, so a stable state produces silence, not one log line per 5 s. 6. Sleep POLL\_INTERVAL\_SEC (5 s) and repeat. There is no exit path except a signal.

### Configuration (all overridable by environment variable)

| Variable          | Code default                                                                                      | Deployed value                                                                                | Meaning                                                          |
|-------------------|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| CAMERA_ID         | worm_cam                                                                                          | moth_cam (exported by the run script)                                                         | Which camera row this instance polls                             |
| POLL_INTERVAL_SEC | 5                                                                                                 | 5                                                                                             | Poll period                                                      |
| API_BASE          | https://vzf17s6z00.execute-api.us-east-1.amazonaws.com (production, since the 2026-08-13 repoint) | VM copy predates the repoint; it polls the development API until the sync pass (Section 6.10) | Backend HTTP API                                                 |
| RTSP_USER         | admin                                                                                             | admin                                                                                         | Hikvision user                                                   |
| RTSP_PASS         | "" (no insecure default)                                                                          | set on VM only                                                                                | Hikvision password. Credential stored on the VM, not reproduced. |
| RTSP_HOST         | 192.168.1.66                                                                                      | same                                                                                          | Camera IP                                                        |
| RTSP_PATH         | /Streaming/channels/101                                                                           | same                                                                                          | RTSP channel path                                                |

| Variable                                     | Code default                                                  | Deployed value | Meaning                                                                                                                                                                                               |
|----------------------------------------------|---------------------------------------------------------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| KVS_SDK_DIR                                  | /home/wilburteo/amazon-kinesis-video-streams-producer-sdk-cpp | same           | Producer SDK build root<br>(GST_PLUGIN_PATH = <sdk>/build, LD_LIBRARY_PATH = <sdk>/open-source/local/lib)                                                                                             |
| AWS_DEFAULT_REGION                           | us-east-1                                                     | same           | Region passed to kvssink                                                                                                                                                                              |
| AWS_ACCESS_KEY_ID /<br>AWS_SECRET_ACCESS_KEY | none                                                          | set on VM only | Producer credentials, consumed by kvssink only. Still the development-account keys until the sync pass swaps in a production-account key (Section 6.10). Credential stored on the VM, not reproduced. |

Note the code default CAMERA\_ID is worm\_cam because the same file serves the Orin instance. The mini PC identity comes entirely from the run script export (Section 6.5).

## Data in / data out

- In: RTSP video from 192.168.1.66; JSON control state from GET /stream/status.
- Out: H.264 fragments into Kinesis Video Streams moth-cam-stream, us-east-1, on whichever account the VM's exported keys belong to. The HLS-validated runs happened on the development account; after the sync pass this is the production account 506868652945, which needs the stream created there first (Section 6.10). The dashboard plays this back over HLS (Chapter 4).

## Gotchas and failure modes

- API unreachable → the daemon logs API poll failed and holds its current state. A dead backend does not stop a running stream.
- stream\_enabled=true with an empty kvs\_stream\_name → logged as an error every cycle, nothing starts. Fix the camera row.
- GStreamer crash → detected next cycle and restarted automatically (desired state still true).

- The camera password appears inside the GStreamer command line (`rtsp://user:pass@...`), so it is visible in `ps` output on the VM. Known and accepted on this single-user machine.
- NOTE: doc lag — the KVS section of `docs/aws.md` still says `moth-cam-stream` is “parked (Hikvision cam repurposed)”. `docs/hardware.md` (updated later) and the 2026-07-14 migration record say the controller is deployed, the `moth_cam` row has `stream_enabled=true`, and HLS playback was verified on the dashboard. Follow `hardware.md` and the code: the controller is in active use, and `docs/state.md` records HLS playback verified on the dashboard through this controller (transcode pipeline). The `aws.md` line is stale doc lag. Physical cable status on any given day can only be confirmed on the machine itself.

## 6.5 `run_kvs_controller.sh` — the wrapper script

The `systemd` service execs this script, not the Python file. It exists to inject environment that must never live in a committed file:

1. `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` / `AWS_DEFAULT_REGION` — the producer keys (still development-account keys on the VM; the Section 6.10 sync pass replaces them with a production-account key). The repo copy carries `<SET_ON_VM>` placeholders; the real values exist only on the VM. Credential stored there, not reproduced. The keys live in the VM copy of the script unless an `EnvironmentFile=` directive, and the script header’s `EnvironmentFile` mention is a recommendation, not a record. Confirm with `systemctl cat kvs-controller` on the VM.
2. `export CAMERA_ID="moth_cam"` — the camera identity override. This line is load-bearing; see the story below.
3. `RTSP_USER` / `RTSP_PASS` / `RTSP_HOST` / `RTSP_PATH` — the Hikvision source. Password placeholder in the repo, real value on the VM only.
4. `exec python3 /home/wilburteo/kvs_controller.py` — replaces the shell, so `systemd` supervises the Python process directly.

## The `CAMERA_ID` override story (2026-07-14 migration)

During the camera-id true migration (`moth_cam_01` → `moth_cam`), the home-directory files on the VM were switched by `sed`. But the deployed `systemd` unit carried its own `Environment=CAMERA_ID=moth_cam_01` line, which `sed` on home files cannot touch and which needs `sudo` to edit. The no-`sudo` fix: add `export CAMERA_ID=moth_cam` to `run_kvs_controller.sh` before the final `exec` line (`hardware.md` records it at line 2 on the VM copy; the repo copy has it at line 17, after the AWS exports — the position does not matter, only that it runs before the `exec`). A shell `export` executed inside `ExecStart` overrides whatever the unit’s `Environment=` line set, because the script runs after `systemd` builds the environment. The service runs as `wilburteo` with `Restart=always`, so a plain

kill <pid> (no sudo) respawned it with the new export. Verified by reading /proc/<new-pid>/environ: CAMERA\_ID=moth\_cam.

The cosmetic residual (the unit file still saying moth\_cam\_01) was cleaned up the same day with sudo:

```
sudo sed -i 's/moth_cam_01/moth_cam/' /etc/systemd/system/kvs-controller.service && sudo systemctl daemon-reload
```

Both sources now say moth\_cam; the script export still wins if they ever differ again. Keep the export line — it makes the camera identity independent of the unit file.

## 6.6 kvs-controller.service — the systemd unit

Deployed at /etc/systemd/system/kvs-controller.service, enabled, so the stream controller survives VM reboots unattended.

### [Unit]

Description=KVS Controller (mini PC) - polls /stream/status and pushes Hikvision RTSP to KVS

After=network-online.target

Wants=network-online.target

### [Service]

Type=simple

User=wilburteo

ExecStart=/home/wilburteo/run\_kvs\_controller.sh

Restart=always

RestartSec=5

### [Install]

WantedBy=multi-user.target

- User=wilburteo — runs unprivileged. This is what makes the no-sudo restart trick work: the process owner can kill it, and Restart=always respawns it within 5 s with any script changes applied.
- Restart=always, RestartSec=5 — crashes and kills alike come back automatically.
- The repo copy above has no Environment= line. The deployed VM unit historically carried Environment=CAMERA\_ID=moth\_cam\_01 (now corrected to moth\_cam, see Section 6.5). If you redeploy from the repo copy, the Environment= line simply does not exist and the script export is the only source — that is fine and simpler.

## 6.7 capture\_and\_upload\_v4\_armyworm.py — the still-frame path

### Purpose and current role

This is the rescued still-capture script: it grabs single JPEG frames from an RTSP camera (or takes a local image file) and puts them into S3 under the exact key shape that triggers

the detection pipeline. It is the first hop only — camera → S3. Everything downstream (the pest-detection-processor Lambda, AWS Rekognition Custom Labels, DynamoDB pest-monitoring-detections, the processed-images bucket, SES alerts, the dashboard gallery) fires automatically off the S3 PutObject event. The script never calls Rekognition, DynamoDB (except the on/off config read), or SES.

Current role: an on-demand tool, not a service. There is no systemd unit for it. It is used for manual captures, smoke tests of the cloud chain (--once), and pushing known images through the live pipeline without any camera at all (--image — this is how holdout and test images get onto the live dashboard). The lineage name says “armyworm” (it was migrated in W7 from the W5 person\_cam version for the armyworm camera), but it is camera-agnostic: --camera moth\_cam routes a frame to the moth model, --camera worm\_cam to the armyworm model.

NOTE: the docstring keeps the W7 migration story as dated development-era history (armyworm\_go2\_a8mini, the old dev-account bucket, dashboard\_v3\_8.html in the flow diagram). A CURRENT TARGET (2026-08) note at the top of the docstring and the DEFAULTS block carry the truth: production account 506868652945, bucket argus-frames-506868652945, profile prod, default camera worm\_cam (post-2026-07-14 migration), and the current dashboard is the v5.2 ARGUS build.

**Production-account repoint (done in the repo 2026-08-13).** The repo copy targets production by default: bucket argus-frames-506868652945, CLI profile prod, camera ids unchanged (worm\_cam / moth\_cam, aligned 2026-08-11). Any copy still sitting on a device predates the repoint and keeps the development-era defaults until the sync pass (Section 6.10). Every value is also settable without editing the file (CLI flags --bucket / --profile / --camera, or the matching environment variables). The moth model the production moth\_cam row points at is moth-prod-20260811 in Rekognition project argus-moth-detection, retrained 2026-08-11 on the production account from the predecessor’s recovered labelled dataset (Chapter 3 covers the rebuild). One deliberate difference from worm\_cam: the production moth\_cam row is seeded with llm\_verify\_enabled=false (migration/migrate\_moth.py), because the LLM verify prompt describes armyworm larvae and this camera’s target is adult moths — moth uploads take the plain Rekognition path with no LLM gate. Do not “fix” that flag. As everywhere, that model’s endpoint must be RUNNING before an upload produces a detection.

## Run modes

```
python capture_and_upload_v4_armyworm.py                # Loop, dashboard-controlled
python capture_and_upload_v4_armyworm.py --ignore-config --interval 10 # Loop, fixed interval
python capture_and_upload_v4_armyworm.py --once          # one frame, then exit
python capture_and_upload_v4_armyworm.py --image some.jpg # upload a local file, no camera
```

Camera modes need RTSP\_URL exported in the terminal first  
(rtsp://<user>:<pass>@192.168.1.66:554/Streaming/Channels/101 — password from the VM notes, never hardcoded). --image mode needs no camera and no OpenCV.

## Key functions

**make\_session(profile, region) — pick the AWS identity.** Purpose: build the one boto3 Session every AWS call in the script uses (S3, and DynamoDB in loop mode). How it works: a non-empty profile name gives boto3.Session(profile\_name=..., region\_name=...); an empty or None profile falls back to the default credential chain (environment variables, then shared config). Defaults since the 2026-08-13 repoint: profile prod, region us-east-1. This is the single place where “which account” is decided.

**get\_capture\_settings(config\_t, fallback\_interval) — the remote on/off switch.** Purpose: let the dashboard start and stop loop-mode capture without touching the process. How it works: one get\_item on the detection\_settings row of DynamoDB pest-monitoring-system-config (a missing item is tolerated and reads as an empty dict), then returns (bool(auto\_capture), interval) where the interval is capture\_interval from the row (or the fallback) clamped to at least MIN\_INTERVAL (5 s), so a bad config value can never make the loop hammer the camera. Data flow: dashboard Settings page → pest-monitoring-api → this row → this function. Failure behaviour: any BotoCoreError, ClientError, or ValueError logs [Config] read failed and returns (False, fallback\_interval) — a broken config read idles the loop rather than capturing blind.

**capture\_frame(rtsp\_url, attempts=3) — one fresh JPEG from the camera.** Purpose: turn the RTSP stream into a single current frame as JPEG bytes. How it works: cv2 is imported lazily inside the function, so --image mode runs on a machine with no OpenCV; a failed import logs an install hint and returns None. Then up to 3 attempts. Each attempt: open the stream with cv2.VideoCapture(rtsp\_url, cv2.CAP\_FFMPEG); a failed open releases the handle, sleeps 1 s, and retries. On a good open it calls cap.read() twice and throws the results away — RTSP delivers buffered stale frames first, and this flush makes the next read current — then reads the real frame and releases the capture. A failed grab also sleeps 1 s and retries. A good frame is JPEG-encoded at quality JPEG\_QUALITY (90). Returns the encoded bytes, or None once all attempts are spent; the caller skips that cycle and nothing is uploaded.

**load\_local\_image(path) — file input for --image mode.** Purpose: exercise the whole cloud chain with a known image and no camera. How it works: maps the file extension to a content type (.jpg/.jpeg → image/jpeg, .png → image/png); any other extension is rejected with a log line, because the processor only accepts these. A missing file is rejected too. Returns (bytes, content\_type) on success, (None, None) on rejection; main() exits with code 1 on the latter.

**upload\_frame(s3, bucket, camera, waypoint, body, content\_type, basename) — the trigger point.** Purpose: put the image into S3 under the exact key shape the processor expects; this single put\_object starts the entire downstream pipeline. How the key is built:

```
ts = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
ext = ".png" if content_type == "image/png" else ".jpg"
filename = f"{ts}{ext}" if not basename else f"{ts}_{basename}"
key = f"frames/{camera}/{waypoint}/{filename}"
```

Microseconds in the timestamp guarantee uniqueness even for rapid --once runs; in --image mode the original basename rides along after the timestamp. One s3.put\_object with the matching ContentType writes it (default bucket argus-frames-506868652945) and returns the key. No retry inside: a BotoCoreError/ClientError propagates to the caller — loop mode logs it and carries on; --once/--image exit 1.

**parse\_args() — runtime config resolution.** Purpose: decide every setting once, at start. Precedence for each: CLI flag > environment variable (CAMERA\_ID, WAYPOINT\_ID, S3\_BUCKET, AWS\_PROFILE, AWS\_REGION, CAPTURE\_INTERVAL) > DEFAULTS. Mode flags: --once, --image PATH, --ignore-config.

**main() — mode dispatch and the capture loop.** Purpose: pick a mode, then run it. It builds the session and S3 client once, then dispatches. --image: load the file, upload once, exit (1 on any failure). Camera modes require RTSP\_URL in the environment; if it is missing the script exits 1 with an export hint instead of limping on. --once: capture one frame, upload, exit. Otherwise loop mode, where the DynamoDB table handle is created (it is needed nowhere else). Each loop cycle: (1) with --ignore-config use the fixed interval, floored at 5 s; otherwise call get\_capture\_settings — if auto\_capture is false, log [Idle], sleep out the interval, and skip the cycle; (2) capture\_frame — a None result logs [Skip] and sleeps out the interval; (3) upload\_frame — an upload failure is logged but never breaks the loop; (4) sleep the interval and repeat. There is no exit path in loop mode except Ctrl-C, which the KeyboardInterrupt handler at the bottom of the file turns into a clean [Stop] message. (log(msg) is the only helper: a one-line timestamped, flushed print.)

## Constants

DEFAULTS: camera worm\_cam, waypoint fixed\_cam, bucket argus-frames-506868652945, profile prod, region us-east-1, interval 60. (The development-era values — bucket frames-armyworm-366356442579, profile nbk2 — survive only in the docstring's W7 history block.) CONFIG\_TABLE=pest-monitoring-system-config, CONFIG\_KEY=detection\_settings, JPEG\_QUALITY=90, MIN\_INTERVAL=5.

## Gotchas and failure modes

- The S3 key shape is a contract. The processor's parse\_s3\_key requires the first segment to be frames and at least 4 segments; the {camera} segment picks the model. A wrong key either fires nothing or falls back to the manual\_upload camera and runs the wrong model.
- The Rekognition Custom Labels model for the target camera must be RUNNING (hourly-billed). If it is stopped, uploads still succeed but the processor's detect\_custom\_labels call fails: no detection row, no gallery image, no email.



- SES on the production account is in sandbox mode: alert emails reach verified addresses only. A detection that lands on the dashboard but sends no email to a new recipient is this, not a pipeline fault.
- The bucket must be the one carrying the S3 → processor event trigger (argus-frames-506868652945 on production). Uploading anywhere else does nothing.
- This script always uploads the whole, uncropped frame. Do not add cropping or resizing here. Whether the processor (v6.3) then tiles the frame is a per-camera cloud setting (tiling\_enabled on the camera row, Chapter 2) — true on worm\_cam (verified against the live production row 2026-08-14) and seeded false on moth\_cam (migration/migrate\_moth.py). Both states are deliberate; do not “fix” either from this end.

### capture\_and\_upload\_v3\_person\_cam.py — history only

The W5 predecessor. It points at the old practice account’s bucket fyp-practice-qrz and the old system-config table, uses camera id person\_cam (deleted from the product 2026-07-13), and has Chinese comments. It also hardcodes the Hikvision RTSP credential inline in its RTSP\_URL constant — credential stored there, not reproduced; this is exactly the practice v4 was written to end. Keep the file as history. Never run it and never copy its pattern.

## 6.8 reverse-tunnel-fyp.service — the self-healing reverse tunnel

### Purpose

The VM is NAT’d and unreachable inbound (Section 6.3). This unit makes the VM keep an outbound SSH connection to the Jetson Orin with a reverse forward: -R 2222:localhost:22. While it is up, anyone on the Orin can reach the VM’s sshd at localhost:2222. Installed and adversarially verified 2026-07-14: all tunnel connections were killed on the Orin, the port went down within 5 s, and the tunnel self-healed by T+45 s.

### Unit facts (from the deployed VM unit; there is no repo copy)

- systemd service, User=wilburteo, enabled at boot.
- Restart=always with RestartSec=15 — any exit retries every 15 s, forever.
- SSH options: ConnectTimeout=10, ServerAliveInterval 30 / ServerAliveCountMax 3 (a dead link is declared after ~90 s of silence and the client exits, triggering a restart).
- StartLimitIntervalSec=0 — disables systemd’s start-rate limiter.

The exact ExecStart line was never captured into the repo; the flags above are recorded in docs/hardware.md and docs/state.md, and the reconstruction in Section 6.10 step 7 is derived from them. Capture the real unit with `systemctl cat reverse-tunnel-fyp` next time the VM is reachable.



## Why the two hardening flags exist

The first build of this unit had neither `ConnectTimeout` nor `StartLimitIntervalSec=0`. During a dog-wifi flap (the Orin's network dropping and returning), the ssh client hung inside the TCP connect. The unit showed active (running) while doing nothing, indefinitely — a silent failure that looked healthy.

- `ConnectTimeout=10` bounds the connect attempt: a hung connect now fails in 10 s instead of hanging forever, so the exit/restart cycle actually happens.
- `StartLimitIntervalSec=0` removes systemd's default start-rate limit. Without it, a fast fail loop (network down for a while → many restarts) trips the limiter and systemd permanently gives up on the unit. With it set to 0, the unit retries every 15 s forever, and comes back on its own the moment both machines are up.

## Two-way key auth

Both directions are key-only, no passwords in the path:

- The VM's ed25519 public key is in the Orin's `/home/unitree/.ssh/authorized_keys` — this lets the tunnel's outbound ssh log in to the Orin unattended.
- The Orin's `~/ .ssh/id_ed25519` public key is in the VM's `/home/wilburteo/.ssh/authorized_keys` — this lets an operator on the Orin hop into the VM through the tunnel.

The VM sudo password was used transiently during the 2026-07-14 install and is not recorded anywhere.

## Known dependency (accepted)

The remote path requires the Orin to be up. Orin off → no tunnel → no remote access to the VM. This is accepted: the KVS streaming path does not depend on the Orin at all (it is direct VM → AWS over NAT), so a down Orin costs administration convenience only, never the moth stream. The escape hatch is the VMware NAT port-forward (Section 6.9), which needs one-time GUI setup on the Win11 host.

## 6.9 Access recipe: laptop → Orin → tunnel → VM

Standard remote path (automatic since 2026-07-14, nothing to start manually):

1. SSH from the laptop to the Orin:
  - Campus wifi: `ssh unitree@10.1.125.24`
  - Dog wired net (laptop static 192.168.123.50): `ssh unitree@192.168.123.18` (alias `go2`)
2. From the Orin, hop through the tunnel into the VM:

```
ssh -i ~/.ssh/id_ed25519 -p 2222 wilburteo@localhost
```

Failure modes on this path:

- Tunnel port not listening on the Orin → the VM or its service is down, or the tunnel is mid-heal. Wait 45 s and retry; then check from the VM side if you have physical access (`systemctl status reverse-tunnel-fyp`).
- Port listens but the connect hangs with “banner exchange” timeouts → stale sshd processes on the Orin are holding the forwarded port. On the Orin: `sudo ss -tnp | grep<mini-pc-host-ip>`, kill the stale pids, and the service rebinds within 15 s.

Optional direct-from-laptop path (no Orin needed; one-time GUI setup on the Win11 host):

1. On the Win11 host: VMware Workstation → Edit → Virtual Network Editor → select VMnet8 → NAT Settings → Add a port forward: host port 2222 → 192.168.189.130, guest port 22.
2. Then from the laptop: `ssh -p 2222 wilburteo@<win11-host-campus-ip>` (e.g. 10.1.67.21; the host IP is DHCP-assigned, check it on the host with `ipconfig`).

## 6.10 Operations / reproduction

### Day-to-day operations (on the VM, as wilburteo)

Status and logs:

```
systemctl status kvs-controller
journalctl -u kvs-controller -f          # Live log; state transitions + GStreamer PIDs
systemctl status reverse-tunnel-fyp
journalctl -u reverse-tunnel-fyp -n 50
```

Restart the controller without sudo (the standard move after editing the run script):

```
pgrep -af kvs_controller.py          # find the PID
kill <pid>                          # Restart=always respawns it in ~5 s with changes applied
```

With sudo (equivalent): `sudo systemctl restart kvs-controller`.

Confirm the running identity after any restart:

```
tr '\0' '\n' < /proc/$(pgrep -f kvs_controller.py)/environ | grep CAMERA_ID
# expect moth_cam
```

### Start / stop the stream (the normal control path)

- Console/dashboard: ARGUS dashboard → Live stream tab → Moth Cam card → toggle Stream on/off. The daemon reacts within ~5 s; HLS fragments appear in ~10 s.
- CLI (writes the same DynamoDB field the API serves — note the project quirk: DynamoDB CLI JSON quoting mangles under PowerShell; run this from bash, or use

boto3). The table's partition key attribute is camera\_id (confirmed in datasets/archive/experiments/one\_off\_scripts/migrate\_camera\_ids.py):

```
aws dynamodb update-item --table-name pest-monitoring-cameras \
--key '{"camera_id": {"S": "moth_cam"}}' \
--update-expression "SET stream_enabled = :v" \
--expression-attribute-values '{":v": {"BOOL": true}}' \
--profile prod --region us-east-1
```

## Verify the stream is landing in AWS

These checks were validated end-to-end on the development account; on production they only work after the moth-cam-stream stream has been created there (reproduction step 1 — the account currently has zero video streams).

- Console: AWS Console → Kinesis Video Streams → Video streams → moth-cam-stream → Media playback (live video should render).
- CLI:

```
aws kinesisanalytics describe-stream --stream-name moth-cam-stream --profile prod
--region us-east-1
EP=$(aws kinesisanalytics get-data-endpoint --stream-name moth-cam-stream \
--api-name LIST_FRAGMENTS --profile prod --region us-east-1 --query DataEnd
point --output text)
aws kinesisanalytics-archived-media list-fragments --stream-name moth-cam-stream
--endpoint-url "$EP" --max-results 5 --profile prod --region us-east-1
```

Fresh fragment timestamps prove production is live.

## Verify the control route

- Console: AWS Console → API Gateway → APIs → the HTTP API with id vzfl7s6z00 → Routes → GET /stream/status → confirm no authorizer is attached (every other route shows cognito-dashboard).
- CLI:

```
aws apigatewayv2 get-routes --api-id vzfl7s6z00 --profile prod --region us-east-1 \
--query "Items[?RouteKey=='GET /stream/status']"
curl "https://vzfl7s6z00.execute-api.us-east-1.amazonaws.com/stream/status?camera=moth_cam"
```

The curl works from anywhere with no token — that is the design.

## What breaks when the Orin is down

- Lost: remote SSH access to the VM (the tunnel has no endpoint). Accepted limitation.
- NOT lost: the moth stream. The controller polls the API and pushes to Kinesis Video Streams directly over the NAT uplink. Detection uploads via the v4 script also work.

- Recovery is automatic: when the Orin comes back, the tunnel unit's 15 s retry loop reconnects with no action on either machine.

## On-device production sync pass (status 2026-08-14: pending hardware return)

The cloud side is done: the ARGUS stack has run on the NP production account 506868652945 since 2026-08-10, both models are trained there, and the repo copies of the mini PC scripts were repointed to production 2026-08-13 (bucket argus-frames-506868652945, profile prod, API vzf17s6z00). What remains is a device-local file-and-key sync on the VM, to run when the hardware returns:

1. **Copy the repointed scripts** from minipc/ over the VM's home-directory copies (kvs\_controller.py, run\_kvs\_controller.sh, capture\_and\_upload\_v4\_armyworm.py if a copy lives there). Camera ids are unchanged (worm\_cam / moth\_cam); the S3-key contract and everything downstream are identical on production — the three ARGUS deploy fixes of 2026-08-10 mean the fresh stack seeds the same tuned detection config the development account ran.
2. **Swap the credentials.** The VM still carries development-account keys. Replace them with a production-account key (Section 6.10 reproduction step 2 gives the minimal policy: s3:PutObject on the frames bucket, plus the KVS producer permissions if streaming is revived).
3. **Check the unit environment.** The deployed kvs-controller.service may carry its own Environment= lines that would override the script's exports (the CAMERA\_ID story in Section 6.5 shows the failure shape). Run `systemctl cat kvs-controller` on the VM during the pass and clean any old-API or old-account value found there.

The streaming leg stays parked until KVS exists on the production account: the deploy ran without `--live-view`, so the account has zero video streams, even though the moth\_cam row is already seeded with `kvs_stream_name = moth-cam-stream` and `stream_enabled = false` (migration/migrate\_moth.py). That is a deliberate open decision (does the demo need live view?), not an omission. The daemon is safe to sync now: it will poll the production API, read `stream_enabled=false`, and start nothing. Create the stream (reproduction step 1) before anyone flips the dashboard toggle.

## Camera-id migration residuals (state as of 2026-07-22)

- The moth\_cam\_01 → moth\_cam migration is complete and verified on all three legs (Orin, cloud, mini PC VM).
- The VM unit's Environment= line and the run-script export both say moth\_cam. The export is the authoritative one if they ever diverge.
- .bak\_premigration files in /home/wilburteo are the rollback copies of the sed-switched files. They predate the 2026-07-30 “backups local only” rule (no .bak/scratch files on production devices). Runzhe's call: keep them as a migration-rollback exception, or archive them locally and delete them from the VM.

- Historical S3 keys under old camera-id prefixes were left untouched on purpose; detection records point at them.
- The Go2 ROS 2 workspaces in the VM home (unitree\_ros2, go2\_\*, ~640 MB, from the old drive-the-dog-from-the-VM era) are kept intact deliberately. Do not tidy them away.

## Reproducing this node on a fresh AWS account

Prerequisites on the cloud side (Chapter 2): the pest-monitoring-cameras table with a moth\_cam row, the pest-monitoring-api Lambda + HTTP API with the unauthenticated GET /stream/status route, and an IAM user whose keys the producer will use.

1. Create the video stream.
  - Console: AWS Console → Kinesis Video Streams → Video streams → Create → name moth-cam-stream, data retention as desired (the project uses the default).
  - CLI: `aws kinesismedia create-stream --stream-name moth-cam-stream --data-retention-in-hours 24 --profile<your-profile>--region<your-region>`
2. Give the producer credentials. Create an IAM user (Console: IAM → Users → Create; CLI: `aws iam create-user --user-name<producer-user>`) with a policy allowing `kinesismedia:PutMedia`, `kinesismedia:GetDataEndpoint`, `kinesismedia:DescribeStream` on the stream ARN, plus `s3:PutObject` on the frames bucket if the v4 script will run here. Generate an access key and place it ONLY in the VM's `run_kvs_controller.sh` (or a root-owned `systemd EnvironmentFile`). Never commit it.
3. Set the camera row: in pest-monitoring-cameras, the moth\_cam item needs `kvs_stream_name = moth-cam-stream` and `stream_enabled` present (start false).
4. Build the KVS Producer SDK on the VM: clone `amazon-kinesis-video-streams-producer-sdk-cpp` into `/home/<user>/`, build per the SDK's README with GStreamer plugin support, so that `<sdk>/build` contains `libgstkvssink.so`. Install GStreamer (`gstreamer1.0-tools`, `gstreamer1.0-plugins-base/good/bad`, `gstreamer1.0-libav`) and x264 support.
5. Deploy the three files from the repo's `minipc/` directory to the VM home, fill the `<SET_ON_VM>` placeholders in the script, adjust `API_BASE` to **your API id**, and fix `KVS_SDK_DIR` if the username differs.
6. Install the unit: copy `kvs-controller.service` to `/etc/systemd/system/` (fix `User=` and paths for your username), then `sudo systemctl daemon-reload && sudo systemctl enable --now kvs-controller`.
7. Rebuild the tunnel for your topology (or skip it and use the VMware NAT port-forward): a `systemd` unit running `ssh -N -R 2222:localhost:22 -o ConnectTimeout=10 -o ServerAliveInterval=30 -o ServerAliveCountMax=3<user>@<jump-host>` with `Restart=always`, `RestartSec=15`, `StartLimitIntervalSec=0`, and key auth pre-installed in both directions. Do not

omit `ConnectTimeout` or `StartLimitIntervalSec=0`; Section 6.8 explains the hang they prevent.

8. Test end to end: toggle `stream_enabled` on, wait 10 s, run the fragment check from this section.

## 6.11 Cross-references

- Chapter 2 (Cloud backend) — the `pest-monitoring-api` Lambda, the `/stream/status` route and its Cognito exception, DynamoDB tables, the per-camera `tiling_enabled` setting, and the S3 → `pest-detection-processor` trigger that the v4 script feeds.
- Chapter 3 (Models and training) — what the moth and armyworm Rekognition Custom Labels models are and why they must be RUNNING for detections; also the production-account moth rebuild (`moth-prod-20260811`, project `argus-moth-detection`, retrained 2026-08-11 from the predecessor's recovered labelled dataset).
- Chapter 4 (Dashboard frontend) — the Live stream tab (HLS playback, Stream on/off toggle) and the Settings page that drives `auto_capture` for the v4 loop mode.
- Chapter 5 (Edge platform: Go2 / Jetson Orin) — the twin `kvs_controller.py` instance for `worm_cam` (passthrough pipeline, no transcode; broken since the 2026-07-29 move to an HDMI-USB capture card, see Chapter 5) and the Orin end of the reverse tunnel.
- Chapter 8 (Reproduction runbook) — the full-system rebuild order; the steps in Section 6.10 slot in after the cloud backend exists.

## Chapter 7 — ARGUS one-click deployer

This chapter documents the ARGUS desktop deployer: a single Windows .exe that stands up the complete detection stack in a customer's own AWS account, trains their model from a local YOLO folder, and can audit or destroy the whole deployment.

*As of 2026-08-11.*

### 7.1 Role in the system

Everything in Chapters 2–4 was built by hand in the development/reference account 366356442579. ARGUS is the productized replay of that work. It takes a fresh AWS account and recreates the whole cloud stack — 6 IAM roles, 4 DynamoDB tables, 3 S3 buckets, a Lambda layer, 5 Lambdas, Cognito, the HTTP API with 21 routes, CloudFront, the cost-guard schedule, an empty Rekognition Custom Labels project, and SES identities — in a single unattended run, with live progress in a desktop window. The run is fast: the measured production figure is **103 seconds for all 15 stages** (2026-08-10); CloudFront's global propagation continues for a few minutes after the run itself completes.

That figure is not a projection. On 2026-08-10 ARGUS performed the project's real account migration: a headless CLI invocation of the stage engine (`python deploy.py --profile prod --prefix argus --target-label armyworm-larva ...`) stood up the production stack in the NP production account 506868652945 — **15 of 15 stages, 103 seconds, zero errors, first attempt, no resume needed**. The KVS stage was skipped by design because `-live-view` was not passed. The resulting stack is live: dashboard <https://d1dtoxef7qmugl.cloudfront.net> (CloudFront E1YADURLSAVNFA), API `vzf17s6z00`, Cognito pool `us-east-1_9selFDHpc` with client `6vebotf45bp8u46cnnraddiaplv`, Rekognition project `argus-detection`, and buckets `argus-frames-506868652945` / `argus-processed-506868652945` / `argus-dashboard-506868652945`.

The system therefore now spans two accounts. 366356442579 is the development/reference account where the system was built and validated. 506868652945 is the NP production account being stood up: the ARGUS stack is deployed and verified, the training data has been copied server-side, both models have been retrained there (`moth moth-prod-20260811 F1 0.991`; `armyworm v9r-prod-20260810 F1 0.613`) and wired into the camera rows, and the SES sender address is verified; KVS / live view is not migrated.

ARGUS is deliberately a generic product, not an armyworm tool. Nothing armyworm-specific ships (product scope ruling, 2026-07-08). No dataset migrates. The detection target is a per-camera `target_label` string the customer chooses in the wizard. The customer brings their own labeled images and trains their own model through the in-app training pipeline. Pest monitoring at Jewel Changi is the first instance of the product, not the product itself.

The tool has three engines behind one shell. `app.py` is the desktop shell and the JS-to-Python bridge. `deploy.py` is the stage engine that creates, verifies, and destroys cloud resources. `training.py` is the pipeline that turns a local YOLO folder into a trained, wired-in Rekognition Custom Labels model. All three run from source (`python app.py`) or frozen into `dist\ARGUS.exe` by `build.ps1`.

## 7.2 Inventory

| Item                              | Location                                   | Purpose                                                                                                                                                      |
|-----------------------------------|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>app.py</code>               | <code>deployer/app.py</code>               | pywebview (Edge WebView2) desktop shell + the Api JS-Python bridge                                                                                           |
| <code>deploy.py</code>            | <code>deployer/deploy.py</code>            | 15-stage idempotent stack deployer; also <code>verify()</code> audit and <code>destroy()</code> teardown; runnable standalone from the CLI                   |
| <code>training.py</code>          | <code>deployer/training.py</code>          | YOLO folder -> Rekognition Custom Labels training pipeline, watch, wire-in, rollback                                                                         |
| <code>web/index.html</code>       | <code>deployer/web/index.html</code>       | The ARGUS UI. Self-contained (inline CSS/JS, no CDNs). UI is frozen — do not restyle                                                                         |
| <code>audit/</code>               | <code>deployer/audit/</code>               | Raw per-resource config JSONs audited from the live reference account; <code>deploy.py</code> consumes them with <code>account/region/bucket</code> rewrites |
| <code>STACK_MANIFEST.md</code>    | <code>deployer/STACK_MANIFEST.md</code>    | The recreate-level bill of materials the deployer was built from                                                                                             |
| <code>layer/fyp-pillow.zip</code> | <code>deployer/layer/fyp-pillow.zip</code> | Prebuilt Pillow 12.2.0 Lambda layer (manylinux2014, cp312) — required by the frozen exe                                                                      |
| <code>legal/</code>               | <code>deployer/legal/</code>               | Terms of Use + Privacy Policy shown on the consent screen                                                                                                    |
| <code>build.ps1</code>            | <code>deployer/build.ps1</code>            | PyInstaller onefile packaging                                                                                                                                |



| Item                | Location                                                                                       | Purpose                                                                                                                                                                           |
|---------------------|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| build_installer.ps1 | deployer/build_installer.ps1                                                                   | script -> dist\ARGUS.exe<br>Wraps dist\ARGUS.exe into a Windows installer via Inno Setup 6 -> dist\installer\ARGUS-Setup-<version>.exe                                            |
| installer/argus.iss | deployer/installer/argus.iss                                                                   | Inno Setup script: Program Files install, shortcuts, uninstaller, silent WebView2 runtime install from the bundled bootstrapper (installer\redist\MicrosoftEdgeWebView2Setup.exe) |
| requirements.txt    | deployer/requirements.txt                                                                      | boto3, pywebview, keyring, Pillow — runtime deps for source runs; baked into the exe by build.ps1 (pyinstaller itself is build-time only)                                         |
| REHEARSAL.md        | deployer/REHEARSAL.md                                                                          | The two-round deployment rehearsal protocol                                                                                                                                       |
| README.md           | deployer/README.md                                                                             | Component map, the one-stop rule, packaging notes                                                                                                                                 |
| Design doc          | docs/deployer_training_pipeline.md                                                             | Training pipeline spec (2026-07-16); code is ground truth where they differ                                                                                                       |
| State file          | %LOCALAPPDATA%\ARGUS\out\deploy_state.json (frozen) or<br>deployer/out/deploy_state.json (dev) | Every generated id; drives resume, verify, destroy, training                                                                                                                      |
| WebView2 profile    | %LOCALAPPDATA%\ARGUS\webview_profile\                                                          | Persistent browser profile for the embedded windows                                                                                                                               |
| Stored credentials  | Windows Credential Manager, service name ARGUS-deployer                                        | The customer's IAM access key, DPAPI-encrypted via keyring                                                                                                                        |

## 7.3 app.py — the desktop shell and the Api bridge

### Purpose

One window, one exe. app.py creates a pywebview window on the Edge WebView2 backend (gui="edgechromium"), loads web/index.html, and exposes a Python class Api to the page as window.pywebview.api. The UI calls bridge methods; long jobs run in background threads and stream events back by calling `window.evaluate_js('ARGUS.onDeployEvent({...})')`.

The `js_api=api` argument in `webview.create_window` is what makes the bridge exist. It was missing until 2026-07-15, so the shipped exe silently fell back to the UI's simulated preview mode with every button dead. Do not remove it.

### The trust boundary

Enforced in code, not just claimed:

- Card numbers and AWS passwords are typed into AWS's own pages, rendered in a separate embedded WebView2 window (`open_aws_pane`). That window has no JS bridge and no injected script. ARGUS never reads its DOM. It observes only the navigation URL (the loaded event) to advance the guidance rail in the main window. This is the same pattern Stripe and Plaid use for onboarding.
- The IAM access key the customer pastes is validated against STS and then stored through keyring, which on Windows means the Windows Credential Manager (DPAPI encryption). Never plaintext, never logged, never echoed back to the UI. The UI only ever sees the account id and ARN.
- Root account keys are refused outright (see `verify_credentials` below).

### Exposed bridge methods

Every public method on Api is callable from the page as `window.pywebview.api.<name>(...)`. All return JSON-serializable dicts.

### Credentials

- `verify_credentials(access_key_id, secret_access_key)` — validates the pasted key with `sts:GetCallerIdentity`. Refusals and checks, in order:
  1. STS failure returns a friendly error (`_friendly_aws_error` maps `InvalidClientTokenId / SignatureDoesNotMatch` to “copy/paste slip”, `AccessDenied` to “use an administrator key”).
  2. Root-key refusal: if the caller ARN ends in `:root`, the key is rejected before anything is stored. Root keys cannot be permission-scoped or safely rotated; the keys screen exists to teach the IAM-user alternative.
  3. AdministratorAccess preflight (added 2026-07-15): a bare IAM user passes STS but fails every deploy stage with `AccessDenied` eight stages in. If the

ARN contains `:user/`, the method lists the user's attached policies, and if `AdministratorAccess` is not directly attached it walks the user's groups (`list_groups_for_user -> list_attached_group_policies`) to catch inherited grants. Result `admin` is `True` (found), `False` (definitely missing), or `None` (key lacks IAM read — shown as a warning, not a block).

4. On success the key pair is stored in the Credential Manager under service `ARGUS-deployer` (keys `aws_access_key_id`, `aws_secret_access_key`). If keyring is absent (dev only) it degrades to an in-memory store. Returns `{ok, account, arn, admin}`. The secret never appears in the return value.
- `has_credentials()` — true if a key pair is already stored; lets the UI mark the keys step done on relaunch.
  - `forget_credentials()` — deletes both entries from the credential store.
  - `account_alive()` — STS liveness probe. Distinguishes auth-dead (`InvalidClientTokenId`, `UnrecognizedClientException`, `SignatureDoesNotMatch`, `ExpiredToken` -> account closed or key revoked, the UI offers a fresh start) from network errors (must never be mistaken for a closed account).

## Embedded AWS window

- `open_aws_pane(which)` — opens AWS's own page in the separate embedded window (reuses the window if already open). Targets: `signup` (`portal.aws.amazon.com/billing/signup`), `console`, `iam-keys` (the IAM **create-user** page, deliberately not `#/security_credentials` — that page is the signed-in root user's own credentials and nudges toward creating root keys, the exact thing the product forbids; fixed 2026-07-15), `ses` (verified identities), `rekognition` (Custom Labels projects), `close-account` (billing account page — account closure is root-plus-console-only; AWS provides no API for standalone accounts, so ARGUS guides and never automates it, and root never touches the product).
- `close_aws_pane()` — destroys the embedded window.

## Deployment

- `preflight(cfg)` — cheap checks before deploying: credentials on file, prefix matches `[a-z][a-z0-9-]{1,20}`, sender email present.
- `start_deploy(cfg)` — refuses if a deploy or a training run is active, runs `preflight`, then launches `_deploy_worker` in a daemon thread and returns immediately. The worker builds a `deploy.Ctx.from_params(...)` from the stored key and the wizard config, calls `deploy.set_emitter(self._push)`, then `deploy.run_plan(ctx, list(deploy.STAGES))`. Progress arrives in the UI as `stage / log / done / error` events.
- `deployment_status()` — reads `deploy_state.json` and reports `none` (never deployed), `partial` (started, no completion stamp — crashed or quit mid-run), or `complete` (stamp present), plus the last stage, dashboard URL, and API URL. Drives the “Resume deployment” offer and the jump to the done screen.

- `verify_deployment(cfg)` — runs `deploy.verify()` with the stored key. The state file, not the UI form, is the source of truth for region and prefix; otherwise a healthy **ap-southeast-1** stack gets audited in `us-east-1` and reads as broken. Older state files without `deploy:region` recover the region from the saved API Gateway URL.
- `destroy_deployment(confirm_text)` — full teardown. Guards: refuses while a deploy runs; refuses while a training run is active (destroy would delete the Rekognition project under the trainer, and the training thread's next whole-dict state save would resurrect the archived state file); requires the typed confirmation to equal the deployment prefix exactly. Then runs `deploy.destroy(ctx)` in a thread, and afterwards re-runs `deploy.verify(ctx)` as proof of absence — success now means 0 resources remain (`destroy_verified` event with any leftovers).

### Dashboard accounts (Cognito CRUD, 2026-07-16)

- `list_dashboard_users()` — lists users in the deployed pool.
- `create_dashboard_user(email, password)` — `admin_create_user` with `MessageAction="SUPPRESS"` followed by `admin_set_user_password(..., Permanent=True)`. The permanent password matters: the dashboard login intentionally does not handle the `FORCE_CHANGE_PASSWORD` state.
- `delete_dashboard_user(email)` — `admin_delete_user`.

All three build their session via `_mgmt_session()`, which reads the deployed region from the state file (same source-of-truth rule as `verify`).

**Training bridges** (all drive `training.py`; deterministic automation only — the 2026-07-16 architecture ruling: no LLM ever holds credentials or mutates cloud resources)

- `pick_training_folder()` — native folder picker (`create_file_dialog(FOLDER_DIALOG)`); the customer never types a path.
- `analyze_training_folder(path)` — local fail-fast dataset report; no AWS, no upload.
- `plan_training(report, class_id)` — train/test division and cost estimate for the cost gate.
- `start_training(params)` — claims the `_training` flag atomically under `_flag_lock` before any slow work (the STS call inside `Ctx.from_params` is slow enough for a double-click to race and launch two pipelines), then runs `training.run_training` in a thread. Refuses while a deployment runs.
- `watch_training()` — re-attaches a watcher to an in-flight run after a relaunch or crash; if training finished while the app was closed, the wire-in completes instantly.
- `training_status()` — `none` / `preparing` (local pipeline running in-process, nothing submitted to AWS yet) / `in_progress` / `needs_wire` / `trained`, plus active F1, rollback availability, and last failure. Answers from local state unless a run is pending.

- `rollback_model()` — swaps every custom camera row back to the previous model ARN.

## Lifecycle and misc

- `on_closing()` — window-close guard. Once training is submitted (`train:pending` on disk) closing is safe; the cloud continues alone. During the local `convert/upload/datasets` phases a close hard-kills the run, so the first close attempt is blocked with an explanation; a second close in the same session is honored.
- `quit()` — decline-and-exit. Repeats the `on_closing` guard (`window.destroy()` bypasses the closing event), closes the AWS pane, destroys the window.
- `legal_text(which)` — returns the bundled Terms of Use / Privacy Policy markdown.
- `open_external(url)` — opens the user's real browser; used only for the final dashboard URL.

## WebView2 profile persistence

`webview.start(...)` pins the WebView2 profile to `%LOCALAPPDATA%\ARGUS\webview_profile` via `storage_path`, with `private_mode=False` (required for the profile to be written at all; added 2026-07-15). Effects: the embedded AWS window keeps the customer's console session across relaunches (AWS's ~12 h session window), and the wizard's own `localStorage` resume state persists. The profile is always per-user under `%LOCALAPPDATA%`, never inside the project tree: it holds console session cookies, autofill and history, and in dev runs a project-tree location would ship the customer's browser profile inside any later zip or handoff of the folder.

## Known gotchas

- Without keyring installed the credential store degrades to in-memory (lost on exit). The packaged build always includes it (`--hidden-import keyring.backends.Windows`).
- `run_plan` already emits the error event itself; the deploy worker must not push it again (doing so printed every failure twice in the feed).
- The `Api` methods return error dicts, not exceptions — the UI renders `{ok: false, error}` directly.

## 7.4 `deploy.py` — the stage engine

### Purpose

Boto3 orchestration of 15 idempotent stages that recreate the whole stack in dependency order in a fresh account. Source of truth: `STACK_MANIFEST.md` plus the raw config JSONs in `audit/`, audited from the live reference account. Runnable standalone from the CLI or driven by `app.py`.

## Context and plumbing

- `Ctx` — the resolved deployment context: a boto3 session, the caller account (from STS), region, prefix, target label, emails, and the state dict. Built from a named AWS profile (`Ctx(args)`, CLI path) or from a pasted key (`Ctx.from_params(...)`, app path — same post-conditions).
- Naming: buckets are `{prefix}-frames-{account}`, `{prefix}-processed-{account}`, `{prefix}-dashboard-{account}`. Table, Lambda, and role names are fixed (`pest-*`) — the prefix parameterizes only the globally-namespaced resources.
- `ctx.rewrite(text)` / `ctx.load_audit_json(name)` — rewrite the reference-account literals (366356442579, us-east-1, the three reference bucket names `frames-armyworm-366356442579`, `processed-images-armyworm-366356442579`, `pest-dashboard-366356442579`) into this deployment's values before applying an audited policy document.
- `ctx.save(key, value)` — writes the whole state dict to `deploy_state.json` after every mutation. When frozen, state lives in `%LOCALAPPDATA%\ARGUS\out` (the `PyInstaller_MEIPASS` temp dir is wiped on exit); in dev it is `deployer/out/`.
- `set_emitter(fn)` / `_emit(kind, **payload)` — optional event sink so the desktop app can stream progress. Kinds: `log`, `stage`, `done`, `error`, plus `destroy`, `destroy_done`, `destroy_verified` from teardown.
- `STAGE_LABELS` — human-facing stage names for the UI theater (e.g. `iam` -> “Identity & permissions”, `rekognition` -> “Vision engine”).
- `run_plan(ctx, plan)` — runs a list of `(name, fn)` stages with structured events per stage. Before the first stage it pops any previous `deploy:completed_at` and `deploy:last_stage` stamps — otherwise a second deployment that dies mid-run still reads as complete and the app resumes to the wrong deployment's summary — and records `deploy:started_at`, `deploy:region`, `deploy:prefix`, `deploy:account`. After each stage it stamps `deploy:last_stage`; after all stages, `deploy:completed_at`. Returns `(ok, error_message)`.
- `wait_for(fn, desc)` — polls for IAM eventual consistency and similar; `already_exists(err)` maps the family of “already there” error codes that make every stage adoptive.

## The three deploy-breaking gaps (found and fixed 2026-08-10)

A pre-migration audit of `deploy.py` against the validated production configuration found three gaps. Any fresh stack would have deployed “successfully” and then run as the untuned shell of the system, not the system Chapters 2–3 validated. All three were fixed as surgical edits before the production deploy, and the 2026-08-10 run confirmed them working on a real fresh account:

1. **Processor sizing.** `pest-detection-processor` deployed at 512 MB / 60 s while validated production runs **1024 MB / 600 s**. One tiled frame plus up to 120 LLM

verify crops measures 24–54 s of runtime, so the 60 s timeout would have failed intermittently on real frames.

2. **lambda\_env carried no detection tuning.** The processor's environment held only table and bucket names, so a fresh stack ran the retired trust-Rekognition mode on Haiku. The env builder now ships the full validated tuning: `TILE_MIN_CONFIDENCE=8`, `LLM_VERIFY_ALL_BOXES=true`, `LLM_VERIFY_MODEL_ID=us.anthropic.claude-sonnet-4-6`, `LLM_VERIFY_MAX_BOXES=120`, `LLM_VERIFY_MAX_TOKENS=300`, `LLM_VERIFY_WORKERS=3`, `LLM_VERIFY_PAD=0.6`, `POST_NMS_IΟΥ=0.1`, `POST_NMS_CONTAIN=0.1`, `POST_MAX_BOX_AREA=0.05`, `POST_VERIFY_FLOOR=49`.
3. **The worst: seeded camera rows had no llm\_verify\_enabled.** The LLM gate is per-camera opt-in, so on a fresh deployment the entire verification layer would simply never run. The rows also seeded `min_confidence=30` — the candidate-floor trap that strangles recall before the LLM ever sees a box. Rows now seed `min_confidence=10`, `llm_verify_enabled=true`, `post_verify_floor=49`, plus `max_runtime_min=45` on the template camera.

These are product fixes, not migration patches: they make any fresh ARGUS stack reproduce the validated system, tuning included.

Two further alignment fixes landed 2026-08-11:

- The processor role gained two inline policies captured from the live reference role: `bedrock-verify` (`bedrock:InvokeModel` + `bedrock:Converse` on the account's inference profiles and underlying foundation models — without it every crop verdict gets `AccessDenied`, the processor's total-gate-failure guard trips, and the stack FAILS OPEN to plain `min_confidence` thresholding at the candidate floor, flooding the dashboard with unverified boxes that look like confident detections; exactly this happened in production 2026-08-12, 321 boxes across 13 images — see Chapter 2 §2.5.6 and 9.3.14) and `s3-frames-write` (`s3:PutObject` on `frames/*` for the processor's EXIF-normalised frame write-back). Audit JSONs: `deployer/audit/iam__pest-detection-processor-role__inline__bedrock-verify.json` and `...__s3-frames-write.json`.
- The watchdog now deploys from the repo mirror `lambda/pest-model-watchdog.py` (v6.2, which honours per-camera `max_runtime_min`) instead of the pre-v6.2 `audit/pest-model-watchdog_src/` snapshot; `MAX_RUNTIME_MIN=75` stays in its env as the global fallback.

Timing caveat on the 2026-08-10 production run: it PREDATED these role-policy fixes. The production processor role came up without `bedrock-verify` and `s3-frames-write` (and without `rekognition:DetectLabels`) and had to be patched separately on 2026-08-12 by `migration/fix_processor_iam.py`, after the first validation run exposed the fail-open flood described above (9.3.14). Future deployments get the policies from the deployer's role build; but validate ANY deployment by result — push a known image through the pipeline and check the detections — never by green stages alone.

The packaged artifacts were rebuilt the same day (2026-08-11) so the frozen build carries all of the above: `dist\ARGUS.exe` (57.4 MB) and `dist\installer\ARGUS-Setup-1.0.0.exe` both ship the Bedrock policy JSONs, the v6.2 watchdog mirror, and the v6.3 Lambda sources. An exe built before 2026-08-11 deploys the untuned stack — do not distribute stale builds.

### The 15 stages, in order

Every stage is idempotent: existing resources are adopted (matched by name), never duplicated. Safe to re-run.

| # | Stage | UI label               | Creates / does                                                                                                                                                                                                                                                                                                                                                                                                                           | Adopt behavior                                             |
|---|-------|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| 1 | iam   | Identity & permissions | 5 Lambda execution roles (pest-detection-processor-role, pest-monitoring-api-role, pest-camera-scheduler-role, kvs-hls-handler-role, pest-model-watchdog-role) with AWSLambdaBasicExecutionRole attached and inline policies loaded from the rewritten audit JSONs (the processor role's set includes bedrock-verify — required by the LLM verify gate — and s3-frames-write; both captured from the live reference role 2026-08-11, see | create_role tolerated on exists; policies re-put every run |



| # | Stage    | UI label | Creates / does                                                                                                                                                                                                                                                                                                                                                                                                  | Adopt behavior                                     |
|---|----------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
|   |          |          | the fix list<br>above); plus<br>pest-<br>scheduler-<br>invocation-<br>role (trusted by<br>scheduler.amaz<br>onaws.com,<br>scoped<br>lambda:InvokeF<br>unction on the<br>watchdog and<br>camera-<br>scheduler).<br>Creating this<br>stably named<br>role also<br>resolves the<br>reference<br>account's latent<br>bug where the<br>api-role's<br>iam:PassRole<br>pointed at a role<br>that did not<br>exist live |                                                    |
| 2 | dynamodb | Database | 4 tables, all<br>PAY_PER_REQU<br>EST: pest-<br>monitoring-<br>cameras (HASH<br>camera_id),<br>pest-<br>monitoring-<br>detections<br>(HASH<br>image_id +<br>RANGE<br>detection_time<br>, GSI by-pest-<br>time on<br>pest_type+det<br>ection_time,<br>projection ALL),                                                                                                                                            | create<br>tolerated;<br>existing tables<br>adopted |

| # | Stage | UI label | Creates / does                                                                                                                                                                                                                                                                                                                                                                                        | Adopt behavior                  |
|---|-------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
|   |       |          | pest-monitoring-system-config (HASH config_key),<br>pest-monitoring-schedule-logs (HASH log_id + RANGE timestamp).<br>Waits for table_exists on all                                                                                                                                                                                                                                                   |                                 |
| 3 | s3    | Storage  | 3 buckets with SSE-S3 (AES256).<br>Frames + processed: versioning on, public access block all-true.<br>Dashboard: public static website — public access block turned OFF first, then the public-read policy, then website config (index and error both index.html, the SPA fallback).<br>The ordering is a trap: PutBucketPolicy fails while the block is on.<br>Skips LocationConstraint in us-east- | BucketAlreadyOwnedByYou adopted |

| # | Stage  | UI label                 | Creates / does                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Adopt behavior                                                             |
|---|--------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| 4 | layer  | Image-processing library | <p>1 (passing it errors)</p> <p>Publishes Lambda layer fyp-pillow (Pillow 12.2.0, cp312, manylinux2014_x86_64). Prefers the prebuilt layer/fyp-pillow.zip bundled beside the app; a frozen exe has no pip toolchain, so if the zip is missing it dies with a pointer to build.ps1. In dev it pip-builds the layer and asserts PIL/_imaging*.so exists — the exact defect that broke layer v1 in the reference account (no compiled binaries, no boxes drawn)</p> | publishes a new version each run (versions are cheap; destroy deletes all) |
| 5 | lambda | Cloud functions          | <p>The 5 functions, python3.12, handler lambda_function.lambda_handler: pest-camera-scheduler first (the api's env</p>                                                                                                                                                                                                                                                                                                                                           | <p>on exists: update_function_code + update_function_configuration</p>     |

| # | Stage | UI label | Creates / does                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | Adopt behavior |
|---|-------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
|   |       |          | <p>references its ARN), then pest-detection-processor (1024 MB/600 s — the validated production sizing — and the only one with the Pillow layer; its env ships the full detection tuning listed in the fix section above), pest-monitoring-api (256/30), kvs-hls-handler (128/30), pest-model-watchdog (128/30, MAX_RUNTIME_MIN=75 global fallback; the v6.2 code honours per-camera max_runtime_min). Source zips are built in-memory; the kvs-hls-handler source comes from audit/kvs-hls-handler_src/, the watchdog from the repo mirror lambda/pest-model-watchdog.py. Retries create</p> |                |

| # | Stage           | UI label        | Creates / does                                                                                                                                                                                                                                                                                                             | Adopt behavior                                                                |
|---|-----------------|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| 6 | s3-notification | Event wiring    | <p>on<br/>InvalidParameterException (IAM role not yet propagated)</p> <p>lambda:AddPermission for s3.amazonaws.com on the processor FIRST, then the frames-bucket notification: s3:ObjectCreated:*, prefix frames/, target pest-detection-processor. Order matters — the notification put fails without the permission</p> | <p>permission add tolerated on exists; notification is a whole-config put</p> |
| 7 | cognito         | Sign-in service | <p>Pool pest-dashboard-users (email as username, auto-verified email, admin-create-only, password minimum 8 with lowercase+number) and SPA client dashboard-web (no secret, ALLOW_USER_PASSWORD_AUTH + refresh, 12 h tokens / 30 d refresh). Zero</p>                                                                      | <p>matched by name via list, adopted</p>                                      |

| # | Stage | UI label   | Creates / does                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Adopt behavior                                                                                                                                                                                                                                              |
|---|-------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 8 | apigw | Secure API | <p>users seeded — the customer creates accounts from the done screen</p> <p>HTTP API pest-monitoring-api-gateway with CORS (*origins, GET/POST/DELETE/OPTIONS); JWT authorizer cognito-dashboard bound to the pool/client; 2 AWS_PROXY integrations (api, hls); the 21 routes — JWT on all except GET /stream/status (device pollers, unauthenticated); all routes target the api integration except GET /video-playback, which targets the HLS integration (still JWT-protected); \$default auto-deploy stage; 2 invoke permissions (api-wide for pest-monitoring-</p> | <p>api/authorizer/integrations matched by name/URI; existing routes go through update_route — which requires ApiId and only RouteKey stripped from the kwargs (stripping both made every resume fail on the first pre-existing route; fixed 2026-07-16)</p> |

| #  | Stage      | UI label             | Creates / does                                                                                                                                                                                                                                                                                                                                                                                                                 | Adopt behavior                                                                                        |
|----|------------|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| 9  | cloudfront | Dashboard & delivery | <p>api, route-scoped<br/>/*/*/video-playback for<br/>kvs-hls-handler)</p> <p>Distribution in front of the dashboard bucket's <b>website endpoint</b> (custom origin, http-only — website endpoints have no HTTPS), viewer redirect-to-https, AWS-managed CachingDisabled policy 4135ea2d-6df8-44a3-9df3-4b5a84be39ad (same id in every account), compress on, fresh UUID caller reference. Takes minutes to reach Deployed</p> | <p>matched by the distribution comment ({prefix} dashboard frontend (S3 website origin)), adopted</p> |
| 10 | writeback  | Configuration        | <p>Templates web/dashboard_v4/js/config.js in memory (HTTP_API, COGNITO_REGION, COGNITO_CLIENT_ID), uploads</p>                                                                                                                                                                                                                                                                                                                | <p>full re-sync every run</p>                                                                         |

| #  | Stage       | UI label      | Creates / does                                                                                                                                                                            | Adopt behavior                                                     |
|----|-------------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
|    |             |               | every dashboard file to the dashboard bucket with correct content types (.js forced to text/javascript), then sets the frames-bucket CORS to the website endpoint + the CloudFront domain |                                                                    |
| 11 | kvs         | Live video    | Optional. Only if the wizard enabled live view: a Kinesis Video Streams stream {prefix}-cam-stream, 2 h retention                                                                         | create tolerated on exists; skipped entirely when live view is off |
| 12 | scheduler   | Cost guard    | EventBridge Scheduler schedule pest-model-watchdog-15min, rate(15 minutes), ENABLED, target = watchdog Lambda via pest-scheduler-invocation-role, zero retries                            | create tolerated on exists                                         |
| 13 | rekognition | Vision engine | Empty Custom Labels project                                                                                                                                                               | matched by project name,                                           |



| #  | Stage | UI label     | Creates / does                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Adopt behavior                                              |
|----|-------|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
|    |       |              | {prefix}-<br>detection. No<br>datasets, no<br>model —<br>trained Custom<br>Labels models<br>are account-<br>bound (no<br>export/copy<br>API), so the<br>customer trains<br>later with their<br>own images                                                                                                                                                                                                                                                                                                    | adopted                                                     |
| 14 | seed  | Initial data | System-config<br>row<br>detection_sett<br>ings (email<br>enabled,<br>recipient, auto-<br>capture, 60 s<br>interval) and 2<br>camera rows:<br>manual_upload<br>(required by the<br>dashboard Test-<br>upload feature)<br>and camera-1<br>(the template<br>camera: the<br>wizard's<br>deployment<br>name as label,<br>target_label<br>from the wizard,<br>tiling_enabled<br>=True,<br>max_runtime_mi<br>n=45 — bounds<br>an unattended<br>model start via<br>the watchdog).<br>Both rows seed<br>the validated | put_item<br>overwrites — re-<br>running resets<br>the seeds |

| # | Stage | UI label | Creates / does                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Adopt behavior |
|---|-------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
|   |       |          | per-camera<br>detection<br>config:<br>model_type=cus<br>tom,<br>min_confidence<br>=10 (the<br>candidate floor<br>feeding the LLM<br>gate, NOT the<br>display<br>threshold —<br>setting it high<br>silently<br>strangles recall<br>before the LLM<br>sees a box),<br>llm_verify_ena<br>bled=True (the<br>gate is per-<br>camera opt-in;<br>left unset it<br>never runs),<br>post_verify_fl<br>oor=49 (the<br>user-facing<br>threshold the<br>dashboard<br>edits), empty<br>custom_model_a<br>rn — filled by<br>training. The<br>tiling default is<br>deliberate: the<br>earlier whole-<br>image-only<br>ruling was<br>superseded in<br>late July 2026<br>(Rekognition<br>finds<br>candidates on<br>tiles, the LLM |                |

| #  | Stage | UI label | Creates / does                                                                                                                                                                                                                                                                                                                                                              | Adopt behavior                   |
|----|-------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| 15 | ses   | Alerts   | <p>verifier filters them — see Chapter 2)</p> <p>verify_email_identity for sender and recipient (each gets a confirmation email that must be clicked). Then checks sesv2 get_account: if the account is in the SES sandbox it logs the warning — alert emails only reach verified addresses until production access is granted (manual AWS request, ~1–2 business days)</p> | verification re-send is harmless |

## State file, resume, and stamps

deploy\_state.json accumulates every generated id: role:\* ARNs, bucket:\* names, layer:fyp-pillow, lambda:\* ARNs, cognito:pool/client, apigw:id/url, cloudfront:id/domain, kvs:stream, rekognition:project, plus the deploy:\* run stamps and every train:\* key from Chapter 7.5’s pipeline.

Resume semantics:

- deploy:last\_stage lets a later launch distinguish “never ran” / “died at stage N” / “completed” and offer the right action.
- The completion stamp deploy:completed\_at means the whole stack ran. A --only subset run never writes it — the CLI logs “completion stamp not written” instead. The app’s resume logic trusts the stamp, so a subset must not forge it.

- Stale-stamp invalidation: both `main()` and `run_plan()` pop `deploy:completed_at` and `deploy:last_stage` at the start of every run, so a re-deploy that dies mid-run cannot masquerade as the previous complete run.
- Because every stage adopts existing resources, “resume” is simply “run the plan again” — completed stages fly through as adoptions.

CLI flags: `--only s3`, `lambda` runs selected stages; `--dry-run` prints the plan and changes nothing; `--verify` audits instead of deploying.

## verify(ctx) — the live audit

Proves every resource recorded in the state file exists in the account right now. Read-only, costs nothing, safe on a partial deploy (missing resources come back `ok=False`). For each state key it makes the matching describe/head call: `iam.get_role`, `s3.head_bucket`, `lambda.get_function` (detail = function state), `get_layer_version_by_arn`, `describe_user_pool/_client`, `apigw.get_api`, `cloudfront.get_distribution` (detail = Deployed vs InProgress), `kinesisvideo.describe_stream`, Rekognition project lookup. It also checks resources the stages never wrote into state: the 4 DynamoDB tables and the `pest-model-watchdog-15min` schedule. Returns `[{name, ok, detail}]`; the CLI prints OK/MISS per line and “deployment is INCOMPLETE” if anything is missing. In the app this is the “System check” behind `verify_deployment`, and it doubles as the post-destroy proof of absence.

## destroy(ctx) — teardown

Tears the whole deployment down in reverse dependency order, driven by the state file plus the fixed naming. Every step is best-effort: a missing resource is reported “(already gone)” and counted as done, never fatal, so a partial stack tears down cleanly too. Not-found error codes (`ResourceNotFoundException`, `NoSuchEntity`, `NotFoundException`, `NoSuchBucket`, `NoSuchDistribution`, `404`, `ValidationException`) are benign; anything else marks the step failed and continues.

Safety rails:

- Account-match guard: refuses to run if `deploy:account` in the state file differs from the account the stored key resolves to — you cannot destroy account B’s stack with account A’s state file. One gap: `deploy:account` is written by `run_plan` (the app path) but not by the CLI’s `main()` loop, and the guard skips silently when the key is absent. A CLI-deployed state file therefore does not get this protection; the typed-confirmation and best-effort semantics still apply.
- Typed confirmation: the app requires the deployment prefix typed exactly (`destroy_deployment`).
- Never under training: the app refuses while `_training` is set.

Order: (1) the watchdog schedule; (2) Rekognition — stop RUNNING project versions, wait out STOPPING, delete versions, delete the project; (3) the KVS stream; (4) CloudFront —

disable, wait for Deployed (the slow one, minutes), delete; (5) API Gateway; (6) the Cognito pool (clients and users die with it); (7) the 5 Lambdas; (8) every fyp-pillow layer version; (9) the 3 buckets (empty each via paginated delete\_objects, then delete); (10) the 4 tables; (11) the 6 IAM roles (inline policies deleted and managed policies detached first); (12) SES identities. Finally the state file is renamed to destroyed\_{timestamp}.json — the deployment no longer exists, but the record of what it was is archived.

After the engine finishes, the app re-runs verify() on the still-in-memory state as proof of absence: the pass condition is “0/N remain”. Leftovers are listed in the destroy\_verified event. Only then does the UI treat the deployment as gone.

Account closure is not part of destroy: closing an AWS account is root-plus-console-only (no API for standalone accounts). ARGUS opens the billing page in the embedded AWS window and guides; root credentials never touch the product.

#### 7.4.9 Function reference: every function in deploy.py

The stage list tells you the order. This table tells you what each function is responsible for and the rule inside it, so the source can be read without tracing every call.

| Function            | Job                                              | Rule or algorithm inside                                                                                                                                                          |
|---------------------|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| set_emitter / _emit | Send structured progress to whatever is watching | The desktop app registers a callback; headless runs get plain log lines. Every stage reports through the same channel, so the UI and the CLI never diverge                        |
| log / die           | The only two output paths                        | die raises after emitting, so a stage cannot half-fail silently                                                                                                                   |
| run_plan            | Drive a list of (name, fn) stages                | Runs stages in order, emits a start and an end event per stage, and returns (ok, error_message). The desktop app uses it; main() keeps its own loop so the CLI can support --only |
| wait_for            | Absorb eventual consistency                      | Polls a predicate with retries and swallows ClientError between attempts. Needed because a role is not immediately usable by Lambda after create_role                             |
| already_exists      | Decide whether a failure is                      | A whitelist of the “already                                                                                                                                                       |

| Function                 | Job                                    | Rule or algorithm inside                                                                                                                                                                                                                                                                        |
|--------------------------|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                          | really a failure                       | there” error codes the various services return (EntityAlreadyExists, ResourceConflictException, BucketAlreadyOwnedByYou, and four more). This one function is what makes every stage idempotent                                                                                                 |
| stage_iam                | Create the six roles                   | For each role: create_role, adopt on already_exists, attach the basic execution policy, attach any managed policies, then put every inline policy from the audited JSON with the account and region rewritten to the target account                                                             |
| stage_dynamodb           | Create the four tables                 | Declares each table’s key schema inline, all pay-per-request, including the by-pest-time index on the detections table; adopts an existing table by name                                                                                                                                        |
| create_bucket / stage_s3 | Create and configure the three buckets | create_bucket handles the us-east-1 special case, where a LocationConstraint must NOT be sent. stage_s3 then applies SSE-S3 encryption, versioning, and the public-access settings each bucket needs — the dashboard bucket has its block cleared so the public-read website policy is accepted |
| stage_layer              | Publish the Pillow layer               | Prefers a prebuilt fyp-pillow.zip shipped with the app, because a packaged executable has no pip toolchain. From source it builds the layer with the                                                                                                                                            |

| Function                           | Job                                | Rule or algorithm inside                                                                                                                                                                                                       |
|------------------------------------|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                    |                                    | manylinux target flags and asserts the compiled <code>_imaging</code> object is present before publishing                                                                                                                      |
| <code>zip_lambda</code>            | Package one function in memory     | Writes the source into a zip <b>renamed to <code>lambda_function.py</code></b> , because that is the handler name every function is deployed with. No temporary files touch disk                                               |
| <code>lambda_env</code>            | Supply each function's environment | A per-function dictionary. The processor branch carries the validated detection tuning, so a fresh account comes up in the measured two-stage configuration rather than on code defaults                                       |
| <code>stage_lambda</code>          | Create the five functions          | Deploys in a fixed order that satisfies the cross-references: the scheduler first, because the API needs its ARN. Sets memory, timeout and layer per function; updates code and configuration when the function already exists |
| <code>stage_s3_notification</code> | Wire the trigger                   | Permission first, notification second — the reverse order fails. The notification carries the frames/ prefix filter that keeps training uploads from starting detection runs                                                   |
| <code>stage_cognito</code>         | Create the pool and app client     | Finds an existing pool by name before creating one, so a re-run does not orphan users. The client is a public single-page client with no secret                                                                                |
| <code>stage_apigw</code>           | Build the HTTP API                 | Adopts an existing API by                                                                                                                                                                                                      |

| Function         | Job                                  | Rule or algorithm inside                                                                                                                                                                                                                                                                             |
|------------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  |                                      | name, creates the JWT authorizer bound to the Cognito pool and client, creates the two Lambda integrations, then creates all 21 routes from the route table — authorization is a field in that table, which is why the one unauthenticated route is a single visible line                            |
| stage_cloudfront | Put HTTPS in front of the dashboard  | Identifies its own distribution by a generated comment string, since CloudFront has no name field. Uses the managed CachingDisabled policy so a plain sync is a full redeploy                                                                                                                        |
| stage_writeback  | Make the deployment self-configuring | Templates the API URL, Cognito region and client id into config.js in memory, uploads every dashboard file with the right content type, then adds the CloudFront domain and website endpoint to the frames-bucket CORS list. Without that last step the Test upload panel fails on its presigned PUT |
| stage_kvs        | The optional live-view stream        | Returns immediately unless --live-view was passed. This is why the production run completed 15 of 15 stages with KVS skipped by design                                                                                                                                                               |
| stage_scheduler  | Install the cost guard               | Creates the EventBridge schedule pest-model-watchdog-15min at rate(15 minutes), targeting the watchdog through the scheduler invocation role,                                                                                                                                                        |



| Function          | Job                                   | Rule or algorithm inside                                                                                                                                                                                                                                              |
|-------------------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| stage_rekognition | Create an empty Custom Labels project | with retries disabled<br>Adopts by project name if it exists. Deliberately empty: models are account-bound and cannot be shipped, so the customer trains their own                                                                                                    |
| stage_seed        | Write the starting configuration      | Puts the global config row and the two camera rows, carrying the validated per-camera detection settings. This is what makes the stack usable the moment it finishes                                                                                                  |
| stage_ses         | Start email verification              | Sends a verification email to the sender and recipient addresses, then checks ProductionAccessEnabled and warns when the account is still in the SES sandbox — the state the production account is in today                                                           |
| destroy           | Tear the whole deployment down        | Reverse dependency order, driven by <code>deploy_state.json</code> . Every step is best-effort so a missing resource is reported and skipped rather than aborting the teardown. The state file is renamed rather than deleted, so the record of what existed survives |
| verify            | Prove the deployment is real          | Read-only post-deployment audit: for every resource recorded in the state file, ask the account whether it exists right now, and return per-resource evidence. Safe on a partial deploy, where missing resources simply come back not-ok. It is also                  |

| Function | Job                          | Rule or algorithm inside                                                                                                                                                                                        |
|----------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| main     | The command-line entry point | <p>what proves a destroy actually removed everything</p> <p>Parses the arguments, builds the context, and runs the stages, honouring --only for a subset and the resume stamp for a restart after a failure</p> |

## 7.5 training.py — the in-app training pipeline

### Purpose

Takes the customer’s local YOLO-labeled image folder and produces a trained, wired-in Rekognition Custom Labels model: analyze -> validate -> convert (YOLO -> Ground Truth manifest) -> upload to S3 -> create/append Rekognition datasets -> train -> watch -> write custom\_model\_arn onto every custom camera row. The converter math is the production-proven datasets/convert\_to\_manifest.py logic (models v3–v6 all trained through it), ported here. Design doc: docs/deployer\_training\_pipeline.md.

### Folder analysis and validation (analyze, \_find\_splits, \_class\_names, parse\_yolo\_line)

\_find\_splits auto-detects three layouts: yolov8 (root/images/{split}/ + root/labels/{split}/), flat (root/images/ + root/labels/, single split), and roboflow (root/{split}/images/ + root/{split}/labels/). \_class\_names best-effort parses data.yaml without a YAML dependency (inline list, block list, and id-map forms). parse\_yolo\_line accepts detection lines (5 numbers) and segmentation lines (1 + 2N numbers, reduced to the bounding box).

analyze(folder) is local, fast, and fail-fast: no AWS calls, no image decoding (label text only). It reports layout, per-split counts, per-class stats with per-split image counts, warnings, and hard blockers:

- Blocker: no readable YOLO boxes anywhere.
- Blocker: pixel-coordinate labels. YOLO coords are normalized [0,1]; any value above 1.5 (tolerating rounding slop) means pixel units. If more than half the boxes are out of range, the whole set is blocked with “re-export with normalized coordinates” — catching it here instead of letting them clamp to zero-size boxes deep in convert. A minority is only a warning (those boxes get dropped).
- Warnings: orphan label files (no matching image), unlabeled images, zero-box labeled images, malformed lines. Zero-box images are excluded up front and reported, never uploaded — Rekognition Custom Labels object detection ignores them (the v6 negatives lesson).

`plan_for_class(report, class_id)` is a pure function feeding the cost gate: if the folder has an explicit test split with at least 10 images, it is used; otherwise a test set of  $\max(10, 15\%)$  is carved from the pool. Blocked if fewer than 10 train or 10 test images survive. Cost estimate:  $\text{est\_hours} = \max(1.0, \text{round}(0.8 + \text{train\_n}/400, 1))$  at ~US\$4 per training hour (reference run: 1,148 images -> 3.5 h billable). Deliberately uncapped: a 10k-image set really does train for a day and the gate must say so.

NOTE: doc lag — `docs/deployer_training_pipeline.md` says “~\$1/h” and “poll every 5 min”; the code bills the estimate at US\$4/h and `watch_and_wire` polls every 60 seconds.

## Convert and stage (`_convert`)

Walks every (image, label) pair, keeps boxes of the chosen class, and produces the upload list plus per-split Ground Truth manifest entries. Key behaviors:

- Dedupe by stem: `_existing_stems` lists every dataset entry already in the project (excluding the current version’s own prefix, so a retry replaces its own earlier upload instead of being deduped against it) and `_convert` skips those stems. Re-picking an old folder cannot duplicate entries or leak former TRAIN images into TEST. If every image is excluded, the run fails with “pick a folder of NEW photos”.
- Deterministic split assignment mirroring `plan_for_class`: explicit test split honored if  $\geq 10$ , otherwise an evenly spaced carve from the sorted pool.
- Images that need work — any dimension over 4096 px (Custom Labels rejects them) or a non-JPEG/PNG format — are re-encoded into a staging dir as JPEG quality 90, with oversize images LANCZOS-downscaled to 4096 max edge. Everything else uploads untouched. Box pixel coordinates are computed from the final saved dimensions, and since YOLO boxes are normalized, the downscale is lossless for labels.
- Degenerate boxes (under 1 px after rounding) are dropped; images left with no annotations are dropped; unreadable images are logged and skipped. After conversion, both splits must still have  $\geq 10$  images or the run fails with a pointer to the warnings.

Manifest entries are Ground Truth object-detection format: source-ref to the S3 key, bounding-box with image size and pixel annotations, `class-map {"0": class_name}`, human-annotated: yes, job name `argus-training`. The class name is sanitized to `[A-Za-z0-9_.\- ]`, max 64 chars.

## Upload and bucket grant (`_grant_rekognition_access, _upload`)

Before uploading, the frames bucket gets a policy so the Rekognition service principal can read `training-data/*` and write evaluation output to `training-output/*` (mirroring what the Custom Labels console generates). Idempotent: the three ARGUS Sids (`ArgusRekognitionBucketRead`, `ArgusRekognitionObjectRead`, `ArgusRekognitionOutputWrite`) are replaced in place, all other statements preserved.

Images upload to `training-data/v{N}/{split}/images/`, manifests to `training-data/manifests/v{N}/{train,test}.manifest`, with progress events every 20 files.

## Datasets (`_load_datasets`, `_wait_dataset`)

If the project has no TRAIN/TEST dataset yet, each is created from its manifest (`create_dataset` with the Ground Truth S3 source). On iteration, entries are appended with `update_dataset_entries`, chunked under 4 MB per call (the API cap is 5 MB). `_wait_dataset` polls to the terminal status with a freshness guard: it ignores a terminal status recorded before our own submission (`after_ts`), because a multi-chunk append could otherwise read the previous chunk's `UPDATE_COMPLETE` and fire the next chunk into a dataset still ingesting. Dataset `ErrorEntries` are reported but non-fatal — the engine skips those images at training.

One engineering lesson from the 2026-08-10 account migration belongs here for anyone moving a Custom Labels project between accounts: after any labelling done in the Rekognition console, the **dataset — not the S3 manifest it was created from — is the source of truth**. Console edits live only inside the dataset. Export the live entries with `ListDatasetEntries` before rebuilding a dataset elsewhere; rebuilding from the original manifest silently drops every console edit.

## Train, watch, wire (`_next_version_number`, `_start_version`, `watch_and_wire`, `_wire_in`)

`train:next_n` is persisted at run start, not on success: a retry after any failure re-uses the same `n`, so the same images land on the same S3 keys and replace their dataset entries instead of duplicating (`update_dataset_entries` matches on `source-ref`). The `train:attempt` stamp lets a relaunched app say “an earlier attempt was interrupted” if the process is hard-killed before submit.

`_start_version` calls `create_project_version` (version name `v{n}-{UTCdate-time}`, output to `training-output/v{n}` in the frames bucket). A `LimitExceededException` becomes a clear message: delete old versions in the Rekognition console, then train again — the pipeline never deletes anything itself. On submit it swaps the local attempt stamp for the durable `train:pending` record; from here the cloud continues without the app.

`watch_and_wire` polls `describe_project_versions` every 60 s. Transient network/API errors are tolerated up to 30 consecutive failures (~30 min) before giving up with “training itself continues in the cloud — re-attach from the Train screen”; `train:pending` survives the give-up. Status handling is deliberate: `TRAINING_COMPLETED`, `STARTING`, `RUNNING`, `STOPPING`, `STOPPED` all mean training succeeded (a model that completed while the app was closed may already have been started or stopped from the dashboard); `TRAINING_FAILED`, `FAILED`, `DELETING` are dead (recorded in `train:last_failure`).

`_wire_in` is the critical step (as the `training.py` docstring puts it, a trained but unwired model is useless): it scans `pest-monitoring-cameras` and sets `custom_model_arn` to the

new version ARN on every row with `model_type == "custom"`, via boto3 only — model ARNs contain colons that shell quoting mangles (see docs/aws.md). It records the version under `train:v{n}` (ARN, F1, billable seconds), saves the previous ARN as `train:rollback_arn`, sets `train:active_arn/train:active_f1`, and bumps `train:next_n`. If no camera row is custom, it warns loudly instead of pretending success.

`rollback(ctx)` codifies the v6-to-v5 hand-rollback of 2026-07-15: one click swaps every custom camera row back to `train:rollback_arn`, recovers that model's F1 from its version record, and makes the swap reversible (current becomes the new rollback target).

`status(ctx)` gives the UI a one-call summary, making a single describe call only when a run is pending. A pending run that vanished from the project (deleted in the console) is cleared to `train:last_failure` status MISSING so the UI cannot wedge.

### Deliberate defaults (settled 2026-07-17)

- **No auto-start.** The model is left STOPPED after training. The done panel says so: start detection from the dashboard, inference costs ~US\$4/h, and the watchdog stops forgotten models.
- **No auto-delete.** Old project versions stay until the customer deletes them in the console; on the version limit the pipeline stops with a clear message.
- **No training-data prefix deletion.** Iteration appends to the same Rekognition datasets, so every past `v{N}` prefix stays referenced by dataset entries — deleting one would break the next retrain. Storage cost is cents; correctness wins.

## 7.6 The UI — screens map

`web/index.html` is self-contained (inline CSS and JS, WebGL + Canvas2D hero, no CDNs). It doubles as a browser preview with simulated actions when `window.pywebview.api` is absent, and the real app when it is present. The UI is frozen by ruling (2026-07-21) — do not restyle it.

| Screen id | Name        | What happens there                                                                                                                                                |
|-----------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| s-welcome | Welcome     | Product intro, start                                                                                                                                              |
| s-consent | Consent     | Terms of Use + Privacy Policy ( <code>legal_text</code> ), accept or decline-and-exit (quit)                                                                      |
| s-account | AWS account | Guidance rail + <code>open_aws_pane('signup'/'console')</code> for creating or signing into the AWS account; card and passwords go into AWS's embedded pages only |
| s-keys    | Access keys | <code>open_aws_pane('iam-</code>                                                                                                                                  |

| Screen id | Name               | What happens there                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| s-config  | Configuration      | <p>keys ' ) walks the customer through creating the deployer IAM user; pasted key goes through verify_credentials (root refusal, admin preflight)</p> <p>Deployment id (prefix), deployment name, target label, region, sender/recipient email, live view toggle</p>                                                                                                                                                                     |
| s-review  | Review             | The plan summary; hold-to-deploy                                                                                                                                                                                                                                                                                                                                                                                                         |
| s-theater | Deployment theater | The 15 stages streaming live (stage/log events); camera-aperture progress motif ending at f/1.4                                                                                                                                                                                                                                                                                                                                          |
| s-done    | Done               | <p>Dashboard URL (opens via open_external), API URL, system check (verify_deployment), SES/Rekognition console shortcuts, plus two embedded panels:</p> <p><b>Dashboard accounts</b> (the pool starts with zero users — nobody can sign in until the first is created here; list/create/delete_dashboard_user) and the <b>Danger zone</b> (Delete deployment: typed-prefix confirmation, streaming destroy log, post-destroy verify)</p> |
| s-train   | Train your model   | Folder pick, analysis report, class choice, cost gate with typed TRAIN, phase rail (validate/convert/upload/datasets/train/wire), resume bar that re-attaches to an in-flight run on relaunch, F1                                                                                                                                                                                                                                        |

| Screen id | Name | What happens there |
|-----------|------|--------------------|
|           |      | result, rollback   |

## 7.7 build.ps1 — packaging

build.ps1 packages everything into dist\ARGUS.exe. Run it from deployer/:

```
powershell -ExecutionPolicy Bypass -File build.ps1
```

What it does, in order:

1. Installs/upgrades build deps: pyinstaller, boto3, pywebview, keyring, Pillow.
2. Prebuilds the Pillow Lambda layer zip once if layer\fyp-pillow.zip is missing: `pip install --platform manylinux2014_x86_64 --implementation cp --python-version 3.12 --only-binary=:all: Pillow==12.2.0`, asserts `PIL\_imaging*.so` exists (the layer-v1 defect), zips it. A frozen exe cannot pip-build (no toolchain, and `sys.executable` is the exe itself), so the zip must ship inside the bundle; `stage_layer` publishes it when present.
3. Runs PyInstaller: `python -m PyInstaller app.py --onefile --noconsole --name ARGUS` with the liquid-glass icon (`assets\argus_B.ico`, Runzhe's pick 2026-07-21) and a version resource, plus `--hidden-import keyring.backends.Windows` and `--collect-data botocore --collect-data boto3`.

Hard-won build facts, all encoded in the script:

- **python -m PyInstaller, not the bare pyinstaller exe** — the exe lives in a Scripts dir that is not reliably on PATH (failed 2026-07-17); the module form always works.
- **Pure ASCII requirement:** the script must stay pure ASCII. Windows PowerShell 5.1 reads BOM-less files as ANSI; a UTF-8 em-dash decodes into a smart quote and breaks the parser (bit the project 2026-07-17).
- **The writeback-bundle bug and fix:** `stage_writeback` reads the dashboard from `REPO / "web" / "dashboard_v4"`, and in a frozen exe `REPO` is the `PyInstaller\_MEIPASS` unpack dir. The exe therefore must bundle the repo's dashboard: `--add-data "..\web\dashboard_v4;web\dashboard_v4"`. Without that entry the writeback stage has nothing to upload and the deployment dies at stage 10. Similarly the app's own UI packs as `"web;web"` (not `"web;."`) because the code expects it at `_MEIPASS\web`. Windows' `--add-data` separator is `;`.
- **Assets bundled:** `audit;audit` (the policy JSONs and two Lambda sources), `legal;legal`, `layer;layer` (the prebuilt Pillow zip), `..\lambda;lambda` (the three core Lambda sources).
- **WebView2 runtime:** handled by the installer (next section). The `MicrosoftEdgeWebView2Setup.exe` evergreen bootstrapper (~2 MB) is bundled at `installer\redist\` and auto-invoked by the installer on machines without the runtime. Win11 and current Win10 preinstall it. Only when shipping the bare onefile

dist\ARGUS.exe (no installer) does the old rule apply: ship the bootstrapper next to the exe as a fallback.

## build\_installer.ps1 — the Windows installer

build\_installer.ps1 wraps the finished dist\ARGUS.exe into a proper Windows installer. Run it from deployer/ after build.ps1:

```
powershell -ExecutionPolicy Bypass -File build_installer.ps1
```

Prereqs: dist\ARGUS.exe exists, Inno Setup 6 installed (winget install JRSoftware.InnoSetup), and the WebView2 evergreen bootstrapper present at installer\redist\MicrosoftEdgeWebView2Setup.exe. The script locates ISCC.exe and compiles installer\argus.iss; output is dist\installer\ARGUS-Setup-<version>.exe.

What the installed app gets, per argus.iss:

- Program Files\ARGUS\ARGUS.exe, a desktop shortcut (checked by default — Runzhe’s spec), a Start Menu entry, and an uninstaller.
- **WebView2 auto-install:** the bundled bootstrapper is staged to {tmp} and run /silent /install before app launch, gated by a registry check (WebView2Installed reads the evergreen runtime’s pv value under HKLM WOW6432Node / HKCU). Machines that already have the runtime skip it entirely.
- License page = the bundled Terms of Use; uninstall also removes the app’s working data under %LOCALAPPDATA%\ARGUS.
- The AppId GUID must never change after first release — it is how upgrades find the install.

The script is pure ASCII for the same PowerShell 5.1 encoding reason as build.ps1.

NOTE: app.py’s docstring says “Package: see build\_exe.md”; that file does not exist — build.ps1, build\_installer.ps1, and README.md are the packaging references.

## 7.8 The two-round rehearsal plan

deployer/REHEARSAL.md defines the acceptance protocol. Round 1 shakes out bugs cheaply; round 2 is the customer-realism dress rehearsal. Do not skip round 1 — burning the fresh-account experience on a packaging bug wastes it.

**Round 1 — cheap shakeout (dev machine, old account).** Target: the dormant old account 396278862184 (queued for closure; its Gen-1 residue usefully exercises the adopt-existing paths). Run from source (python app.py --debug), paste an IAM key for that account, run the full wizard, watch all 15 stages. PASS = the theater completes, the done screen shows a real CloudFront URL, and deploy\_state.json holds every id. Then the training pipeline: pick a small YOLO folder (~30 images), confirm the analysis report and cost gate, type TRAIN, let it run to TRAINING\_COMPLETED (~1 h, ~US\$4), confirm F1 appears and custom\_model\_arn landed on both camera rows, and close/reopen the app



mid-watch once to prove re-attach. “Exists, adopting” logs from residue are acceptable noise. Bugs found here get fixed before round 2.

**Round 2 — full dress rehearsal.** Realism on all four axes: build the real `dist\ARGUS.exe`; run it on a clean machine (the mini PC’s Win11 host — no Python, no AWS CLI, no cached credentials); create a fresh AWS account through the app’s embedded signup (email verification, card typed into AWS’s embedded page, phone code, Basic plan — the only true test of the one-stop UX); IAM key via the embedded console flow, confirmed to land in the Windows Credential Manager. Then a real deploy, and the day-1 customer smoke test: dashboard over HTTPS with login, first Cognito user, test upload appears in the gallery, train screen renders, SES verification email confirmed, watchdog schedule ENABLED. Cost check: 24 h idle should bill ~\$0. The account is kept afterwards as the demo/handoff account for Dr. Li, or closed (one console action).

**What actually happened (2026-08-10): the production deployment became the acceptance run.** The account migration to the NP production account 506868652945 merged into the rehearsal plan. The stage engine ran headless from the CLI — `python deploy.py --profile prod --prefix argus --target-label armyworm-larva --sender-email ... --deployment-name "Jewel Forest Valley"` — against the brand-new account and completed **all 15 stages in 103 seconds with zero errors**, first attempt, no resume needed. The KVS stage was skipped by design (`--live-view` not passed; live view is not migrated). Post-deploy verification confirmed the three 2026-08-10 gap fixes landed on a real fresh account: processor at 1024 MB / 600 s with the Pillow layer attached; env carrying Sonnet 4.6, `LLM_VERIFY_ALL_BOXES=true`, tile floor 8, `POST_VERIFY_FLOOR=49`, area cap 0.05; both camera rows reading `llm_verify_enabled=true`, `min_confidence=10`, `post_verify_floor=49`. The dashboard, API, Cognito pool, and Rekognition project ids it produced are listed in section 7.1.

This run covers the **account axis** of Round 2 — fresh account, no adoption residue, real CloudFront/Cognito/SES from zero. It exercised `deploy.py`, not the frozen exe: the **clean-machine axis** (packaged `dist\ARGUS.exe` on a no-Python machine, embedded signup and key flow, Credential Manager storage) remains untested and still deserves Round 2’s dress-rehearsal treatment if the product is taken further.

**Success criteria:** zero terminal windows and zero external-browser visits during the whole flow; every user-typed secret went either into AWS’s own embedded page (card, passwords) or into the OS credential store (IAM key), nothing in plaintext; the done screen’s URL serves the dashboard over HTTPS with login; the state file contains 6 role ARNs, 3 buckets, the layer ARN, 5 Lambdas, pool+client ids, API id/url, CloudFront id/domain, and the project ARN.

NOTE: doc lag — `REHEARSAL.md` lists “4 tables” among the state-file contents. `deploy.py` never writes table keys into `deploy_state.json`; the tables have fixed names and `verify()` checks them directly. The state-file checklist above is the code-accurate one.

## 7.9 Security doctrine summary

- **Card data never touches the product** (PCI-DSS). It is typed into AWS's own embedded pages; ARGUS observes navigation URLs only, never the DOM, never any field.
- **Root keys are refused** before storage, with the IAM-user path taught instead. Root is used exactly once in the customer's life with this product — account creation and (if ever) account closure — both inside AWS's own pages.
- **The IAM key lives in the OS credential store** (Windows Credential Manager via keyring, DPAPI). Never plaintext on disk, never logged, never returned to the UI.
- **Permission preflight over late failure:** the AdministratorAccess check (direct and group-inherited) names the fix at the keys screen instead of letting a deployment die 8 stages in.
- **Destroy is guarded three ways:** typed prefix confirmation, account-match between state file and credentials, and a refusal while training is live. Its result is proven by a live audit, not trusted.
- **Deterministic automation only:** no LLM ever holds credentials or mutates cloud resources (architecture ruling, 2026-07-16).
- **No secrets in the repo:** the deployer tree contains identifiers only (account ids, ARNs, bucket names). One known exception elsewhere in the project: `datasets/archive/experiments/pre_v3_abandoned/download.py` contains an inline Roboflow API key — credential stored there, not reproduced here. Treat that file as sensitive; do not publish it.
- **The WebView2 profile stays in %LOCALAPPDATA%,** never the project tree, so console session cookies can never ride along in a zip or handoff.

## 7.10 Operations / reproduction

### 7.10.1 Run the deployer from source

From the repository root:

```
cd deployer
pip install -r requirements.txt
python app.py --debug
```

--debug opens WebView2 devtools. Without pywebview, serving web/ from any static server gives the simulated preview (all actions fake).

### 7.10.2 Deploy a stack from the CLI (no app)

Bootstrap credentials must already be configured (`aws configure --profile **customer**`).

```
python deploy.py --profile **customer** --region **us-east-1** ^
--prefix **vision** --target-label **my-pest** ^
```

```
--sender-email **alerts@example.com** --recipient-email **me@example.com**  
*
```

Useful variants: --dry-run (print the plan, change nothing), --only \*\*s3,lambda\*\* (subset; never stamps the run complete), --live-view (adds the KVS stage's stream).

The 2026-08-10 production deployment is a worked example of exactly this path: --profile prod --prefix argus --target-label armyworm-larva plus the emails and deployment name, 15/15 stages in 103 seconds on account 506868652945, KVS skipped because --live-view was omitted.

### 7.10.3 Verify a deployment

App: done screen -> System check. CLI:

```
python deploy.py --profile **customer** --region **us-east-1** --prefix **vision** ^  
--target-label x --sender-email x@x --verify
```

Expect verify: N/N present. Any MISS line names the missing resource.

Console spot checks (region **us-east-1** unless you deployed elsewhere):

- Lambda: console -> Lambda -> Functions -> expect the 5 pest-\* functions. CLI: `aws lambda list-functions --profile **customer** --query "Functions[].FunctionName"`
- Watchdog: console -> Amazon EventBridge -> Scheduler -> Schedules -> pest-model-watchdog-15min ENABLED. CLI: `aws scheduler get-schedule --name pest-model-watchdog-15min --profile **customer**`
- CloudFront: console -> CloudFront -> Distributions -> status Deployed. CLI: `aws cloudfront get-distribution --id **<cloudfront:id from deploy_state.json>** --profile **customer**`

### 7.10.4 Inspect the state file

Frozen exe: %LOCALAPPDATA%\ARGUS\out\deploy\_state.json. Dev: deployer/out/deploy\_state.json. It should contain 6 role:\*, 3 bucket:\*, layer:fyp-pillow, 5 lambda:\*, cognito:pool/client, apigw:id/url, cloudfront:id/domain, rekognition:project, and the deploy:\* stamps.

### 7.10.5 Confirm the stored credential

Console path (Windows): Control Panel -> Credential Manager -> Windows Credentials -> look for entries under ARGUS-deployer. PowerShell check that the entry exists (does not reveal the secret):

```
cmdkey /list | Select-String ARGUS
```

### 7.10.6 Create the first dashboard sign-in account

App: done screen -> Dashboard accounts -> add email + password. Console: Amazon Cognito -> User pools -> pest-dashboard-users -> Users -> Create user. CLI:

```
aws cognito-idp admin-create-user --user-pool-id **<cognito:pool>** --username **user@example.com** --user-attributes Name=email,Value=**user@example.com*  
* Name=email_verified,Value=true --message-action SUPPRESS --profile **customer**  
aws cognito-idp admin-set-user-password --user-pool-id **<cognito:pool>** --username **user@example.com** --password "***<password>*" --permanent --profile **customer**
```

The permanent flag matters: the dashboard login does not handle FORCE\_CHANGE\_PASSWORD.

### 7.10.7 Confirm SES identities

Each address gets a verification email that must be clicked. Console: Amazon SES -> Verified identities -> status Verified. CLI:

```
aws ses get-identity-verification-attributes --identities **alerts@example.com** --profile **customer**
```

While the account is in the SES sandbox, alerts only reach verified addresses. Production access is a manual AWS request (~1–2 business days).

### 7.10.8 Train a model

App: done screen -> Train your model -> pick the YOLO folder -> review the analysis -> choose the class -> pass the cost gate (typed TRAIN). Training runs 1–4 h in AWS; the app can be closed after submit and re-attached later (the Train screen's resume bar, or watch\_training). Watch progress in the console: Amazon Rekognition -> Custom Labels -> Projects -> \*\*<prefix>\*\* -detection -> the training version. CLI:

```
aws rekognition describe-project-versions --project-arn **<rekognition:project>** --profile **customer** --region **us-east-1**
```

On completion the app wires custom\_model\_arn onto every model\_type=custom camera row automatically. Verify:

```
aws dynamodb get-item --table-name pest-monitoring-cameras --key "{\"camera_id\":{\"S\":\"camera-1\"}}\" --profile **customer** --region **us-east-1**
```

The model is left STOPPED by design. Start it from the dashboard when needed; the watchdog stops it after the camera row's max\_runtime\_min (seeded 45). The env MAX\_RUNTIME\_MIN=75 is only the global fallback for rows without the field.

### 7.10.9 Destroy a deployment

App: done screen -> Danger zone -> Delete deployment -> type the deployment prefix exactly -> watch the destroy log stream -> the final line must read 0 resources remaining. The state file is archived as `destroyed_{timestamp}.json` in the same `out/` folder. Account closure afterwards (optional) is guided through the embedded AWS window: Billing -> Account -> Close account, as root, in AWS's own page.

### 7.10.10 Rebuild the exe

From the repository root:

```
cd deployer
powershell -ExecutionPolicy Bypass -File build.ps1
powershell -ExecutionPolicy Bypass -File build_installer.ps1
```

Output: `dist\ARGUS.exe` (onefile) and `dist\installer\ARGUS-Setup-<version>.exe` (installer; auto-installs the WebView2 runtime if missing). If shipping only the bare exe, put `MicrosoftEdgeWebView2Setup.exe` next to it for machines without the runtime.

## 7.11 Cross-references

- Chapter 1 — where the deployer sits in the overall system, and the production-vs-testbed framing.
- Chapter 2 — the cloud stack the deployer recreates (Lambdas, tables, buckets, API Gateway, Cognito, CloudFront, SES, the watchdog); `STACK_MANIFEST.md` is the deployer-side mirror of that chapter's live-account facts.
- Chapter 3 — detection models: what the training pipeline produces, F1 history, Rekognition Custom Labels limits (4096 px, zero-box images, account-bound models).
- Chapter 4 — the dashboard that stage 10 (writeback) templates and syncs; its Cognito auth is what the done screen's account panel manages.
- Chapter 8 — reproducing the whole system on a fresh AWS account end to end; ARGUS is the fast path for that chapter, and the 2026-08-10 production deployment to 506868652945 is its proof. Chapter 8 also lists the account ids in play (development/reference 366356442579, NP production 506868652945, old 396278862184).

## Chapter 8 — Reproducing the system from scratch

This chapter is the from-zero runbook: how to stand up the complete ARGUS stack in a NEW AWS account, by the automated path (the ARGUS deployer) or the manual path (console + CLI, resource by resource).

*As of 2026-08-11.*

### 8.1 Role in the system

This runbook is no longer theory. On 2026-08-10 it was executed for real: the full stack was deployed into the NP production account 506868652945 (IAM user Student\_QianRunzhe, CLI profile prod) — all 15 stages green in 103 seconds, zero errors — and the days after ran the complete post-deploy sequence: SES identity verification, Cognito user creation, server-side training-data copy, and retraining of both detection models on the new account. The manual therefore describes two accounts:

- **366356442579** (us-east-1, CLI profile prod) — the development/reference account where the system was built and validated. Chapters 1–7 describe it.
- **506868652945** — the NP production account being stood up (in progress; the cloud stack and both retrained models are already live there). Device repointing is the main remaining step (Section 8.9.3).

Every account-bound value changes between accounts: account ID, bucket names, API Gateway ID, Cognito IDs, CloudFront domain, and — critically — the Rekognition Custom Labels models, which cannot be copied between accounts at all (Section 8.9).

Two reproduction paths exist and produce the same stack:

- **Path A (recommended): the ARGUS deployer.** One Windows executable runs 15 idempotent stages end to end and writes every generated ID back into the dashboard config before syncing it. This path has been executed for real twice: a full 15/15-stages-green deployment into a fresh, self-registered AWS account (CAG\_Test, 324908170757) on 2026-07-16, and the production migration into 506868652945 on 2026-08-10 (Section 8.4.4).
- **Path B: manual.** The same 15 steps done by hand, console click-path plus CLI for each, in the dependency order from `deployer/STACK_MANIFEST.md` §2. Use this if the exe is unavailable, if a stage needs debugging, or to understand exactly what Path A does.

Both paths end at the same place: a live dashboard URL over HTTPS, an empty Rekognition project waiting for training, and a day-1 task list (Section 8.5). The cloud stack is complete and testable without any robot or camera — the dashboard's Test upload exercises the full detection rail. Device provisioning (Go2/Orin, mini PC) is a separate, optional layer (Section 8.8).

## 8.2 Inventory

Paths are relative to the project repository root.

| Component             | Location                                      | Purpose                                                                                                                                                                            |
|-----------------------|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ARGUS.exe             | deployer/dist/ARGUS.exe                       | The one-file deployer app (pywebview shell + deploy engine). Built by build.ps1. Rebuilt 2026-08-11 with the Bedrock policy, the per-camera watchdog, and the v6.3 Lambda sources. |
| ARGUS-Setup-1.0.0.exe | deployer/dist/installer/ARGUS-Setup-1.0.0.exe | Inno Setup installer for the exe: Program Files install, shortcuts, uninstaller, auto-installs the WebView2 runtime when missing. Built by build_installer.ps1.                    |
| deploy.py             | deployer/deploy.py                            | The 15-stage boto3 engine. Also runnable headless (python deploy.py ...). Contains verify() (post-deploy audit) and destroy() (teardown).                                          |
| training.py           | deployer/training.py                          | The Train-screen engine: YOLO folder → validated Rekognition dataset → training → custom_model_arn write-back.                                                                     |
| STACK_MANIFEST.md     | deployer/STACK_MANIFEST.md                    | The recreate-level bill of materials, audited from the live account. Path B follows its §2 creation order.                                                                         |
| deployer/audit/*.json | deployer/audit/                               | Raw per-resource config JSONs (IAM policies, table specs, CORS, CloudFront config). Path B reuses them as CLI inputs after rewriting account/region/bucket literals.               |
| Lambda code mirror    | lambda/                                       | Source of 4 functions (pest-detection-processor.py,                                                                                                                                |

| Component              | Location                                                                                    | Purpose                                                                                                                                                                   |
|------------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                        |                                                                                             | pest-monitoring-api.py, pest-camera-scheduler.py, pest-model-watchdog.py) plus reusable policy files.                                                                     |
| kvs-hls-handler source | deployer/audit/kvs-hls-handler_src/lambda_function.py                                       | The fifth function's code. NOT in lambda/ — it lives under deployer/audit/.                                                                                               |
| lambda/cors.json       | lambda/cors.json                                                                            | API Gateway CORS configuration, reusable as-is for create-api.                                                                                                            |
| lambda/ddb-policy.json | lambda/ddb-policy.json                                                                      | The api role's 6-verb DynamoDB policy (pest-monitoring-ddb-full-access). Account ID inside must be rewritten.                                                             |
| Bedrock verify policy  | deployer/audit/iam__pest-detection-processor-role__inline__bedrock-verify.json              | Bedrock invoke policy for the LLM verification gate (Sonnet 4.6 + Haiku 4.5 inference profiles). Since 2026-08-11 the deployer attaches it automatically (Section 8.9.4). |
| REHEARSAL.md           | deployer/REHEARSAL.md                                                                       | The two-round rehearsal protocol and its success criteria; the 2026-08-10 production migration served as Round 2 on the fresh-account axis.                               |
| Dashboard source       | web/dashboard_v4/                                                                           | The 15 static files the deployer templates and syncs to the new dashboard bucket.                                                                                         |
| Deploy state           | deployer/out/deploy_state.json (dev)<br>or %LOCALAPPDATA%\ARGUS\out\deploy_state.json (exe) | Every generated ID from a run. Input to verify/resume/teardown.                                                                                                           |
| Migration toolkit      | migration/                                                                                  | The scripts that executed the 2026-08-10/11 production migration: copy_training_data.py (server-side training-data                                                        |



| Component | Location | Purpose                                                                                                                                                                                                                                                                                                                                                       |
|-----------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|           |          | copy + grant/revoke),<br>train_v9r_on_prod.py<br>(dataset-export retrain),<br>poll_training.py (training-<br>status poller),<br>migrate_moth.py (moth<br>model rebuild),<br>migrate_jewel_records.py<br>(on-site detection-record<br>migration),<br>diff_accounts.py (old-vs-<br>new audit),<br>prod_baseline_20260810.j<br>son (frozen reference<br>config). |

### 8.3 Prerequisites (both paths)

1. **A Windows machine.** Windows 10/11. The app uses Edge WebView2 (preinstalled on Windows 11 and current Windows 10). Two artifacts exist: the bare ARGUS.exe (deployer/dist/), which assumes WebView2 is present, and the installer ARGUS-Setup-1.0.0.exe (deployer/dist/installer/), which auto-installs the WebView2 runtime when missing (bundled evergreen bootstrapper). On an unknown machine, use the installer.
2. **The ARGUS.exe** (Path A), or Python 3.12+ with boto3 and the AWS CLI v2 (Path B / headless Path A). Rebuild the exe with powershell -ExecutionPolicy Bypass -File build.ps1 in deployer/ if needed; the script prebuilds layer/fyp-pillow.zip on first run. Close any running ARGUS.exe first — PyInstaller cannot replace a running file.
3. **A fresh AWS account.** Path A's embedded AWS window can guide account signup inside the app (email verification, payment card typed directly into AWS's own page, phone code, Basic plan). AWS signup cannot be automated — CAPTCHA, card, and phone are always manual. An institutionally issued account works the same way: the NP production account 506868652945 arrived with an IAM user (Student\_QianRunzhe) already in an AdministratorAccess group, so step 4 was already satisfied there.
4. **An IAM user with AdministratorAccess — never root.** Create a user (e.g. deployer), attach the AWS-managed AdministratorAccess policy, create an access key (CLI type), and keep the secret for the paste step. The ARGUS keys screen walks this end to end and **refuses root keys outright**; it also preflights that the pasted key actually has AdministratorAccess (including via group membership) so a rights-less key fails at paste time, not 8 stages in.

- Console: **IAM** → **Users** → **Create user** → name deployer → **Attach policies directly** → AdministratorAccess → create → **Security credentials** → **Create access key** → **Command Line Interface**.

- CLI (from any machine already holding credentials for the new account):

```
aws iam create-user --user-name deployer --profile **prod**
aws iam attach-user-policy --user-name deployer --policy-arn arn:aws:iam::aws:policy/AdministratorAccess --profile **prod**
aws iam create-access-key --user-name deployer --profile **prod**
```

The secret access key is shown once. Store it in the app (it lands in Windows Credential Manager via DPAPI, never plaintext on disk) or in an AWS CLI profile. Never write it into any file in this repo.

5. **An alert email address** you can open — SES sends a verification link to it during deployment.
6. **Read Section 8.9 (account-portability caveats) before starting.** In particular: no trained model migrates; the new account must retrain.

## 8.4 Path A — automated deployment with ARGUS

### 8.4.1 The wizard flow

Run ARGUS.exe. The screens, in order:

1. **Welcome** — the opening screen the app lands on; Start continues to Consent.
2. **Consent** — checkbox + Terms/Privacy folds. Decline exits.
3. **Account** — optional embedded AWS signup (skip if the account exists).
4. **Keys** — the 4-step IAM rail; paste the access key ID + secret. STS liveness check + root-key refusal + AdministratorAccess preflight run here.
5. **Config** — deployment name, region, resource prefix (lowercase, 2–21 chars, [a-z][a-z0-9-]\*), detection target label (the object class you will later train, e.g. armyworm-larva), alert email (required, validated), live-view on/off.
6. **Review** — the plan plus the cost estimate (pricing appears here and nowhere else).
7. **Hold-to-deploy** — the 15 stages stream into the aperture “theater” UI.
8. **Done** — real dashboard URL, accounts panel, Train screen entry, “Run a system check”, Danger zone (teardown).

### 8.4.2 The 15 stages (what actually happens)

Stage order is fixed in `deploy.py` (STAGES); every stage is idempotent — an existing resource is adopted, not duplicated, so re-running after a failure resumes safely.

| # | Stage | UI label               | Creates                                                        |
|---|-------|------------------------|----------------------------------------------------------------|
| 1 | iam   | Identity & permissions | 5 Lambda execution roles + inline policies (from the rewritten |

| # | Stage    | UI label | Creates                                                                                                                                                                                                                                                                                                                                                                       |
|---|----------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   |          |          | <p>deployer/audit/documents) + the pest-scheduler-invocation-role (trust: scheduler.amazonaws.com, invoke on both pest-model-watchdog and pest-camera-scheduler). Since 2026-08-11 the processor role also gets the live bedrock-verify and s3-frames-write inline policies, so the LLM verification gate works on a fresh stack without manual IAM work (Section 8.9.4).</p> |
| 2 | dynamodb | Database | <p>4 tables, PAY_PER_REQUEST: pest-monitoring-cameras, pest-monitoring-detections (+ GSI by-pest-time), pest-monitoring-system-config, pest-monitoring-schedule-logs.</p>                                                                                                                                                                                                     |
| 3 | s3       | Storage  | <p>3 buckets: **PREFIX**-frames- **ACCT**, **PREFIX**-processed- **ACCT** (private, versioned, SSE-S3), **PREFIX**-dashboard- **ACCT** (public-read website, PAB disabled first —</p>                                                                                                                                                                                         |

| # | Stage  | UI label                 | Creates                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---|--------|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4 | layer  | Image-processing library | <p>ordering trap).</p> <p>Publishes the fyp-pillow layer (Pillow 12.2.0, cp312, manylinux2014). The exe uses the prebuilt bundled zip; a dev run pip-builds it and asserts PIL/_imaging*.so exists (the exact defect that broke layer v1).</p>                                                                                                                                                                                                                                          |
| 5 | lambda | Cloud functions          | <p>The 5 functions, pest-camera-scheduler first (the api's SCHEDULE_EXECUTOR_ARN env references it). Retries through IAM eventual consistency. Since 2026-08-10 the processor deploys at the validated production sizing (1024 MB / 600 s — a tiled frame plus up to 120 verify crops measures 24–54 s) with the full detection-tuning env: Sonnet 4.6 verifier, TILE_MIN_CONFIDENCE=8, LLM_VERIFY_ALL_BOXES=true, MAX_BOXES=120, POST_NMS_IOU/CONTAIN=0.1, POST_MAX_BOX_AREA=0.05,</p> |

| # | Stage           | UI label             | Creates                                                                                                                                                                                                                                                                                        |
|---|-----------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 6 | s3-notification | Event wiring         | <p>POST_VERIFY_FLOOR=49. The watchdog ships from the repo mirror lambda/pest-model-watchdog.py (per-camera max_runtime_min).</p> <p>lambda add-permission for s3.amazonaws.com FIRST, then the frames-bucket s3:ObjectCreated:* notification on prefix frames/ → pest-detection-processor.</p> |
| 7 | cognito         | Sign-in service      | <p>Pool pest-dashboard-users (email sign-in, admin-create-only) + SPA client dashboard-web (no secret, USER_PASSWORD_AUTH, 12 h tokens). Zero users seeded.</p>                                                                                                                                |
| 8 | apigw           | Secure API           | <p>HTTP API pest-monitoring-api-gateway: CORS, JWT authorizer cognito-dashboard, 2 integrations, 21 routes (JWT on all but GET /stream/status), \$default auto-deploy stage, 2 invoke permissions.</p>                                                                                         |
| 9 | cloudfront      | Dashboard & delivery | <p>Distribution over the dashboard bucket's <b>S3 website</b></p>                                                                                                                                                                                                                              |

| #  | Stage       | UI label      | Creates                                                                                                                                                                                                                                                                                                                                           |
|----|-------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 10 | writeback   | Configuration | <p><b>endpoint</b> (http-only origin), Managed-CachingDisabled policy, fresh CallerReference. Takes minutes to reach Deployed.</p> <p>Rewrites js/config.js (HTTP_API, COGNITO_REGION, COGNITO_CLIENT_ID) , uploads all web/dashboard_v4/ files to the dashboard bucket, sets the frames-bucket CORS to the new website + CloudFront origins.</p> |
| 11 | kvs         | Live video    | <p>Only with live-view enabled: stream <b>**PREFIX**-cam-stream</b>, 2 h retention.</p>                                                                                                                                                                                                                                                           |
| 12 | scheduler   | Cost guard    | <p>EventBridge Scheduler pest-model-watchdog-15min, rate(15 minutes), target = the watchdog via the invocation role.</p>                                                                                                                                                                                                                          |
| 13 | rekognition | Vision engine | <p>Empty Custom Labels project <b>**PREFIX**-detection</b>. No model — training is a later, customer-driven step.</p>                                                                                                                                                                                                                             |
| 14 | seed        | Initial data  | <p>1 system-config row (detection_settings:</p>                                                                                                                                                                                                                                                                                                   |

| #  | Stage | UI label | Creates                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----|-------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    |       |          | email_enabled=true<br>, recipient_email,<br>auto_capture=true,<br>capture_interval=6<br>0) + 2 camera rows:<br>manual_upload and<br>camera-1 (label =<br>deployment name,<br>custom_model_arn='<br>' until training).<br>Since 2026-08-10<br>both rows seed the<br>validated detection<br>config:<br>min_confidence=10<br>(the CANDIDATE<br>floor feeding the gate<br>— seeding it high<br>silently strangles<br>recall),<br>llm_verify_enabled<br>=true,<br>post_verify_floor=<br>49; the real camera<br>also gets<br>tiling_enabled=tru<br>e and<br>max_runtime_min=45<br>. The code is the<br>truth if any doc lags.<br>verify-email-<br>identity for sender<br>+ recipient (sends<br>the confirmation<br>email) and prints the<br>sandbox warning. It<br>does not wait for you<br>to click the link. |
| 15 | ses   | Alerts   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

Every generated ID is written to `deploy_state.json` as it appears. On the exe the file lives at `%LOCALAPPDATA%\ARGUS\out\deploy_state.json`.

### 8.4.3 Headless equivalent

The same engine runs without the app on any machine with Python + boto3 and an AWS CLI profile for the new account. This is exactly how the 2026-08-10 production migration ran:

```
python deployer/deploy.py --profile **prod** --prefix argus --target-label ar  
myworm-larva --sender-email **alerts@np.edu.sg** --recipient-email **you@np.e  
du.sg** --deployment-name "Jewel Forest Valley"
```

(Substitute your own profile, emails, and deployment name; add `--live-view` if the KVS live-video layer is wanted — the production run omitted it.)

Useful flags: `--dry-run` (print the plan, change nothing), `--only s3,lambda` (run selected stages; a subset run never writes the completion stamp), `--verify` (read-only audit of an existing deployment: prints [OK ]/[MISS] per resource, then N/N present, plus “deployment is INCOMPLETE” if anything is missing).

### 8.4.4 The verified precedents

**First precedent — CAG\_Test (2026-07-15/16).** The full Path A flow first ran on a fresh self-registered account (CAG\_Test, 324908170757):

- Run 1: 9/15 stages created live (iam through cloudfront), then writeback failed — the exe bundle was missing web/dashboard\_v4 (build bug, fixed in build.ps1).
- Run 2 (resume): failed at apigw — the route-adopt branch stripped a required parameter, a path only reachable on resume. Fixed 2026-07-16.
- Run 3 (resume): **15/15 complete** — the product’s first successful end-to-end deployment. Existing resources adopted, the remaining six stages green first time.

Post-deploy management (accounts panel, teardown, verify audit) shipped and was exercised the same day. The CAG\_Test account was later closed.

**Second precedent — the real production migration (2026-08-10).** The headless command in 8.4.3 deployed the stack into the NP production account 506868652945: **all 15 stages, 103 seconds, zero errors, first run.** This served as REHEARSAL.md Round 2 on the fresh-account axis (though not the clean-machine axis). What it created:

- Dashboard: <https://d1dtoxef7qmug1.cloudfront.net> (CloudFront E1YADURLSAVNFA)
- API: <https://vzf17s6z00.execute-api.us-east-1.amazonaws.com> (21 routes, JWT on all but GET /stream/status)
- Cognito pool us-east-1\_9selFDHpc, client 6vebotf45bp8u46cnraddiaplv
- Buckets argus-frames-506868652945, argus-processed-506868652945, argus-dashboard-506868652945
- Empty Rekognition project argus-detection; 6 roles, 4 tables, fyp-pillow layer v1, 5 Lambdas, the S3→processor notification, the watchdog schedule



- KVS skipped (deployed without `--live-view`)

Post-deploy verification passed on every point that had previously been a gap: the processor came up at 1024 MB / 600 s with the layer attached, the env carried the full Sonnet 4.6 detection tuning, and both camera rows read `llm_verify_enabled=true`, `min_confidence=10`, `post_verify_floor=49`. Section 8.5 walks the post-deploy sequence this migration then executed.

## 8.5 Path A — day-1 post-deploy tasks (worked example: the 2026-08-10/11 production migration)

The deployer ends with a live but empty system. Four tasks make it usable. Every one of them has now been executed for real on the production account, so each subsection below pairs the generic procedure with what actually happened there.

### 8.5.1 Confirm the SES identity, note the sandbox

Open the “Amazon Web Services – Email Address Verification Request” email and click the link. Then check:

- Console: **Amazon SES → Identities** — the address must show **Verified**.
- CLI:

```
aws ses get-identity-verification-attributes --identities **alerts@np.edu.sg*
* --profile **prod** --region **us-east-1**
```

A fresh account is in the **SES sandbox**: max 200 messages/24 h, 1/sec, and mail can only be sent **to verified addresses**. For a demo this is fine (verify every recipient). To lift it, request production access — a manual AWS review, ~1–2 business days:

- Console: **Amazon SES → Account dashboard → Request production access**.
- CLI:

```
aws sesv2 put-account-details --production-access-enabled --mail-type TRANSACTIONAL --website-url **https://your-dashboard-url** --use-case-description "**pest detection alert emails to site staff**" --profile **prod** --region **us-east-1**
```

**Worked example:** the production account’s alert identity was verified 2026-08-11 (the owner clicked the link; the identity read SUCCESS on the next check). The account remains in the sandbox — the reference account is too, so this is not a regression — meaning alert mail only reaches verified recipients until production access is requested.

### 8.5.2 Create dashboard users (the pool starts empty)

A fresh Cognito pool has zero users and self-signup is disabled, so nobody can log in until an account is admin-created. The ARGUS done screen’s **accounts panel** lists/creates/removes users and sets the password as **permanent** (avoiding the `FORCE_CHANGE_PASSWORD` limbo that the dashboard’s login flow does not handle).

- Console: **Amazon Cognito → User pools → pest-dashboard-users → Users → Create user** (email as username; then still set a permanent password via CLI below, or the user is stuck in the temporary-password state).
- CLI:

```
aws cognito-idp admin-create-user --user-pool-id **NEW_POOL_ID** --username *
*user@np.edu.sg** --message-action SUPPRESS --profile **prod** --region **us-
east-1**
aws cognito-idp admin-set-user-password --user-pool-id **NEW_POOL_ID** --user
name **user@np.edu.sg** --password "***ChosenPassword1**" --permanent --profil
e **prod** --region **us-east-1**
```

Then open the CloudFront URL from the done screen and sign in.

**Worked example:** the production migration admin-created one user in pool us-east-1\_9selFDHpc with a permanent password (status CONFIRMED, no forced change), and sign-in against <https://d1dtoxef7qmugl.cloudfront.net> was confirmed 2026-08-11. One lesson from that check is worth keeping: the dashboard shows the camera row's label, not its camera\_id. The deployer had written the deployment name into the label, so the operator opened the new dashboard, saw no familiar camera name, and reasonably concluded the row was missing — the backend was fine the whole time. A display name IS functionality when it is the only handle the user has on a resource; set labels to what the operator expects.

### 8.5.3 Train a model (mandatory — no model migrates)

The new account's Rekognition project is empty. Use the ARGUS **Train screen** (done screen → "Train your model"): pick a labeled image folder (YOLOv8, Roboflow export, or flat layout auto-detected), review the analysis report (pairing, normalized-coords check, zero-box exclusion, >4096 px downscale), pass the typed-TRAIN cost gate, and watch the 6-phase rail to TRAINING\_COMPLETED. The pipeline then **auto-wires the resulting custom\_model\_arn onto every model\_type=custom camera row** and offers one-click rollback. Training is left STOPPED afterwards by design; start detection from the dashboard when needed (~US\$4/hr while running; the watchdog stops idle models). Closing and reopening the app mid-training re-attaches to the watch. (The ARGUS.exe on disk was rebuilt 2026-08-11, so the built exe contains the Train screen; if a future rebuild's done screen ever lacks "Train your model", rebuild via build.ps1.)

The manual/CLI equivalent of this whole step is the Chapter 3 training pipeline (dataset build → GroundTruth manifests → create-dataset → create-project-version → write the ARN into the camera row).

**Worked example — how the production migration actually retrained both models.** The migration did not start from a local image folder; it reused the training data already in the reference account's S3, entirely server-side. The sequence generalizes to any account-to-account move:

**(a) Copy the training data server-side** (`migration/copy_training_data.py`). A temporary, prefix-scoped statement is merged into the OLD frames bucket's policy (by Sid, preserving existing statements) granting the NEW account's IAM user `s3:GetObject` on `training-data/v9/*` only, plus a prefix-conditioned `s3:ListBucket`. The new account then `copy_objects` every key under that prefix into `argus-frames-506868652945/training-data/v9/...` — the bytes never touch a laptop. The script also rewrites every manifest `source-ref` to point at the new bucket and verifies object counts. Result on 2026-08-10: **36,641/36,641 objects (4.27 GB) copied, 0 failed, counts match; 36,634 manifest source-refs repointed**. The read grant stays in place until the new model is signed off, then `python migration/copy_training_data.py --revoke` removes it (and only it).

**(b) The trap that would have wasted a multi-hour training run — the dataset, not the manifest, is the source of truth.** Any labels added or corrected in the **Rekognition console** after a manifest was uploaded exist only inside that account's DATASET; the S3 manifest is stale from that moment. Building the new account's dataset from the copied manifest would have silently trained an older labelling state. The correct method, scripted in `migration/train_v9r_on_prod.py`: export the live entries with `ListDatasetEntries` (the labelled truth), repoint every `source-ref`, upload the result as new manifests, then `create-dataset` from those. Generalize this: **after ANY console labelling, export the dataset — never trust the original manifest**.

**(c) Armyworm retrain.** With datasets built the correct way (TRAIN 32,986 labelled images, TEST 3,653), `v9r-prod-20260810` — the v9-generation model with added data augmentation (flips, rotations, exposure jitter; the documented 13x build) — was submitted on project `argus-detection` on 2026-08-10. At ~33k images this is a run of many hours; poll with `DescribeProjectVersions` (`migration/poll_training.py`) rather than watching the console. Outcome: `TRAINING_COMPLETED` 2026-08-12 10:04 SGT, Rekognition test-split F1 0.613 (development-account equivalent 0.599 — same recipe, treat as equivalent), and the version ARN was written into the `worm_cam` camera row. The post-training holdout validation re-push must be repeated: the first 2026-08-12 attempt was invalidated by the processor-role IAM gap fixed the same day (see 9.3.14).

**(d) Moth model rebuild** (`migration/migrate_moth.py`). The predecessor's moth model (`SmartPestProject`) is account-bound and cannot move, but its DATA survived intact in the old account's Rekognition console bucket (`custom-labels-console-us-east-1-d1abc2aed2`): 116 TRAIN + 29 TEST labelled images. The script copies them server-side (same temporary-grant pattern, `--revoke` after), exports the labelled dataset entries (`dataset-is-truth` again), creates project **argus-moth-detection**, trains **moth-prod-20260811**, and seeds the `moth_cam` camera row. Trained 2026-08-11 with **F1 = 0.991** vs the original's 0.988 — same data, so treat them as equivalent, and as proof that a Custom Labels capability is fully reconstructible on a new account from data alone. `l1m_verify_enabled` is deliberately FALSE on `moth_cam`: the verification prompt describes larvae and this camera's target is adult moths.

When each training completes, write its version ARN into the matching camera row's `custom_model_arn` (the Train screen does this automatically; scripted runs do it with a `file:///JSON update-item`).

#### 8.5.4 Wire cameras (optional, demo layer)

The stack detects on anything uploaded to `frames/**camera_id**/...` in the frames bucket — the dashboard Test upload already proves the rail. To attach real capture devices (Go2 + SIYI A8 Mini via the Jetson Orin, or the mini PC's Hikvision), provision the devices per Section 8.8 and repoint their config at the new account (Section 8.9.3).

**Worked example:** this is the one step the production migration has NOT yet completed — the Orin and mini PC still point at the reference account. During the migration the production camera rows were also aligned to the reference-account ids the devices and operators expect (`worm_cam`, `moth_cam`, `manual_upload`) with the reference labels and schedule restored, so a repointed device drops straight into a matching row. The KVS live-view layer was deliberately not deployed (Section 8.4.4), so `kvs_stream_name` is empty on the new account; create the stream (Path B Step 11) before repointing any `kvs_controller` if live view is wanted.

### 8.6 Path B — manual resource creation, in order

Follow the order exactly — it is the dependency order from `STACK_MANIFEST.md` §2 and the `deploy.py` stage list. Everything below assumes region **us-east-1**; substitute yours consistently. Placeholders in bold are yours to fill: **YOUR\_ACCOUNT\_ID**, profile **np**, prefix **vision** (or keep the reference names, e.g. `frames-armyworm-**YOUR_ACCOUNT_ID**` — names must be consistent everywhere, including Lambda env vars).

Before starting, prepare the policy inputs: copy the needed `deployer/audit/iam__*.json` files plus `lambda/ddb-policy.json` to a working folder and rewrite every occurrence of `366356442579` → **YOUR\_ACCOUNT\_ID**, `us-east-1` → **YOUR\_REGION**, and the three reference bucket names → your bucket names. (`deploy.py Ctx.rewrite()` does exactly this mechanically.)

#### Step 1 — IAM roles (everything else references these)

Six roles. The 5 Lambda execution roles share one trust policy (`deployer/audit/iam__trust-policy-lambda.json`); each gets `AWSLambdaBasicExecutionRole` attached plus its inline policies.

Console: **IAM** → **Roles** → **Create role** → **AWS service** → **Lambda** → attach `AWSLambdaBasicExecutionRole` → name → after creation, **Add permissions** → **Create inline policy** → **JSON** and paste each rewritten document.

CLI pattern (repeat per role/policy):

```
aws iam create-role --role-name pest-detection-processor-role --assume-role-policy-document file:///deployer/audit/iam__trust-policy-lambda.json --profile
```

```

**prod**
aws iam attach-role-policy --role-name pest-detection-processor-role --policy
-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole --profi
le **prod**
aws iam put-role-policy --role-name pest-detection-processor-role --policy-na
me pest-detection-processor-policy --policy-document file:///**rewritten\iam__
pest-detection-processor-role__inline__pest-detection-processor-policy.json**
--profile **prod**

```

| Role                          | Inline policies (rewritten audit file / source)                                                                                                                           |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pest-detection-processor-role | pest-detection-processor-policy, read-system-config, bedrock-verify (LLM gate — Sonnet 4.6 + Haiku 4.5 inference-profile ARNs, account/region rewritten), s3-frames-write |
| pest-monitoring-api-role      | pest-monitoring-api-policy, pest-monitoring-ddb-full-access ( <b>reuse lambda/ddb-policy.json</b> , account rewritten), read-processed-armyworm                           |
| pest-camera-scheduler-role    | pest-camera-scheduler-policy                                                                                                                                              |
| kvs-hls-handler-role          | kvs-hls-policy (from the managed kvs-hls-handler-rolePolicy audit file; inline is fine)                                                                                   |
| pest-model-watchdog-role      | rekognition-db-handler                                                                                                                                                    |

Then the **scheduler invocation role** pest-scheduler-invocation-role: trust scheduler.amazonaws.com with condition aws:SourceAccount = **\*\*YOUR\_ACCOUNT\_ID\*\***; one inline policy invoke-scheduled-lambdas allowing lambda:InvokeFunction on **both** pest-model-watchdog(:\*) **and** pest-camera-scheduler(:\*). This fixes two live-account name gaps on purpose: the reference account's watchdog schedule uses a console-generated role, and its api policy's iam:PassRole points at a role name that does not exist there. In the new account, make sure the rewritten pest-monitoring-api-policy's iam:PassRole Resource is this role's ARN.

## Step 2 — DynamoDB (4 tables)

Console: **DynamoDB** → **Tables** → **Create table**, On-demand capacity, keys per the table below; on pest-monitoring-detections add GSI by-pest-time under **Indexes**. Do NOT enable TTL, PITR, or streams.

CLI:

```

aws dynamodb create-table --table-name pest-monitoring-cameras --billing-mode
PAY_PER_REQUEST --attribute-definitions AttributeName=camera_id,AttributeTyp
e=S --key-schema AttributeName=camera_id,KeyType=HASH --profile **prod** --re
gion **us-east-1**

```

```
aws dynamodb create-table --table-name pest-monitoring-detections --billing-mode PAY_PER_REQUEST --attribute-definitions AttributeName=image_id,AttributeType=S AttributeName=detection_time,AttributeType=S AttributeName=pest_type,AttributeType=S --key-schema AttributeName=image_id,KeyType=HASH AttributeName=detection_time,KeyType=RANGE --global-secondary-indexes "[{\"IndexName\":\"by-pest-time\",\"KeySchema\":[{\"AttributeName\":\"pest_type\",\"KeyType\":\"HASH\"},{\"AttributeName\":\"detection_time\",\"KeyType\":\"RANGE\"}],\"Projection\":{\"ProjectionType\":\"ALL\"}}]" --profile **prod** --region **us-east-1**
```

```
aws dynamodb create-table --table-name pest-monitoring-system-config --billing-mode PAY_PER_REQUEST --attribute-definitions AttributeName=config_key,AttributeType=S --key-schema AttributeName=config_key,KeyType=HASH --profile **prod** --region **us-east-1**
```

```
aws dynamodb create-table --table-name pest-monitoring-schedule-logs --billing-mode PAY_PER_REQUEST --attribute-definitions AttributeName=log_id,AttributeType=S AttributeName=timestamp,AttributeType=S --key-schema AttributeName=log_id,KeyType=HASH AttributeName=timestamp,KeyType=RANGE --profile **prod** --region **us-east-1**
```

### Step 3 — S3 (3 buckets)

**us-east-1 quirk:** create-bucket must be called WITHOUT --create-bucket-configuration in us-east-1 (passing a LocationConstraint there errors); in any other region you must pass it.

Console: **S3 → Create bucket** ×3; for frames/processed leave Block Public Access ON and enable **Versioning**; set default SSE-S3 encryption on all three. For the dashboard bucket: **Permissions → Block public access → uncheck all** FIRST, then add the public-read bucket policy, then **Properties → Static website hosting** with index and error document both index.html (the SPA fallback).

CLI (frames shown; repeat for processed; dashboard differs as noted):

```
aws s3api create-bucket --bucket **vision**-frames-**YOUR_ACCOUNT_ID** --profile **prod**
aws s3api put-bucket-encryption --bucket **vision**-frames-**YOUR_ACCOUNT_ID** --server-side-encryption-configuration "{\"Rules\":[{\"ApplyServerSideEncryptionByDefault\":{\"SSEAlgorithm\":\"AES256\"}}]}" --profile **prod**
aws s3api put-bucket-versioning --bucket **vision**-frames-**YOUR_ACCOUNT_ID** --versioning-configuration Status=Enabled --profile **prod**
aws s3api put-public-access-block --bucket **vision**-frames-**YOUR_ACCOUNT_ID** --public-access-block-configuration BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true --profile **prod**
```

Dashboard bucket (ordering trap — PAB off BEFORE the policy or PutBucketPolicy fails):

```
aws s3api create-bucket --bucket **vision**-dashboard-**YOUR_ACCOUNT_ID** --p
rofile **prod**
aws s3api put-public-access-block --bucket **vision**-dashboard-**YOUR_ACCOUNT_ID** --public-access-block-configuration BlockPublicAcls=false,IgnorePublicAcls=false,BlockPublicPolicy=false,RestrictPublicBuckets=false --profile **prod**
aws s3api put-bucket-policy --bucket **vision**-dashboard-**YOUR_ACCOUNT_ID** --policy '{"Version":"2012-10-17","Statement":[{"Sid":"PublicReadDashboard","Effect":"Allow","Principal":{"*"},"Action":["s3:GetObject"],"Resource":["arn:aws:s3:::**vision**-dashboard-**YOUR_ACCOUNT_ID**/*"]}]} ' --profile **prod**
aws s3api put-bucket-website --bucket **vision**-dashboard-**YOUR_ACCOUNT_ID** --website-configuration '{"IndexDocument":{"Suffix":"index.html"},"ErrorDocument":{"Key":"index.html"}}' --profile **prod**
```

The frames-bucket CORS and event notification come later (Steps 6 and 10) — they depend on the processor Lambda and the CloudFront domain.

## Step 4 — Lambda layer fyp-pillow

No source zip exists in any bucket — the layer is BUILT. On a machine with Python + pip:

```
pip install --platform manylinux2014_x86_64 --implementation cp --python-version 3.12 --only-binary=:all: --target **work\python** Pillow==12.2.0
```

**Before zipping, verify python/PIL/\_imaging\*.so and python/pillow.libs/ exist** — their absence is the exact defect that made layer v1 in the reference account silently draw no boxes. Zip with python/ at the archive root, then publish:

```
aws lambda publish-layer-version --layer-name fyp-pillow --zip-file fileb://**work\fyp-pillow.zip** --compatible-runtimes python3.12 --compatible-architectures x86_64 --profile **prod** --region **us-east-1**
```

Capture the returned LayerVersionArn — on a fresh account it ends :1, not :2; never hardcode the reference account’s version number. (Shortcut: deployer/layer/fyp-pillow.zip is the prebuilt, verified zip the exe bundles — publish that directly.)

Console alternative: **Lambda → Layers → Create layer**, upload the zip, runtime python3.12, arch x86\_64.

## Step 5 — Lambda functions (5)

See Section 8.7 for where each function’s code comes from and how to zip it. Create pest-camera-scheduler FIRST (the api function’s env references its ARN). Common settings: runtime python3.12, handler lambda\_function.lambda\_handler, arch x86\_64. If creation fails right after Step 1 with an invalid-role error, wait ~10 s and retry — IAM eventual consistency.

| Function | Mem / timeout | Role | Layer | Env vars |
|----------|---------------|------|-------|----------|
|----------|---------------|------|-------|----------|

| Function                 | Mem / timeout                                                                                                                                                     | Role                          | Layer            | Env vars                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pest-camera-scheduler    | 128 / 60                                                                                                                                                          | pest-camera-scheduler-role    | —                | TABLE_CAMERAS=pest-monitoring-cameras,<br>TABLE_SCHEDULE_LOGS=pest-monitoring-schedule-logs                                                                                                                                                                                                                                                                                                                                                                                                                            |
| pest-detection-processor | 1024 / 600 (the validated production sizing — a tiled frame plus up to 120 verify crops measures 24–54 s; the old 512/60 times out intermittently on real frames) | pest-detection-processor-role | fyp-pillow:**N** | TABLE_CAMERAS,<br>TABLE_SYSTEM_CONFIG=pest-monitoring-system-config,<br>TABLE_DETECTIONS=pest-monitoring-detections,<br>SENDER_EMAIL=*alerts@np.edu.sg**,<br>S3_PROCESSED_BUCKET=**vision**-processed-**YOUR_ACCOUNT_ID**, plus the detection tuning:<br>TILE_MIN_CONFIDENCE=8,<br>LLM_VERIFY_ALL_BOXES=true,<br>LLM_VERIFY_MODEL_ID=us.anthropic.claude-sonnet-4-6,<br>LLM_VERIFY_MAX_BOXES=120,<br>LLM_VERIFY_MAX_TOKENS=300,<br>LLM_VERIFY_WORKERS=3,<br>LLM_VERIFY_PAD=0.6,<br>POST_NMS_IOU=0.1,<br>POST_NMS_CONTA |



| Function            | Mem / timeout | Role                     | Layer | Env vars                                                                                                                                                                                                                                                                                                                                                                                           |
|---------------------|---------------|--------------------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pest-monitoring-api | 256 / 30      | pest-monitoring-api-role | —     | IN=0.1,<br>POST_MAX_BOX_AREA=0.05,<br>POST_VERIFY_FLOOR=49<br><br>TABLE_CAMERAS,<br>TABLE_SYSTEM_CONFIG,<br>TABLE_DETECTIONS,<br>TABLE_SCHEDULE_LOGS,<br>S3_FRAMES_BUCKET=**vision**-frames-<br>**YOUR_ACCOUNT_ID**,<br>S3_PROCESSED_BUCKET=**vision**-processed-<br>**YOUR_ACCOUNT_ID**,<br>SCHEDULE_EXECUTOR_ARN=arn:aws:lambda:**us-east-1**:**YOUR_ACCOUNT_ID**:function:pest-camera-scheduler |
| kvs-hls-handler     | 128 / 30      | kvs-hls-handler-role     | —     | none                                                                                                                                                                                                                                                                                                                                                                                               |
| pest-model-watchdog | 128 / 30      | pest-model-watchdog-role | —     | TABLE_CAMERAS,<br>MAX_RUNTIME_MIN=75                                                                                                                                                                                                                                                                                                                                                               |

Console: **Lambda** → **Create function** → **Author from scratch** per row, then **Code** → **Upload from** → **.zip file**, **Configuration** → **Environment variables / General configuration / Permissions**; attach the layer on the processor under **Code** → **Layers** → **Add a layer** → **Custom layers**.

CLI example (processor):

```
aws lambda create-function --function-name pest-detection-processor --runtime python3.12 --handler lambda_function.lambda_handler --role arn:aws:iam:**YO
```

```
UR_ACCOUNT_ID**:role/pest-detection-processor-role --memory-size 1024 --timeo
ut 600 --architectures x86_64 --layers arn:aws:lambda:**us-east-1**:**YOUR_AC
COUNT_ID**:layer:fyp-pillow:**N** --zip-file fileb://**work\pest-detection-pr
ocessor.zip** --environment "Variables={TABLE_CAMERAS=pest-monitoring-cameras,
TABLE_SYSTEM_CONFIG=pest-monitoring-system-config,TABLE_DETECTIONS=pest-monit
oring-detections,SENDER_EMAIL=**alerts@np.edu.sg**,S3_PROCESSED_BUCKET=**visi
on**-processed-**YOUR_ACCOUNT_ID**,TILE_MIN_CONFIDENCE=8,LLM_VERIFY_ALL_BOXES
=true,LLM_VERIFY_MODEL_ID=us.anthropic.claude-sonnet-4-6,LLM_VERIFY_MAX_BOXES
=120,LLM_VERIFY_MAX_TOKENS=300,LLM_VERIFY_WORKERS=3,LLM_VERIFY_PAD=0.6,POST_N
MS_I0U=0.1,POST_NMS_CONTAIN=0.1,POST_MAX_BOX_AREA=0.05,POST_VERIFY_FLOOR=49}"
--profile **prod** --region **us-east-1**
```

## Step 6 — S3 → processor event wiring

Permission FIRST or the notification call fails:

```
aws lambda add-permission --function-name pest-detection-processor --statemen
t-id s3-frames-invoke --action lambda:InvokeFunction --principal s3.amazonaws.
com --source-arn arn:aws:s3:::**vision**-frames-**YOUR_ACCOUNT_ID** --source-
account **YOUR_ACCOUNT_ID** --profile **prod** --region **us-east-1**
```

```
aws s3api put-bucket-notification-configuration --bucket **vision**-frames-**
YOUR_ACCOUNT_ID** --notification-configuration '{"LambdaFunctionConfiguratio
ns\":[{"Id\":"frames-to-processor\","LambdaFunctionArn\":"arn:aws:lambda:
**us-east-1**:**YOUR_ACCOUNT_ID**:function:pest-detection-processor","Event
s\":[{"s3:ObjectCreated:*"},""],"Filter\":{"Key\":{"FilterRules\":[{"Name\":"
Prefix"},"Value\":"frames/"}]}]}' --profile **prod**
```

Console: **S3 → the frames bucket → Properties → Event notifications → Create event notification** (All object create events, prefix frames/, destination the processor). The console adds the permission for you; emit only one.

## Step 7 — Cognito

```
aws cognito-idp create-user-pool --pool-name pest-dashboard-users --username-
attributes email --auto-verified-attributes email --admin-create-user-config
AllowAdminCreateUserOnly=true --policies "PasswordPolicy={MinimumLength=8,Req
uireUppercase=false,RequireLowercase=true,RequireNumbers=true,RequireSymbols=
false}" --profile **prod** --region **us-east-1**
```

```
aws cognito-idp create-user-pool-client --user-pool-id **NEW_POOL_ID** --clie
nt-name dashboard-web --no-generate-secret --explicit-auth-flows ALLOW_USER_P
ASSWORD_AUTH ALLOW_REFRESH_TOKEN_AUTH --id-token-validity 12 --access-token-v
alidity 12 --refresh-token-validity 30 --token-validity-units "IdToken=hours,
AccessToken=hours,RefreshToken=days" --profile **prod** --region **us-east-1**
*
```

Capture **NEW\_POOL\_ID** and **NEW\_CLIENT\_ID**. Console: **Amazon Cognito → User pools → Create user pool** (email sign-in, no self-registration) then **App clients → Create app client** (public client, no secret).

## Step 8 — API Gateway (HTTP API)

lambda/cors.json is the exact CORS shape:

```
aws apigatewayv2 create-api --name pest-monitoring-api-gateway --protocol-type HTTP --cors-configuration file://lambda/cors.json --profile **prod** --region **us-east-1**
```

```
aws apigatewayv2 create-authorizer --api-id **NEW_API_ID** --name cognito-dashboard --authorizer-type JWT --identity-source "$request.header.Authorization" --jwt-configuration Audience=**NEW_CLIENT_ID**,Issuer=https://cognito-idp.*us-east-1*.amazonaws.com/**NEW_POOL_ID** --profile **prod**
```

```
aws apigatewayv2 create-integration --api-id **NEW_API_ID** --integration-type AWS_PROXY --integration-uri arn:aws:lambda:**us-east-1**:**YOUR_ACCOUNT_ID*:function:pest-monitoring-api --integration-method POST --payload-format-version 2.0 --profile **prod**
```

```
aws apigatewayv2 create-integration --api-id **NEW_API_ID** --integration-type AWS_PROXY --integration-uri arn:aws:lambda:**us-east-1**:**YOUR_ACCOUNT_ID*:function:kvs-hls-handler --integration-method POST --payload-format-version 2.0 --profile **prod**
```

Then the 21 routes. All target the api integration except GET /video-playback (the hls integration); all carry --authorization-type JWT --authorizer-id \*\*AUTH\_ID\*\* except GET /stream/status, which must be --authorization-type NONE (device pollers hit it unauthenticated). Route list: GET /cost, GET /settings, POST /settings, GET /history, GET /presigned-url, POST /detection/verify, DELETE /detection, GET /model/status, POST /model/start, POST /model/stop, GET /identities, POST /identities, DELETE /identities, GET /schedule, POST /schedule, DELETE /schedule, GET /schedule-logs, POST /stream/start, POST /stream/stop, GET /stream/status, GET /video-playback.

```
aws apigatewayv2 create-route --api-id **NEW_API_ID** --route-key "GET /settings" --target integrations/**INT_API_ID** --authorization-type JWT --authorizer-id **AUTH_ID** --profile **prod**
```

```
aws apigatewayv2 create-route --api-id **NEW_API_ID** --route-key "GET /stream/status" --target integrations/**INT_API_ID** --authorization-type NONE --profile **prod**
```

```
aws apigatewayv2 create-route --api-id **NEW_API_ID** --route-key "GET /video-playback" --target integrations/**INT_HLS_ID** --authorization-type JWT --authorizer-id **AUTH_ID** --profile **prod**
```

Stage and invoke permissions:

```
aws apigatewayv2 create-stage --api-id **NEW_API_ID** --stage-name "$default" --auto-deploy --profile **prod**
```

```
aws lambda add-permission --function-name pest-monitoring-api --statement-id apigw-invoke --action lambda:InvokeFunction --principal apigateway.amazonaws.com --source-arn "arn:aws:execute-api:**us-east-1**:**YOUR_ACCOUNT_ID**:**NEW_API_ID**/*" --profile **prod**
```

```
aws lambda add-permission --function-name kvs-hls-handler --statement-id apigw-invoke-videoplayback --action lambda:InvokeFunction --principal apigateway.amazonaws.com --source-arn "arn:aws:execute-api:**us-east-1**:**YOUR_ACCOUNT_ID**:**NEW_API_ID**/**/video-playback" --profile **prod**
```

Base URL (no stage suffix): `https://**NEW_API_ID**.execute-api.**us-east-1**.amazonaws.com`. Console path for all of it: **API Gateway** → **Create API** → **HTTP API**, then Authorization, Integrations, Routes, and CORS tabs.

## Step 9 — CloudFront

Console: **CloudFront** → **Create distribution**. Origin domain = the dashboard bucket's **static website endpoint** `**vision**-dashboard-**YOUR_ACCOUNT_ID**.s3-website-**us-east-1**.amazonaws.com` — type it; do NOT accept the console's suggested REST endpoint. Protocol **HTTP only** (website endpoints have no HTTPS; https origin → 502). Viewer protocol redirect-to-HTTPS, allowed methods GET/HEAD, cache policy **CachingDisabled** (AWS-managed, same ID in every account: 4135ea2d-6df8-44a3-9df3-4b5a84be39ad), default root object `index.html`, no OAC/OAI, no aliases. No CloudFront custom error responses — the SPA 404 fallback is the S3 website ErrorDocument.

CLI: take `deployer/audit/cloudfront__E1YADURLSAVNFA_distribution-config.json`, rewrite the origin DomainName to your website endpoint, set a fresh unique CallerReference (CloudFront rejects duplicates), then:

```
aws cloudfront create-distribution --distribution-config file://**rewritten\cf-config.json** --profile **prod**
```

Capture the new distribution ID and `*.cloudfront.net` domain. The distribution takes minutes to reach Deployed.

## Step 10 — Config write-back + dashboard sync + frames CORS

Edit `web/dashboard_v4/js/config.js` (a copy — do not commit the new account's values over the reference ones): set `HTTP_API` to the Step 8 base URL, `COGNITO_REGION` to **us-east-1**, `COGNITO_CLIENT_ID` to **NEW\_CLIENT\_ID** (the pool ID is not referenced by the dashboard). Then sync:

```
aws s3 sync web/dashboard_v4/ s3://**vision**-dashboard-**YOUR_ACCOUNT_ID**/ --cache-control "no-cache, must-revalidate" --profile **prod**
```

Then the frames-bucket CORS, with the new origins:

```
aws s3api put-bucket-cors --bucket **vision**-frames-**YOUR_ACCOUNT_ID** --cors-configuration "{\"CORSRules\": [{\"AllowedHeaders\": [\"*\"], \"AllowedMethod s\": [\"GET\", \"PUT\", \"POST\", \"HEAD\"], \"AllowedOrigins\": [\"https://**NEW_C F_DOMAIN**.cloudfront.net\", \"http://**vision**-dashboard-**YOUR_ACCOUNT_ID**. s3-website-**us-east-1**.amazonaws.com\"], \"ExposeHeaders\": [\"ETag\"], \"MaxA geSeconds\": 3000}]}" --profile **prod**
```

## Step 11 — Kinesis Video Streams (only if live view is wanted)

```
aws kinesisanalytics create-stream --stream-name **vision**-cam-stream --data-retention-in-hours 2 --profile **prod** --region **us-east-1**
```

Console: **Kinesis Video Streams** → **Create video stream**. The HLS data endpoint is per-account/per-stream; kvs-hls-handler resolves it at runtime — never hardcode it anywhere.

## Step 12 — EventBridge Scheduler (the cost-guard watchdog)

```
aws scheduler create-schedule --name pest-model-watchdog-15min --schedule-expression "rate(15 minutes)" --flexible-time-window Mode=OFF --state ENABLED --target "{\"Arn\":\"arn:aws:lambda:**us-east-1**:**YOUR_ACCOUNT_ID**;function:pest-model-watchdog\",\"RoleArn\":\"arn:aws:iam:**YOUR_ACCOUNT_ID**;role/pest-scheduler-invocation-role\",\"RetryPolicy\":{\"MaximumEventAgeInSeconds\":86400,\"MaximumRetryAttempts\":0}}\" --profile **prod** --region **us-east-1**
```

Console: **Amazon EventBridge** → **Scheduler** → **Schedules** → **Create schedule** (rate 15 minutes, target the watchdog Lambda, execution role = the invocation role, retries 0). Note the mechanism: EventBridge Scheduler invokes Lambda by assuming the role — no lambda add-permission is involved (unlike classic EventBridge rules).

## Step 13 — Rekognition Custom Labels project (skeleton only)

```
aws rekognition create-project --project-name **vision**-detection --profile **prod** --region **us-east-1**
```

Console: **Amazon Rekognition** → **Custom Labels** → **Projects** → **Create project**. Capture the ProjectArn (the numeric suffix is AWS-assigned). No model exists yet; training is Section 8.5.3 / Chapter 3. Do not recreate the reference account's SmartPestProject (moth demo) or the dead v1.

## Step 14 — DynamoDB seeds

Write the JSON item files and use file:// inputs — raw inline JSON with colons breaks PowerShell quoting (the same trap as the model-ARN write in Chapter 3).

System config (1 row):

```
aws dynamodb put-item --table-name pest-monitoring-system-config --item file://**seed-config.json** --profile **prod** --region **us-east-1**
```

seed-config.json:

```
{\"config_key\": {\"S\": \"detection_settings\"}, \"email_enabled\": {\"BOOL\": true}, \"recipient_email\": {\"S\": \"**you@np.edu.sg**\"}, \"additional_recipients\": {\"L\": []}, \"auto_capture\": {\"BOOL\": true}, \"capture_interval\": {\"N\": \"60\"}}
```

Camera rows (2): manual\_upload (required by the dashboard Test-upload feature) and one real camera. Fields per row, matching the deploy.py seed: camera\_id, label,

default\_waypoint\_id, target\_label = **your training class**, model\_type = custom, min\_confidence (N) = 10 (the CANDIDATE floor feeding the LLM gate — setting it high silently strangles recall before the LLM ever sees a box; the user-facing threshold is post\_verify\_floor), llm\_verify\_enabled (BOOL) = true, post\_verify\_floor (N) = 49, stream\_enabled (BOOL), model\_running (BOOL) = false, custom\_model\_arn = "" (empty until training writes it), kvs\_stream\_name (\*\*vision\*\*-cam-stream on the real camera if Step 11 ran, else NULL), schedule (empty map on manual\_upload; {enabled:false, days:[], start\_time:"09:00", end\_time:"17:00"} on the camera), and on the real camera tiling\_enabled (BOOL) = true and max\_runtime\_min (N) = 45 (the watchdog's per-camera auto-stop bound).

## Step 15 — SES identities

```
aws ses verify-email-identity --email-address **alerts@np.edu.sg** --profile
**prod** --region **us-east-1**
```

Repeat for the recipient if different. Then Section 8.5.1 (confirm the link, sandbox note).  
Console: **Amazon SES → Identities → Create identity → Email address**.

**Do not create anything else.** CloudWatch log groups auto-create on first invoke. No SNS, SQS, EC2, classic EventBridge rules, or KMS CMKs are part of this stack.

## 8.7 Path B — Lambda code upload from the local mirror

Each function's zip must contain exactly one file named `lambda_function.py` at the archive root. The mirrors in `lambda/` are named `pest-*.py` — **rename inside the zip**, not on disk.

| Function                 | Source file                                                                                                                                                                                                                   |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pest-detection-processor | lambda/pest-detection-processor.py                                                                                                                                                                                            |
| pest-monitoring-api      | lambda/pest-monitoring-api.py                                                                                                                                                                                                 |
| pest-camera-scheduler    | lambda/pest-camera-scheduler.py                                                                                                                                                                                               |
| pest-model-watchdog      | lambda/pest-model-watchdog.py (the repo mirror — it honours per-camera max_runtime_min; do NOT use the older snapshot under <code>deployer/audit/pest-model-watchdog_src/</code> , which only knows the global 75-minute cap) |
| kvs-hls-handler          | deployer/audit/kvs-hls-handler_src/lambda_function.py (already correctly named)                                                                                                                                               |

PowerShell zip recipe (per core function):

```
Copy-Item lambda/pest-detection-processor.py **work**\lambda_function.py
Compress-Archive -Path **work**\lambda_function.py -DestinationPath **work**\
pest-detection-processor.zip -Force
```

Upload at creation (Step 5) or update later:

```
aws lambda update-function-code --function-name pest-detection-processor --zip-file fileb://**work\pest-detection-processor.zip** --profile **prod** --region **us-east-1**
```

Console: **Lambda → the function → Code → Upload from → .zip file**. Two cautions: the function name and runtime are fixed at creation (changing them means delete + recreate, which wipes the resource policy — re-grant the API Gateway and S3 invoke permissions); and the mirror files contain no secrets, but always deploy from the mirror, never from a console-edited copy, so the repo stays the source of truth.

## 8.8 Device provisioning (checklist level)

The cloud stack works with zero devices — skip this section for a dashboard-only reproduction. The Go2 is a demo/testbed only; the production vision is fixed cameras.

**Jetson Orin (Go2 patrol + SIYI A8 Mini capture) — Chapter 5 has every step:** - [ ] Ubuntu + ROS 2 Foxy installed; cyclonedds\_ws workspace built (the Unitree Go2 DDS interface). - [ ] setup\_go2.sh environment sourced (network interface + CYCLONEDDS\_URI / ROS env for the Go2 link). - [ ] KVS Producer SDK compiled for ARM64; GStreamer passthrough pipeline (rtspsrc → rtph264depay → h264parse → kvssink) for the A8 Mini feed — RTSP, or the 2026-07-29 mini-HDMI → USB capture-card path now in use (Chapter 5). - [ ] Patrol + capture scripts present: go2\_patrol\_gated.py (capture and upload are built into it — capture\_frame/upload\_frame) and capture\_4k\_hdmi.py (the 2026-07-29 capture-card path); kvs\_controller.py service polling GET /stream/status. - [ ] patrol-scheduler.service (systemd, Restart=always) running run\_patrol\_scheduler.sh / patrol\_scheduler.py, polling GET /schedule?camera=worm\_cam and launching go2\_patrol\_gated.py on the dashboard's schedule (Chapter 5, Section 5.13) — **not yet dry-run on real hardware**; confirm the systemd unit starts, and that a scheduled test run actually launches (armed via ~/go2/.patrol\_armed) before relying on it. - [ ] AWS credentials for the NEW account on the device, and every script's API URL / bucket / camera\_id repointed (Section 8.9.3).

**Mini PC Win11 host + Ubuntu VM (fixed Hikvision camera / moth stream) — Chapter 6 has every step:** - [ ] VM up (VMware, NAT — inbound unreachable by design). - [ ] kvs-controller.service (systemd, Restart=always) running run\_kvs\_controller.sh; CAMERA\_ID set correctly (the systemd unit env overrides home-dir edits — the Chapter 6 gotcha). Capture/upload script capture\_and\_upload\_v4\_armyworm.py present (repo mirror: minipc/). - [ ] reverse-tunnel-fyp.service up (VM → Orin:2222 reverse SSH; self-heals within ~45 s) if remote administration is needed. - [ ] Same repointing as the Orin: new API base URL for /stream/status, new stream name, new account credentials.



## 8.9 Account-portability caveats (critical — read before either path)

Every caveat in this section has now been confronted for real by the 2026-08-10/11 production migration; where a caveat has been resolved or confirmed by that migration, the subsection says so.

### 8.9.1 Rekognition Custom Labels models are account-bound — but their data is portable

There is **no export, import, copy, or cross-account API** for trained Custom Labels models. The armyworm models (current generation: the v9 retrain with added data augmentation) and the predecessor's moth model (SmartPestProject, F1 0.988) exist only in account 366356442579 and cannot follow you. On the new account you **MUST** retrain: the ARGUS Train screen (Section 8.5.3) with a labeled image folder, the Chapter 3 pipeline, or — when the data already lives in the old account's S3 — the executed migration route in Section 8.5.3 (server-side copy + ListDatasetEntries export). Training runs from ~1 h (a ~30-image proof run) to many hours at full scale (the ~33k-image armyworm run). After training, the new ProjectVersionArn must land in the camera row's custom\_model\_arn — the Train screen does this automatically; manually it is one boto3/file://-JSON field write (the ARN contains colons; raw PowerShell update-item mangles the quoting).

This caveat is no longer theory: both capabilities were rebuilt on 506868652945 from data alone — armyworm v9r-prod-20260810 (project argus-detection) and moth moth-prod-20260811 (project argus-moth-detection, F1 0.991 vs the original 0.988 on the same data). The model is account-bound; the capability is not, as long as you keep the labelled data.

### 8.9.2 Every hardcoded identifier changes

The account ID is embedded in bucket names, every ARN, and the IAM trust conditions; the API Gateway ID, Cognito pool/client IDs, and CloudFront domain are server-assigned and new. Path A rewrites all of them mechanically and records them in deploy\_state.json — the production migration confirmed this end to end (the templated js/config.js served through the new CloudFront carried the correct new API and client IDs on first check). Path B: the complete inventory of reference-account identifiers — which files hold them and their PROD/DEMO/DEAD classification — is the Chapter 9 registry. Work through it once after deployment and confirm nothing still points at 366356442579. For a migration between two live accounts, migration/diff\_accounts.py automates the old-vs-new comparison (Lambda config, camera rows, SES identities, streams) and is how the remaining gaps in Section 8.9.3 were enumerated.

### 8.9.3 Device-side config references the old account

The edge devices carry the old API and bucket coordinates. At minimum, on each device: the API base URL (currently https://vzf17s6z00.execute-api.us-east-1.amazonaws.com → your https://\*\*NEW\_API\_ID\*\*...) used by kvs\_controller.py for GET /stream/status; the frames bucket name in the capture/upload scripts; the KVS stream



name; the region; and the device's AWS credentials, which must belong to the new account. Camera IDs must match the rows you seeded. Chapters 5 and 6 list the exact files per device.

**Migration status:** this is the remaining open step of the production migration. For that account the concrete targets are: API base `https://vzf17s6z00.execute-api.us-east-1.amazonaws.com`, frames bucket `argus-frames-506868652945`, camera ids unchanged (`worm_cam` / `moth_cam` — the rows were aligned to the reference ids, Section 8.5.4), and new-account credentials on each device. KVS was not deployed there, so any `kvs_controller` repointing waits until a stream exists.

#### 8.9.4 The LLM verification gate needs one Bedrock step the deployer cannot do

The processor mirror in `lambda/` runs the Bedrock LLM verification gate as part of the standard pipeline (v6.3: tiling → suppression → LLM verify → post-gate cleanup), and since the 2026-08-10/11 deployer fixes a fresh stack comes up with the gate fully wired: the camera rows seed `llm_verify_enabled=true`, the env carries `LLM_VERIFY_MODEL_ID=us.anthropic.claude-sonnet-4-6` (note the `us.-`prefixed inference-profile ID, not the bare foundation-model ID), the processor deploys at 1024 MB / 600 s, and the `bedrock-verify` inline policy (Sonnet 4.6 + Haiku 4.5 profile and foundation-model ARNs) is attached to `pest-detection-processor-role` automatically. Without that policy the fail-closed gate would reject every detection — this was a real deployer defect, found and fixed 2026-08-11.

What no deployer can do: **Bedrock model access is a per-account console grant.** A fresh account gets `AccessDeniedException` on Anthropic models until the use-case form is accepted (**Bedrock → Model access → Modify model access**; approval can lag, so submit it first thing). On the NP production account this cost nothing — Sonnet 4.6 and Haiku 4.5 were already authorized when the account was audited on 2026-08-10 — but never assume that; check before deploying. Details in Chapter 2's processor section and `docs/aws.md`.

#### 8.9.5 SES sandbox and other fresh-account frictions

SES starts sandboxed (Section 8.5.1). Cognito starts with zero users (Section 8.5.2). CloudFront takes minutes to deploy. New accounts occasionally carry low default service quotas; none have bitten this stack in practice (neither the `CAG_Test` run nor the production migration needed a quota increase). An institutionally issued account may also not be empty — the NP production account carried a previous student's Amplify relics (13 roles, one bucket); they are harmless and were left untouched. Do not clean up resources you did not create.

### 8.9.6 One known credential in the repo

datasets/archive/experiments/pre\_v3\_abandoned/download.py contains an inline Roboflow API key from an abandoned experiment — credential stored there, not reproduced here. It has nothing to do with reproduction; just do not copy that file into anything you hand over or publish.

### 8.9.7 Cross-account S3 grants must be revoked after sign-off

The server-side data copies in Section 8.5.3 work by adding temporary, prefix-scoped read statements to the OLD account's bucket policies. They are deliberately left in place until the new models are signed off, and each script removes exactly its own statements with `--revoke`. As of 2026-08-11 three grants are outstanding on the reference account: the `training-data/v9/*` grant on the frames bucket (python `migration/copy_training_data.py --revoke`), a `frames/*` grant on the same bucket added for the on-site records migration (Sid MigrationReadJewelFrames — remove the statement from the bucket policy directly), and the moth console-bucket grant (python `migration/migrate_moth.py --revoke`). Whoever finishes the migration must run the revokes — a standing cross-account read grant is not something to hand over.

## 8.10 Smoke-test suite (run after either path)

Run these in order on the new account. They are the day-1 customer test from REHEARSAL.md Round 2, expanded.

### 8.10.1 Detection rail: upload → DynamoDB record → dashboard card

Sign in to the dashboard (CloudFront URL) → **Settings** → **Test upload** → upload any image. Within ~10–30 s a record must appear in the Gallery. With no model trained yet this is a **clean record** (zero detections) — that is the expected pass; clean images write a DynamoDB record too, by design. CLI equivalent:

```
aws s3 cp **test.jpg** s3://**vision**-frames-**YOUR_ACCOUNT_ID**/frames/manual_upload/**smoke-1**.jpg --profile **prod**
aws dynamodb query --table-name pest-monitoring-detections --key-condition-expression "image_id = :k" --expression-attribute-values "{\":k\":{\"S\":\\\"frames/manual_upload/**smoke-1**.jpg\\\"}}}" --profile **prod** --region **us-east-1**
```

If no record appears: check `/aws/lambda/pest-detection-processor` in CloudWatch Logs (console: **CloudWatch** → **Log groups**); if the log group itself does not exist, the S3 notification (Step 6) never fired.

After training a model and starting it, repeat with a known-positive image and expect a card with boxes.

### 8.10.2 KVS live check (only if live view was deployed)

With a producer streaming (device or the GStreamer test pipeline):

- Console: **Kinesis Video Streams** → vision-**cam-stream** → **Media playback** shows live video.
- CLI:

```
aws kinesisanalytics get-data-endpoint --stream-name **vision**-cam-stream --api-name GET_HLS_STREAMING_SESSION_URL --profile **prod** --region **us-east-1**
```

Then the dashboard Live tab must play (it calls GET /video-playback → kvs-hls-handler). With no producer, the correct behavior is a clean “stream not active” state, not an error page.

### 8.10.3 Scheduler fire (cost guard)

The watchdog must run every 15 minutes from minute zero:

```
aws scheduler get-schedule --name pest-model-watchdog-15min --profile **prod** * --region **us-east-1**
```

Expect "State": "ENABLED". Then confirm real invocations: console **CloudWatch** → **Log groups** → /aws/lambda/pest-model-watchdog — log streams appearing on a 15-minute cadence. This is the guard that stops a forgotten Rekognition endpoint (per-camera max\_runtime\_min — 45 min on the seeded real camera; the env MAX\_RUNTIME\_MIN=75 is the fallback for rows without one), so prove it before ever starting a model.

### 8.10.4 Alert email

Prerequisites: SES identity verified, a trained model RUNNING, email\_enabled=true in system config (the seed default). Upload a known-positive image via Test upload; a detection at/above the camera's min\_confidence sends an SES alert to recipient\_email. In sandbox the recipient must itself be verified — if the mail does not arrive, check **SES** → **Account dashboard** send statistics and the processor's CloudWatch log before suspecting the pipeline.

### 8.10.5 Full-stack audit

ARGUS done screen → **Run a system check**, or headless:

```
python deployer/deploy.py --profile **prod** --region **us-east-1** --prefix **vision** --target-label **x** --sender-email **x@x** --verify
```

Every line must read OK; the summary must read all resources present. (--verify is read-only; the dummy label/email arguments are required by the parser but unused.)

## 8.11 Operations — cost profile, idle check, teardown

### 8.11.1 What the stack costs

| Item                                                | Billed when        | Approximate rate                                                                                                                                                                 | Notes                                                                                                                                                 |
|-----------------------------------------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rekognition Custom Labels <b>inference endpoint</b> | Only while RUNNING | ~US\$4/hour per model                                                                                                                                                            | The dominant cost. The watchdog auto-stops running endpoints after the camera row's <code>max_runtime_min</code> (45 min as seeded; env fallback 75). |
| Rekognition Custom Labels <b>training</b>           | Per training run   | billed per hour; a v5-scale run $\approx$ 3.5 h; a ~30-image proof run $\approx$ 1 h $\approx$ US\$4                                                                             | One-off per model version.                                                                                                                            |
| One-model duty cycle                                | Daily              | $\approx$ US\$2/day (30 min warm/day) $\rightarrow \approx$ <b>US\$60/month</b>                                                                                                  | The project's headline figure; roughly 2× if both models run.                                                                                         |
| S3 (3 buckets)                                      | Storage            | cents/month at a few GB                                                                                                                                                          | Free tier covers the first 12 months at this scale.                                                                                                   |
| DynamoDB (4 tables, on-demand)                      | Per request        | negligible                                                                                                                                                                       | Detection volume is hundreds of rows/week.                                                                                                            |
| Lambda (5 functions)                                | Per invoke         | free tier covers it                                                                                                                                                              | Watchdog = 2,880 invokes/month; the rest are event-driven.                                                                                            |
| API Gateway HTTP API                                | Per request        | ~US1/<br><i>millionrequests Dash request negligible/ freetier Staticdashb day Bedrock(theLLi onbydefaultsince202 08 — 10) Per call afewUS /month at FYP volume on Sonnet 4.6</i> | Section 8.9.4.                                                                                                                                        |
| EventBridge                                         | Per invocation     | negligible                                                                                                                                                                       |                                                                                                                                                       |

| Item      | Billed when | Approximate rate | Notes |
|-----------|-------------|------------------|-------|
| Scheduler |             |                  |       |

### 8.11.2 Idle-cost check

A correctly deployed stack idles at effectively **\$0/day**: nothing bills by the hour except a RUNNING Rekognition endpoint, and the watchdog kills those. The acceptance test (REHEARSAL.md Round 2, step 7): leave the fresh account untouched for 24 h and read the bill — console **Billing and Cost Management → Bills**; expect ~\$0. If it is not ~\$0, the usual culprit on the reference account’s history was exactly a running model endpoint or stray legacy schedules — check **Rekognition → Projects → your project → Use model** for anything RUNNING, and `aws scheduler list-schedules --profile **prod** --region **us-east-1**` for anything unexpected.

### 8.11.3 Teardown

Full removal is one guarded action in ARGUS (**Danger zone → Delete deployment**): reverse-dependency deletion of everything (schedule, Rekognition versions + project, KVS, CloudFront disable→wait→delete, API, Cognito pool, 5 Lambdas, layer versions, 3 emptied buckets, 4 tables, 6 roles, SES identities), followed by the same `verify()` audit in reverse expectation — success means “0/N resources remain”, independently confirmed, not assumed. Headless: `deploy.destroy(ctx)` in `deploy.py`; it refuses to run if the stored state belongs to a different account than the credentials. Closing the AWS account itself is always a manual root console action (Account → Close account; 90-day reopen window). Full teardown detail is Chapter 7.

## 8.12 Cross-references

- Chapter 1 — architecture overview and the two-camera model; Section 1.10.7 is the short form of this chapter.
- Chapter 2 — what each cloud resource does once it exists; the processor version history and the Bedrock gate this chapter’s caveat 8.9.4 refers to.
- Chapter 3 — the training pipeline (the manual equivalent of the Train screen), the model history, and the ARN write-back procedure.
- Chapter 4 — dashboard modules, Cognito user management in depth, the redeploy command with the cache-control flag.
- Chapter 5 — Jetson Orin / Go2 provisioning in full (the checklist in 8.8 expanded).
- Chapter 6 — mini PC VM provisioning in full, including the reverse tunnel and the systemd CAMERA\_ID gotcha.
- Chapter 7 — the ARGUS deployer internals: `deploy.py`, `app.py`, `training.py`, the stack manifest, rehearsal protocol, and teardown.
- Chapter 9 — the registry of every ID, ARN, IP, and endpoint that must change on a new account, plus the PROD/DEMO/DEAD classification.

## Chapter 9 — Appendix: registries and reference tables

Single-source reference tables for the whole ARGUS Smart Pest Monitoring System: every AWS resource id and ARN, every device, every environment variable, the repository map, the camera registry, and the project glossary. *As of 2026-08-11.* (Entries verified against a live account carry their own inline date. The code in `lambda/` is ground truth; where a doc lagged the code, the row says so. All repository paths in this chapter are relative to the project repository root.)

The system runs on the NP production account 506868652945, stood up 2026-08-10/11 by the ARGUS deployer and validated end to end on 2026-08-12. Every row below describes that account unless it is explicitly marked as retired development-era history. The development account 366356442579, where the system was built, is retired from operation; its identifiers are kept only where a historical record needs them (the model version table in 9.3.9 is the main case, because those versions were trained there and cannot be moved between accounts).

### 9.1 Role in the system

This chapter is the lookup layer of the manual. Chapters 1 to 8 explain how each part of the system works. This chapter answers a different question: “what is the exact name, id, or ARN of that thing?” Every identifier used anywhere in the manual appears here once, in a table, with its location and status.

Two audiences use it. An operator running the live stack uses it to find the resource they need to inspect (which Lambda, which table, which schedule). An engineer reproducing the stack on a fresh AWS account (Chapter 8) uses it as the checklist of what must exist when the rebuild is done, and as the map of which values are account-bound and must be replaced.

Nothing in this chapter contains a secret. AWS account ids, ARNs, bucket names, API ids, Cognito ids, IPs, and SSH usernames are non-secret identifiers and are listed in full. Passwords, access keys, and API keys are never reproduced; where one exists inside a source file, the table says “credential stored here, not reproduced”.

### 9.2 Inventory

| Registry              | Section | Covers                                                                                                                                                        |
|-----------------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AWS resource registry | 9.3     | Account, S3, DynamoDB, Lambda, API Gateway, Cognito, CloudFront, Rekognition, Bedrock, KVS, EventBridge, SES, IAM — development account (9.3.1-9.3.13) and NP |

| Registry                 | Section | Covers                                                                                                 |
|--------------------------|---------|--------------------------------------------------------------------------------------------------------|
| Device registry          | 9.4     | production account (9.3.14)<br>Go2, Jetson Orin, SIYI A8<br>Mini, mini PC, Hikvision<br>camera, laptop |
| Environment variables    | 9.5     | Per-Lambda env vars and<br>per-edge-service config,<br>with meanings                                   |
| Repository file map      | 9.6     | Top-level directories of the<br>project repository                                                     |
| Camera registry          | 9.7     | The pest-monitoring-<br>cameras rows on both<br>accounts and their settings<br>fields                  |
| Glossary                 | 9.8     | Project terms: batch_2,<br>gate, gen-1, nbk2, and the<br>rest                                          |
| Conventions + change log | 9.9     | How this manual writes<br>commands; document<br>history stub                                           |
| Operations               | 9.10    | Console + CLI inspection<br>procedure for every registry                                               |
| Cross-references         | 9.11    | Pointers to Chapters 1-8                                                                               |

## 9.3 AWS resource registry

### 9.3.1 Account and region

| Item                          | Value                                                                                                                                                                 |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Production account id</b>  | <b>506868652945</b> — the live system; everything<br>in this chapter refers to it                                                                                     |
| <b>CLI profile / IAM user</b> | <b>prod</b> / Student_QianRunzhe (group<br>CAG_Proj = AdministratorAccess, no<br>permission boundary, no region lock)                                                 |
| Region                        | us-east-1                                                                                                                                                             |
| Retired development account   | 366356442579 (profile nbk2, console user<br>cag_user) — where the system was built;<br>retired from operation 2026-08-10, kept<br>read-only while the report is filed |
| Older account                 | 396278862184 — dormant, to be closed                                                                                                                                  |

The production account is not empty: a previous student’s Amplify “moobusapps” relics (13 roles, 1 deployment bucket) exist there. They are harmless; do not touch them. The retired development account was shared with unrelated NP projects (amplify chatbot, projassist, sagemaker, lights-out, spf\* chatbot), and CloudShell writes were blocked there by a VPC endpoint — history only, no longer operative.

### 9.3.2 S3 buckets

| Bucket                                                                                                                        | Status                  | Purpose                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| argus-frames-506868652945                                                                                                     | LIVE                    | Raw frames under frames/<camera_id>/...; upload here triggers the processor. Also holds training data under training-data/, training output under training-output/, and the handover snapshot under handover/ |
| argus-processed-506868652945                                                                                                  | LIVE (empty since v4.2) | Legacy annotated-image bucket. The processor stopped writing it at v4.2; kept because Lambda cold-start requires the env var and the delete path references it                                                |
| argus-dashboard-506868652945                                                                                                  | LIVE                    | Static website hosting for the dashboard (web/dashboard_v4/), public read, fronted by CloudFront E1YADURLSAVNFA                                                                                               |
| frames-armyworm-366356442579, processed-images-armyworm-366356442579, pest-dashboard-366356442579, lambda-layers-366356442579 | RETIRED (dev account)   | The development-era equivalents. Training data was copied out of the first of these to production on 2026-08-10; nothing writes to them now                                                                   |
| streaming-buckets                                                                                                             | DEAD                    | Gen-1                                                                                                                                                                                                         |
| custom-labels-console-us-east-1-d1abc2aed2                                                                                    | Auto-created            | Made by the Rekognition Custom Labels console; do not hand-manage                                                                                                                                             |
| processed-images-moth                                                                                                         | Legacy (moth demo)      | No account suffix — global-name collision risk if ever                                                                                                                                                        |



| Bucket | Status | Purpose   |
|--------|--------|-----------|
|        |        | recreated |

S3 event notification: argus-frames-506868652945, event s3:ObjectCreated:\*, prefix frames/, target pest-detection-processor. The prefix filter is load-bearing: without it every training-data upload would start a detection run.

### 9.3.3 DynamoDB tables

All pest tables: PAY\_PER\_REQUEST, TTL off, PITR off, no streams, tags Project=armyworm-v2, Owner=Runzhe-armyworm, ManagedBy=Runzhe.

| Table                         | Key schema                                                                                                                            | Status                                                                                                                                    |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| pest-monitoring-cameras       | camera_id (S) HASH                                                                                                                    | PROD — 3 rows (see 9.7)                                                                                                                   |
| pest-monitoring-detections    | image_id (S) HASH +<br>detection_time (S) RANGE;<br>GSI by-pest-time<br>(pest_type HASH +<br>detection_time RANGE,<br>projection ALL) | PROD. image_id = the exact<br>S3 object key. Row stores<br>bboxes (structured),<br>verifications map,<br>llm_verify_model,<br>llm_scan    |
| pest-monitoring-system-config | config_key (S) HASH                                                                                                                   | PROD — 1 row<br>detection_settings<br>(email_enabled,<br>recipient_email,<br>additional_recipients,<br>auto_capture,<br>capture_interval) |
| pest-monitoring-schedule-logs | log_id (S) HASH +<br>timestamp (S) RANGE                                                                                              | PROD — runtime logs                                                                                                                       |
| pest-monitoring-config        | config_id                                                                                                                             | DEAD (Gen-1, only<br>referenced by the archived<br>Flutter app)                                                                           |
| websocket-connections         | —                                                                                                                                     | DEAD (retired WebSocket<br>store)                                                                                                         |

### 9.3.4 Lambda functions and layer

All PROD functions: runtime **Python 3.12**, handler lambda\_function.lambda\_handler, x86\_64. Runtime and name are fixed at creation — changing either means delete + recreate, which wipes the resource policy (API Gateway invoke must be re-granted).

| Function        | Mem / timeout                              | Role            | Trigger             | Source mirror          |
|-----------------|--------------------------------------------|-----------------|---------------------|------------------------|
| pest-detection- | 1024 MB / <b>600 s</b><br>(live read 2026- | pest-detection- | S3<br>ObjectCreated | lambda/pest-detection- |

| Function                  | Mem / timeout                                                        | Role                                                                                                                                                           | Trigger                                                                                                                                               | Source mirror                                                                                                                                                                                                      |
|---------------------------|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| processor                 | 08-07/10;<br>earlier docs say<br>512/60 or<br>1024/180 — doc<br>lag) | processor-role                                                                                                                                                 | on frames/                                                                                                                                            | processor.py<br>(v6.3 — dead<br>code stripped<br>2026-08-07,<br>3266 -> 1545<br>lines; pre-strip<br>source in<br>lambda/archive<br>)                                                                               |
| pest-<br>monitoring-api   | 256 MB / 30 s                                                        | pest-<br>monitoring-<br>api-role                                                                                                                               | API Gateway<br>vzfl7s6z00 (21<br>routes)                                                                                                              | lambda/pest-<br>monitoring-<br>api.py                                                                                                                                                                              |
| pest-camera-<br>scheduler | 128 MB / 60 s                                                        | pest-camera-<br>scheduler-role                                                                                                                                 | Invoked by the<br>api<br>(SCHEDULE_EX<br>ECUTOR_ARN)<br>and by runtime<br>pest-sched-*<br>rules                                                       | lambda/pest-<br>camera-<br>scheduler.py                                                                                                                                                                            |
| kvs-hls-<br>handler       | 128 MB / 30 s                                                        | kvs-hls-<br>handler-role                                                                                                                                       | API Gateway<br>route GET<br>/video-<br>playback only                                                                                                  | deployer/audit<br>/kvs-hls-<br>handler_src/la<br>mbda_function.<br>py                                                                                                                                              |
| pest-model-<br>watchdog   | 128 MB / 30 s                                                        | pest-model-<br>watchdog-role-<br>78asdw2b<br>(service-role<br>path; per the<br>2026-08-10<br>frozen baseline<br>migration/prod<br>_baseline_2026<br>0810.json) | EventBridge<br>schedule pest-<br>model-<br>watchdog-15min<br>(re-enabled<br>2026-08-05 with<br>the v6.2 per-<br>camera auto-<br>close, see<br>9.3.10) | lambda/pest-<br>model-<br>watchdog.py<br>(v6.2 — honours<br>per-camera<br>max_runtime_mi<br>n; the older<br>deployer/audit<br>/pest-model-<br>watchdog_src/<br>snapshot is the<br>pre-v6.2 global-<br>cap version) |

Layer: fyp-pillow:2 (Pillow 12.2.0, cp312, manylinux2014\_x86\_64, ~7.9 MB), attached to the processor only. Version :1 is DEAD (missing the compiled \_imaging\*.so).

DEAD Lambdas (Gen-1 / Wilbur — do not recreate): image-upload-handler, pest-detection-http, FrameExtractionControl, websocket-handler, kvs-hls-url-generator,

ImageProcessing, pest-model-control, Scheduling, Extraction, ses-identity-manager.  
Not pest: lightsOutFunc.

### 9.3.5 API Gateway

| Item                  | Value                                                                                                                                   |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| HTTP API v2 name / id | pest-monitoring-api-gateway /<br><b>vzf17s6z00</b> (retired dev: zwpcbivmsj)                                                            |
| Base URL              | https://vzf17s6z00.execute-api.us-east-1.amazonaws.com (single \$default auto-deploy stage, no stage suffix)                            |
| JWT authorizer        | cognito-dashboard, id enxa26, IdentitySource<br>\$request.header.Authorization, Issuer = the Cognito pool, Audience = the app client id |
| Integrations          | t7ggvzn → pest-monitoring-api; 33f69kt → kvs-hls-handler (both AWS_PROXY, payload 2.0, 30 s)                                            |
| CORS                  | AllowOrigins *, AllowMethods GET, POST, DELETE, OPTIONS, AllowHeaders content-type, authorization, MaxAge 3600                          |
| DEAD APIs             | REST 3go4jj1698 (PestDetectionAPI, already deleted); WebSocket j4v2m5cbte (dropped for polling in v3.7)                                 |

Full route table — 21 routes. All require the JWT authorizer EXCEPT GET /stream/status (open — the Orin and mini PC poll it unauthenticated). All target pest-monitoring-api except GET /video-playback:

| Route              | Handler function (in pest-monitoring-api.py unless noted) |
|--------------------|-----------------------------------------------------------|
| GET /settings      | read global config + camera list                          |
| POST /settings     | write global config / camera fields (allowed, see 9.5)    |
| POST /model/start  | StartProjectVersion for a camera's model                  |
| POST /model/stop   | StopProjectVersion                                        |
| GET /model/status  | DescribeProjectVersions state                             |
| GET /presigned-url | presigned S3 PUT for the dashboard Test upload            |
| DELETE /detection  | permanent delete of all rows for an                       |

| Route                                                 | Handler function (in <code>pest-monitoring-api.py</code> unless noted)        |
|-------------------------------------------------------|-------------------------------------------------------------------------------|
|                                                       | image_id + its S3 frames (S3 deletes gated to frames/ prefix in code AND IAM) |
| POST /detection/verify                                | write a manual verification verdict on a record                               |
| GET /history                                          | detection records for the gallery                                             |
| GET /cost                                             | billing summary                                                               |
| GET /identities, POST /identities, DELETE /identities | SES recipient management                                                      |
| GET /schedule, POST /schedule, DELETE /schedule       | per-camera capture schedules (creates runtime pest-sched-* EventBridge rules) |
| GET /schedule-logs                                    | schedule run history                                                          |
| POST /stream/start, POST /stream/stop                 | flip stream_enabled on a camera row                                           |
| GET /stream/status                                    | <b>no auth</b> — polled by edge devices every ~5 s                            |
| GET /video-playback                                   | → kvs-hls-handler: resolves the KVS HLS streaming URL                         |

### 9.3.6 Cognito and CloudFront

| Item                    | Value                                                                                                                                                                                                                                |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cognito User Pool       | pest-dashboard-users, id <b>us-east-1_9se1FDHpc</b> , app client 6vebotf45bp8u46cnraddiaplv (dashboard-web), email sign-in, admin-create-only (no self sign-up), MFA off (retired dev pool: us-east-1_ea0aJdus1)                     |
| App client              | dashboard-web, id <b>4husu6afr835e235eu9dqp8av6</b> , no secret, USER_PASSWORD_AUTH + refresh, 12 h id/access tokens, 30-day refresh                                                                                                 |
| CloudFront distribution | <b>E1YADURLSAVNFA</b> → <a href="https://d1dtoxef7qmug1.cloudfront.net">https://d1dtoxef7qmug1.cloudfront.net</a> (retired dev: E1423RGLAXWNSI → <a href="https://d1twcdquexdgj8.cloudfront.net">d1twcdquexdgj8.cloudfront.net</a> ) |
| CloudFront config       | Origin = S3 website endpoint of argus-dashboard-506868652945 (http-only), cache policy Managed-CachingDisabled (4135ea2d-6df8-44a3-9df3-4b5a84be39ad, same id in every account), redirect-to-https, default CloudFront certificate   |

The dashboard JS references only the region and the client id (direct USER\_PASSWORD\_AUTH); the pool id is used server-side by the API Gateway authorizer.

### 9.3.7 Rekognition Custom Labels — projects and versions

Hard constraint: trained Custom Labels models are **account-bound**. There is no export or cross-account copy. On a new account, retrain (Chapter 3, Chapter 8) — this is exactly what the production-account migration does (9.3.14). A model must be RUNNING to serve detections and bills per running hour — stop it after testing. The live model ARN lives in exactly ONE runtime place: the custom\_model\_arn field of the camera row in pest-monitoring-cameras; all Lambdas resolve it from there, nothing hardcodes an ARN.

Development-account versions (production-account versions are in 9.3.14):

| Version                                                                                                                            | F1    | Status                                                            | ARN                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------|-------|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>armyworm v9r-prod — LIVE on worm_cam, trained on the production account 2026-08-10</b>                                          | 0.613 | STOPPED between windows (watchdog auto-close, max_runtime_min 45) | arn:aws:rekognition:us-east-1:506868652945:project/argus-detection/version/v9r-prod-20260810/1786377372187 |
| <b>moth-prod — LIVE on moth_cam, trained on the production account 2026-08-11</b>                                                  | 0.991 | STOPPED between windows                                           | project argus-moth-detection, version moth-prod-20260811 (account 506868652945)                            |
| <i>Rows below are development-account history.</i>                                                                                 |       |                                                                   |                                                                                                            |
| Custom Labels versions are account-bound and cannot be moved, so the production models above were retrained from the same recipes. |       |                                                                   |                                                                                                            |
| armyworm v9 retrain “v9r” (dev, live on worm_cam 2026-08-07 to the migration)                                                      | 0.599 | STOPPED                                                           | arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection-v9/version/v9-20260805-              |

| Version                                               | F1    | Status  | ARN                                                                                                                                   |
|-------------------------------------------------------|-------|---------|---------------------------------------------------------------------------------------------------------------------------------------|
| armyworm v9 first cut (live 2026-07-29 to 2026-08-07) | 0.591 | STOPPED | 0713/1785913987295<br>arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection-v9/version/v9-20260725-1746/1785001598671 |
| armyworm v8                                           | 0.739 | STOPPED | arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection-v8/version/v8-20260723-1703/1784826241523                       |
| armyworm v7.5                                         | 0.737 | STOPPED | arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection-v7-5/version/v7-5-20260723-0317/1784776673711                   |
| armyworm v7.4                                         | 0.720 | STOPPED | arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection-v7-4/version/v7-4-20260723-0259/1784775551708                   |
| armyworm v7.3                                         | 0.822 | STOPPED | arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection-v7-3/version/v7-3-20260722-0610/1784700642273                   |
| armyworm v7.2                                         | 0.794 | STOPPED | arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection-v7-2/version/v7-2-                                              |

| Version                                | F1    | Status                                                                                      | ARN                                                                                                                                              |
|----------------------------------------|-------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| armyworm v7.1                          | 0.744 | STOPPED                                                                                     | 20260721-0301/1784602920896<br>arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection-v7/version/v7-1-20260720-0604/1784527489460 |
| armyworm v6<br>(REJECTED 2026-07-15)   | 0.839 | STOPPED                                                                                     | arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection/version/v6-2026-07-14/1784011732429                                        |
| armyworm v5 (live until the v9 switch) | 0.852 | STOPPED                                                                                     | arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection/version/v5-2026-07-07/1783394123547                                        |
| armyworm v4 (oldest kept rollback)     | 0.719 | STOPPED                                                                                     | arn:aws:rekognition:us-east-1:366356442579:project/armyworm-detection/version/armyworm-detection.2026-05-21T12.46.19/1779338780450               |
| moth (Wilbur's, label Moths)           | 0.988 | STOPPED (verified 2026-08-07 audit — moth_cam cannot detect until this endpoint is started) | arn:aws:rekognition:us-east-1:366356442579:project/SmartPestProject/version/SmartPestProject.2026-02-15T01.13.58/1771089238911                   |

Reading note: the v9-family low F1 is by design — the detector runs as a high-recall front end that gathers candidate boxes down to very low confidence, and the LLM gate does the rejecting. Do not read 0.599 as pipeline accuracy. Endpoint policy since v6.2: nothing stays RUNNING permanently — the watchdog stops an endpoint max\_runtime\_min minutes (45

for worm\_cam) after it comes up, and any concurrently RUNNING endpoints bill concurrently.

Rollback = write the wanted ARN into worm\_cam.custom\_model\_arn. Gotcha: the ARN contains colons — a raw aws dynamodb update-item from PowerShell mangles the quoting; use boto3 or a JSON file.

### 9.3.8 Bedrock (LLM verification)

| Item                                          | Value                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Live verifier model id                        | us.anthropic.claude-sonnet-4-6 (Claude Sonnet 4.6; live since the v9-era denoiser config — earlier docs say Haiku 4.5, doc lag)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Code default (if env unset)                   | us.anthropic.claude-haiku-4-5-20251001-v1:0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Routable aliases (v4.9, S3 key __llm-<alias>) | sonnet46, haiku45, novapro (us.amazon.nova-pro-v1:0), llama4 (us.meta.llama4-maverick-17b-instruct-v1:0), pixtral (us.mistral.pixtral-large-2502-v1:0)                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| API                                           | bedrock-runtime Converse, raw JPEG bytes (no pre-base64)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| IAM                                           | Inline policy bedrock-verify on pest-detection-processor-role: one statement on the inference-profile ARN, one on the region-wildcarded foundation-model ARNs.<br><b>Doc lag CLOSED 2026-08-11:</b> the live policy (covering the Sonnet 4.6 verifier) was dumped from the account into deployer/audit/iam__pest-detection-processor-role__inline__bedrock-verify.json (alongside ...__inline__s3-frames-write.json) and wired into the deployer's ROLE_POLICIES, so a fresh deployment now grants the verifier correctly. The older lambda/bedrock-policy.json only grants Haiku 4.5 + Nova Lite — do not rebuild from it |

The #1 gotcha: the us. prefix is REQUIRED — it is a cross-region inference-profile id, not a foundation-model id. The bare id fails with “on-demand throughput isn’t supported”. Claude Sonnet 5 / Opus 5 remain AccessDenied on this account (AWS-side entitlement, not IAM) and are off the critical path.



### 9.3.9 Kinesis Video Streams

| Stream              | Status | Purpose                                                                                                |
|---------------------|--------|--------------------------------------------------------------------------------------------------------|
| armyworm-cam-stream | PROD   | Live view of the A8 feed<br>(GStreamer rtspsrc →<br>rtph264depay → h264parse<br>→ kvssink on the Orin) |
| moth-cam-stream     | Parked | Moth Hikvision live view via<br>the mini PC VM                                                         |
| FYP-PROJECT         | DEAD   | Gen-1, 168 h retention                                                                                 |

Retention 2 h on the PROD streams, AWS-managed KMS. The HLS DataEndpoint is per-account/per-stream — kvs-hls-handler resolves it at runtime via get-data-endpoint; never hardcode it.

### 9.3.10 EventBridge schedules

EventBridge **Scheduler** schedules (group default; Scheduler invokes Lambda by assuming a role, not via lambda add-permission):

| Schedule                  | Expression                             | State                                                                                                                                                                                                                                                                                                                             | Target                       |
|---------------------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| pest-model-watchdog-15min | rate(15 minutes),<br>tz Asia/Singapore | <b>ENABLED</b> (disabled<br>2026-07-28 during<br>testing, re-enabled<br>with the 2026-08-05<br>v6.2 closeout: the<br>watchdog now<br>honours per-camera<br>max_runtime_min —<br>worm_cam 45 — with<br>env<br>MAX_RUNTIME_MIN=75<br>as the global<br>fallback; this is the<br>auto-close half of<br>the unattended<br>morning run) | pest-model-<br>watchdog      |
| model-start-schedule      | daily 05:53 SGT                        | <b>DELETED 2026-08-05</b> (Gen-1; the<br>legacy start/stop<br>chain was broken —<br>fixed-time stop killed<br>the model while still<br>STARTING)                                                                                                                                                                                  | Gen-1 pest-model-<br>control |

| Schedule                  | Expression      | State                     | Target                   |
|---------------------------|-----------------|---------------------------|--------------------------|
| model-stop-schedule       | daily 06:01 SGT | <b>DELETED 2026-08-05</b> | Gen-1 pest-model-control |
| frame-extraction-schedule | daily 06:00 SGT | <b>DELETED 2026-08-05</b> | Gen-1 Extraction         |
| ExtractFrameEveryMinute   | every minute    | <b>DELETED 2026-08-05</b> | Gen-1                    |

Only the watchdog schedule remains on the development account. Since v6.2, per-camera capture scheduling is START-only: the dashboard schedule sets a start time, the API creates one classic rule, and the watchdog closes the endpoint after `max_runtime_min` — no stop time exists anywhere.

Classic EventBridge **rules** are created at runtime by `pest-monitoring-api` for per-camera capture schedules: name pattern `pest-sched-{camera_id}-{action}`, expression `cron(mm hh ? * DAYS *)` in UTC (the API converts SGT to UTC with day-shift), target `pest-camera-scheduler`, plus a per-rule `lambda:AddPermission (StatementId sched-invoke-<rule>)`. None are pre-created, and the API deletes the rule when a schedule is disabled — `worm_cam`'s schedule is currently saved as 05:40 SGT daily but disabled (2026-08-05), so no `pest-sched-*` rule exists live.

Re-enabling a disabled schedule needs the FULL existing definition, not just the state: `aws scheduler update-schedule --name --state ENABLED ...` with the whole schedule body.

### 9.3.11 SES

| Identity                | Status                                                                                                                                        |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| rex2956550768@gmail.com | PROD — both sender ( <code>SENDER_EMAIL</code> on the processor) and recipient ( <code>recipient_email</code> in <code>system-config</code> ) |
| teowilbur@gmail.com     | DEAD (predecessor)                                                                                                                            |
| neobkee@gmail.com       | Not pest                                                                                                                                      |

The account is in the SES **sandbox** (200 msgs/24 h, verified addresses only). Production access is a manual AWS Support request. No SNS, no SQS, no configuration sets.

### 9.3.12 IAM roles

| Role                          | Used by   | Notable policies                                                                                                                          |
|-------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------|
| pest-detection-processor-role | processor | inline <code>pest-detection-processor-policy</code> (S3 frames/processed, DDB detections/cameras, <code>rekognition:DetectCustom</code> ) |

| Role                                           | Used by                       | Notable policies                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------------------------------------------------|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pest-monitoring-api-role                       | api                           | <p>Labels + DetectLabels, SES send); inline read-system-config; inline bedrock-verify<br/>           (bedrock:InvokeModel + bedrock:Converse)</p> <p>inline pest-monitoring-api-policy (S3 both buckets<br/>           incl. s3:DeleteObject scoped to frames/*; DDB; Rekognition<br/>           start/stop/describe; scheduler + events CRUD<br/>           scoped to pest-camera-* / pest-sched-*;<br/>           iam:PassRole to the scheduler invocation role;<br/>           kinesisvideo read); inline pest-monitoring-ddb-full-access; inline read-processed-armyworm; managed<br/>           AmazonSESFullAccess, AWSBillingReadOnlyAccess, AWSLambdaBasicExecutionRole</p> |
| pest-camera-scheduler-role                     | scheduler Lambda              | inline pest-camera-scheduler-policy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| kvs-hls-handler-role                           | kvs-hls-handler               | customer-managed kvs-hls-handler-rolePolicy (KVS GetDataEndpoint + GetHLSStreamingSessionURL)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| pest-model-watchdog-role                       | watchdog                      | inline Rekognition-DB-handler                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Amazon_EventBridge_Scheduler_LAMBDA_d585792c00 | watchdog schedule target role | console-generated (the stably-named pest-scheduler-invocation-role in the deployer spec does NOT exist live)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Amazon_EventBridge_Sched                       | Gen-1 schedules               | legacy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

| Role                                | Used by | Notable policies |
|-------------------------------------|---------|------------------|
| uler_LAMBDA_4c3f0b5d17              |         |                  |
| kvs-hls-url-generator-role-013j8khq | Gen-1   | legacy           |

### 9.3.13 Other

- EC2 instance MonitoringSystem (i-0750c917a1f038d4d, t3.micro, running): believed to be the older EC2+nginx dashboard host, superseded by S3 + CloudFront. OPEN ITEM: confirm it no longer serves anything, then stop or terminate it; it bills while running.
- CloudWatch: log groups /aws/lambda/<function> are auto-created, retention Never-Expire, no alarms, no metric filters.
- Empty on this account: SNS, SQS, Secrets Manager, SSM Parameter Store, ACM, Route53, WAF, KMS CMKs, custom event buses.

### 9.3.14 NP production account 506868652945 (migration in progress)

Stood up by the ARGUS deployer, 2026-08-10: full 15-stage headless deploy, 103 seconds, zero errors (python deploy.py --profile prod --prefix argus ...). This run doubles as the deployer's Round-2 rehearsal on a real fresh account. Post-deploy verification passed: processor 1024 MB / 600 s with the Pillow layer, env carries Sonnet 4.6 + LLM\_VERIFY\_ALL\_BOXES=true + tile floor 8 + post-verify floor 49 + area cap 0.05; seeded camera rows read llm\_verify\_enabled=true, min\_confidence=10, post\_verify\_floor=49. All CLI commands against this account use --profile prod.

| Item                    | Value                                                                                                                                   |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Account / IAM / profile | 506868652945 / Student_QianRunzhe (group CAG_Proj = AdministratorAccess) / prod                                                         |
| S3 buckets              | argus-frames-506868652945 (frames + training-data/), argus-processed-506868652945, argus-dashboard-506868652945                         |
| DynamoDB                | same 4 pest-monitoring-* table names as the development account (the argus prefix renames buckets and Rekognition projects, not tables) |
| Lambdas                 | the same 5 functions, deployed from the repo lambda/ sources (processor v6.3, watchdog v6.2)                                            |
| API Gateway             | HTTP API id <b>vzf17s6z00</b> —<br>https://vzf17s6z00.execute-api.us-east-1.amazonaws.com, 21 routes, JWT on all but GET /stream/status |

| Item                           | Value                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cognito                        | pool <b>us-east-1_9se1FDHpc</b> , app client <b>6vebotf45bp8u46cnraddiaplv</b> ; sign-in user created 2026-08-11 (admin-create-only, CONFIRMED)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| CloudFront                     | <b>E1YADURLSAVNFA</b> →<br><a href="https://d1dtoxef7qmugl.cloudfront.net">https://d1dtoxef7qmugl.cloudfront.net</a><br>(Deployed; serves the ARGUS dashboard, verified 2026-08-11)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Rekognition project (armyworm) | arn:aws:rekognition:us-east-1:506868652945:project/argus-detection/1786376502421                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Armyworm model version         | <b>v9r-prod-20260810</b> —<br>arn:aws:rekognition:us-east-1:506868652945:project/argus-detection/version/v9r-prod-20260810/1786377372187. Retrain of the v9 retrain recipe (added data augmentation: flips, rotations, exposure jitter). Datasets CREATE_COMPLETE: TRAIN 32,986 labelled, TEST 3,653 labelled. Submitted 2026-08-10 23:56; TRAINING_COMPLETED 2026-08-12, console F1 0.613 (the development account's equivalent version reads 0.599 — same data and recipe, so treat them as equivalent; a poller, migration/poll_training.py, watched the run). The version ARN is written into worm_cam (verified by live scan 2026-08-12; manual_upload keeps an empty ARN and takes the generic path). Validation against the development account's threshold-study zones is in flight — see the IAM-gap note below the table |
| Rekognition project (moth)     | argus-moth-detection — version <b>moth-prod-20260811</b> , arn:aws:rekognition:us-east-1:506868652945:project/argus-moth-detection/version/moth-prod-20260811/1786412382515. <b>TRAINED 2026-08-11, F1 = 0.991</b> (statistically the same as the development-account moth model's 0.988 — same data). ARN already written into moth_cam.custom_model_arn; endpoint                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

| Item                | Value                                                                                                                                                                                                                   |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                     | not yet started. Rebuilt from the surviving labelled data (116 TRAIN + 29 TEST images) because trained models are account-bound                                                                                         |
| EventBridge         | watchdog schedule rate(15 minutes) created by the deployer; no pest-sched-* rules (worm schedule saved 05:40 daily, disabled)                                                                                           |
| SES                 | rex2956550768@gmail.com VERIFIED 2026-08-11 (sender + recipient). Account is in the SES <b>sandbox</b> , same as the development account                                                                                |
| KVS / live view     | <b>NOT migrated</b> — deploy ran without --live-view; no streams exist, worm_cam.kvs_stream_name is empty. The dashboard Live tab and the devices' kvs_controller cannot work until streams are created (open decision) |
| Cameras             | worm_cam (deployer-seeded as camera-1, re-keyed to the reference id 2026-08-11), manual_upload, moth_cam (seeded by migration/migrate_moth.py). Full rows in 9.7                                                        |
| Migrated records    | 40 detection records: the complete 36-record Jewel on-site set (frames included, S3 keys byte-identical, model_arn left pointing at the development-account model as provenance) + 4 requested gallery captures         |
| Bedrock             | Sonnet 4.6 and Haiku 4.5 both authorized (verified 2026-08-10); no use-case form wait                                                                                                                                   |
| Pre-existing relics | a previous student's Amplify "moobusapps" leftovers (13 roles, 1 deployment bucket) — harmless, do not touch                                                                                                            |

IAM gap found by the first validation run (2026-08-12): the deployer had created pest-detection-processor-role WITHOUT the bedrock-verify and s3-frames-write inline policies (so no bedrock:InvokeModel/Converse, no rekognition:DetectLabels, no s3:PutObject on frames/\*). The failure is silent and looks like success — every Bedrock call raises, the gate reports zero verdicts, the processor fails open to plain min\_confidence (10), and frames come back covered in hundreds of unverified boxes. Fixed on the account by copying the development account's known-good policies

(migration/fix\_processor\_iam.py); the role now matches the development account action-for-action, and deployer/deploy.py ROLE\_POLICIES ships both policies so future deployments are correct (9.3.8). Lesson for any redeployment: validate by RESULT (push a known image, count surviving boxes), never by “all stages went green”.

Training-data migration: migration/copy\_training\_data.py copied 36,641 objects / 4.27 GB server-side into argus-frames-506868652945/training-data/v9/ (counts verified, manifest source-refs repointed). The copy uses a temporary prefix-scoped read grant on the old bucket; run the scripts’ --revoke mode after model sign-off (three grants are outstanding: training data, Jewel frames, moth console bucket).

Engineering lesson from this migration, generalised: **after any Rekognition console labelling, the S3 manifest is stale — the dataset is the source of truth**. Export the live entries with ListDatasetEntries, repoint every source-ref, upload as new manifests, then create the datasets (migration/train\_v9r\_on\_prod.py and migration/migrate\_moth.py both do this). Building from the copied S3 manifest alone silently trains a different model.

Remaining before the development account can be retired: the armyworm endpoint validation run (same holdout images, same detections — the ARN writeback is done; the 2026-08-12 first attempt was invalidated by the IAM gap above and must be repeated), restore worm\_cam.max\_runtime\_min from the temporary 240 to 45 after sign-off, repoint the Orin / mini PC (bucket → argus-frames-506868652945, API base → the new API GW URL; camera ids unchanged), revoke the three cross-account read grants. The development account stays untouched until the new one is proven end to end, and holds the threshold-study evidence the final report cites.

## 9.4 Device registry

No passwords in this table. SSH is key-auth where stated; remaining passwords are held by Runzhe outside the repo.

| Host                              | IP / network                        | OS              | User                       | Services / notes                                                                                                                                                           |
|-----------------------------------|-------------------------------------|-----------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unitree Go2<br>EDU — Sport<br>MCU | 192.168.123.16<br>1 (wired dog net) | vendor firmware | none (DDS only,<br>no SSH) | <b>USLAM runs here</b> , auto-starts at power-on, auto-loads the last map. Control surface: plaintext std_msgs/String on /uslam/client_command + /uslam/server_log. An MCU |

| Host                          | IP / network                                                                                                                                                                                                       | OS                       | User                    | Services / notes                                                                                                                                                                                                                                                                                      |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Jetson Orin Nano (on the Go2) | 192.168.123.18<br>dog net; WiFi<br>10.1.125.24 at NP<br>(npwireless),<br>10.38.19.10/23<br>gw 10.38.18.1<br>at Jewel (apps-jewel, hidden SSID<br>Apps@Jewel, client isolation<br>— no inbound SSH over Jewel WiFi) | Ubuntu 20.04, ROS 2 Foxy | unitree (SSH alias go2) | wedge needs a full power cycle<br>Patrol<br>~/go2/go2_patrol_gated.py, survey<br>~/go2/pose.py, ~/setup_go2.sh (binds CycloneDDS), SIYI SDK at ~/a8/siyi_sdk, KVS producer SDK (ARM64). AWS CLI is v1. eth0 = a8-link (192.168.144.30, MAC-bound to ASIX AX88179B 6C:1F:F7:28:CD:0C), eth1 = dog link |
| SIYI A8 Mini gimbal camera    | 192.168.144.25 ; RTSP<br>rtsp://192.168.144.25:8554/main.264 (1080p cap); UDP control port 37260                                                                                                                   | fw 0.2.8                 | —                       | FOLLOW mode set in SIYI PC Assistant, persists. Pitch is INVERTED (+ = lens down, clamp [-90, +25]); LOCK mode required for absolute angles. Back on ethernet since 2026-07-29; the MS2109 USB capture card detour is retired (card maxes at 1080p, no 4K)                                            |
| Mini PC —                     | campus IP                                                                                                                                                                                                          | Windows 11               | —                       | Runs VMware                                                                                                                                                                                                                                                                                           |



| Host                     | IP / network                                                                                                           | OS           | User                            | Services / notes                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------|------------------------------------------------------------------------------------------------------------------------|--------------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Win11 host               | (e.g. 10.1.67.21)                                                                                                      |              |                                 | hosting the VM below                                                                                                                                                                                                                                                                                                                                                                             |
| Mini PC —<br>Ubuntu VM   | ens33 NAT<br>192.168.189.x<br>(no inbound);<br>ens37 bridged<br>192.168.123.99<br>/24 (dog net,<br>only when<br>wired) | Ubuntu 22.04 | wilburteo                       | systemd kvs-<br>controller.ser<br>vice<br>(kvs_controlle<br>r.py via<br>run_kvs_contro<br>ller.sh,<br>Restart=always,<br>polls GET<br>/stream/status<br>every 5 s);<br>systemd<br>reverse-<br>tunnel-<br>fyp.service<br>(self-healing -R<br>2222:localhost<br>:22 to the Orin;<br>from the laptop:<br>SSH to the Orin,<br>then ssh -p<br>2222<br>wilburteo@loca<br>lhost, key auth<br>both ways) |
| Hikvision moth<br>camera | 192.168.1.66                                                                                                           | —            | —                               | Feeds moth-<br>cam-stream<br>(parked). RTSP<br>credential<br>stored on the<br>mini PC, not<br>reproduced                                                                                                                                                                                                                                                                                         |
| Laptop<br>(ZenBook Air)  | dock ASIX<br>adapter static<br>192.168.123.50<br>/24 for the dog<br>net                                                | Windows 11   | Zenbook Air<br>(note the space) | Holds the<br>working clone of<br>the project<br>repository; CLI<br>profiles nbk2<br>(dev) + prod (NP<br>production),<br>Posh-SSH for<br>remote ops                                                                                                                                                                                                                                               |

## 9.5 Environment-variable tables

Names and meanings only. No values are secret; email addresses and ids are shown, credentials never appear in env vars in this stack (AWS access on devices comes from per-device AWS CLI profiles — credential stored on each device, not reproduced).

`pest-detection-processor` (code defaults in brackets; live overrides noted)

Core:

| Var                                                                                           | Meaning                                                                                                                                                  |
|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>SENDER_EMAIL</code>                                                                     | <b>required</b> — SES-verified alert sender (live <code>rex2956550768@gmail.com</code> )                                                                 |
| <code>TABLE_DETECTIONS</code> / <code>TABLE_CAMERAS</code> / <code>TABLE_SYSTEM_CONFIG</code> | DynamoDB table names [defaults = the real names]                                                                                                         |
| <code>S3_PROCESSED_BUCKET</code>                                                              | set on the live function but <b>not read by current code</b> (v4.2 stopped writing processed images) — NOTE: doc lag in older docs that call it required |

Tiling (cloud-side zoom scan, per-camera opt-in via `tiling_enabled`):

| Var [default]                                                                           | Meaning                                                                                                                                                            |
|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>TILING_ENABLED</code> [true]                                                      | global kill switch                                                                                                                                                 |
| <code>TILE_COLS</code> / <code>TILE_ROWS</code> [4 / 4]                                 | grid                                                                                                                                                               |
| <code>TILE_OVERLAP</code> [0.15]                                                        | overlap fraction per tile                                                                                                                                          |
| <code>TILE_UPSCALE_LONG_EDGE</code> [1920]                                              | per-tile upscale target                                                                                                                                            |
| <code>TILE_INCLUDE_FULL_FRAME</code> [true]                                             | also detect on the whole frame                                                                                                                                     |
| <code>TILE_MIN_CONFIDENCE</code> [30; <b>live 8</b> per the 2026-08-10 frozen baseline] | per-tile gather floor — a box under this never exists downstream (the first of the two stacked floors; the camera row's <code>min_confidence</code> is the second) |
| <code>TILE_NMS_IΟΥ</code> [0.5], <code>TILE_MAX_WORKERS</code> [4]                      | NMS + parallelism                                                                                                                                                  |

Non-vegetation false-positive suppression (v4.3):

| Var [default]                                                                | Meaning                                                                              |
|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <code>SUPPRESS_NONVEG</code> [true]                                          | drop worm boxes $\geq$ coverage inside hard-object <code>DetectLabels</code> regions |
| <code>SUPPRESS_MIN_CONF</code> [55],<br><code>SUPPRESS_COVERAGE</code> [0.5] | <code>DetectLabels</code> floor; coverage fraction                                   |
| <code>SUPPRESS_LABELS</code> / <code>SUPPRESS_PROTECT</code>                 | hard-object list / never-suppress plant list                                         |

LLM verification gate (v4.5+) and its extensions:

| Var [default]                                                                                                           | Meaning                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| LLM_VERIFY [true]                                                                                                       | global kill switch for the gate                                                                            |
| LLM_VERIFY_MODEL_ID [Haiku 4.5 profile id;<br><b>live us.anthropic.claude-sonnet-4-6</b> ]                              | Bedrock inference-profile id (us . prefix required)                                                        |
| LLM_VERIFY_ALL_BOXES [false; <b>live true</b> ]                                                                         | judge every box (denoiser mode), not only sub-min_confidence ones                                          |
| LLM_VERIFY_MAX_BOXES [5; <b>live 120</b> ]                                                                              | cap on clusters judged per frame                                                                           |
| LLM_VERIFY_PAD [0.6],<br>LLM_VERIFY_LONG_EDGE [672],<br>LLM_VERIFY_MIN_CONTEXT_PX [32],<br>LLM_VERIFY_MAX_UPSCALE [8.0] | crop recipe (measured well-tuned — do not widen)                                                           |
| LLM_VERIFY_WORKERS [4; <b>live 3</b> ],<br>LLM_VERIFY_TIMEOUT [12],<br>LLM_VERIFY_MAX_TOKENS [100; <b>live 300</b> ]    | parallelism / per-call limits                                                                              |
| LLM_VERIFY_TEMPERATURE [unset]                                                                                          | keep UNSET — a “0” here 400s on newer models                                                               |
| LLM_MERGE [false; live false]                                                                                           | cluster-merge gate (v4.8) — measured harmful; code path DELETED in v6.3                                    |
| LLM_SCAN [live false]                                                                                                   | v4.6 whole-frame recovery scan — code path DELETED in v6.3                                                 |
| LLM_FIRST [false]                                                                                                       | v5.1 LLM-first mode — measured worse; code path DELETED in v6.3                                            |
| LLM_LEAD [false; <b>live false</b> ]                                                                                    | v5.2 Sonnet-leads experiment — retired; code path DELETED in v6.3                                          |
| POST_NMS_IOU [0; <b>live 0.1</b> ]                                                                                      | v5.0 post-gate NMS (0 = off)                                                                               |
| POST_NMS_CONTAIN [0; <b>live 0.1</b> ]                                                                                  | post-gate containment-NMS (drop a box mostly contained in a kept box)                                      |
| POST_MAX_BOX_AREA [0; <b>live 0.05</b> ]                                                                                | the 5% area cap — drop boxes covering more than this fraction of the frame (too big to be a larva)         |
| POST_VERIFY_FLOOR [0; <b>live 49</b> — flipped 34→49 on 2026-08-10, the delivered gate]                                 | confidence floor applied AFTER the LLM verdict; the camera row’s post_verify_floor overrides it per camera |

v6.3 (2026-08-07) deleted the scan / merge / LLM-first / LLM-lead code paths and their LLM\_SCAN\_\* / LLM\_MERGE\_\* / LLM\_FIRST\_\* / LLM\_LEAD\_\* geometry knobs. The development-account function still carries some of those vars (set to false) — they are ignored. The production-account function omits them; behaviour is identical. The

authoritative dev-account env read as of the migration is  
migration/prod\_baseline\_20260810.json.

Stateless S3-key overrides (not env vars, but part of the same contract): a key segment  
\_\_conf<N> overrides the camera's min\_confidence for that one run; \_\_llm-<alias> picks  
the verifier model for that one run (aliases in 9.3.8).

#### pest-monitoring-api

| Var                                                                                | Meaning                                                                           |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| S3_FRAMES_BUCKET                                                                   | <b>required</b> — frames-armyworm-366356442579                                    |
| S3_PROCESSED_BUCKET                                                                | <b>required</b> — legacy bucket (cold-start check + delete path)                  |
| TABLE_CAMERAS / TABLE_DETECTIONS /<br>TABLE_SYSTEM_CONFIG /<br>TABLE_SCHEDULE_LOGS | table names                                                                       |
| SCHEDULE_EXECUTOR_ARN                                                              | ARN of pest-camera-scheduler (filled after that function exists — 2-phase deploy) |

Writable-field allow-lists in this Lambda (code constants, not env): GLOBAL\_ALLOWED =  
email\_enabled, recipient\_email, additional\_recipients, auto\_capture, capture\_interval;  
CAMERA\_ALLOWED = label, target\_label, model\_type, custom\_model\_arn, min\_confidence,  
model\_running, default\_waypoint\_id, kvs\_stream\_name, stream\_enabled, tiling\_enabled,  
llm\_verify\_enabled.

#### pest-camera-scheduler

| Var                                | Meaning                             |
|------------------------------------|-------------------------------------|
| TABLE_CAMERAS, TABLE_SCHEDULE_LOGS | table names (defaults = real names) |

#### kvs-hls-handler

No env vars. Region comes from the Lambda's AWS\_REGION default.

#### pest-model-watchdog

| Var             | Meaning                                                                                                                                                                                                                                                              |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TABLE_CAMERAS   | table name                                                                                                                                                                                                                                                           |
| MAX_RUNTIME_MIN | live 75 — the GLOBAL FALLBACK: stop a RUNNING Rekognition endpoint after this many minutes. Since watchdog v6.2 a per-camera max_runtime_min field wins where set (worm_cam 45). The watchdog schedule was re-ENABLED 2026-08-05 with the v6.2 closeout (see 9.3.10) |

## Edge services

| Service (host)                         | Config item                                  | Meaning                                                                                                                                                                                                      |
|----------------------------------------|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| kvs-controller.service<br>(mini PC VM) | CAMERA_ID                                    | which camera row to poll —<br>set BOTH in the systemd<br>unit Environment= line AND<br>as export<br>CAMERA_ID=moth_cam on line<br>2 of<br>run_kvs_controller.sh;<br>the script export wins if they<br>differ |
| reverse-tunnel-<br>fyp.service (VM)    | tunnel spec                                  | -R 2222:localhost:22 to<br>the Orin, key auth,<br>Restart=15 s self-healing                                                                                                                                  |
| go2_patrol_gated.py (Orin<br>~/go2/)   | INITIAL_POSE, WAYPOINTS                      | per-map parameter sets live<br>in robot/map_profiles.md;<br>waypoints with "capture":<br>False navigate only                                                                                                 |
| go2_patrol_gated.py                    | FOLLOW_SETTLE_S = 2.0                        | gimbal settle wait per<br>waypoint                                                                                                                                                                           |
| go2_patrol_gated.py                    | send_verb(repeat=1) /<br>send_goal(repeat=1) | MUST stay 1 — repeated<br>USLAM verbs wedge the<br>MCU (Chapter 5)                                                                                                                                           |
| setup_go2.sh (Orin)                    | CycloneDDS binding                           | binds DDS to the dog link by<br>IP 192.168.123.18; source it<br>in every Orin terminal                                                                                                                       |
| Capture script (Orin)                  | camera id worm_cam                           | uploads to<br>frames/worm_cam/. . . in the<br>frames bucket                                                                                                                                                  |

## 9.6 Repository file map

All paths are relative to the project repository root.

| Path      | Contents                                                                                                                                                                                                                                           |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CLAUDE.md | behavioral brief for the AI assistant                                                                                                                                                                                                              |
| docs/     | state.md (living state + roadmap — start<br>here), aws.md, hardware.md, detection.md,<br>dashboard.md, go2_demo_commands.md,<br>project_timeline.md, model_ladder.md,<br>deployer_training_pipeline.md;<br>docs/history/ = frozen weekly records + |

| Path       | Contents                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            | retired code.md                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| lambda/    | deployed source of the 3 core Lambdas (pest-detection-processor.py, pest-monitoring-api.py, pest-camera-scheduler.py) + cors.json, ddb-policy.json, bedrock-policy.json. The other two live Lambdas are mirrored under deployer/audit/*_src/                                                                                                                                                                                                                                                                                                              |
| robot/     | mirror of the Orin ~/go2/ set + setup_go2.sh, pose.py, map_profiles.md; tests/ = nav validation harness + run logs                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| minipc/    | mini-PC stack: kvs_controller.py, run_kvs_controller.sh, kvs-controller.service, capture_and_upload_v4_armyworm.py, capture_and_upload_v3_person_cam.py                                                                                                                                                                                                                                                                                                                                                                                                   |
| web/       | dashboard_v4/ (current dashboard) + _archive/ (holds dashboard_v3_9.html, the fallback, alongside v3_4-v3_8; unrelated to the top-level archive/)                                                                                                                                                                                                                                                                                                                                                                                                         |
| deployer/  | ARGUS one-click deployer: deploy.py, STACK_MANIFEST.md (authoritative BOM), audit/ (raw live-account specs, incl. the live bedrock-verify + s3-frames-write role policies dumped 2026-08-11), web/ (frozen deployer UI), REHEARSAL.md                                                                                                                                                                                                                                                                                                                     |
| migration/ | account-migration toolkit (dev 366356442579 → NP production 506868652945):<br>prod_baseline_20260810.json (frozen dev-account config baseline — Lambdas, cameras, system-config, schedules; no secrets), copy_training_data.py (server-side S3 copy + manifest repoint + --revoke), train_v9r_on_prod.py (dataset export via ListDatasetEntries + retrain on the new account), migrate_moth.py (moth data + project + moth_cam row rebuild), migrate_jewel_records.py + migrate_v9r49_picks.py (detection-record migration), diff_accounts.py (old-vs-new |

| Path                 | Contents                                                                                                                                                                                                                                                                                                                                                              |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      | account diff), poll_training.py (training-status poller), fix_processor_iam.py (prod processor-role policy repair, 2026-08-12), plus survey/verify helpers                                                                                                                                                                                                            |
| datasets/            | training data pipeline, restructured into live / staging / sources / holdout / archive (+ current/ in transition); holdout/cag/ = the sacred CAG holdout. Full map: datasets/README.md. <b>Caution: datasets/archive/experiments/pre_v3_abandoned/download.py contains a live Roboflow API key — credential stored here, not reproduced; do not publish this path</b> |
| reports/             | deliverables: proposal/, interim/, weekly/ (W1+ docx), presentation/, manual/ (this manual), final/ (final report: REPORT_PLAN.md, figures/ + build_figures.py, draft/ = full chapter drafts ch00-ch10 + appendix, 2026-08-11)                                                                                                                                        |
| reference/wilbur/    | predecessor material (final report, decks, old Flutter dashboard). Reference only. <b>Caution: Wilbur's final-report docx has an AWS secret key in cleartext in its appendix — credential stored here, not reproduced; never publish unscrubbed</b>                                                                                                                   |
| context/claude-chat/ | immutable archives of prior Claude chat accounts (historical evidence, not current state)                                                                                                                                                                                                                                                                             |
| archive/             | ALL other historical material (chat imports, snapshots, old lambda snapshot). Nothing load-bearing                                                                                                                                                                                                                                                                    |

## 9.7 Camera registry

### Development account 366356442579

Three rows in pest-monitoring-cameras (test/legacy rows notile\_test, batch2\_notile, person\_cam deleted 2026-07-13; ids migrated 2026-07-14: armyworm\_go2\_a8mini → worm\_cam, moth\_cam\_01 → moth\_cam; historical S3 keys under frames/armyworm\_go2\_a8mini/... untouched, records still point at them). Live values

below verified against the frozen 2026-08-10 baseline  
migration/prod\_baseline\_20260810.json plus the 2026-08-10 floor flip.

| Field              | worm_cam                                                                                                                   | moth_cam                                        | manual_upload                           |
|--------------------|----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|-----------------------------------------|
| Display label      | Worm Cam                                                                                                                   | Moth Cam                                        | Manual test upload<br>(hidden fallback) |
| Device             | SIYI A8 Mini on the<br>Go2, captured by the<br>Orin                                                                        | Hikvision<br>192.168.1.66 via<br>the mini PC VM | dashboard Test<br>upload panel          |
| target_label       | armyworm-larva                                                                                                             | Moths                                           | Person                                  |
| model_type         | custom                                                                                                                     | custom                                          | general (plain<br>DetectLabels)         |
| custom_model_arn   | armyworm <b>v9</b><br><b>retrain v9-</b><br><b>20260805-0713</b> ARN<br>(9.3.7)                                            | moth<br>SmartPestProject<br>ARN (9.3.7)         | empty                                   |
| min_confidence     | <b>10</b> (lowered from 75<br>— the candidate<br>floor; the LLM gate<br>and<br>post_verify_floor<br>do the rejecting)      | 80                                              | 80                                      |
| post_verify_floor  | <b>49</b> (34→49, decided +<br>flipped live 2026-08-<br>10 — the delivered<br>gate)                                        | —                                               | —                                       |
| max_runtime_min    | <b>45</b> (restored from a<br>temporary 240 on<br>2026-08-10;<br>watchdog v6.2<br>honours it)                              | —                                               | —                                       |
| tiling_enabled     | <b>true</b>                                                                                                                | false                                           | false                                   |
| llm_verify_enabled | <b>true</b> (only camera<br>opted in — the verify<br>prompt asks about<br>larvae, so it must<br>never run for<br>moth_cam) | false / unset                                   | false / unset                           |
| kvs_stream_name    | armyworm-cam-<br>stream                                                                                                    | moth-cam-stream                                 | none                                    |
| stream_enabled     | false (patrol uploads<br>stills; stream on                                                                                 | false                                           | false                                   |



| Field    | worm_cam                            | moth_cam                 | manual_upload |
|----------|-------------------------------------|--------------------------|---------------|
|          | demand)                             |                          |               |
| schedule | 05:40 SGT daily,<br><b>disabled</b> | 09:00-17:00,<br>disabled | —             |

## NP production account 506868652945

Three rows, seeded 2026-08-10/11 (the worm camera + manual\_upload by the ARGUS deployer, moth\_cam by migration/migrate\_moth.py). The deployer seeded the worm camera as camera-1; the row was re-keyed to **worm\_cam** in the 2026-08-11 reconciliation, so device upload scripts keep their camera id.

| Field             | worm_cam                                                                                                                                                   | moth_cam                                                   | manual_upload                                                                                                                 |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Display label     | Worm Cam<br>(deployer wrote the deployment name into it; restored to “Worm Cam” 2026-08-11 — the dashboard shows the label, so the label IS functionality) | Moth Cam                                                   | Manual test upload                                                                                                            |
| target_label      | armyworm-larva                                                                                                                                             | Moths                                                      | armyworm-larva<br>(differs from the dev row’s Person/general by design; behaviour is generic while custom_model_arn is empty) |
| model_type        | custom                                                                                                                                                     | custom                                                     | custom                                                                                                                        |
| custom_model_arn  | <b>v9r-prod-20260810</b> ARN (9.3.14), written after training completed                                                                                    | <b>moth-prod-20260811</b> ARN (9.3.14), written 2026-08-11 | empty                                                                                                                         |
| min_confidence    | <b>10</b>                                                                                                                                                  | 80                                                         | <b>10</b>                                                                                                                     |
| post_verify_floor | <b>49</b>                                                                                                                                                  | —                                                          | <b>49</b>                                                                                                                     |
| max_runtime_min   | seeded <b>45</b> ; live read 2026-08-12 shows <b>240</b> (temporarily raised for the post-training validation window — restore to                          | 45                                                         | — (not seeded; global watchdog fallback 75 applies)                                                                           |

| Field              | worm_cam                                                                                                                       | moth_cam                                                                                             | manual_upload |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|---------------|
|                    | 45 after sign-off, or the watchdog lets a \$4/hr endpoint run 4 hours)                                                         |                                                                                                      |               |
| tiling_enabled     | <b>true</b>                                                                                                                    | false                                                                                                | —             |
| llm_verify_enabled | <b>true</b>                                                                                                                    | <b>false — on purpose</b> (the verify prompt asks about larvae; this camera’s target is adult moths) | <b>true</b>   |
| kvs_stream_name    | empty (KVS not migrated)                                                                                                       | moth-cam-stream (stream does not exist on this account yet)                                          | none          |
| schedule           | 05:40 SGT daily, <b>disabled</b> (deployer default 09:00-17:00 corrected 2026-08-11 to match the development account’s intent) | 09:00-17:00, disabled                                                                                | —             |

Field semantics (both accounts): min\_confidence is NOT a plain detection floor any more — since v4.5 it is “the point above which Rekognition’s word is final”; below it the LLM gate decides (and in live denoiser mode every box is judged). post\_verify\_floor is the confidence floor applied AFTER the LLM verdict — the delivered 49 gate. max\_runtime\_min is how long the watchdog lets that camera’s Rekognition endpoint run. tiling\_enabled and llm\_verify\_enabled are per-camera opt-ins on this shared processor. llm\_verify\_enabled is in the API’s CAMERA\_ALLOWED so it CAN be flipped via POST /settings, but no dashboard control exists — DynamoDB/CLI is the practical toggle.

## 9.8 Glossary of project terms

| Term           | Meaning                                                                                                                                                                                                                                                           |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ARGUS</b>   | Project name of the deliverable system: the v5.2 vanilla-JS dashboard (cloud-deployed with Cognito) + the one-click deployer. The dashboard IS the final deliverable (settled 2026-07-15; the old “Flutter Web target” claim was a miscapture — do not resurrect) |
| <b>batch_1</b> | 13 CAG images (23 worms), hand-held close-ups. Used for internal spot checks                                                                                                                                                                                      |

| Term                      | Meaning                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>batch_2</b>            | <p>only — evaluation numbers in outward-facing documents are quoted solely from never-trained images: batch_2 102-109, Jewel_1/2, and the 4 field-realistic photos</p> <p>The sacred CAG holdout: originally 7 images, grown to 10-11 labelled images / 11 worms (110/111 added 2026-07-28). <b>Never train on it.</b> The only fair evaluation set; contains NO true negatives, so image-level FP rate is not measurable on it</p> |
| <b>gate (patrol gate)</b> | <p>The Orin patrol's per-waypoint wait: capture + upload → poll DynamoDB<br/> get_item(image_id=key) every ~2 s → record appears (clean frames also write a record, so no deadlock) → next waypoint</p>                                                                                                                                                                                                                             |
| <b>gate (LLM gate)</b>    | <p>The Bedrock verification step in the processor: Rekognition proposes boxes, the LLM judges which survive. In the live denoiser mode survival is fail-closed per box (rejected AND un-judged boxes are dropped); the fail-open path exists only for total gate failure (zero verdicts returned, infrastructure down), when all boxes are kept and the caller falls back to plain thresholding</p>                                 |
| <b>denoiser mode</b>      | <p>LLM_VERIFY_ALL_BOXES=true: Rekognition runs as a high-recall front end (low tile floor), the LLM rejects the noise. Explains v9's deliberate low F1</p>                                                                                                                                                                                                                                                                          |
| <b>gen-1 / legacy</b>     | <p>Wilbur's original stack (Gen-1 Lambdas, REST + WebSocket APIs, FYP-PROJECT stream, EC2 dashboard). Dead; listed in 9.3 so nobody recreates it</p>                                                                                                                                                                                                                                                                                |
| <b>prod</b>               | <p>The AWS CLI profile name for the production account 506868652945. Every operative command in this manual uses it</p>                                                                                                                                                                                                                                                                                                             |
| <b>nbk2</b>               | <p>The CLI profile of the retired development account 366356442579. It appears only in historical notes; nothing in the live system uses it</p>                                                                                                                                                                                                                                                                                     |

| Term                                 | Meaning                                                                                                                                                                      |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>whole-image protocol</b>          | The retired “never tile, judge the whole frame” evaluation rule. SUPERSEDED: tiling_enabled=true on worm_cam (2026-07-27/28); Rekognition finds, the LLM verifies            |
| <b>tiling / zoom scan</b>            | Cloud-side split of the frame into an overlapping grid, per-tile upscale + detect, global-coordinate NMS. Dashboard name: “Zoom scan”                                        |
| <b>two stacked floors</b>            | TILE_MIN_CONFIDENCE (inside tiling, before NMS) and the camera row’s min_confidence (after tiling). Lowering only the second cannot recover what the first already discarded |
| <b>__confN / __llm-&lt;alias&gt;</b> | Stateless S3-key overrides: per-upload min_confidence / per-upload verifier model. Lets N experiment arms run concurrently with no Lambda config swaps                       |
| <b>Zone</b>                          | Dashboard grouping field (waypoint id). Experiment arms write to Zones named after themselves, so the gallery labels the arm                                                 |
| <b>holdout</b>                       | Images deliberately excluded from all training, used only for evaluation (datasets/holdout/cag/)                                                                             |
| <b>manual_upload</b>                 | Hidden camera row backing the dashboard Settings → Test upload panel — the way live detection is demonstrated, since there are no worms at Jewel                             |
| <b>USLAM</b>                         | Unitree’s SLAM/nav stack. Runs on the Go2 Sport MCU (not the Orin), driven by plaintext string commands. See Chapter 5 for its failure modes (wedge, stale localization)     |
| <b>wedge (MCU wedge)</b>             | USLAM failure state from rapid repeated control verbs: thousands of TIMEOUT_ODOMETRY events, odom publishers drop to 0, only a power cycle clears it. Send every verb once   |
| <b>CL</b>                            | Rekognition Custom Labels. Cannot be trained on negatives; models are account-bound                                                                                          |

| Term                     | Meaning                                                                                                                                                                                                                                   |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>inference profile</b> | The <code>us.-</code> prefixed Bedrock model id required for cross-region on-demand invocation. The bare foundation-model id fails                                                                                                        |
| <b>NOMERGE</b>           | Experiment arms with <code>LLM_MERGE=false</code> (per-box judging). The merge gate was measured to destroy detections; live config keeps it off                                                                                          |
| <b>LLM_LEAD</b>          | v5.2 experiment: Sonnet 4.6 swept native-resolution tiles and led detection, with Rekognition refining box geometry. RETIRED — code path deleted in v6.3; the delivered architecture is the reverse (Rekognition finds, the LLM verifies) |
| <b>clean image</b>       | A frame with zero surviving boxes. Clean frames still write a DynamoDB record (unconditional <code>put_item</code> ) — the patrol gate depends on this                                                                                    |
| <b>sandbox (SES)</b>     | SES restricted mode: send only to/from verified identities                                                                                                                                                                                |
| <b>Gen-2</b>             | The current 5-Lambda stack described in this manual, as opposed to gen-1                                                                                                                                                                  |

## 9.9 Document conventions and change log

Conventions used throughout this manual:

- English only, plain and short. Real proper nouns are kept exactly: SIYI A8 Mini, Unitree Go2, Jetson Orin, AWS Rekognition Custom Labels, USLAM, ARGUS, CAG, Jewel Changi.
- Every AWS procedure gives BOTH the console click-path and the CLI command. In commands, **bold** marks a value the reader must fill in themselves; plain code is a literal value of this deployment.
- The code is ground truth. `docs/*.md` provide context and sometimes lag; rows above marked “doc lag” record known disagreements, resolved in the code’s favor.
- “OPEN ITEM” marks a fact this chapter could not resolve from the repository alone; each has a one-command check listed in 9.10.
- Secrets never appear. “Credential stored here, not reproduced” marks a known secret location.

Change log (append one row per revision):

| Date | Version | Author | Change |
|------|---------|--------|--------|
|------|---------|--------|--------|

| Date       | Version | Author               | Change                                                                                                                                                                                                                                            |
|------------|---------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2026-07-22 | 1.0     | Runzhe (with Claude) | First complete appendix: all registries, glossary, conventions                                                                                                                                                                                    |
| 2026-08-11 | 1.1     | Runzhe (with Claude) | Dual-account update: NP production registry (9.3.14) and prod camera rows (9.7); dev rows refreshed to the v9-retrain / floor-49 / v6.3 state; migration/ and reports/final/draft/ added to the file map; all repository paths made repo-relative |

## 9.10 Operations / reproduction

Concrete inspection procedures for every registry above. The commands are written against the live production account 506868652945 with CLI profile prod. On a stack reproduced elsewhere, replace `--profile prod` with **your own profile**, 506868652945 with **your account id**, and the `argus-*` bucket names with the ones your deployment prefix produced; DynamoDB table names and Lambda function names are the same on every deployment.

**Lambda inventory and live config.** Console: Lambda → Functions → filter pest. CLI:

```
aws lambda list-functions --profile prod --region us-east-1 --query "Functions[?starts_with(FunctionName, 'pest')].[FunctionName, Runtime, MemorySize, Timeout]" --output table
aws lambda get-function-configuration --function-name pest-detection-processor --profile prod --region us-east-1
```

The second command is the authoritative read of the processor's live env vars (this is how the live values in 9.5 were captured).

**API Gateway routes and authorizer.** Console: API Gateway → HTTP APIs → pest-monitoring-api-gateway (vzf17s6z00) → Routes / Authorization. CLI:

```
aws apigatewayv2 get-routes --api-id vzf17s6z00 --profile prod --region us-east-1 --output table
```

```
aws apigatewayv2 get-authorizers --api-id vzfl7s6z00 --profile prod --region us-east-1
```

**DynamoDB tables and camera rows.** Console: DynamoDB → Tables → pest-monitoring-cameras → Explore table items. CLI:

```
aws dynamodb list-tables --profile prod --region us-east-1
aws dynamodb scan --table-name pest-monitoring-cameras --profile prod --region us-east-1
```

The scan is the definitive live read of the camera rows in 9.7: it shows `custom_model_arn`, `min_confidence`, `tiling_enabled`, `llm_verify_enabled` per camera.

**Rekognition versions and full ARNs.** Console: Rekognition → Use custom labels → Projects → pick the project → Models. CLI (this is how the full ARN suffixes in 9.3.7 are verified):

```
aws rekognition describe-projects --profile prod --region us-east-1 --query "ProjectDescriptions[0].ProjectArn"
aws rekognition describe-project-versions --project-arn **<project ARN from the previous command>** --profile prod --region us-east-1 --query "ProjectVersionDescriptions[0].[ProjectVersionArn,Status,EvaluationResult.F1Score]"
```

**Endpoint state before/after testing.** Same describe-project-versions call; Status RUNNING means it is billing. Stop with:

```
aws rekognition stop-project-version --project-version-arn "**<version ARN>**" --profile prod --region us-east-1
```

**EventBridge schedules.** Console: Amazon EventBridge → Scheduler → Schedules. CLI:

```
aws scheduler list-schedules --profile prod --region us-east-1 --output table
aws scheduler get-schedule --name pest-model-watchdog-15min --profile prod --region us-east-1
```

Runtime per-camera rules: Console: EventBridge → Rules (filter pest-sched-). CLI: `aws events list-rules --name-prefix pest-sched- --profile prod --region us-east-1`.

**Cognito.** Console: Cognito → User pools → pest-dashboard-users. CLI:

```
aws cognito-idp describe-user-pool --user-pool-id us-east-1_9selFDHpc --profile prod --region us-east-1
aws cognito-idp describe-user-pool-client --user-pool-id us-east-1_9selFDHpc --client-id 6vebotf45bp8u46cnraddiapl --profile prod --region us-east-1
```

**CloudFront.** Console: CloudFront → Distributions → E1YADURLSAVNFA. CLI: `aws cloudfront get-distribution --id E1YADURLSAVNFA --profile prod`.

**S3 buckets, CORS, notification.** Console: S3 → pick the bucket → Permissions / Properties. CLI:

```
aws s3api get-bucket-notification-configuration --bucket argus-frames-506868652945 --profile prod
```

```
aws s3api get-bucket-cors --bucket argus-frames-506868652945 --profile prod
```

**KVS.** Console: Kinesis Video Streams → Video streams. CLI: `aws kinesisanalytics list-streams --profile prod --region us-east-1`.

**SES.** Console: Amazon SES → Identities. CLI: `aws ses list-identities --profile prod --region us-east-1` and `aws sesv2 get-account --profile prod --region us-east-1` (shows sandbox state).

**IAM roles.** Console: IAM → Roles → filter pest. CLI: `aws iam list-role-policies --role-name pest-detection-processor-role --profile prod` then `aws iam get-role-policy --role-name pest-detection-processor-role --policy-name **<policy name>** --profile prod`.

**Bedrock model access.** Console: Bedrock → Model access. CLI: `aws bedrock list-inference-profiles --profile prod --region us-east-1` (the valid us. ids) and `aws bedrock get-foundation-model-availability --model-id **<model id>** --profile prod --region us-east-1`.

**Reproduction.** This chapter is the target-state checklist; the build procedure itself is Chapter 8 (manual) and Chapter 7 (the ARGUS deployer, driven by `deployer/STACK_MANIFEST.md` — the authoritative creation order, parameter table, and per-resource raw specs under `deployer/audit/`). On a fresh account, everything in 9.3 marked PROD must exist; everything marked DEAD/legacy must not be recreated; Rekognition models must be retrained (account-bound); and every value in the `STACK_MANIFEST` parameter table (`ACCOUNT_ID`, `REGION`, pool/client/API ids, emails, ARNs) must be replaced with the new account's values. This is no longer theoretical: the 2026-08-10 deploy onto the NP production account (9.3.14) executed exactly this procedure — 15 stages, 103 seconds — and its section is what a correct fresh deployment looks like.

## 9.11 Cross-references

- Chapter 1 — where each registry entry sits in the end-to-end chain.
- Chapter 2 — the AWS cloud backend: the Lambdas, tables, buckets, and API of 9.3 in working detail.
- Chapter 3 — the model versions of 9.3.7: what each was trained on and why v9 + LLM gate is the live pair.
- Chapter 4 — the ARGUS dashboard: consumer of the API routes (9.3.5) and Cognito/CloudFront ids (9.3.6).
- Chapter 5 — the Go2 + Orin devices of 9.4 and the patrol constants of 9.5.
- Chapter 6 — the mini PC devices and services of 9.4/9.5.
- Chapter 7 — the ARGUS deployer that recreates 9.3 from `deployer/STACK_MANIFEST.md`.
- Chapter 8 — the from-scratch reproduction runbook that 9.10 points into.